geospatial-analytics-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `geospatial_analytics_ai999x.about()` for the same answer.

Every capability geospatial-analytics-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import geospatial_analytics_ai999x

result = geospatial_analytics_ai999x.solve('geo_geodesic_destination', lat1=52.2, lon1=0.12,
azimuth=45, distance='100000 m', ellipsoid='WGS84')
result.show()
# lat2 = 52.8308, lon2 = 1.16915, azimuth2 = 45.8325
```

By topic

topic	capabilities
Cartography	Ground distance from a map distance and a representative fraction, Map scale (representative fraction) from a map and a ground distance
Coordinate notation	Degrees, minutes and seconds to decimal degrees, Decimal degrees to degrees, minutes and seconds, Geohash of a point and its cell, Decode a geohash: cell centre and size, MGRS grid reference of a point (1 m)
Coordinate systems and conversions	Geodetic to geocentric (ECEF) coordinates, Geocentric (ECEF) to geodetic coordinates, Geodetic to local east-north-up (ENU) coordinates, Local east-north-up (ENU) to geodetic coordinates, Look angles: azimuth, elevation and range from an observer to a target, Auxiliary latitudes: geocentric, reduced, conformal, authalic and rectifying, UTM zone, latitude band and central meridian
Ellipsoidal geodesy	Geodesic destination point on the ellipsoid (direct problem), Midpoint of a geodesic on the ellipsoid, Area and perimeter of a geodesic polygon on the ellipsoid, Meridian arc length between two latitudes, Arc length along a parallel of latitude, Area of a latitude-longitude cell on the ellipsoid, Radii of curvature of the ellipsoid at a latitude, Rhumb-line (loxodrome) distance and bearing on the ellipsoid, Destination along a rhumb line (loxodrome) on the ellipsoid
Facility location	Closest facility for every node of a network, P-median facility location on a network, Maximal covering location problem (MCLP) on a network
Geostatistics	Inverse distance weighted (IDW) interpolation at a point, Empirical (Matheron) semivariogram in distance bins, Fit an exponential or Gaussian variogram model (nugget, partial sill, range), Ordinary kriging estimate and kriging variance at a point, Polynomial trend surface (first or second order) by least squares
Map projections	Transverse Mercator projection: coordinates, scale factor and grid convergence, Latitude and longitude to UTM coordinates (with scale factor and convergence), UTM coordinates to latitude and longitude, Latitude and longitude to Web Mercator (EPSG:3857), Web Mercator (EPSG:3857) to latitude and longitude, Web-map tile (slippy map / XYZ) and quadkey containing a point, Mercator projection (ellipsoidal): coordinates and scale factor, Inverse Mercator projection (ellipsoidal), Lambert conformal conic projection (two standard parallels): coordinates, scale factor and convergence, Inverse Lambert conformal conic projection (two standard parallels), Albers equal-area conic projection: coordinates and scale factors, Inverse Albers equal-

topic	capabilities
	area conic projection, Polar stereographic projection (standard parallel): coordinates and scale factor, Lambert azimuthal equal-area projection (ellipsoidal), Azimuthal equidistant projection (ellipsoidal, geodesic), Sinusoidal equal-area projection (ellipsoidal), Mollweide equal-area projection (sphere), Orthographic projection (sphere), Gnomonic projection (sphere), Equirectangular (equidistant cylindrical / plate carree) projection, Lambert cylindrical equal-area projection (sphere), Tissot's indicatrix: scale factors and distortion of a projection at a point
Network accessibility	Gravity-based (Hansen) accessibility on a road network, Two-step floating catchment area (2SFCA) accessibility, Network service area (isochrone) of a facility, Network detour index (circuity) between two nodes, Network connectivity indices: cyclomatic number, alpha, beta and gamma, Shimbel accessibility index of every node
Planar vector analysis	Douglas-Peucker line simplification, Discrete Frechet distance between two polylines, Shape compactness indices of a polygon (Polsby-Popper, Schwartzberg, convex hull, Reock)
Point pattern analysis	Nearest-neighbour index (Clark-Evans ratio) of a point pattern, Quadrat analysis: chi-square test of complete spatial randomness, Ripley's K and L functions of a point pattern, Mean centre and standard distance of a point pattern, Weighted mean centre and weighted standard distance, Standard deviational ellipse of a point pattern, Kernel density estimate of a point pattern at a location
Remote sensing	Vegetation indices: NDVI, EVI, SAVI, OSAVI, MSAVI and EVI2 from band reflectances, Water, moisture, burn and built-up indices: NDWI, MNDWI, NDMI, NBR and NDBI, Top-of-atmosphere reflectance from at-sensor radiance, At-sensor brightness temperature from thermal radiance, Ground sample distance and swath of a nadir-looking camera, Thematic map accuracy: overall accuracy, kappa, producer's and user's accuracy
Spatial statistics	Global Moran's I spatial autocorrelation with normal and randomization inference, Geary's C spatial autocorrelation with normal and randomization inference, Getis-Ord general G (global high/low clustering), Local Getis-Ord Gi and Gi* hot-spot statistics, Local Moran's I (LISA) and Moran scatterplot quadrants, Bivariate Moran's I (spatial cross-correlation), Join count statistics of a binary map (BB, BW, WW), Spatial lag of a variable, K-nearest-neighbour spatial weights, Distance-band spatial weights
Terrain analysis	Slope and aspect of a 3x3 elevation window (Horn's method), Hillshade (analytical relief shading) of a 3x3 elevation window, Terrain curvature: total, profile and plan curvature (Zevenbergen-Thorne), Terrain ruggedness index (Riley), topographic position index and local relief, D8 flow direction of every cell of a small DEM, D8 flow accumulation of a small DEM, Watershed (catchment) of an outlet cell on a small DEM, Line-of-sight visibility along a terrain profile (with Earth curvature and refraction)
Urban geography	Rank-size rule (Zipf) fit of a settlement system, Urban population density gradient (Clark's negative exponential model), Spatial interaction (gravity) model fitted by Poisson regression, Huff model: probabilities of shopping at each store from an origin (network distances), Residential segregation (evenness): dissimilarity, Gini and Theil entropy indices, Residential segregation (exposure): isolation, interaction and correlation ratio

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below

outcome	meaning	what to do
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
geo_geodesic_destination	Geodesic destination point on the ellipsoid (direct problem)	lat1, lon1, azimuth, distance, ellipsoid	lat2, lon2, azimuth2
geo_geodesic_midpoint	Midpoint of a geodesic on the ellipsoid	lat1, lon1, lat2, lon2, ellipsoid	lat_mid, lon_mid, distance
geo_geodesic_polygon_area	Area and perimeter of a geodesic polygon on the ellipsoid	lats, lons, ellipsoid	area, perimeter
geo_meridian_arc	Meridian arc length between two latitudes	lat1, lat2, ellipsoid	distance, m1, m2
geo_parallel_arc	Arc length along a parallel of latitude	lat, lon1, lon2, ellipsoid	distance, radius

id	operation	inputs	outputs
geo_graticule_cell_area	Area of a latitude-longitude cell on the ellipsoid	lat1, lat2, lon1, lon2, ellipsoid	area, fraction_of_ellipsoid
geo_radii_of_curvature	Radii of curvature of the ellipsoid at a latitude	lat, ellipsoid	M, N, R_gauss
geo_rhumb_line_inverse	Rhumb-line (loxodrome) distance and bearing on the ellipsoid	lat1, lon1, lat2, lon2, ellipsoid	distance, bearing
geo_rhumb_line_destination	Destination along a rhumb line (loxodrome) on the ellipsoid	lat1, lon1, bearing, distance, ellipsoid	lat2, lon2
geo_geodetic_to_ecef	Geodetic to geocentric (ECEF) coordinates	lat, lon, h, ellipsoid	X, Y, Z
geo_ecef_to_geodetic	Geocentric (ECEF) to geodetic coordinates	X, Y, Z, ellipsoid	lat, lon, h
geo_geodetic_to_enu	Geodetic to local east-north-up (ENU) coordinates	lat0, lon0, h0, lat, lon, h, ellipsoid	e, n, u, slant_range
geo_enu_to_geodetic	Local east-north-up (ENU) to geodetic coordinates	lat0, lon0, h0, e, n, u, ellipsoid	lat, lon, h
geo_look_angles	Look angles: azimuth, elevation and range from an observer to a target	lat0, lon0, h0, lat, lon, h, ellipsoid	azimuth, elevation, slant_range
geo_auxiliary_latitudes	Auxiliary latitudes: geocentric, reduced, conformal, authalic and rectifying	lat, ellipsoid	geocentric, reduced, conformal, authalic, rectifying
geo_utm_zone	UTM zone, latitude band and central meridian	lat, lon	zone, band, hemisphere, central_meridian
geo_transverse_mercator_forward	Transverse Mercator projection: coordinates, scale factor and grid convergence	lat, lon, lat0, lon0, k0, ellipsoid, FE, FN	x, y, k, gamma
geo_utm_forward	Latitude and longitude to UTM coordinates (with scale factor and convergence)	lat, lon, ellipsoid	easting, northing, zone, hemisphere, k, gamma
geo_utm_inverse	UTM coordinates to latitude and longitude	easting, northing, zone, hemisphere, ellipsoid	lat, lon
geo_web_mercator_forward	Latitude and longitude to Web Mercator (EPSG:3857)	lat, lon	x, y
geo_web_mercator_inverse	Web Mercator (EPSG:3857) to latitude and longitude	x, y	lat, lon
geo_web_map_tile	Web-map tile (slippy map / XYZ) and quadkey	lat, lon, zoom	tile_x, tile_y, quadkey

id	operation	inputs	outputs
	containing a point		
geo_mercator_forward	Mercator projection (ellipsoidal): coordinates and scale factor	lat, lon, lon0, lat_ts, ellipsoid	x, y, k
geo_mercator_inverse	Inverse Mercator projection (ellipsoidal)	x, y, lon0, lat_ts, ellipsoid	lat, lon
geo_lambert_conformal_conic_forward	Lambert conformal conic projection (two standard parallels): coordinates, scale factor and convergence	lat, lon, lat1, lat2, lat0, lon0, ellipsoid, FE, FN	x, y, k, gamma
geo_lambert_conformal_conic_inverse	Inverse Lambert conformal conic projection (two standard parallels)	x, y, lat1, lat2, lat0, lon0, ellipsoid, FE, FN	lat, lon
geo_albers_equal_area_forward	Albers equal-area conic projection: coordinates and scale factors	lat, lon, lat1, lat2, lat0, lon0, ellipsoid, FE, FN	x, y, k, h
geo_albers_equal_area_inverse	Inverse Albers equal-area conic projection	x, y, lat1, lat2, lat0, lon0, ellipsoid, FE, FN	lat, lon
geo_polar_stereographic_forward	Polar stereographic projection (standard parallel): coordinates and scale factor	lat, lon, pole, lat_ts, lon0, ellipsoid, FE, FN	x, y, k
geo_lambert_azimuthal_equal_area_forward	Lambert azimuthal equal-area projection (ellipsoidal)	lat, lon, lat0, lon0, ellipsoid, FE, FN	x, y
geo_azimuthal_equidistant_forward	Azimuthal equidistant projection (ellipsoidal, geodesic)	lat, lon, lat0, lon0, ellipsoid, FE, FN	x, y, distance, azimuth
geo_sinusoidal_forward	Sinusoidal equal-area projection (ellipsoidal)	lat, lon, lon0, ellipsoid	x, y
geo_mollweide_forward	Mollweide equal-area projection (sphere)	lat, lon, lon0, R	x, y
geo_orthographic_forward	Orthographic projection (sphere)	lat, lon, lat0, lon0, R	x, y
geo_gnomonic_forward	Gnomonic projection (sphere)	lat, lon, lat0, lon0, R	x, y
geo_equirectangular_forward	Equirectangular (equidistant cylindrical / plate carree) projection	lat, lon, lat_ts, lat0, lon0, R	x, y
geo_cylindrical_equal_area_forward	Lambert cylindrical equal-area projection (sphere)	lat, lon, lat_ts, lon0, R	x, y
geo_tissot_indicatrix	Tissot's indicatrix: scale factors and distortion of a projection at a point	projection, lat, lon	h, k, theta_prime, a, b, omega, s
geo_dms_to_decimal	Degrees, minutes and seconds to decimal	degrees, minutes, seconds,	decimal, radians

id	operation	inputs	outputs
	degrees	hemisphere	
geo_decimal_to_dms	Decimal degrees to degrees, minutes and seconds	decimal, axis	degrees, minutes, seconds, hemisphere
geo_map_scale_ground_distance	Ground distance from a map distance and a representative fraction	map_distance, scale	ground_distance, ground_km
geo_map_scale_from_distances	Map scale (representative fraction) from a map and a ground distance	map_distance, ground_distance	scale, rf
geo_geohash_encode	Geohash of a point and its cell	lat, lon, precision	geohash, lat_center, lon_center, lat_error, lon_error
geo_geohash_decode	Decode a geohash: cell centre and size	geohash	lat_center, lon_center, lat_error, lon_error
geo_mgrs_encode	MGRS grid reference of a point (1 m)	lat, lon	mgrs
geo_douglas_peucker_simplification	Douglas-Peucker line simplification	x, y, tolerance	kept, n_kept
geo_discrete_frechet_distance	Discrete Frechet distance between two polylines	x1, y1, x2, y2	distance
geo_polygon_compactness	Shape compactness indices of a polygon (Polsby-Popper, Schwartzberg, convex hull, Reock)	x, y	area, perimeter, polsby_popper, schwartzberg, convex_hull_ratio, reock
geo_morans_i	Global Moran's I spatial autocorrelation with normal and randomization inference	y, W, transform	I, EI, VI_norm, z_norm, p_norm, VI_rand, z_rand, p_rand
geo_gearys_c	Geary's C spatial autocorrelation with normal and randomization inference	y, W, transform	C, VC_norm, z_norm, VC_rand, z_rand
geo_getis_ord_general_g	Getis-Ord general G (global high/low clustering)	y, W	G, EG, VG, z_norm
geo_local_getis_ord_gi	Local Getis-Ord Gi and Gi* hot-spot statistics	y, W, statistic	Gs, Zs
geo_local_morans_i	Local Moran's I (LISA) and Moran scatterplot quadrants	y, W	Is, quadrant
geo_bivariate_morans_i	Bivariate Moran's I (spatial cross-correlation)	x, y, W	I_xy
geo_join_counts	Join count statistics of a binary map (BB, BW, WW)	y, W	BB, BW, WW, J
geo_spatial_lag	Spatial lag of a variable	y, W, transform	lag

id	operation	inputs	outputs
geo_knn_weights	k-nearest-neighbour spatial weights	x, y, k	W, s0, symmetric
geo_distance_band_weights	Distance-band spatial weights	x, y, threshold	W, neighbours, s0, islands
geo_nearest_neighbour_index	Nearest-neighbour index (Clark-Evans ratio) of a point pattern	x, y, area	d_obs, d_exp, R, z, p_value
geo_quadrat_test	Quadrat analysis: chi-square test of complete spatial randomness	x, y, nx, ny	chi2, df, p_value, vmr, counts
geo_ripley_k	Ripley's K and L functions of a point pattern	x, y, d	K, L, area
geo_mean_center_standard_distance	Mean centre and standard distance of a point pattern	x, y	mean_x, mean_y, std_distance
geo_weighted_mean_center	Weighted mean centre and weighted standard distance	x, y, w	wmean_x, wmean_y, weighted_std_distance
geo_standard_deviational_ellipse	Standard deviational ellipse of a point pattern	x, y	sx, sy, orientation
geo_kernel_density	Kernel density estimate of a point pattern at a location	x, y, px, py, bandwidth, kernel	density, intensity
geo_idw_interpolation	Inverse distance weighted (IDW) interpolation at a point	x, y, z, px, py, power	estimate
geo_empirical_semivariogram	Empirical (Matheron) semivariogram in distance bins	x, y, z, bin_edges	gamma, counts
geo_variogram_model_fit	Fit an exponential or Gaussian variogram model (nugget, partial sill, range)	h, gamma, model	nugget, psill, range, sill
geo_ordinary_kriging	Ordinary kriging estimate and kriging variance at a point	x, y, z, px, py, model, nugget, sill, range	estimate, variance
geo_trend_surface	Polynomial trend surface (first or second order) by least squares	x, y, z, order	coefficients, r_squared
geo_rank_size_rule	Rank-size rule (Zipf) fit of a settlement system	populations	q, P1_fit, r_squared, primacy
geo_density_gradient	Urban population density gradient (Clark's negative exponential model)	r, D	D0, b, r_squared, half_distance
geo_gravity_model_poisson_fit	Spatial interaction (gravity) model fitted by Poisson regression	T, O, D, d	k, alpha, beta, gamma

id	operation	inputs	outputs
geo_vegetation_indices	Vegetation indices: NDVI, EVI, SAVI, OSAVI, MSAVI and EVI2 from band reflectances	red, nir, blue, L	NDVI, EVI, SAVI, OSAVI, MSAVI, EVI2
geo_water_burn_built_indices	Water, moisture, burn and built-up indices: NDWI, MNDWI, NDMI, NBR and NDBI	green, nir, swir1, swir2	NDWI, MNDWI, NDMI, NBR, NDBI
geo_toa_reflectance	Top-of-atmosphere reflectance from at-sensor radiance	radiance, esun, doy, sun_elevation	earth_sun_distance, reflectance
geo_brightness_temperature	At-sensor brightness temperature from thermal radiance	radiance, K1, K2	T
geo_ground_sample_distance	Ground sample distance and swath of a nadir-looking camera	pixel_pitch, focal_length, altitude, pixels	gsd, swath
geo_classification_accuracy	Thematic map accuracy: overall accuracy, kappa, producer's and user's accuracy	matrix	overall, kappa, producers, users
geo_slope_aspect_horn	Slope and aspect of a 3x3 elevation window (Horn's method)	window, cell_size	slope_deg, slope_percent, aspect_deg
geo_hillshade	Hillshade (analytical relief shading) of a 3x3 elevation window	window, cell_size, sun_azimuth, sun_altitude	illumination, hillshade
geo_terrain_curvature	Terrain curvature: total, profile and plan curvature (Zevenbergen-Thorne)	window, cell_size	curvature, profile, plan
geo_terrain_ruggedness	Terrain ruggedness index (Riley), topographic position index and local relief	window	TRI, TPI, relief
geo_d8_flow_direction	D8 flow direction of every cell of a small DEM	dem, cell_size	direction
geo_d8_flow_accumulation	D8 flow accumulation of a small DEM	dem, cell_size	accumulation, max_accumulation
geo_watershed_delineation	Watershed (catchment) of an outlet cell on a small DEM	dem, cell_size, row, col	cells, area, mask
geo_line_of_sight_profile	Line-of-sight visibility along a terrain profile (with Earth curvature and refraction)	d, z, observer_height, target_height, curvature	visible, n_visible
geo_gravity_accessibility	Gravity-based (Hansen) accessibility on a road network	edges, opportunities, beta, impedance	accessibility

id	operation	inputs	outputs
geo_huff_model	Huff model: probabilities of shopping at each store from an origin (network distances)	edges, origin, stores, attractiveness, alpha, beta	probabilities, distances
geo_two_step_floating_catchment	Two-step floating catchment area (2SFCA) accessibility	edges, population, supply_nodes, capacity, threshold	accessibility, ratios
geo_service_area	Network service area (isochrone) of a facility	edges, source, cutoff	reachable, count, farthest
geo_closest_facility	Closest facility for every node of a network	edges, facilities	nearest, distance, mean_distance
geo_p_median	p-median facility location on a network	edges, demand, candidates, p	objective, facilities, mean_distance
geo_maximal_covering_location	Maximal covering location problem (MCLP) on a network	edges, demand, candidates, p, coverage	covered, fraction
geo_detour_index	Network detour index (circuity) between two nodes	coords, edges, origin, destination	network_distance, euclidean_distance, detour_index
geo_network_connectivity_indices	Network connectivity indices: cyclomatic number, alpha, beta and gamma	edges	vertices, edges, components, cyclomatic, alpha, beta, gamma
geo_shimbel_index	Shimbel accessibility index of every node	edges	shimbel, total, most_accessible
geo_segregation_evenness	Residential segregation (evenness): dissimilarity, Gini and Theil entropy indices	x, t	dissimilarity, gini, theil_h
geo_segregation_exposure	Residential segregation (exposure): isolation, interaction and correlation ratio	x, t	isolation, exposure, correlation_ratio

Geodesic destination point on the ellipsoid (direct problem) — `geo_geodesic_destination`

The point reached by travelling a given distance along a geodesic from a start point at a given initial azimuth on the WGS84 or GRS80 ellipsoid, and the azimuth of the geodesic on arrival.

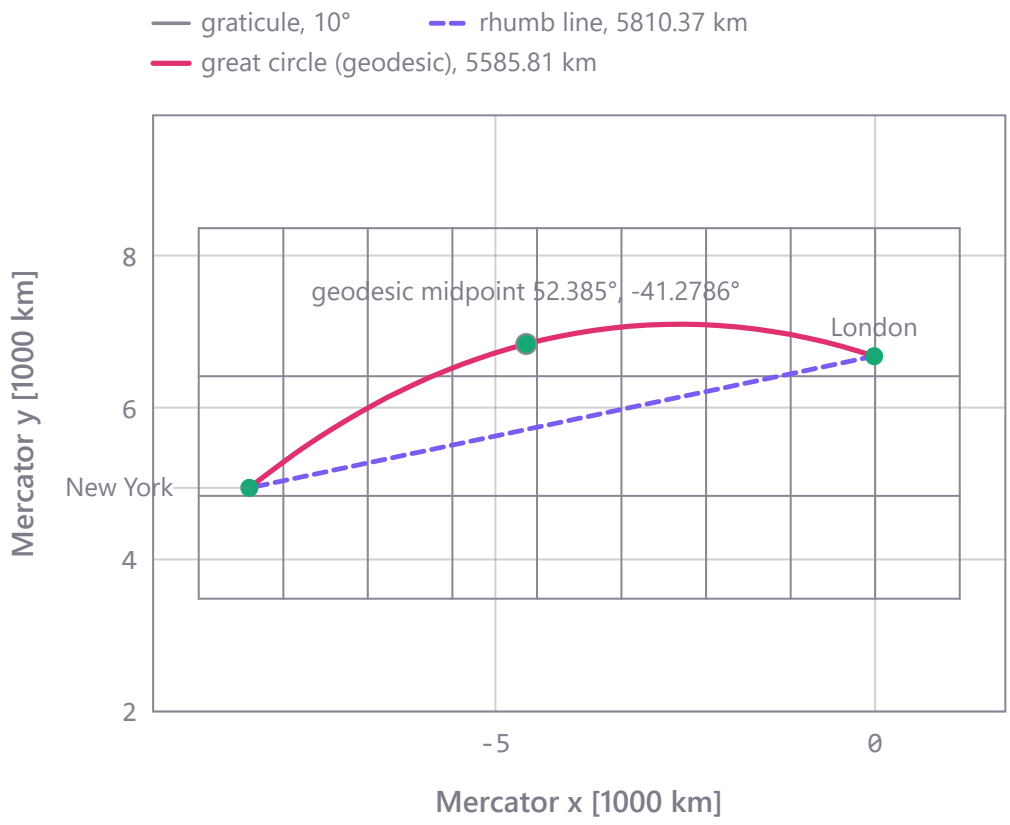
$$\frac{s}{b} = \int_{\sigma_1}^{\sigma_2} \sqrt{1 + k^2 \sin^2\sigma}\, d\sigma, \quad \lambda_{12} = \omega_{12} - f \sin\alpha_0 \int_{\sigma_1}^{\sigma_2} \frac{2 - f}{1 + (1 - f)\sqrt{1 + k^2 \sin^2\sigma}}\, d\sigma$$

input	meaning	condition
`lat1`	latitude of the start point (decimal degrees)	
`lon1`	longitude of the start point (decimal degrees)	
`azimuth`	initial azimuth, clockwise from north (decimal degrees)	
`distance`	geodesic distance travelled	

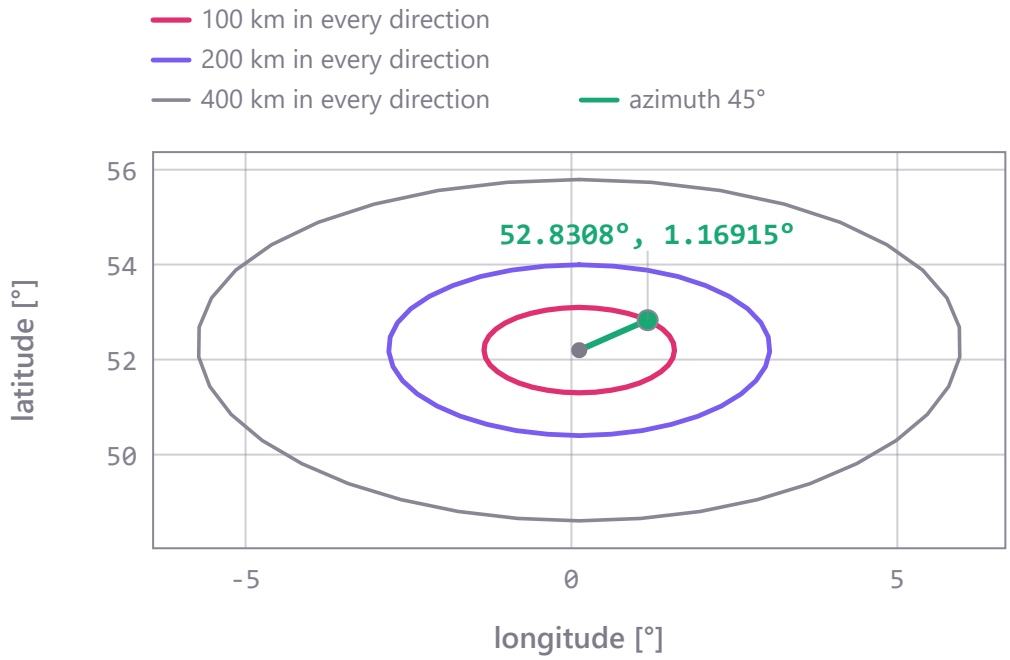
input	meaning	condition
`ellipsoid`	reference ellipsoid	

output	meaning
`lat2`	latitude of the destination (degrees)
`lon2`	longitude of the destination (degrees)
`azimuth2`	forward azimuth at the destination (degrees clockwise from north)

- **Method:** Bessel's auxiliary sphere: solve bI_1(sigma) = s by Newton steps with the arc-length integral by Gauss-Legendre quadrature, then the longitude integral
- **Assumes:** start latitude strictly between -90 and 90; 0 < distance <= 15000 km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



London to New York on a Mercator map (WGS84): the geodesic leaves at azimuth 288.373° and is 5585.81 km long; the rhumb line keeps bearing 258.088° and is 5810.37 km. Each path is 40 destination points computed by the package and projected by it; the rhumb line is straight on this map. Marked: the geodesic midpoint, latitude 52.385°, longitude -41.2786°.



Destinations from latitude 52.2°, longitude 0.12° (WGS84) at every 10° of azimuth for distances 100, 200 and 400 km, each point a run of the package (111 runs), drawn in longitude–latitude (the circles widen east–west because a degree of longitude is shorter here). Marked: the worked example, azimuth 45° for 100 km → 52.8308°, 1.16915°, arriving at azimuth 45.8325°.

```
geospatial_analytics_ai999x.solve('geo_geodesic_destination', lat1=52.2, lon1=0.12, azimuth=45,
distance=100000.0, ellipsoid='WGS84')
# -> lat2 = 52.8308, lon2 = 1.16915, azimuth2 = 45.8325   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_geodesic_destination', lat1=-33.87, lon1=151.21,
azimuth=240, distance=2000000.0, ellipsoid='GRS80')
# -> lat2 = -41.1854, lon2 = 130.45, azimuth2 = 252.759   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geodesic destination: lat1 = 52.2, lon1 = 0.12, azimuth = 45, distance = 100 km, ellipsoid = WGS84	SOLVED	lat2 = 52.8308, lon2 = 1.16915, azimuth2 = 45.8325
Geodesic destination: lat1 = -33.87, lon1 = 151.21, azimuth = 240, distance = 2000 km, ellipsoid = GRS80	SOLVED	lat2 = -41.1854, lon2 = 130.45, azimuth2 = 252.759
Geodesic destination: lon1 = 0.12, azimuth = 45, distance = 100 km, ellipsoid = WGS84	NEEDS_INPUT	geo_geodesic_destination needs lat1 (latitude of the start point (decimal degrees)). Example: solve('geo_geodesic_destin
Geodesic destination: lat1 = 52.2, lon1 = 0.12, azimuth = 45, distance = 100 km, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ
Geodesic destination: lat1 = 52.2°, lon1 = 0.12°, azimuth = 45°, distance = 100000 m, ellipsoid = WGS84	SOLVED	lat2 = 52.8308, lon2 = 1.16915, azimuth2 = 45.8325
Geodesic destination: lat1 = 95, lon1 = 0.12, azimuth = 45, distance = 100	INVALID_INPUT	Geodesic destination point on the ellipsoid (direct problem): lat1 must lie strictly between -90 and 90

question	outcome	result
km, ellipsoid = WGS84		degrees.
Geodesic destination: lat1 = 52.2, lon1 = 0.12, azimuth = 45, distance = 100, ellipsoid = WGS84	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Midpoint of a geodesic on the ellipsoid — `geo_geodesic_midpoint`

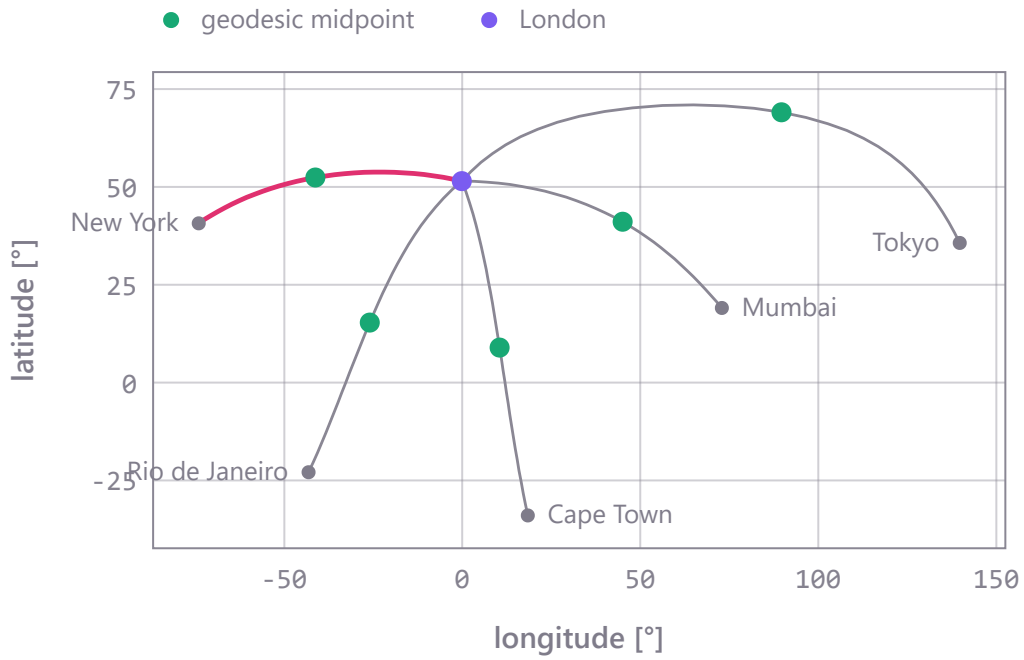
The point halfway along the shortest path (geodesic) between two points on the WGS84 or GRS80 ellipsoid, with the geodesic length.

$$P_{mid} = \mathrm{direct}\,(P_1,\ \alpha_1,\ s_{12}/2)$$

input	meaning	condition
`lat1`	latitude of the first point (decimal degrees)	
`lon1`	longitude of the first point (decimal degrees)	
`lat2`	latitude of the second point (decimal degrees)	
`lon2`	longitude of the second point (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`lat_mid`	latitude of the midpoint (degrees)
`lon_mid`	longitude of the midpoint (degrees)
`distance`	length of the geodesic between the two points

- **Method:** inverse problem (iteration on the auxiliary-sphere longitude, exact integrals) for the azimuth and length, then the direct problem over half the length
- **Assumes:** distinct points at most 150 degrees apart; latitudes strictly between -90 and 90
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Geodesics from London to five cities drawn in longitude–latitude, each path 30 destination points along the initial azimuth the package returns, with the midpoint the package computes: New York: 52.385°, -41.2786° (5585.81 km); Cape Town: 8.94345°, 10.4769° (9635.07 km); Mumbai: 41.0888°, 45.0042° (7199.57 km); Tokyo: 69.0606°, 89.6146° (9583.99 km); Rio de Janeiro: 15.3347°, -25.999° (9251.66 km). Marked (thick): the worked example to New York; on these axes the geodesic bows poleward, so its midpoint lies north of both ends.

```
geospatial_analytics_ai999x.solve('geo_geodesic_midpoint', lat1=51.5, lon1=-0.13, lat2=40.71,
lon2=-74.01, ellipsoid='WGS84')
# -> lat_mid = 52.385, lon_mid = -41.2786, distance = 5.58581e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_geodesic_midpoint', lat1=35.68, lon1=139.69, lat2=-33.87,
lon2=151.21, ellipsoid='GRS80')
# -> lat_mid = 0.912597, lon_mid = 145.513, distance = 7.7921e+06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geodesic midpoint: lat1 = 51.5, lon1 = -0.13, lat2 = 40.71, lon2 = -74.01, ellipsoid = WGS84	SOLVED	lat_mid = 52.385, lon_mid = -41.2786, distance = 5.58581e+06 m
Geodesic midpoint: lat1 = 35.68, lon1 = 139.69, lat2 = -33.87, lon2 = 151.21, ellipsoid = GRS80	SOLVED	lat_mid = 0.912597, lon_mid = 145.513, distance = 7.7921e+06 m
Geodesic midpoint: lon1 = -0.13, lat2 = 40.71, lon2 = -74.01, ellipsoid = WGS84	NEEDS_INPUT	geo_geodesic_midpoint needs lat1 (latitude of the first point (decimal degrees)). Example: solve('geo_geodesic_midpoint'
Geodesic midpoint: lat1 = 51.5, lon1 = -0.13, lat2 = 40.71, lon2 = -74.01, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ
Geodesic midpoint: lat1 = 10, lon1 = 20, lat2 = 10, lon2 = 20, ellipsoid = WGS84	INVALID_INPUT	Midpoint of a geodesic on the ellipsoid: the two points must be distinct.

question	outcome	result
Geodesic midpoint: lat1 = 51.5, lon1 = -0.13, lat2 = 40.71, lon2 = -74.01, ellipsoid = Clarke	INVALID_INPUT	ellipsoid: one of WGS84, GRS80; got 'Clarke'.

Area and perimeter of a geodesic polygon on the ellipsoid — `geo_geodesic_polygon_area`

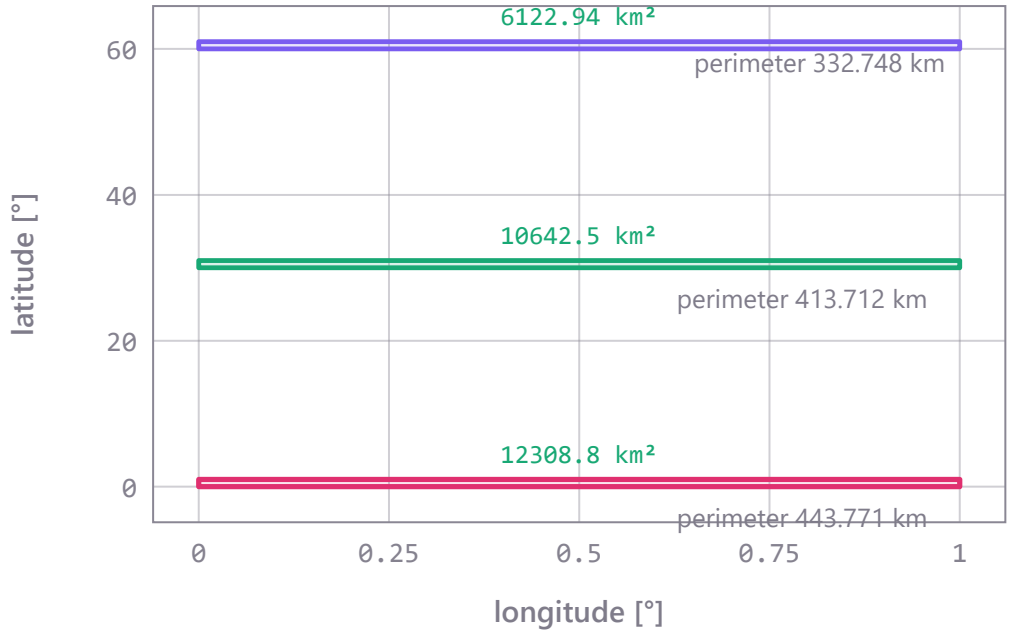
The area enclosed by a polygon whose edges are geodesics on the WGS84 or GRS80 ellipsoid, and its perimeter (vertices given as latitude and longitude lists, in either orientation).

$$S = -\oint \frac{a^2}{2}\left[q(\varphi) - q(\varphi_c)\right] d\lambda, \quad q(\varphi) = (1-e^2)\left[\frac{\sin\varphi}{1-e^2\sin^2\varphi} + \frac{\operatorname{artanh}(e\sin\varphi)}{e}\right]$$

input	meaning	condition
`lats`	latitudes of the vertices in order (decimal degrees)	
`lons`	longitudes of the vertices in the same order (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`area`	enclosed area on the ellipsoid
`perimeter`	perimeter (sum of the geodesic edge lengths)

- **Method:** Green's theorem on the ellipsoid: the authalic strip area a^2 q(phi)/2 integrated against the longitude along each geodesic edge (Gauss-Legendre in the auxiliary-sphere arc), summed over the edges
- **Assumes:** a simple polygon not enclosing a pole; vertices within 88 degrees of the equator; edges at most 90 degrees long
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example's quadrilateral (latitudes [0, 0, 1, 1], longitudes [0, 1, 1, 0]) and the same shape moved 30° and 60° north, each with the geodesic area and perimeter the package returns: shifted by 0°: 12308.8 km², perimeter 443.771 km; shifted by 30°: 10642.5 km², perimeter 413.712 km; shifted by 60°: 6122.94 km², perimeter 332.748 km. The same span of degrees encloses less ground toward the pole.

```
geospatial_analytics_ai999x.solve('geo_geodesic_polygon_area', lats=[0, 0, 1, 1], lons=[0, 1, 1, 0], ellipsoid='WGS84')
# -> area = 1.23088e+10 m^2, perimeter = 443771 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_geodesic_polygon_area', lats=[48.0, 49.5, 47.2], lons=[2.0, 4.5, 6.1], ellipsoid='GRS80')
# -> area = 3.36697e+10 m^2, perimeter = 850958 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geodesic polygon area: lats = [0, 0, 1, 1], lons = [0, 1, 1, 0], ellipsoid = WGS84	SOLVED	area = 1.23088e+10 m^2, perimeter = 443771 m
Geodesic polygon area: lats = [48, 49.5, 47.2], lons = [2, 4.5, 6.1], ellipsoid = GRS80	SOLVED	area = 3.36697e+10 m^2, perimeter = 850958 m
Geodesic polygon area: lons = [0, 1, 1, 0], ellipsoid = WGS84	NEEDS_INPUT	geo_geodesic_polygon_area needs lats (latitudes of the vertices in order (decimal degrees)). Example: solve('geo_geodesi
Geodesic polygon area: lats = [0, 0, 1, 1], lons = [0, 1, 1, 0], ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ
Geodesic polygon area: lats = [0, 0, 1], lons = [0, 1], ellipsoid = WGS84	INVALID_INPUT	lons: needs at least 3 entries, got 2.

Meridian arc length between two latitudes — `geo_meridian_arc`

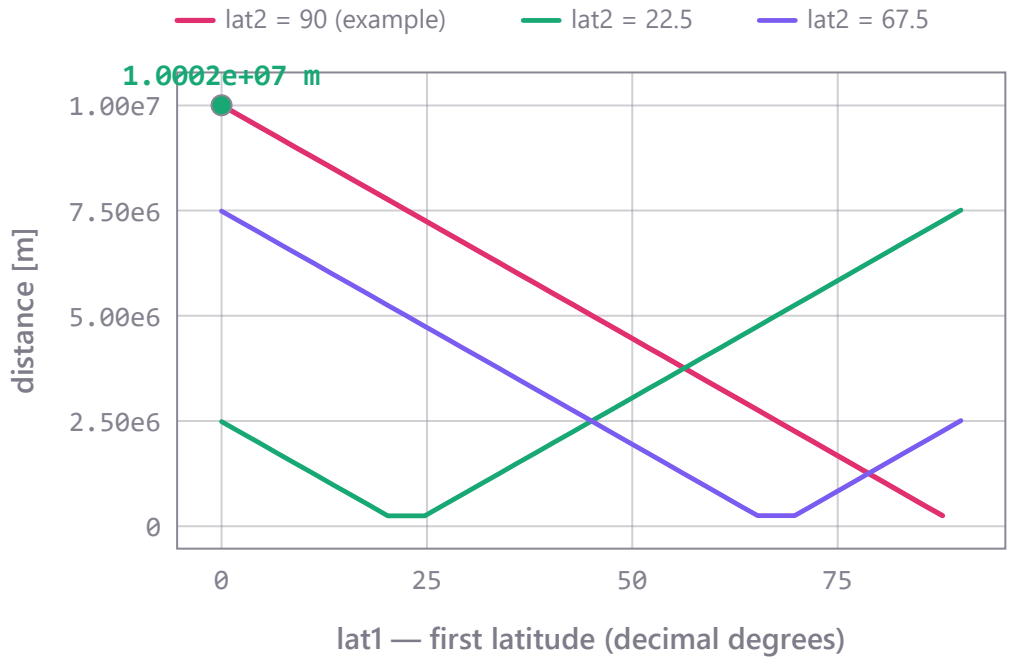
The distance along a meridian of the WGS84 or GRS80 ellipsoid between two latitudes, and the meridian distance of each latitude from the equator.

$$m(\varphi) = \frac{a}{1+n}\left[\left(1+\frac{n^2}{4}+\frac{n^4}{64}\right)\varphi - \frac{3}{2}\left(n-\frac{n^3}{8}\right)\sin 2\varphi + \frac{15}{16}\left(n^2-\frac{n^4}{4}\right)\sin 4\varphi - \frac{35}{48}n^3\sin 6\varphi + \frac{315}{512}n^4 \text{ s}\right.$$

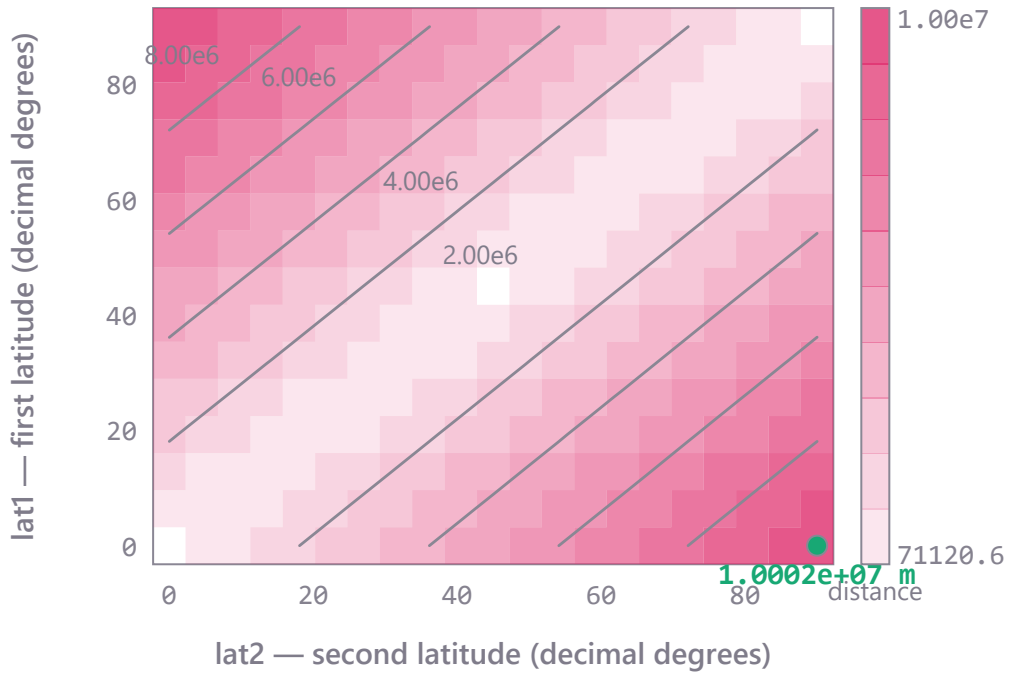
input	meaning	condition
`lat1`	first latitude (decimal degrees)	
`lat2`	second latitude (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`distance`	meridian arc length between the latitudes
`m1`	meridian distance from the equator to lat1 (negative south)
`m2`	meridian distance from the equator to lat2 (negative south)

- **Method:** Helmert's series for the meridian distance in the third flattening n (to n^4)
- **Assumes:** latitudes between -90 and 90 and different
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



distance (meridian arc length between the latitudes) against lat1 (first latitude (decimal degrees)) from 0 to 90. One curve for each value of lat2 (second latitude (decimal degrees)): 90, 22.5, 67.5. Other inputs as in the worked example: ellipsoid = WGS84. 40 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, lat1 = 0, distance = 1.0002e+07 m.



distance (meridian arc length between the latitudes) over lat2 (second latitude (decimal degrees)) from 0 to 90 and lat1 (first latitude (decimal degrees)) from 0 to 90: each of the 21 × 15 cells is one run of geospatial-analytics-ai999x, shaded by its value (light 71120.6 m to dark 1.0002e+07 m); iso-lines at distance = 2.00e6, 4.00e6, 6.00e6, 8.00e6 m; blank cells: inputs the capability refuses. Other inputs as in the worked example: ellipsoid = WGS84. Marked: the worked example, lat2 = 90, lat1 = 0, distance = 1.0002e+07 m.

```
geospatial_analytics_ai999x.solve('geo_meridian_arc', lat1=0, lat2=90, ellipsoid='WGS84')
# -> distance = 1.0002e+07 m, m1 = 0 m, m2 = 1.0002e+07 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_meridian_arc', lat1=40, lat2=41, ellipsoid='GRS80')
# -> distance = 111044 m, m1 = 4.42953e+06 m, m2 = 4.54057e+06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Meridian arc length: lat1 = 0, lat2 = 90, ellipsoid = WGS84	SOLVED	distance = 1.0002e+07 m, m1 = 0 m, m2 = 1.0002e+07 m
Meridian arc length: lat1 = 40, lat2 = 41, ellipsoid = GRS80	SOLVED	distance = 111044 m, m1 = 4.42953e+06 m, m2 = 4.54057e+06 m
Meridian arc length: lat2 = 90, ellipsoid = WGS84	NEEDS_INPUT	geo_meridian_arc needs lat1 (first latitude (decimal degrees)). Example: solve('geo_meridian_arc', lat1=0, lat2=90, elli
Meridian arc length: lat1 = 0, lat2 = 90, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Arc length along a parallel of latitude — `geo_parallel_arc`

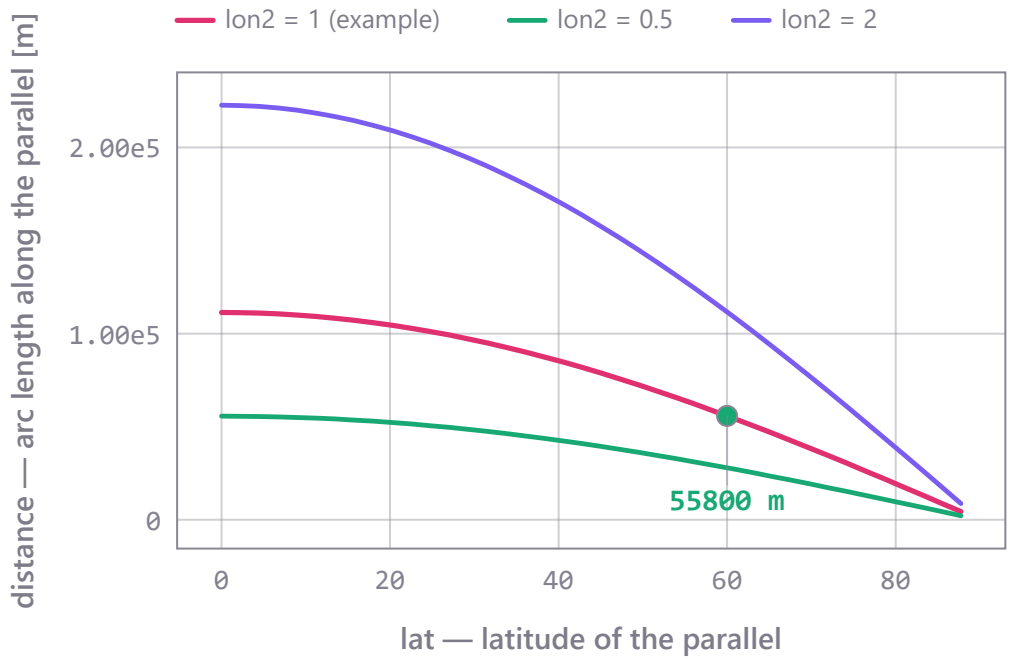
The distance along a parallel of latitude between two longitudes (the shorter way round) on the WGS84 or GRS80 ellipsoid, and the radius of the parallel.

$$s = N(\varphi)\cos\varphi\,|\Delta\lambda|, \quad N = \frac{a}{\sqrt{1 - e^2\sin^2\varphi}}$$

input	meaning	condition
`lat`	latitude of the parallel (decimal degrees)	
`lon1`	first longitude (decimal degrees)	
`lon2`	second longitude (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`distance`	arc length along the parallel
`radius`	radius of the parallel N cos(phi)

- **Method:** radius of the parallel N(phi) cos(phi) times the longitude difference in radians
- **Assumes:** latitude strictly between -90 and 90; different longitudes
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



distance (arc length along the parallel) against lat (latitude of the parallel (decimal degrees)) from 0 to 90. One curve for each value of lon2 (second longitude (decimal degrees)): 1, 0.5, 2. Other inputs as in the worked example: lon1 = 0, ellipsoid = WGS84. 40 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, lat = 60, distance = 55800 m.

```
geospatial_analytics_ai999x.solve('geo_parallel_arc', lat=60, lon1=0, lon2=1, ellipsoid='WGS84')
# -> distance = 55800 m, radius = 3.1971e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_parallel_arc', lat=-23.5, lon1=170, lon2=-170,
ellipsoid='GRS80')
# -> distance = 2.04282e+06 m, radius = 5.85225e+06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Parallel arc length: lat = 60, lon1 = 0, lon2 = 1, ellipsoid = WGS84	SOLVED	distance = 55800 m, radius = 3.1971e+06 m
Parallel arc length: lat = -23.5, lon1 = 170, lon2 = -170, ellipsoid = GRS80	SOLVED	distance = 2.04282e+06 m, radius = 5.85225e+06 m
Parallel arc length: lon1 = 0, lon2 = 1, ellipsoid = WGS84	NEEDS_INPUT	geo_parallel_arc needs lat (latitude of the parallel (decimal degrees)). Example: solve('geo_parallel_arc', lat=60, lon1
Parallel arc length: lat = 60, lon1 = 0, lon2 = 1, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Area of a latitude-longitude cell on the ellipsoid — `geo_graticule_cell_area`

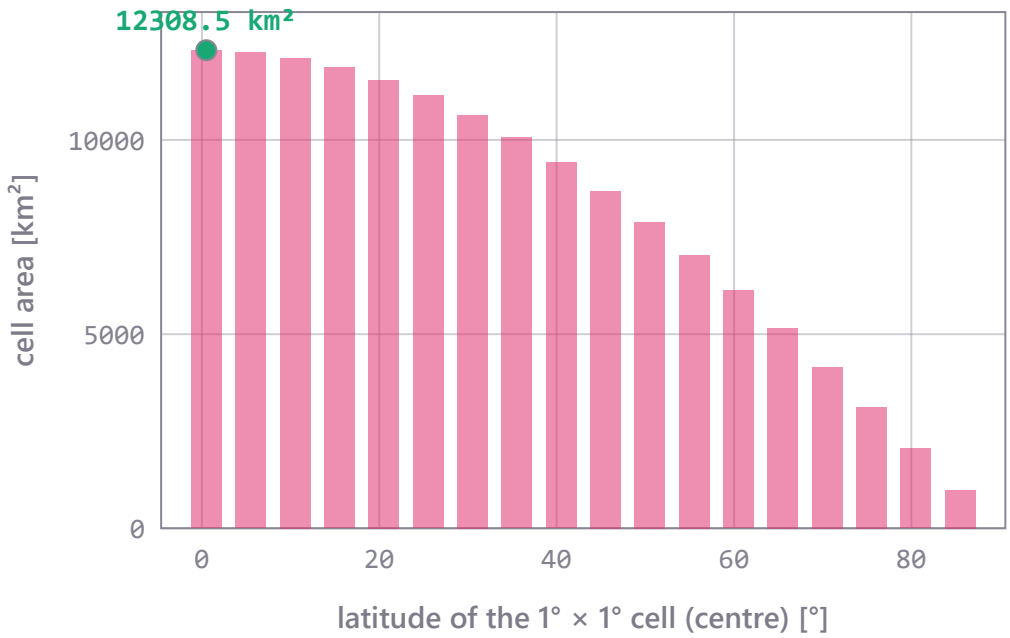
The area of the region between two parallels and two meridians (a graticule cell or grid square) on the WGS84 or GRS80 ellipsoid, and its share of the whole ellipsoid's surface.

$$A = \frac{a^2}{2}[q(\varphi_2) - q(\varphi_1)](\lambda_2 - \lambda_1)$$

input	meaning	condition
`lat1`	southern latitude (decimal degrees)	
`lat2`	northern latitude (decimal degrees)	
`lon1`	western longitude (decimal degrees)	
`lon2`	eastern longitude (east of lon1) (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`area`	area of the cell
`fraction_of_ellipsoid`	area as a fraction of the ellipsoid's total surface

- **Method:** authalic function: area = (a^2/2) [q(phi2) - q(phi1)] $(lambda2 - lambda1)$
- **Assumes:** lat1 < lat2; 0 < lon2 - lon1 <= 360
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The area of a 1° × 1° graticule cell on the WGS84 ellipsoid returned by the package for cells starting every 5° from the equator to 85° N: it falls with the cosine of latitude as the meridians converge. Marked: the worked example, 0°–1° N, 12308.5 km² (2.4131e-05 of the ellipsoid).

```
geospatial_analytics_ai999x.solve('geo_graticule_cell_area', lat1=0, lat2=1, lon1=0, lon2=1,
ellipsoid='WGS84')
# -> area = 1.23085e+10 m^2, fraction_of_ellipsoid = 2.41311e-05   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_graticule_cell_area', lat1=50, lat2=51, lon1=4, lon2=5,
ellipsoid='GRS80')
# -> area = 7.89222e+09 m^2, fraction_of_ellipsoid = 1.54729e-05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Graticule cell area: lat1 = 0, lat2 = 1, lon1 = 0, lon2 = 1, ellipsoid = WGS84	SOLVED	area = 1.23085e+10 m^2, fraction_of_ellipsoid = 2.41311e-05
Graticule cell area: lat1 = 50, lat2 = 51, lon1 = 4, lon2 = 5, ellipsoid = GRS80	SOLVED	area = 7.89222e+09 m^2, fraction_of_ellipsoid = 1.54729e-05
Graticule cell area: lat2 = 1, lon1 = 0, lon2 = 1, ellipsoid = WGS84	NEEDS_INPUT	geo_graticule_cell_area needs lat1 (southern latitude (decimal degrees)). Example: solve('geo_graticule_cell_area', lat1
Graticule cell area: lat1 = 0, lat2 = 1, lon1 = 0, lon2 = 1, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Radii of curvature of the ellipsoid at a latitude — `geo_radii_of_curvature`

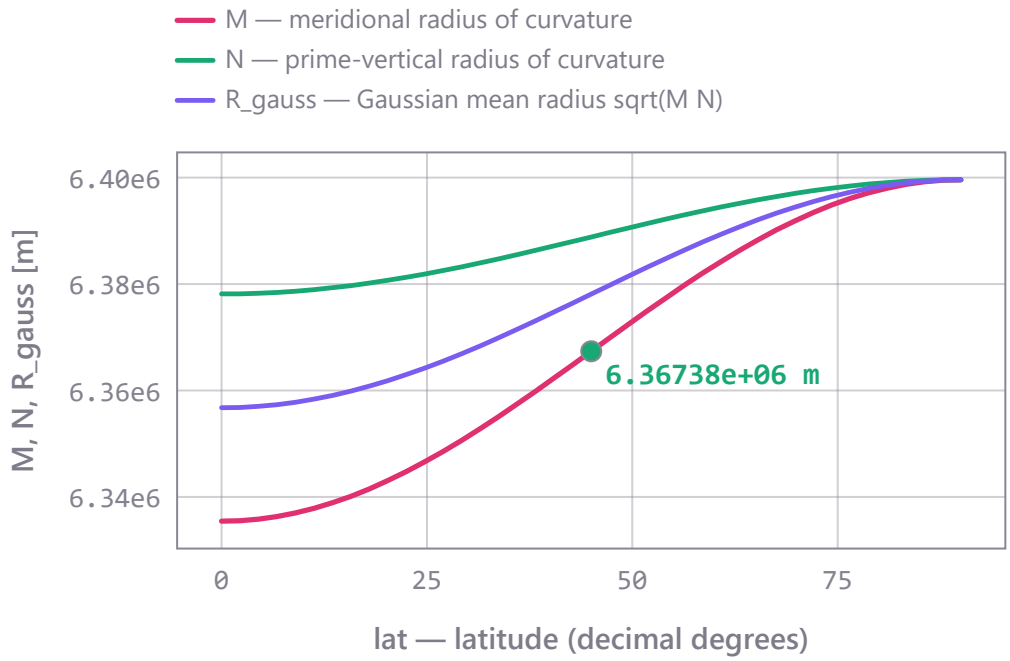
The meridional radius of curvature M, the prime-vertical radius of curvature N and the Gaussian mean radius sqrt(MN) of the WGS84 or GRS80 ellipsoid at a latitude.

$$M = \frac{a(1-e^2)}{(1-e^2\sin^2\varphi)^{3/2}}, \quad N = \frac{a}{\sqrt{1-e^2\sin^2\varphi}}, \quad R_G = \sqrt{MN}$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`M`	meridional radius of curvature
`N`	prime-vertical radius of curvature
`R_gauss`	Gaussian mean radius sqrt(M N)

- **Method:** $M = a(1-e^2)/(1-e^2\ sin^2$ phi)^(3/2), $N = a/sqrt(1-e^2\ sin^2$ phi)
- **Assumes:** latitude between -90 and 90
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



M (meridional radius of curvature) against lat (latitude (decimal degrees)) from 0 to 90. Outputs with the same unit drawn together: N (prime-vertical radius of curvature), R_gauss (Gaussian mean radius sqrt(M N)). Other inputs as in the worked example: ellipsoid = WGS84. 41 points computed by geospatial-analytics-ai999x; marked: the worked example, lat = 45, M = 6.36738e+06 m.

```
geospatial_analytics_ai999x.solve('geo_radii_of_curvature', lat=45, ellipsoid='WGS84')
# -> M = 6.36738e+06 m, N = 6.38884e+06 m, R_gauss = 6.3781e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_radii_of_curvature', lat=0, ellipsoid='GRS80')
# -> M = 6.33544e+06 m, N = 6.37814e+06 m, R_gauss = 6.35675e+06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radii of curvature: lat = 45, ellipsoid = WGS84	SOLVED	M = 6.36738e+06 m, N = 6.38884e+06 m, R_gauss = 6.3781e+06 m
Radii of curvature: lat = 0, ellipsoid = GRS80	SOLVED	M = 6.33544e+06 m, N = 6.37814e+06 m, R_gauss = 6.35675e+06 m
Radii of curvature: ellipsoid = WGS84	NEEDS_INPUT	geo_radii_of_curvature needs lat (latitude (decimal degrees)). Example: solve('geo_radii_of_curvature', lat=45, ellipsoi
Radii of curvature: lat = 45, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Rhumb-line (loxodrome) distance and bearing on the ellipsoid — `geo_rhumb_line_inverse`

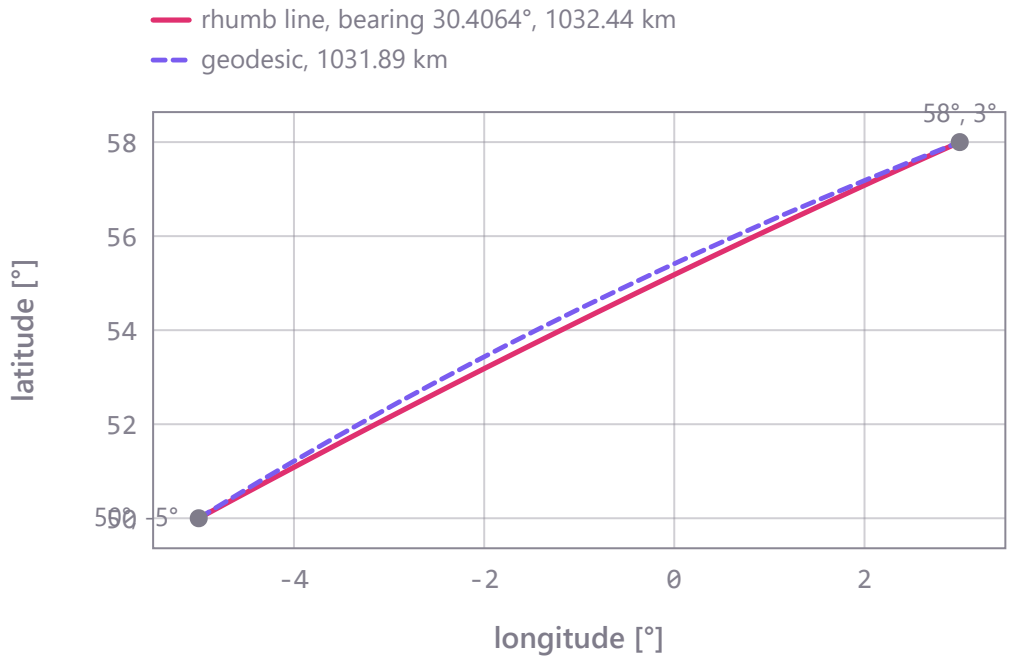
The constant bearing and the length of the rhumb line (loxodrome) between two points on the WGS84 or GRS80 ellipsoid, the shorter way round in longitude.

$$\tan\alpha = \frac{\Delta\lambda}{\Delta\psi}, \quad s = \frac{\Delta m}{\cos\alpha}, \quad \psi = \operatorname{arsinh}(\tan\varphi) - e\operatorname{artanh}(e\sin\varphi)$$

input	meaning	condition
`lat1`	latitude of the start point (decimal degrees)	
`lon1`	longitude of the start point (decimal degrees)	
`lat2`	latitude of the end point (decimal degrees)	
`lon2`	longitude of the end point (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`distance`	rhumb-line length
`bearing`	constant bearing (degrees clockwise from north)

- **Method:** bearing = atan2(dlambda, dpsi) with the isometric latitude psi; length = dm / cos(bearing) with Helmert's meridian distance (N cos(phi) |dlambda| along a parallel)
- **Assumes:** latitudes within 89 degrees of the equator; distinct points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



From 50°, -5° to 58°, 3° (WGS84): the package returns the rhumb line's constant bearing 30.4064° and length 1032.44 km; the path is drawn from 20 rhumb-line destinations along that bearing, and it ends at the target. For comparison, the geodesic (1031.89 km, from the package's geodesic runs) is shorter.

```
geospatial_analytics_ai999x.solve('geo_rhumb_line_inverse', lat1=50.0, lon1=-5.0, lat2=58.0,
lon2=3.0, ellipsoid='WGS84')
# -> distance = 1.03244e+06 m, bearing = 30.4064   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_rhumb_line_inverse', lat1=-34.0, lon1=18.4, lat2=-31.9,
lon2=115.9, ellipsoid='GRS80')
# -> distance = 9.11886e+06 m, bearing = 88.5365   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rhumb line distance and bearing: lat1 = 50, lon1 = -5, lat2 = 58, lon2 = 3, ellipsoid = WGS84	SOLVED	distance = 1.03244e+06 m, bearing = 30.4064
Rhumb line distance and bearing: lat1 = -34, lon1 = 18.4, lat2 = -31.9, lon2 = 115.9, ellipsoid = GRS80	SOLVED	distance = 9.11886e+06 m, bearing = 88.5365
Rhumb line distance and bearing: lon1 = -5, lat2 = 58, lon2 = 3, ellipsoid = WGS84	NEEDS_INPUT	geo_rhumb_line_inverse needs lat1 (latitude of the start point (decimal degrees)). Example: solve('geo_rhumb_line_invers
Rhumb line distance and bearing: lat1 = 50, lon1 = -5, lat2 = 58, lon2 = 3, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Destination along a rhumb line (loxodrome) on the ellipsoid — `geo_rhumb_line_destination`

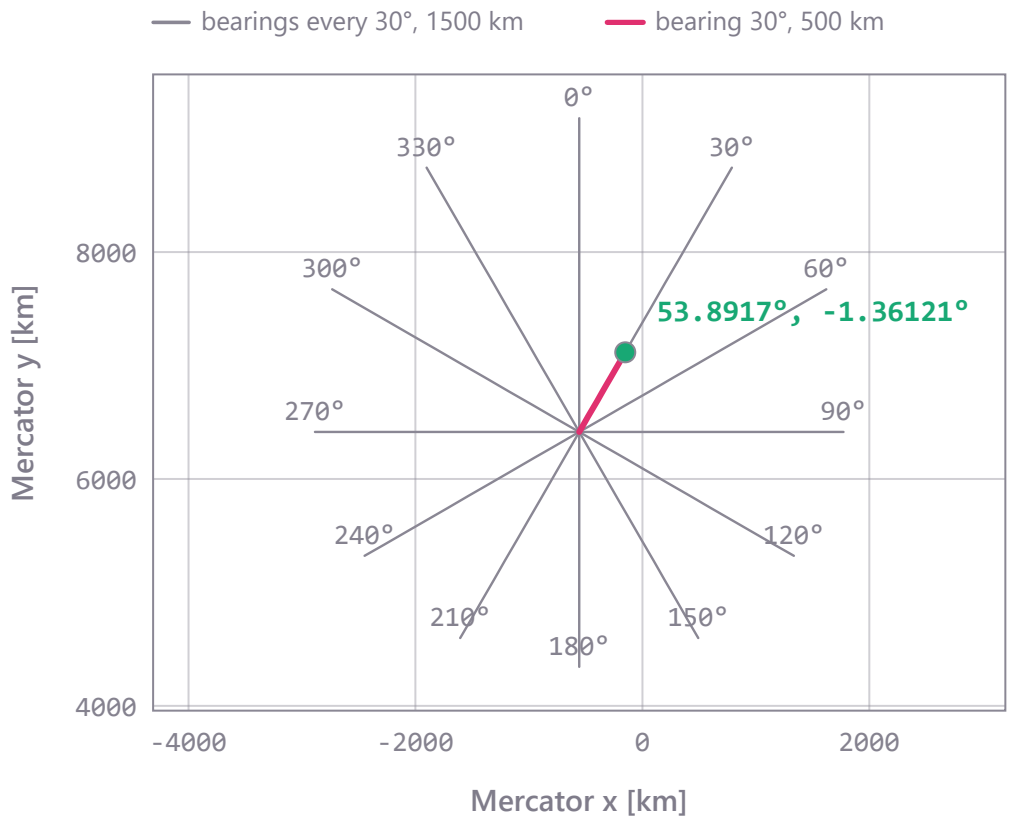
The point reached by travelling a given distance on a constant bearing (rhumb line) from a start point on the WGS84 or GRS80 ellipsoid.

$$m_2 = m_1 + s\cos\alpha, \quad \Delta\lambda = \tan\alpha\,(\psi_2 - \psi_1)$$

input	meaning	condition
`lat1`	latitude of the start point (decimal degrees)	
`lon1`	longitude of the start point (decimal degrees)	
`bearing`	constant bearing, clockwise from north (decimal degrees)	
`distance`	distance travelled	
`ellipsoid`	reference ellipsoid	

output	meaning
`lat2`	latitude of the destination (degrees)
`lon2`	longitude of the destination (degrees)

- **Method:** meridian distance advances by s cos(bearing) (Helmert's series, inverted by Newton steps); the longitude by tan(bearing) times the change of isometric latitude
- **Assumes:** the line stays more than 1 degree from the poles; distance > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Rhumb lines from latitude 50°, longitude -5° at bearings 0° to 330° for 1500 km, every point a destination returned by the package and projected by its Mercator projection (132 points): each constant-bearing path is a straight ray on this map. Marked: the worked example, bearing 30° for 500 km → 53.8917°, -1.36121°.

```
geospatial_analytics_ai999x.solve('geo_rhumb_line_destination', lat1=50.0, lon1=-5.0, bearing=30,
distance=500000.0, ellipsoid='WGS84')
# -> lat2 = 53.8917, lon2 = -1.36121   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_rhumb_line_destination', lat1=-10.0, lon1=100.0,
bearing=200, distance=1200000.0, ellipsoid='GRS80')
# -> lat2 = -20.1908, lon2 = 96.1764   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rhumb line destination: lat1 = 50, lon1 = -5, bearing = 30, distance = 500 km, ellipsoid = WGS84	SOLVED	lat2 = 53.8917, lon2 = -1.36121
Rhumb line destination: lat1 = -10, lon1 = 100, bearing = 200, distance = 1200 km, ellipsoid = GRS80	SOLVED	lat2 = -20.1908, lon2 = 96.1764
Rhumb line destination: lon1 = -5, bearing = 30, distance = 500 km, ellipsoid = WGS84	NEEDS_INPUT	geo_rhumb_line_destination needs lat1 (latitude of the start point (decimal degrees)). Example: solve('geo_rhumb_line_de
Rhumb line destination: lat1 = 50, lon1 = -5, bearing = 30, distance = 500 km, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Geodetic to geocentric (ECEF) coordinates — `geo_geodetic_to_ecef`

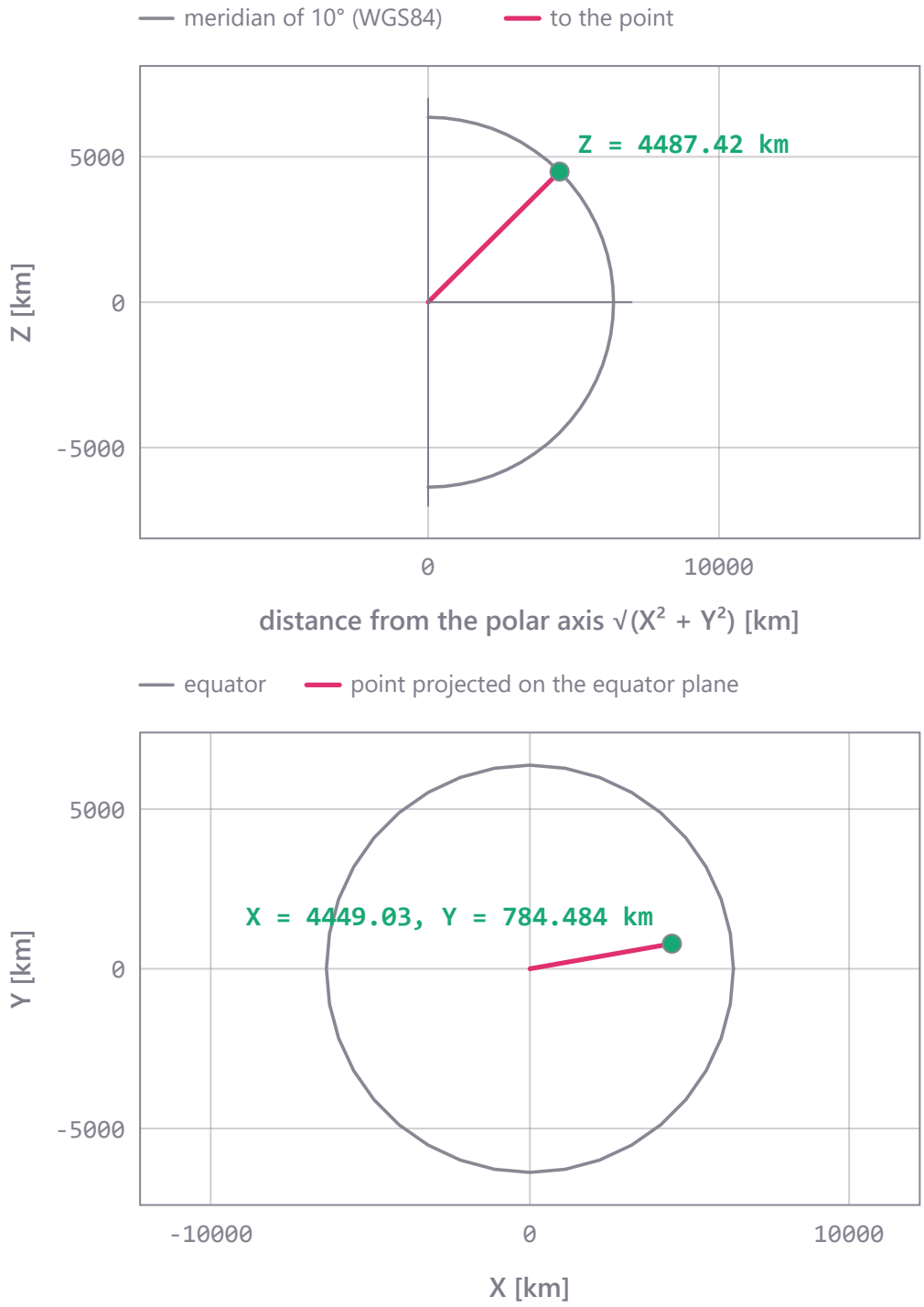
Earth-centred, Earth-fixed Cartesian coordinates X, Y, Z of a point given by geodetic latitude, longitude and ellipsoidal height on the WGS84 or GRS80 ellipsoid.

$$X = (N + h)\cos\varphi\cos\lambda,\ Y = (N + h)\cos\varphi\sin\lambda,\ Z = \left(N(1 - e^2) + h\right)\sin\varphi$$

input	meaning	condition
`lat`	geodetic latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`h`	ellipsoidal height	
`ellipsoid`	reference ellipsoid	

output	meaning
`X`	X (towards latitude 0, longitude 0)
`Y`	Y (towards longitude 90 E)
`Z`	Z (towards the north pole)

- **Method:** $X = (N + h)$ cos(phi) cos(lambda), $Y = (N + h)$ cos(phi) sin(lambda), Z = (N(1 - e^2) + h) sin(phi)
- **Assumes:** latitude between -90 and 90; height between -100 km and 100000 km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The point latitude 45°, longitude 10°, height 100.0 m in Earth-centred, Earth-fixed coordinates as returned: X = 4.44903e+06 m, Y = 784484 m, Z = 4.48742e+06 m. Top: the meridian section through the point (the ellipsoid traced by the package at every 5° of latitude; horizontal axis √(X² + Y²), computed here from the returned X and Y). Bottom: the equatorial plane, the equator traced every 10° of longitude, and the point's X and Y.

```
geospatial_analytics_ai999x.solve('geo_geodetic_to_ecef', lat=45, lon=10, h=100.0,
ellipsoid='WGS84')
# -> X = 4.44903e+06 m, Y = 784484 m, Z = 4.48742e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_geodetic_to_ecef', lat=-33.8568, lon=151.2153, h=50.0,
ellipsoid='GRS80')
# -> X = -4.64701e+06 m, Y = 2.5531e+06 m, Z = -3.53329e+06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geodetic to ECEF: lat = 45, lon = 10, h = 100 m, ellipsoid = WGS84	SOLVED	X = 4.44903e+06 m, Y = 784484 m, Z = 4.48742e+06 m
Geodetic to ECEF: lat = -33.8568, lon = 151.2153, h = 50 m, ellipsoid = GRS80	SOLVED	X = -4.64701e+06 m, Y = 2.5531e+06 m, Z = -3.53329e+06 m
Geodetic to ECEF: lon = 10, h = 100 m, ellipsoid = WGS84	NEEDS_INPUT	geo_geodetic_to_ecef needs lat (geodetic latitude (decimal degrees)). Example: solve('geo_geodetic_to_ecef', lat=45, lon
Geodetic to ECEF: lat = 45, lon = 10, h = 100 m, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Geocentric (ECEF) to geodetic coordinates — `geo_ecef_to_geodetic`

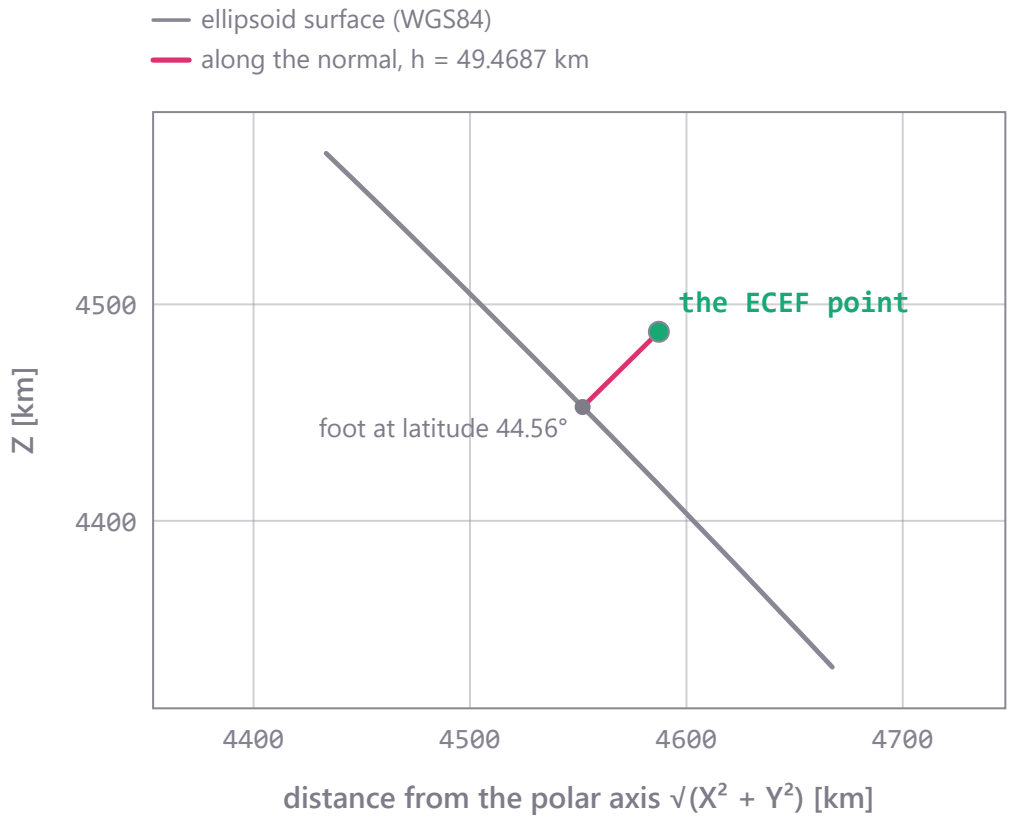
Geodetic latitude, longitude and ellipsoidal height of a point given by its Earth-centred, Earth-fixed coordinates X, Y, Z on the WGS84 or GRS80 ellipsoid.

$$\varphi = \operatorname{atan2}\left(Z + e^2 N \sin\varphi,\ \sqrt{X^2 + Y^2}\right), \quad h = p\cos\varphi + Z\sin\varphi - \frac{a^2}{N}$$

input	meaning	condition
`X`	geocentric X	
`Y`	geocentric Y	
`Z`	geocentric Z	
`ellipsoid`	reference ellipsoid	

output	meaning
`lat`	geodetic latitude (degrees)
`lon`	longitude (degrees)
`h`	ellipsoidal height

- **Method:** fixed-point iteration phi = atan2($Z + e^2$ N sin(phi), p) to convergence; $h = p$ cos(phi) + Z sin(phi) - a^2/N
- **Assumes:** 6200 km <= distance from the centre <= 100000 km; not on the polar axis
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The ECEF point X = 4517590.879 m, Y = 796576.168 m, Z = 4487348.409 m converted by the package to latitude 44.56°, longitude 10°, height 49468.7 m. Drawn in the meridian plane near the point (√(X² + Y²) computed here from the inputs): the surface traced by the package's forward conversion at ±1.5° around the returned latitude, the foot of the normal (returned latitude at h = 0) and the point 49.4687 km above it along the normal.

```
geospatial_analytics_ai999x.solve('geo_ecef_to_geodetic', X=4517590.879, Y=796576.168,
Z=4487348.409, ellipsoid='WGS84')
# -> lat = 44.56, lon = 10, h = 49468.7 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_ecef_to_geodetic', X=-4646678.4, Y=2553002.1, Z=-3534200.6,
ellipsoid='GRS80')
# -> lat = -33.8652, lon = 151.214, h = 278.966 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ECEF to geodetic: X = 4517590.879 m, Y = 796576.168 m, Z = 4487348.409 m, ellipsoid = WGS84	SOLVED	lat = 44.56, lon = 10, h = 49468.7 m
ECEF to geodetic: X = -4646678.4 m, Y = 2553002.1 m, Z = -3534200.6 m, ellipsoid = GRS80	SOLVED	lat = -33.8652, lon = 151.214, h = 278.966 m
ECEF to geodetic: Y = 796576.168 m, Z = 4487348.409 m, ellipsoid = WGS84	NEEDS_INPUT	geo_ecef_to_geodetic needs X (geocentric X). Example: solve('geo_ecef_to_geodetic', X='4.51759e+06 m', Y='796576 m', Z='
ECEF to geodetic: X = 4517590.879 m, Y = 796576.168 m, Z = 4487348.409 m, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Geodetic to local east-north-up (ENU) coordinates — `geo_geodetic_to_enu`

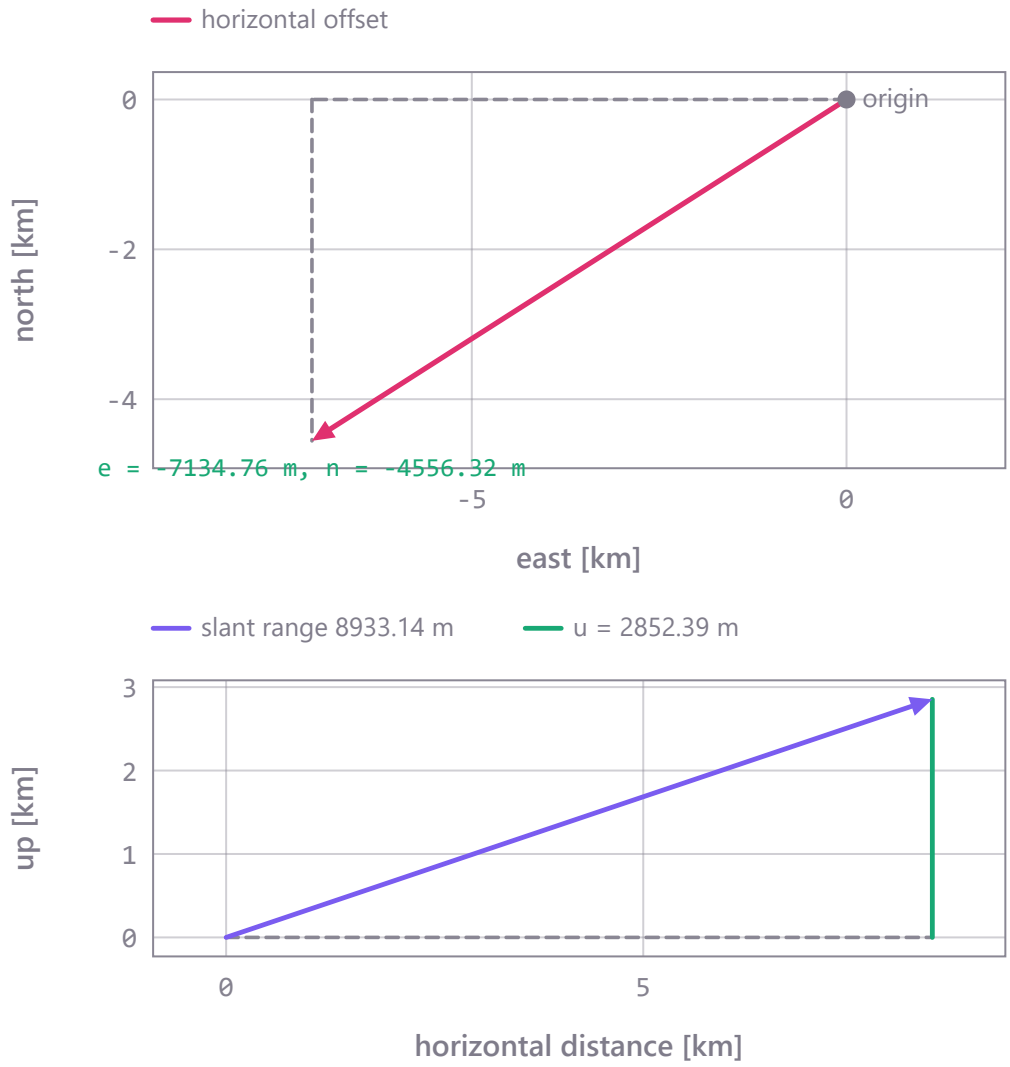
Local tangent-plane coordinates east, north and up of a point relative to an origin (both given by latitude, longitude and ellipsoidal height), and the straight-line (slant) range.

$$\begin{bmatrix} e \\ n \\ u \end{bmatrix} = \begin{bmatrix} -\sin\lambda_0 & \cos\lambda_0 & 0 \\ -\sin\varphi_0\cos\lambda_0 & -\sin\varphi_0\sin\lambda_0 & \cos\varphi_0 \\ \cos\varphi_0\cos\lambda_0 & \cos\varphi_0\sin\lambda_0 & \sin\varphi_0 \end{bmatrix} \begin{bmatrix} \Delta X \\ \Delta Y \\ \Delta Z \end{bmatrix}$$

input	meaning	condition
`lat0`	latitude of the local origin (decimal degrees)	
`lon0`	longitude of the local origin (decimal degrees)	
`h0`	ellipsoidal height of the origin	
`lat`	latitude of the point (decimal degrees)	
`lon`	longitude of the point (decimal degrees)	
`h`	ellipsoidal height of the point	
`ellipsoid`	reference ellipsoid	

output	meaning
`e`	east coordinate
`n`	north coordinate
`u`	up coordinate
`slant_range`	straight-line distance from the origin

- **Method:** ECEF difference rotated into the local frame: [e n u] = R(lat0, lon0) $(X - X_0)$
- **Assumes:** origin not at a pole
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Local east–north–up coordinates of the point 45.976°, 7.658°, 4531.0 m seen from the origin 46.017°, 7.75°, 1673.0 m: returned e = -7134.76 m, n = -4556.32 m, u = 2852.39 m, slant range 8933.14 m. Top: the plan (east, north); bottom: the vertical plane through the target (horizontal distance $\sqrt{(e^2 + n^2)}$ computed here from the returned e and n).

```
geospatial_analytics_ai999x.solve('geo_geodetic_to_enu', lat0=46.017, lon0=7.75, h0=1673.0,
lat=45.976, lon=7.658, h=4531.0, ellipsoid='WGS84')
# -> e = -7134.76 m, n = -4556.32 m, u = 2852.39 m, slant_range = 8933.14 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_geodetic_to_enu', lat0=0, lon0=0, h0=0.0, lat=0.5, lon=0.5,
h=1000.0, ellipsoid='GRS80')
# -> e = 55665.7 m, n = 55295.2 m, u = 515.839 m, slant_range = 78463.3 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geodetic to ENU: lat0 = 46.017, lon0 = 7.75, h0 = 1673 m, lat = 45.976, lon = 7.658, h = 4531 m, ellipsoid = WGS84	SOLVED	e = -7134.76 m, n = -4556.32 m, u = 2852.39 m, slant_range = 8933.14 m
Geodetic to ENU: lat0 = 0, lon0 = 0, h0 = 0 m, lat = 0.5, lon = 0.5, h = 1000 m, ellipsoid = GRS80	SOLVED	e = 55665.7 m, n = 55295.2 m, u = 515.839 m, slant_range = 78463.3 m
Geodetic to ENU: lon0 = 7.75, h0 = 1673 m, lat = 45.976, lon = 7.658, h =	NEEDS_INPUT	geo_geodetic_to_enu needs lat0 (latitude of the local origin (decimal degrees)). Example:

question	outcome	result
4531 m, ellipsoid = WGS84		solve('geo_geodetic_to_enu', l
Geodetic to ENU: lat0 = 46.017, lon0 = 7.75, h0 = 1673 m, lat = 45.976, lon = 7.658, h = 4531 m, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Local east-north-up (ENU) to geodetic coordinates — `geo_enu_to_geodetic`

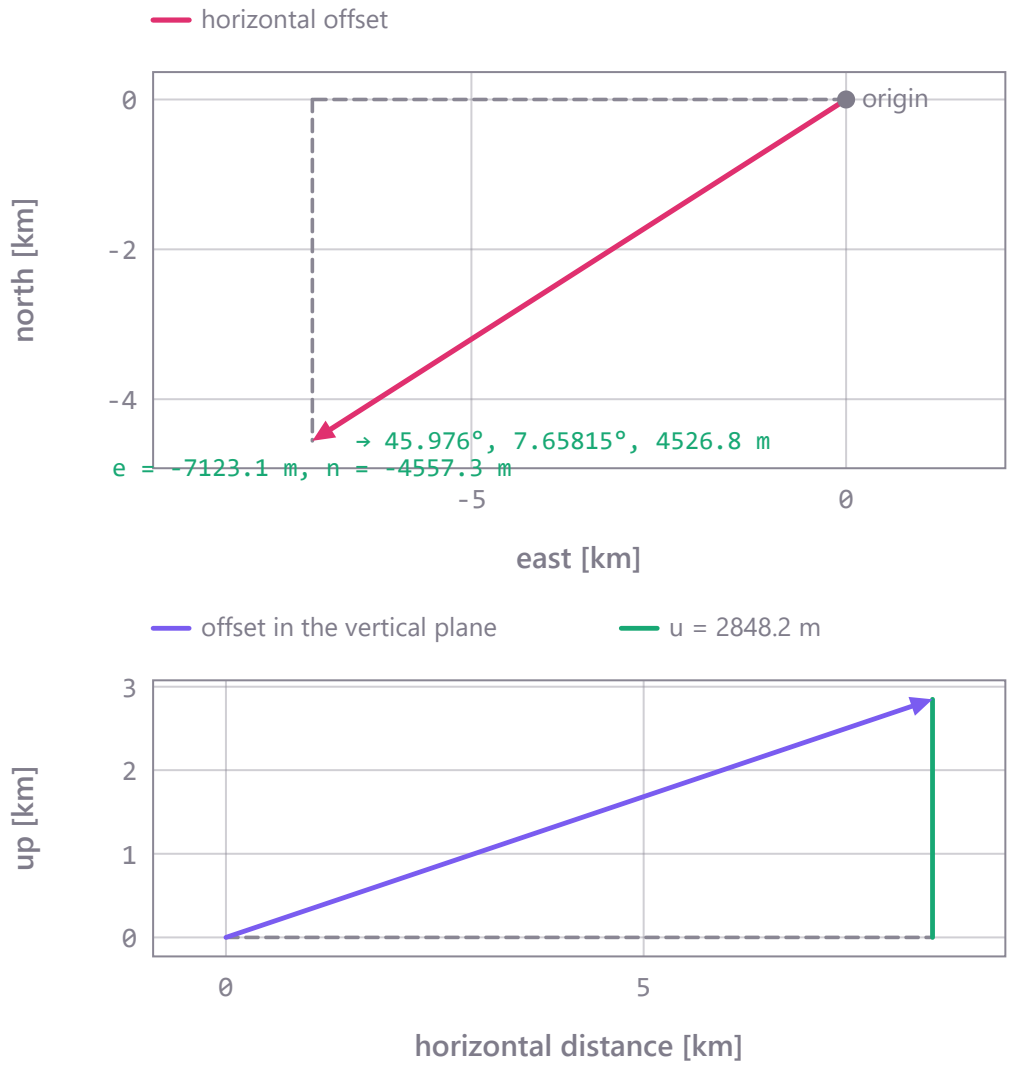
Latitude, longitude and ellipsoidal height of a point given by its local east, north and up coordinates relative to an origin on the WGS84 or GRS80 ellipsoid.

$$\mathbf{X} = \mathbf{X}_0 + R^T(\varphi_0, \lambda_0)\,[e\ n\ u]^T$$

input	meaning	condition
`lat0`	latitude of the local origin (decimal degrees)	
`lon0`	longitude of the local origin (decimal degrees)	
`h0`	ellipsoidal height of the origin	
`e`	east coordinate	
`n`	north coordinate	
`u`	up coordinate	
`ellipsoid`	reference ellipsoid	

output	meaning
`lat`	latitude (degrees)
`lon`	longitude (degrees)
`h`	ellipsoidal height

- **Method:** rotate [e n u] back to an ECEF offset, add the origin's ECEF position, then convert ECEF to geodetic by iteration
- **Assumes:** origin not at a pole; the point not deep inside the Earth
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The local offset e = -7123.1 m, n = -4557.3 m, u = 2848.2 m from the origin 46.017°, 7.75°, 1673.0 m (inputs, drawn in plan and in the vertical plane) and the geodetic position the package returns for it: latitude 45.976°, longitude 7.65815°, height 4526.8 m.

```
geospatial_analytics_ai999x.solve('geo_enu_to_geodetic', lat0=46.017, lon0=7.75, h0=1673.0,
e=-7123.1, n=-4557.3, u=2848.2, ellipsoid='WGS84')
# -> lat = 45.976, lon = 7.65815, h = 4526.8 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_enu_to_geodetic', lat0=0, lon0=0, h0=0.0, e=1000.0,
n=2000.0, u=10.0, ellipsoid='GRS80')
# -> lat = 0.0180874, lon = 0.00898314, h = 10.3941 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ENU to geodetic: lat0 = 46.017, lon0 = 7.75, h0 = 1673 m, e = -7123.1 m, n = -4557.3 m, u = 2848.2 m, ellipsoid = WGS84	SOLVED	lat = 45.976, lon = 7.65815, h = 4526.8 m
ENU to geodetic: lat0 = 0, lon0 = 0, h0 = 0 m, e = 1000 m, n = 2000 m, u = 10 m, ellipsoid = GRS80	SOLVED	lat = 0.0180874, lon = 0.00898314, h = 10.3941 m
ENU to geodetic: lon0 = 7.75, h0 = 1673 m, e = -7123.1 m, n = -4557.3 m,	NEEDS_INPUT	geo_enu_to_geodetic needs lat0 (latitude of the local origin (decimal degrees)). Example:

question	outcome	result
u = 2848.2 m, ellipsoid = WGS84		solve('geo_enu_to_geodetic', l
ENU to geodetic: lat0 = 46.017, lon0 = 7.75, h0 = 1673 m, e = -7123.1 m, n = -4557.3 m, u = 2848.2 m, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Look angles: azimuth, elevation and range from an observer to a target — `geo_look_angles`

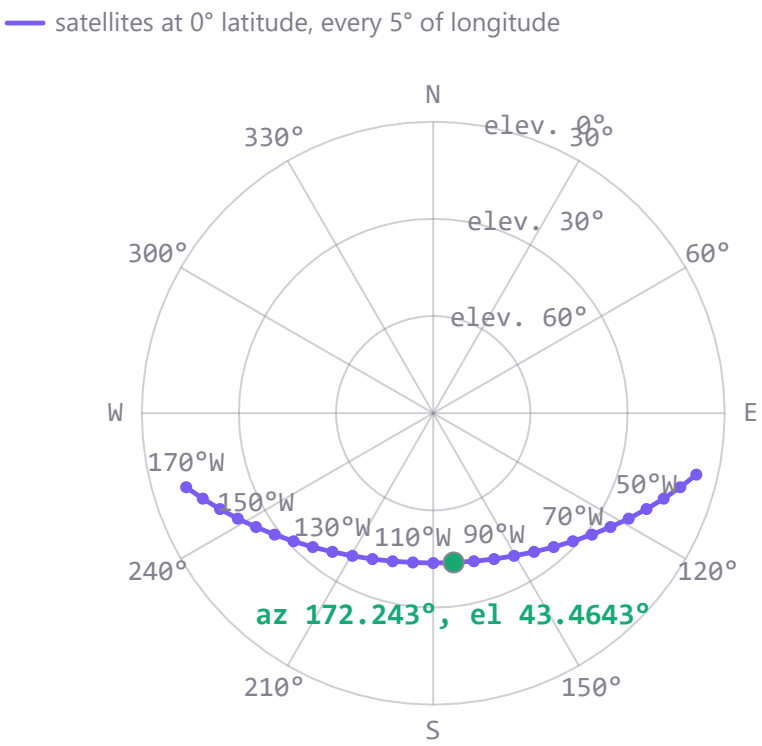
The azimuth (clockwise from true north), elevation angle above the local horizontal plane and slant range from an observer to a target, both given by latitude, longitude and ellipsoidal height (a satellite, aircraft or summit).

$$A = \operatorname{atan2}(e, n), \quad \varepsilon = \operatorname{atan2}\left(u, \sqrt{e^2 + n^2}\right)$$

input	meaning	condition
`lat0`	observer latitude (decimal degrees)	
`lon0`	observer longitude (decimal degrees)	
`h0`	observer ellipsoidal height	
`lat`	target latitude (decimal degrees)	
`lon`	target longitude (decimal degrees)	
`h`	target ellipsoidal height	
`ellipsoid`	reference ellipsoid	

output	meaning
`azimuth`	azimuth (degrees clockwise from north)
`elevation`	elevation angle (degrees, negative below the horizon)
`slant_range`	straight-line distance

- **Method:** ENU vector of the target; azimuth = atan2(e, n), elevation = atan2(u, sqrt(e^2 + n^2))
- **Assumes:** observer not at a pole; target distinct from the observer
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A sky plot seen from 40.0°, -105.0°, 1600.0 m (zenith at the centre, horizon at the edge, azimuth clockwise from north): the look angles the package returns for targets at latitude 0.0°, height 35786 km and longitudes 170° W to 35° W, every 5° (those above the horizon) — the geostationary arc. Marked: the worked example at longitude -100.0°, azimuth 172.243°, elevation 43.4643°, slant range 37513.7 km.

```
geospatial_analytics_ai999x.solve('geo_look_angles', lat0=40.0, lon0=-105.0, h0=1600.0, lat=0.0,
lon=-100.0, h=35786000.0, ellipsoid='WGS84')
# -> azimuth = 172.243, elevation = 43.4643, slant_range = 3.75137e+07 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_look_angles', lat0=46.017, lon0=7.75, h0=1673.0, lat=45.976,
lon=7.658, h=4478.0, ellipsoid='WGS84')
# -> azimuth = 237.437, elevation = 18.2983, slant_range = 8916.29 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Look angles: lat0 = 40, lon0 = -105, h0 = 1600 m, lat = 0, lon = -100, h = 35786000 m, ellipsoid = WGS84	SOLVED	azimuth = 172.243, elevation = 43.4643, slant_range = 3.75137e+07 m
Look angles: lat0 = 46.017, lon0 = 7.75, h0 = 1673 m, lat = 45.976, lon = 7.658, h = 4478 m, ellipsoid = WGS84	SOLVED	azimuth = 237.437, elevation = 18.2983, slant_range = 8916.29 m
Look angles: lon0 = -105, h0 = 1600 m, lat = 0, lon = -100, h = 35786000 m, ellipsoid = WGS84	NEEDS_INPUT	geo_look_angles needs lat0 (observer latitude (decimal degrees)). Example: solve('geo_look_angles', lat0=40, lon0=-105,
Look angles: lat0 = 40, lon0 = -105, h0 = 1600 m, lat = 0, lon = -100, h =	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

question	outcome	result
35786000 m, ellipsoid = WGS84, zz = 1		

Auxiliary latitudes: geocentric, reduced, conformal, authalic and rectifying —

`geo_auxiliary_latitudes`

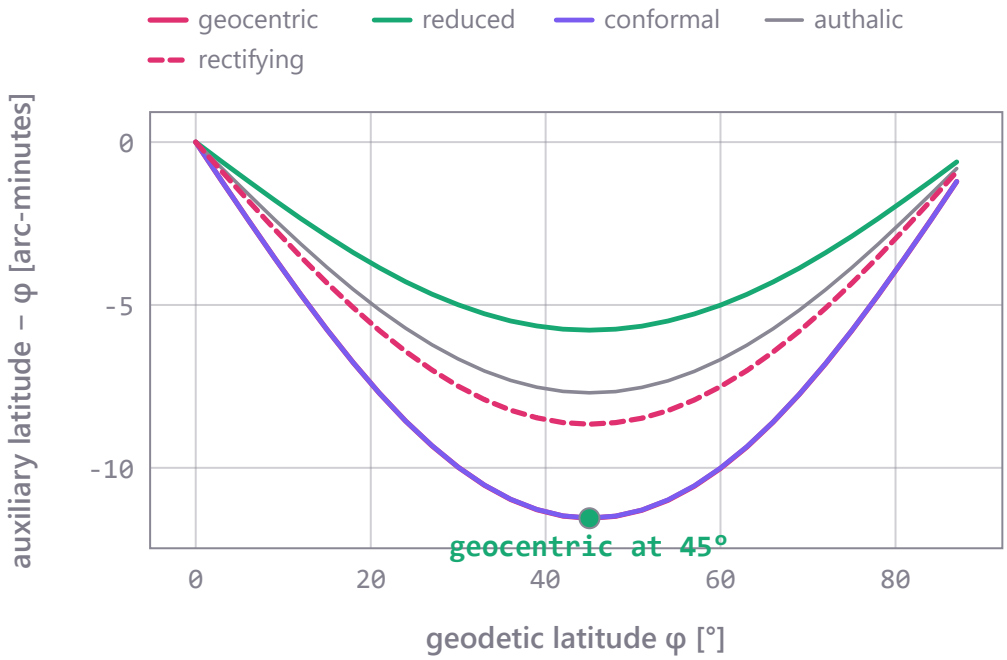
The five auxiliary latitudes of a geodetic latitude on the WGS84 or GRS80 ellipsoid (used by the sphere-based forms of map projections and geodesics).

$$\tan\psi = (1-e^2)\tan\varphi,\ \ \tan\beta = (1-f)\tan\varphi,\ \chi = \operatorname{gd}\psi_{iso},\ \xi = \arcsin\frac{q}{q_p},\ \mu = \frac{\pi}{2}\frac{m(\varphi)}{m(\pi/2)}$$

input	meaning	condition
`lat`	geodetic latitude (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`geocentric`	geocentric latitude (degrees)
`reduced`	reduced (parametric) latitude (degrees)
`conformal`	conformal latitude (degrees)
`authalic`	authalic latitude (degrees)
`rectifying`	rectifying latitude (degrees)

- **Method:** tan(psi) = (1-e^2) tan(phi); tan(beta) = (1-f) tan(phi); chi = atan(sinh(isometric latitude)); xi = asin(q/q_p); $mu = (pi/2)$ m(phi)/m(pi/2)
- **Assumes:** latitude strictly between -90 and 90
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The five auxiliary latitudes the package returns on the WGS84 ellipsoid, every 3° of geodetic latitude, drawn as their difference from the geodetic latitude in arc-minutes (difference computed here): all vanish at the equator and the pole and are largest near 45°. Marked: the worked example at 45°, geocentric latitude 44.8076° (-11.5454′).

```
geospatial_analytics_ai999x.solve('geo_auxiliary_latitudes', lat=45, ellipsoid='WGS84')
# -> geocentric = 44.8076, reduced = 44.9038, conformal = 44.8077, authalic = 44.8717, rectifying =
44.8557   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_auxiliary_latitudes', lat=-30, ellipsoid='GRS80')
# -> geocentric = -29.8336, reduced = -29.9167, conformal = -29.8337, authalic = -29.889,
rectifying = -29.8751   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Auxiliary latitudes: lat = 45, ellipsoid = WGS84	SOLVED	geocentric = 44.8076, reduced = 44.9038, conformal = 44.8077, authalic = 44.8717, rectifying = 44.8557
Auxiliary latitudes: lat = -30, ellipsoid = GRS80	SOLVED	geocentric = -29.8336, reduced = -29.9167, conformal = -29.8337, authalic = -29.889, rectifying = -29.8751
Auxiliary latitudes: ellipsoid = WGS84	NEEDS_INPUT	geo_auxiliary_latitudes needs lat (geodetic latitude (decimal degrees)). Example: solve('geo_auxiliary_latitudes', lat=4
Auxiliary latitudes: lat = 45, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

UTM zone, latitude band and central meridian — `geo_utm_zone`

The Universal Transverse Mercator zone number (with the Norway and Svalbard exceptions), the MGRS latitude band letter, the hemisphere and the zone's central meridian for a latitude and longitude.

$$z = \left\lfloor \frac{\lambda + 180^\circ}{6^\circ} \right\rfloor + 1, \quad \lambda_0 = 6z - 183^\circ$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	

output	meaning
`zone`	UTM zone number $(1 - 60)$
`band`	latitude band letter (C-X)
`hemisphere`	N or S
`central_meridian`	central meridian of the zone (degrees)

- **Method:** zone = floor((lon + 180)/6) + 1 with zone 32 widened over south-west Norway and zones $31/33/35/37$ over Svalbard; band = 8-degree letter from 80 S (X spans $72 - 84$ N)
- **Assumes:** 80 S <= latitude <= 84 N
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



UTM zones (6° of longitude) and latitude bands (8°) over Europe, 18° W to 24° E and 32° N to 72° N: every label is the zone number and band letter the package returns for the cell's centre (35 runs); the grid lines are drawn at the 6° and 8° spacing. Marked: the worked example 51.5°, -0.13° in zone 30, band U (N), central meridian -3°.

```
geospatial_analytics_ai999x.solve('geo_utm_zone', lat=51.5, lon=-0.13)
# -> zone = 30, band = U, hemisphere = N, central_meridian = -3   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_utm_zone', lat=60.0, lon=5.3)
# -> zone = 32, band = V, hemisphere = N, central_meridian = 9   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_utm_zone', lat=78.2, lon=15.6)
# -> zone = 33, band = X, hemisphere = N, central_meridian = 15   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_utm_zone', lat=-33.9, lon=18.4)
# -> zone = 34, band = H, hemisphere = S, central_meridian = 21   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
UTM zone: lat = 51.5, lon = -0.13	SOLVED	zone = 30, band = U, hemisphere = N, central_meridian = -3
UTM zone: lat = 60, lon = 5.3	SOLVED	zone = 32, band = V, hemisphere = N, central_meridian = 9
UTM zone: lat = 78.2, lon = 15.6	SOLVED	zone = 33, band = X, hemisphere = N, central_meridian = 15
UTM zone: lat = -33.9, lon = 18.4	SOLVED	zone = 34, band = H, hemisphere = S, central_meridian = 21
UTM zone: lon = -0.13	NEEDS_INPUT	geo_utm_zone needs lat (latitude (decimal degrees)). Example: solve('geo_utm_zone', lat=51.5, lon=-0.13).
UTM zone: lat = 51.5, lon = -0.13, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ
UTM zone: lat = 85, lon = 10	INVALID_INPUT	UTM zone, latitude band and central meridian: UTM covers latitudes from 80 S to 84 N (polar regions use UPS).

Transverse Mercator projection: coordinates, scale factor and grid convergence — `geo_transverse_mercator_forward`

Easting and northing of a point in a transverse Mercator projection with a given origin, central-meridian scale factor and false origin on the WGS84 or GRS80 ellipsoid, with the point scale factor and the grid convergence.

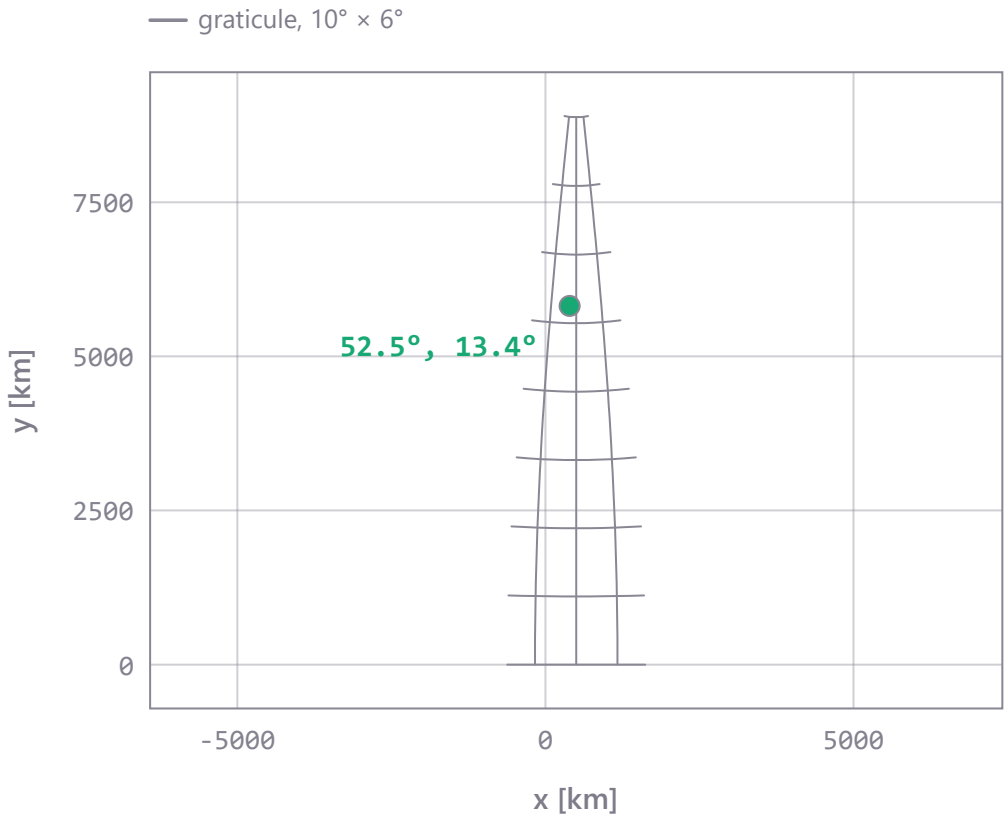
$$\xi = \xi' + \sum_{j=1}^{6} \alpha_j \sin 2j\xi' \cosh 2j\eta', \quad \eta = \eta' + \sum_{j=1}^{6} \alpha_j \cos 2j\xi' \sinh 2j\eta', \quad x = FE + k_0 A\eta,\ y = FN + k_0$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lat0`	latitude of origin (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`k0`	scale factor on the central meridian	
`ellipsoid`	reference ellipsoid	
`FE`	false easting (default 0)	
`FN`	false northing (default 0)	

output	meaning
`x`	easting
`y`	northing
`k`	point scale factor

output	meaning
`gamma`	grid convergence (degrees, positive east of the central meridian in the north)

- **Method:** Krueger's series to sixth order in the third flattening (conformal latitude, then the Gauss-Krueger sums); scale and convergence from the series' derivatives (Karney 2011)
- **Assumes:** within 10 degrees of longitude of the central meridian; latitude within 89 degrees of the equator
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Transverse Mercator about the 15° E meridian: the graticule (parallels every 10°, meridians every 6°) traced through the package's forward projection with lat0 = 0, lon0 = 15, k0 = 0.9996, ellipsoid = GRS80, FE = 500000.0, FN = 0.0 (222 projected points). Marked: the worked example, latitude 52.5°, longitude 13.4° → x = 391391 m, y = 5.81786e+06 m. Scale factor k = 0.999745, convergence γ = -1.26949°.

```
geospatial_analytics_ai999x.solve('geo_transverse_mercator_forward', lat=52.5, lon=13.4, lat0=0,
lon0=15, k0=0.9996, ellipsoid='GRS80', FE=500000.0, FN=0.0)
# -> x = 391391 m, y = 5.81786e+06 m, k = 0.999745, gamma = -1.26949   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_transverse_mercator_forward', lat=51.5, lon=-0.13, lat0=49,
lon0=-2, k0=0.9996012717, ellipsoid='WGS84', FE=400000.0, FN=-100000.0)
# -> x = 529797 m, y = 179632 m, k = 0.999808, gamma = 1.46368   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transverse Mercator forward: lat = 52.5, lon = 13.4, lat0 = 0, lon0 = 15, k0	SOLVED	x = 391391 m, y = 5.81786e+06 m, k = 0.999745, gamma = -1.26949

question	outcome	result
= 0.9996, ellipsoid = GRS80, FE = 500000 m, FN = 0 m		
Transverse Mercator forward: lat = 51.5, lon = -0.13, lat0 = 49, lon0 = -2, k0 = 0.9996012717, ellipsoid = WGS84, FE = 400000 m, FN = -100000 m	SOLVED	x = 529797 m, y = 179632 m, k = 0.999808, gamma = 1.46368
Transverse Mercator forward: lon = 13.4, lat0 = 0, lon0 = 15, k0 = 0.9996, ellipsoid = GRS80, FE = 500000 m, FN = 0 m	NEEDS_INPUT	geo_transverse_mercator_forward needs lat (latitude (decimal degrees)). Example: solve('geo_transverse_mercator_forward'
Transverse Mercator forward: lat = 52.5, lon = 13.4, lat0 = 0, lon0 = 15, k0 = 0.9996, ellipsoid = GRS80, FE = 500000 m, FN = 0 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Latitude and longitude to UTM coordinates (with scale factor and convergence) — `geo_utm_forward`

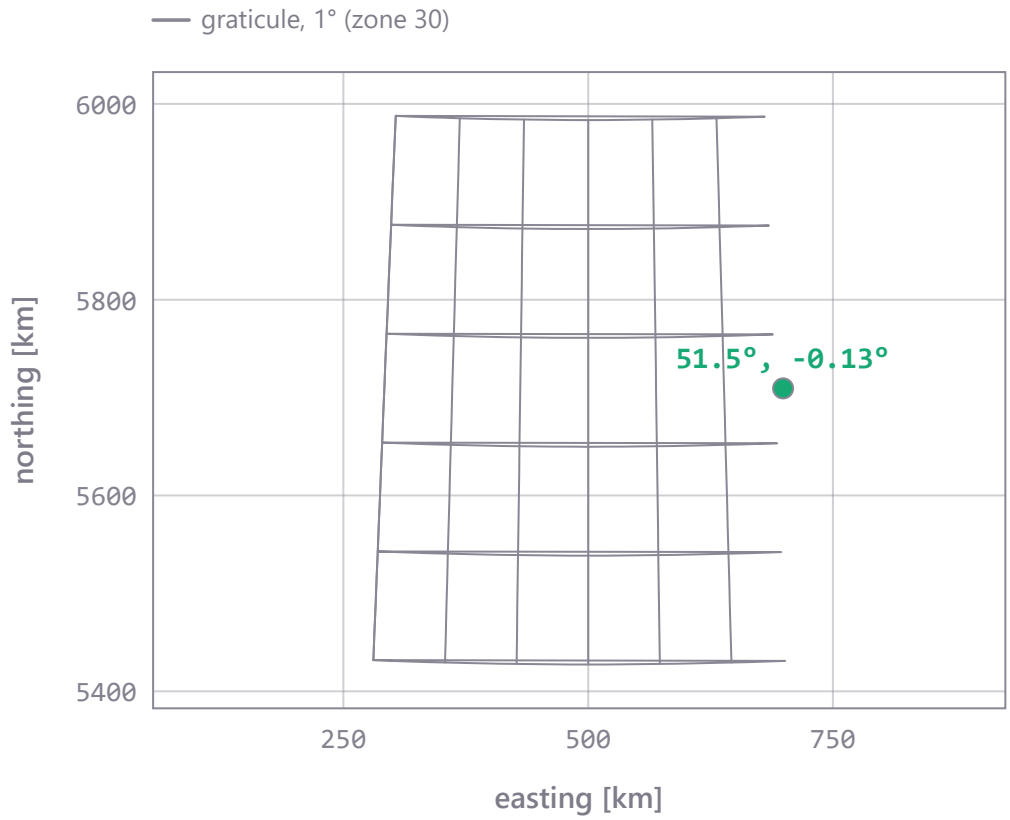
UTM zone, hemisphere, easting and northing of a point on the WGS84 or GRS80 ellipsoid, with the point scale factor and grid convergence.

$$E = 500\,000 + 0.9996\,A\eta, \quad N = N_0 + 0.9996\,A\xi$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`easting`	UTM easting
`northing`	UTM northing
`zone`	UTM zone number
`hemisphere`	N or S
k	point scale factor
`gamma`	grid convergence (degrees)

- **Method:** zone from the longitude (Norway and Svalbard exceptions), then transverse Mercator by Krueger's series with $k_0 = 0.9996$, false easting 500 km, false northing 10000 km in the south
- **Assumes:** 80 S <= latitude <= 84 N
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 1° graticule of UTM zone 30 (6° W to 0°, 49° N to 54° N) traced through the package's UTM projection (297 points; the meridians converge northward, the central meridian 3° W is straight). Marked: latitude 51.5°, longitude -0.13° → zone 30N, easting 699196 m, northing 5.70934e+06 m, scale factor 1.00009, convergence 2.24682°.

```
geospatial_analytics_ai999x.solve('geo_utm_forward', lat=51.5, lon=-0.13, ellipsoid='WGS84')
# -> easting = 699196 m, northing = 5.70934e+06 m, k = 1.00009, gamma = 2.24682, zone = 30,
hemisphere = N   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_utm_forward', lat=-33.9, lon=18.4, ellipsoid='WGS84')
# -> easting = 259583 m, northing = 6.24589e+06 m, k = 1.00031, gamma = 1.45083, zone = 34,
hemisphere = S   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_utm_forward', lat=60.4, lon=5.3, ellipsoid='GRS80')
# -> easting = 296192 m, northing = 6.70168e+06 m, k = 1.00011, gamma = -3.21823, zone = 32,
hemisphere = N   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
UTM forward: lat = 51.5, lon = -0.13, ellipsoid = WGS84	SOLVED	easting = 699196 m, northing = 5.70934e+06 m, k = 1.00009, gamma = 2.24682, zone = 30, hemisphere = N
UTM forward: lat = -33.9, lon = 18.4, ellipsoid = WGS84	SOLVED	easting = 259583 m, northing = 6.24589e+06 m, k = 1.00031, gamma = 1.45083, zone = 34, hemisphere = S
UTM forward: lat = 60.4, lon = 5.3, ellipsoid = GRS80	SOLVED	easting = 296192 m, northing = 6.70168e+06 m, k = 1.00011, gamma = -3.21823, zone = 32, hemisphere = N
UTM forward: lon = -0.13, ellipsoid = WGS84	NEEDS_INPUT	geo_utm_forward needs lat (latitude (decimal degrees)). Example: solve('geo_utm_forward', lat=51.5, lon=-0.13, ellipsoid

question	outcome	result
UTM forward: lat = 51.5, lon = -0.13, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

UTM coordinates to latitude and longitude — `geo_utm_inverse`

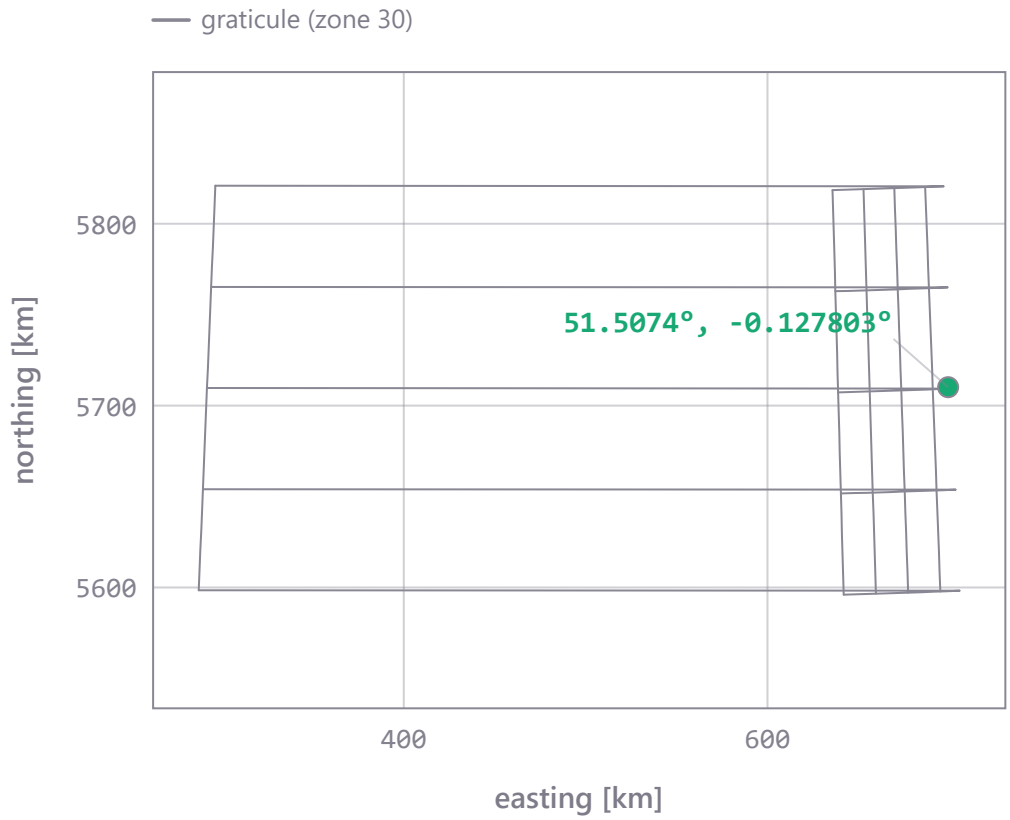
Latitude and longitude on the WGS84 or GRS80 ellipsoid of a point given by its UTM easting, northing, zone and hemisphere.

$$\xi' = \xi - \sum_{j=1}^{6} \beta_j \sin 2j\xi \cosh 2j\eta, \quad \eta' = \eta - \sum_{j=1}^{6} \beta_j \cos 2j\xi \sinh 2j\eta$$

input	meaning	condition
`easting`	UTM easting	
`northing`	UTM northing	
`zone`	UTM zone number	
`hemisphere`	hemisphere of the zone (N: false northing 0; S: false northing 10000 km)	
`ellipsoid`	reference ellipsoid	

output	meaning
`lat`	latitude (degrees)
`lon`	longitude (degrees)

- **Method:** inverse Krueger series to the conformal latitude, then Newton iteration for the geodetic latitude
- **Assumes:** 100 km <= easting <= 900 km; 0 <= northing <= 10000 km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The input UTM coordinates (zone 30N, easting 699316.0 m, northing 5710164.0 m) on the graticule traced by the package's UTM forward projection (160 points, parallels every 0.5°, meridians every 0.25°). Marked: the returned latitude 51.5074° and longitude -0.127803°.

```
geospatial_analytics_ai999x.solve('geo_utm_inverse', easting=699316.0, northing=5710164.0, zone=30,
hemisphere='N', ellipsoid='WGS84')
# -> lat = 51.5074, lon = -0.127803   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_utm_inverse', easting=261878.0, northing=6243186.0, zone=34,
hemisphere='S', ellipsoid='WGS84')
# -> lat = -33.9249, lon = 18.4241   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
UTM inverse: easting = 699316.0 m, northing = 5710164.0 m, zone = 30, hemisphere = N, ellipsoid = WGS84	SOLVED	lat = 51.5074, lon = -0.127803
UTM inverse: easting = 261878.0 m, northing = 6243186.0 m, zone = 34, hemisphere = S, ellipsoid = WGS84	SOLVED	lat = -33.9249, lon = 18.4241
UTM inverse: northing = 5710164.0 m, zone = 30, hemisphere = N, ellipsoid = WGS84	NEEDS_INPUT	geo_utm_inverse needs easting (UTM easting). Example: solve('geo_utm_inverse', easting='699316 m', northing='5.71016e+06
UTM inverse: easting = 699316.0 m, northing = 5710164.0 m, zone = 30, hemisphere = N, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Latitude and longitude to Web Mercator (EPSG:3857) — `geo_web_mercator_forward`

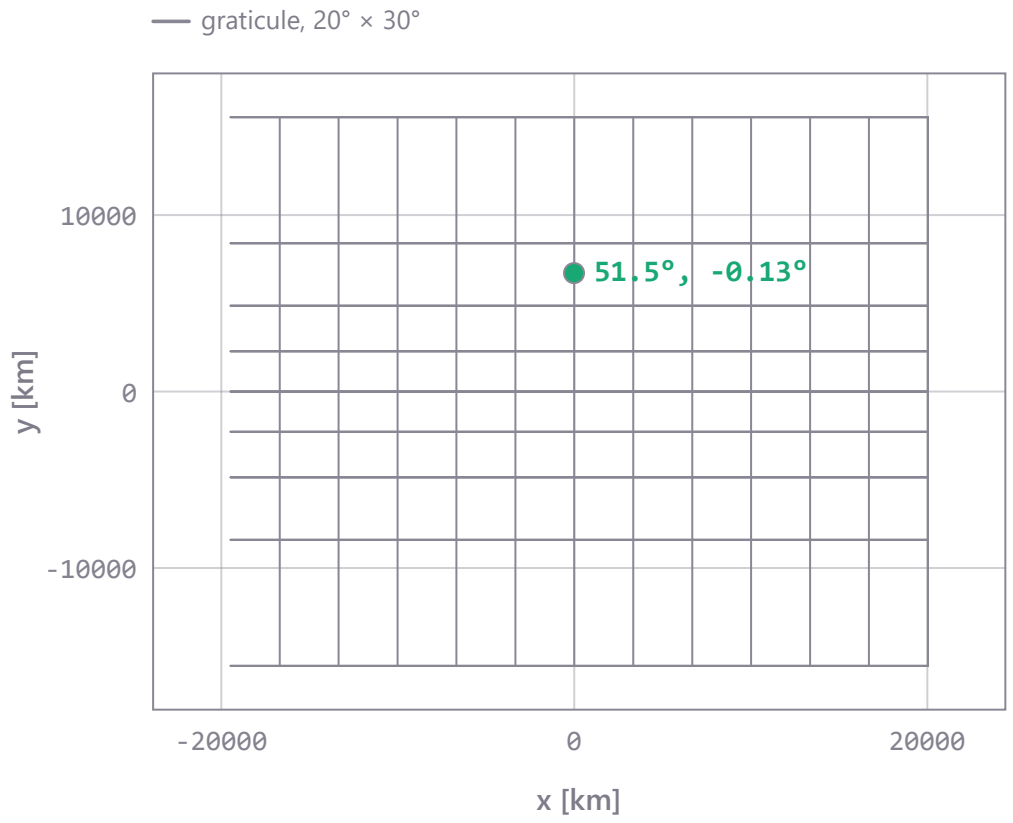
Web Mercator (pseudo-Mercator, EPSG:3857) x and y of a WGS84 latitude and longitude, as used by web maps.

$$x = R\lambda, \quad y = R \ln \tan\left(\frac{\pi}{4} + \frac{\varphi}{2}\right), \quad R = 6378137\ \text{m}$$

input	meaning	condition
`lat`	latitude (WGS84) (decimal degrees)	
`lon`	longitude (WGS84) (decimal degrees)	

output	meaning
`x`	Web Mercator x
`y`	Web Mercator y

- **Method:** spherical Mercator formulas on a sphere of radius 6378137 m applied to WGS84 coordinates
- **Assumes:** |latitude| <= 85.06 degrees (the square web-map extent)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Web Mercator world graticule: the graticule (parallels every 20°, meridians every 30°) traced through the package's forward projection with (1086 projected points). Marked: the worked example, latitude 51.5°, longitude -0.13° → x = -14471.5 m, y = 6.71022e+06 m.

```
geospatial_analytics_ai999x.solve('geo_web_mercator_forward', lat=51.5, lon=-0.13)
# -> x = -14471.5 m, y = 6.71022e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_web_mercator_forward', lat=-33.87, lon=151.21)
# -> x = 1.68326e+07 m, y = -4.01136e+06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Web Mercator forward: lat = 51.5, lon = -0.13	SOLVED	x = -14471.5 m, y = 6.71022e+06 m
Web Mercator forward: lat = -33.87, lon = 151.21	SOLVED	x = 1.68326e+07 m, y = -4.01136e+06 m
Web Mercator forward: lon = -0.13	NEEDS_INPUT	geo_web_mercator_forward needs lat (latitude (WGS84) (decimal degrees)). Example: solve('geo_web_mercator_forward', lat=
Web Mercator forward: lat = 51.5, lon = -0.13, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Web Mercator (EPSG:3857) to latitude and longitude — `geo_web_mercator_inverse`

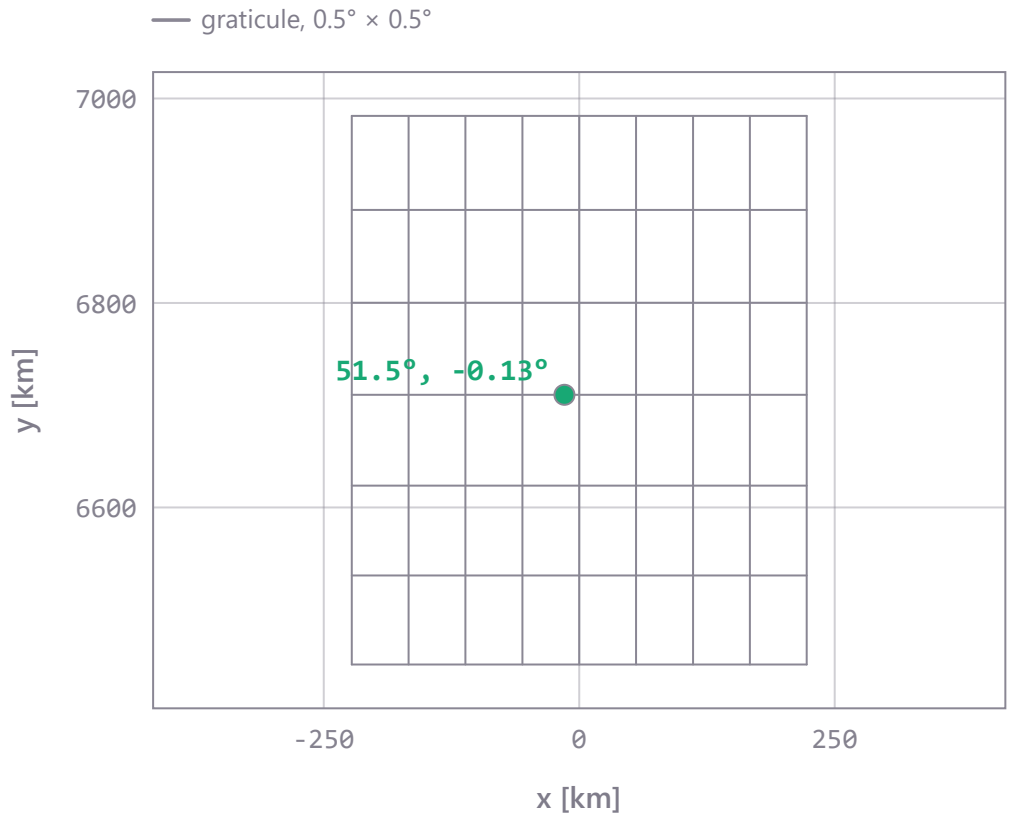
WGS84 latitude and longitude of a point given by its Web Mercator (pseudo-Mercator) x and y.

$$\varphi = \arctan\sinh\frac{y}{R}, \quad \lambda = \frac{x}{R}$$

input	meaning	condition
`x`	Web Mercator x	
`y`	Web Mercator y	

output	meaning
`lat`	latitude (degrees)
`lon`	longitude (degrees)

- **Method:** inverse spherical Mercator: lat = atan(sinh(y/R)), lon = x/R
- **Assumes:** inside the web-map square
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Web Mercator inverse around London: the input point x = -14471.5 m, y = 6710219.1 m placed on the graticule that the package's forward projection traces with the same parameters (236 points, parallels every 0.5°, meridians every 0.5°). Marked: the returned latitude 51.5° and longitude -0.13°, which lie where the point falls among the lines.

```
geospatial_analytics_ai999x.solve('geo_web_mercator_inverse', x=-14471.5, y=6710219.1)
# -> lat = 51.5, lon = -0.13   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_web_mercator_inverse', x=16832556.0, y=-4011000.0)
# -> lat = -33.8673, lon = 151.209   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Web Mercator inverse: x = -14471.5 m, y = 6710219.1 m	SOLVED	lat = 51.5, lon = -0.13
Web Mercator inverse: x = 16832556.0 m, y = -4011000.0 m	SOLVED	lat = -33.8673, lon = 151.209
Web Mercator inverse: y = 6710219.1 m	NEEDS_INPUT	geo_web_mercator_inverse needs x (Web Mercator x). Example: solve('geo_web_mercator_inverse', x='-14471.5 m', y='6.71022
Web Mercator inverse: x = -14471.5 m, y = 6710219.1 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Web-map tile (slippy map / XYZ) and quadkey containing a point — `geo_web_map_tile`

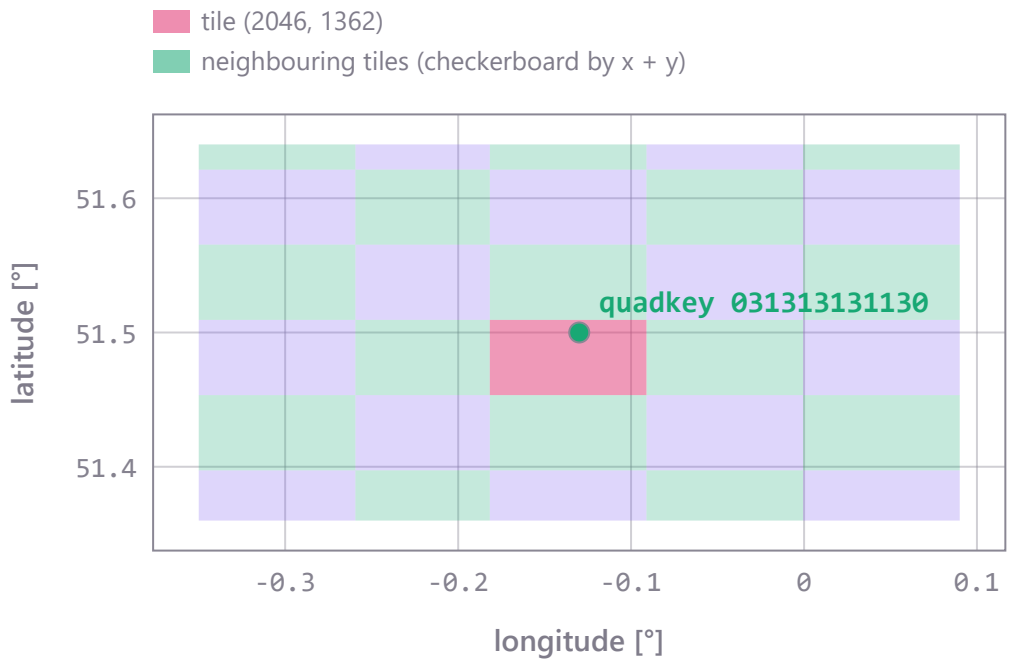
The x and y indices of the web-map (XYZ / slippy map) tile containing a WGS84 point at a zoom level, and the tile's quadkey.

$$x_t = \left\lfloor 2^z \, \frac{\lambda + 180^\circ}{360^\circ} \right\rfloor, \quad y_t = \left\lfloor 2^z \left(\frac{1}{2} - \frac{\operatorname{arsinh}(\tan \varphi)}{2\pi} \right) \right\rfloor$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`zoom`	zoom level $(0 - 24)$	

output	meaning
`tile_x`	tile column (from 180 W)
`tile_y`	tile row (from the north edge)
`quadkey`	quadkey of the tile

- **Method:** Web Mercator fractions x = (lon + 180)/360, y = 1/2 - asinh(tan lat)/(2 pi), scaled by 2^zoom and floored; quadkey digits interleave the bits of x and y
- **Assumes:** |latitude| <= 85.0511; zoom $0 - 24$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Web-map tiles at zoom 12 around latitude 51.5°, longitude -0.13°: each small cell is coloured by the tile the package returns for its centre (1020 runs), alternately by the parity of tile x + y so the tile boundaries show. Marked: the worked example in tile x = 2046, y = 1362 (quadkey 031313131130).

```
geospatial_analytics_ai999x.solve('geo_web_map_tile', lat=51.5, lon=-0.13, zoom=12)
# -> tile_x = 2046, tile_y = 1362, quadkey = 031313131130   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_web_map_tile', lat=-33.87, lon=151.21, zoom=8)
# -> tile_x = 235, tile_y = 153, quadkey = 31123013   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Web map tile: lat = 51.5, lon = -0.13, zoom = 12	SOLVED	tile_x = 2046, tile_y = 1362, quadkey = 031313131130
Web map tile: lat = -33.87, lon = 151.21, zoom = 8	SOLVED	tile_x = 235, tile_y = 153, quadkey = 31123013
Web map tile: lon = -0.13, zoom = 12	NEEDS_INPUT	geo_web_map_tile needs lat (latitude (decimal degrees)). Example: solve('geo_web_map_tile', lat=51.5, lon=-0.13, zoom=12
Web map tile: lat = 51.5, lon = -0.13, zoom = 12, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Mercator projection (ellipsoidal): coordinates and scale factor — `geo_mercator_forward`

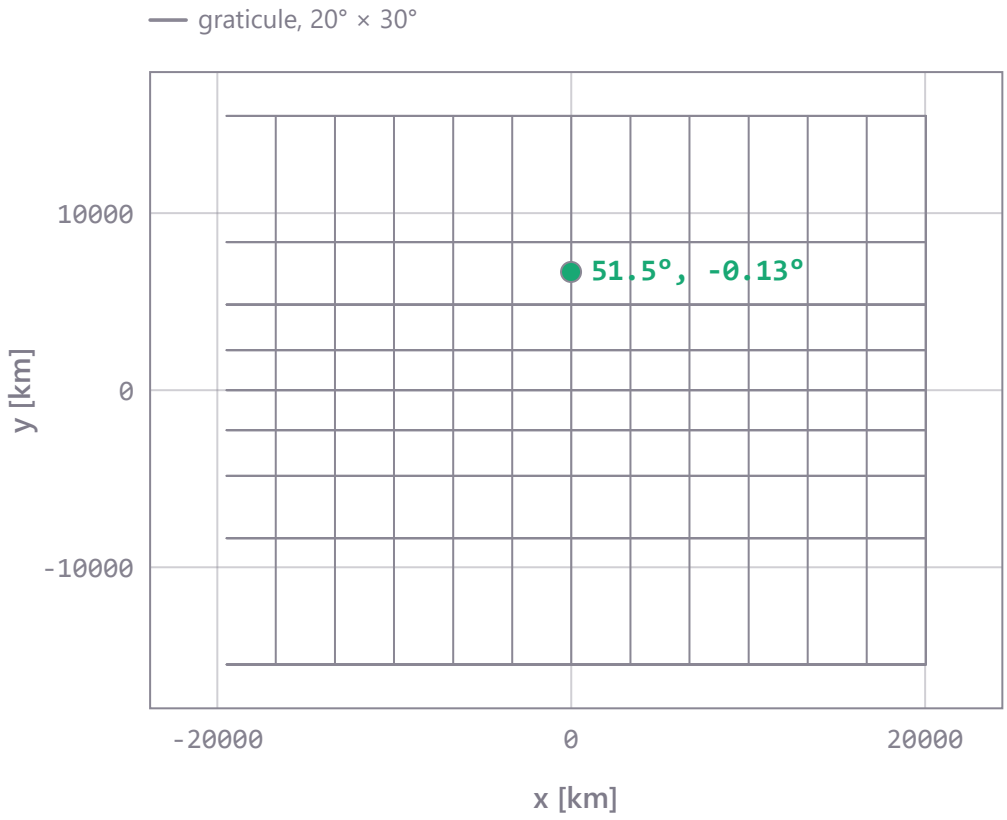
x and y of a point in the ellipsoidal Mercator projection with a central meridian and standard parallel (lat_ts = 0 is World Mercator, EPSG:3395), with the point scale factor.

$$x = k_0 a(\lambda - \lambda_0), \quad y = k_0 a\left[\operatorname{arsinh}(\tan\varphi) - e\operatorname{artanh}(e\sin\varphi)\right], \quad k = \frac{k_0\sqrt{1 - e^2\sin^2\varphi}}{\cos\varphi}$$

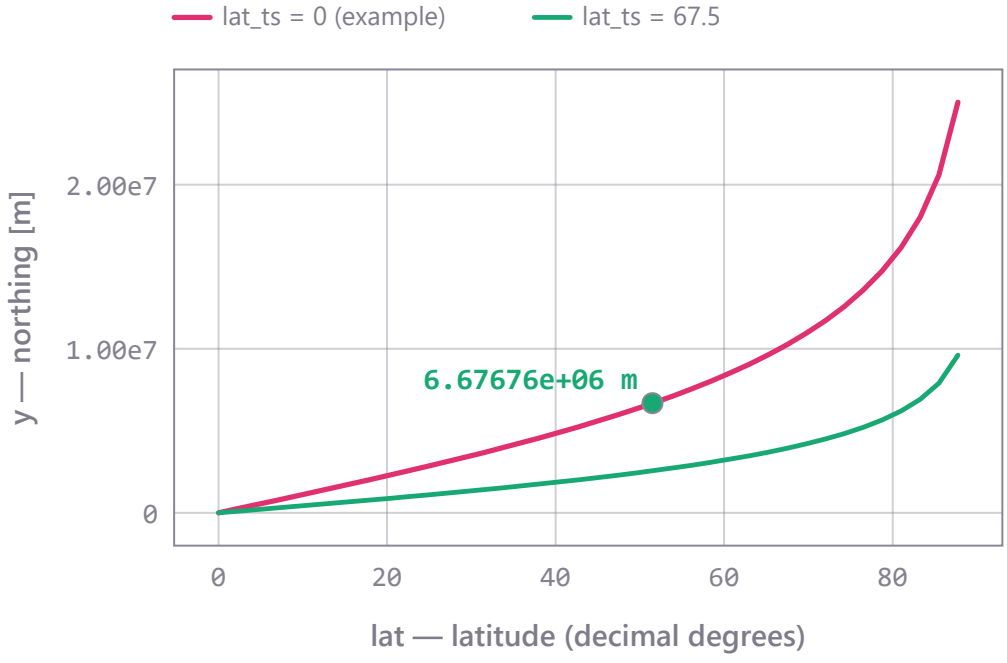
input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`lat_ts`	latitude of true scale (standard parallel) (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
x	easting
y	northing
k	point scale factor

- **Method:** $x = k_0$ a $(\lambda - lambda0)$, $y = k_0$ a psi with the isometric latitude psi; $k_0 = cos$(lat_ts)/sqrt($1 - e^2\ sin^2$ lat_ts)
- **Assumes:** |latitude| <= 89.5; |lat_ts| <= 85
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Mercator world graticule: the graticule (parallels every 20°, meridians every 30°) traced through the package's forward projection with lon0 = 0, lat_ts = 0, ellipsoid = WGS84 (1086 projected points). Marked: the worked example, latitude 51.5°, longitude -0.13° → x = -14471.5 m, y = 6.67676e+06 m. Scale factor at the point k = 1.60309.



y (northing) against lat (latitude (decimal degrees)) from 0 to 90. One curve for each value of lat_ts (latitude of true scale (standard parallel) (decimal degrees)): 0, 67.5. Other inputs as in the worked example: lon = -0.13, lon0 = 0, ellipsoid = WGS84. 40 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, lat = 51.5, y = 6.67676e+06 m.

```
geospatial_analytics_ai999x.solve('geo_mercator_forward', lat=51.5, lon=-0.13, lon0=0, lat_ts=0,
ellipsoid='WGS84')
# -> x = -14471.5 m, y = 6.67676e+06 m, k = 1.60309   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_mercator_forward', lat=-20.0, lon=120.0, lon0=110,
lat_ts=-15, ellipsoid='GRS80')
# -> x = 1.0755e+06 m, y = -2.18196e+06 m, k = 1.02774   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mercator forward: lat = 51.5, lon = -0.13, lon0 = 0, lat_ts = 0, ellipsoid = WGS84	SOLVED	x = -14471.5 m, y = 6.67676e+06 m, k = 1.60309
Mercator forward: lat = -20, lon = 120, lon0 = 110, lat_ts = -15, ellipsoid = GRS80	SOLVED	x = 1.0755e+06 m, y = -2.18196e+06 m, k = 1.02774
Mercator forward: lon = -0.13, lon0 = 0, lat_ts = 0, ellipsoid = WGS84	NEEDS_INPUT	geo_mercator_forward needs lat (latitude (decimal degrees)). Example: solve('geo_mercator_forward', lat=51.5, lon=-0.13,
Mercator forward: lat = 51.5, lon = -0.13, lon0 = 0, lat_ts = 0, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Inverse Mercator projection (ellipsoidal) — `geo_mercator_inverse`

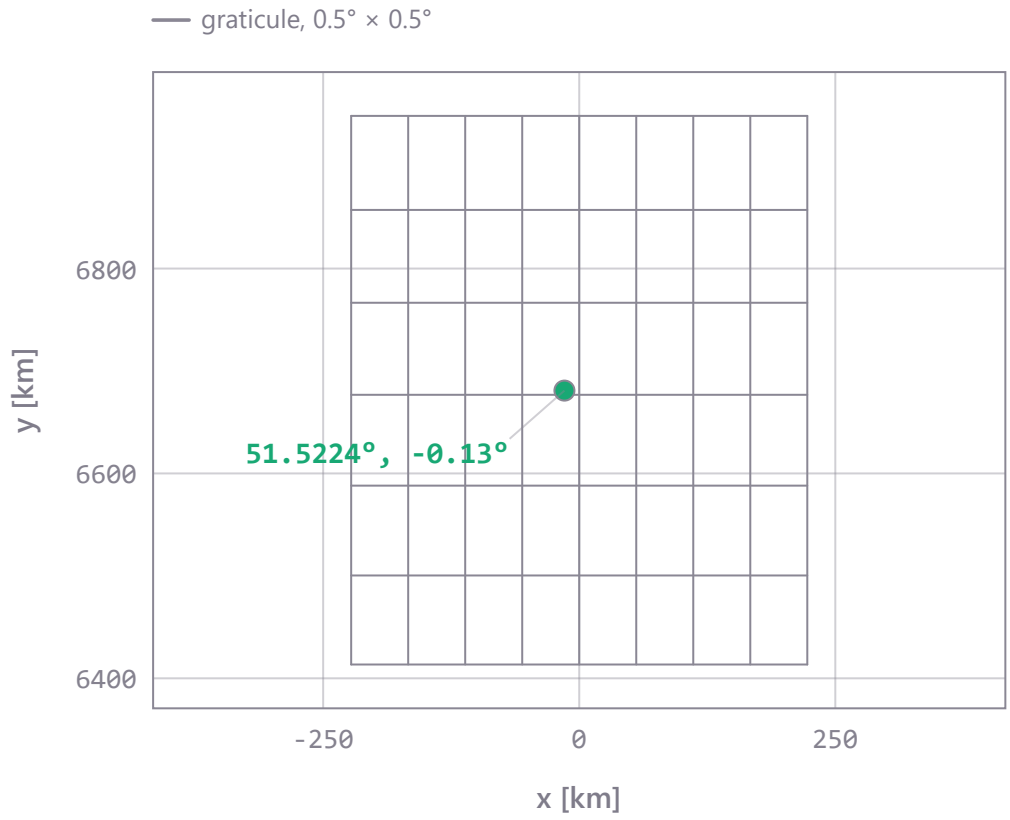
Latitude and longitude of a point given by its x and y in the ellipsoidal Mercator projection with a central meridian and standard parallel.

$$\psi = \frac{y}{k_0 a}, \quad \tau' = \sinh\psi, \quad \tau = \tan\varphi : \ \tau\sqrt{1+\sigma^2} - \sigma\sqrt{1+\tau^2} = \tau'$$

input	meaning	condition
`x`	easting	
`y`	northing	
`lon0`	central meridian (decimal degrees)	
`lat_ts`	latitude of true scale (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`lat`	latitude (degrees)
`lon`	longitude (degrees)

- **Method:** isometric latitude psi = y/(k0 a); conformal tangent sinh(psi); geodetic latitude by Newton iteration (Karney's tau method)
- **Assumes:** |lat_ts| <= 85
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Mercator inverse around London: the input point x = -14471.5 m, y = 6680752.0 m placed on the graticule that the package's forward projection traces with the same parameters (236 points, parallels every 0.5°, meridians every 0.5°). Marked: the returned latitude 51.5224° and longitude -0.13°, which lie where the point falls among the lines.

```
geospatial_analytics_ai999x.solve('geo_mercator_inverse', x=-14471.5, y=6680752.0, lon0=0,
lat_ts=0, ellipsoid='WGS84')
# -> lat = 51.5224, lon = -0.13   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_mercator_inverse', x=1000000.0, y=-2000000.0, lon0=110,
lat_ts=-15, ellipsoid='GRS80')
# -> lat = -18.3927, lon = 119.298   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mercator inverse: x = -14471.5 m, y = 6680752.0 m, lon0 = 0, lat_ts = 0, ellipsoid = WGS84	SOLVED	lat = 51.5224, lon = -0.13
Mercator inverse: x = 1000000 m, y = -2000000 m, lon0 = 110, lat_ts = -15, ellipsoid = GRS80	SOLVED	lat = -18.3927, lon = 119.298
Mercator inverse: y = 6680752.0 m, lon0 = 0, lat_ts = 0, ellipsoid = WGS84	NEEDS_INPUT	geo_mercator_inverse needs x (easting). Example: solve('geo_mercator_inverse', x='-14471.5 m', y='6.68075e+06 m', lon0=0
Mercator inverse: x = -14471.5 m, y = 6680752.0 m, lon0 = 0, lat_ts = 0, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Lambert conformal conic projection (two standard parallels): coordinates, scale factor and convergence — `geo_lambert_conformal_conic_forward`

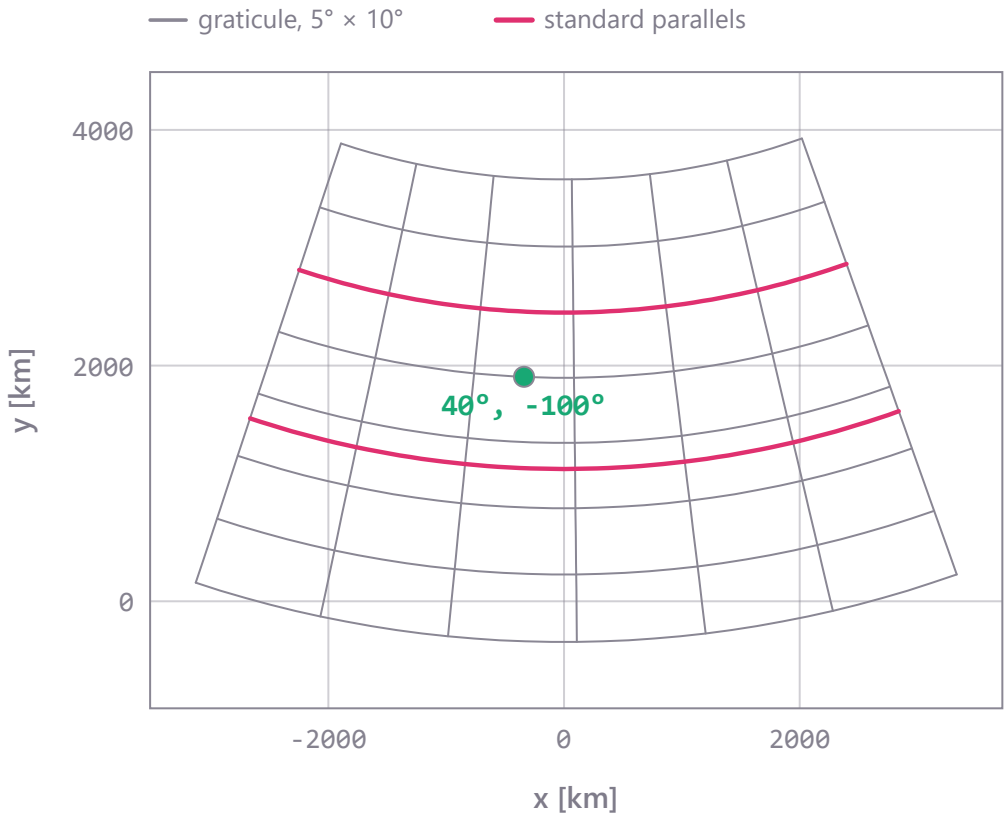
Easting and northing of a point in the ellipsoidal Lambert conformal conic projection with two standard parallels (EPSG method 9802), with the point scale factor and the grid convergence.

$$n = \frac{\ln m_1 - \ln m_2}{\ln t_1 - \ln t_2},\ \rho = aFt^n,\ x = FE + \rho\sin\theta,\ y = FN + \rho_0 - \rho\cos\theta,\ k = \frac{\rho n}{am}$$

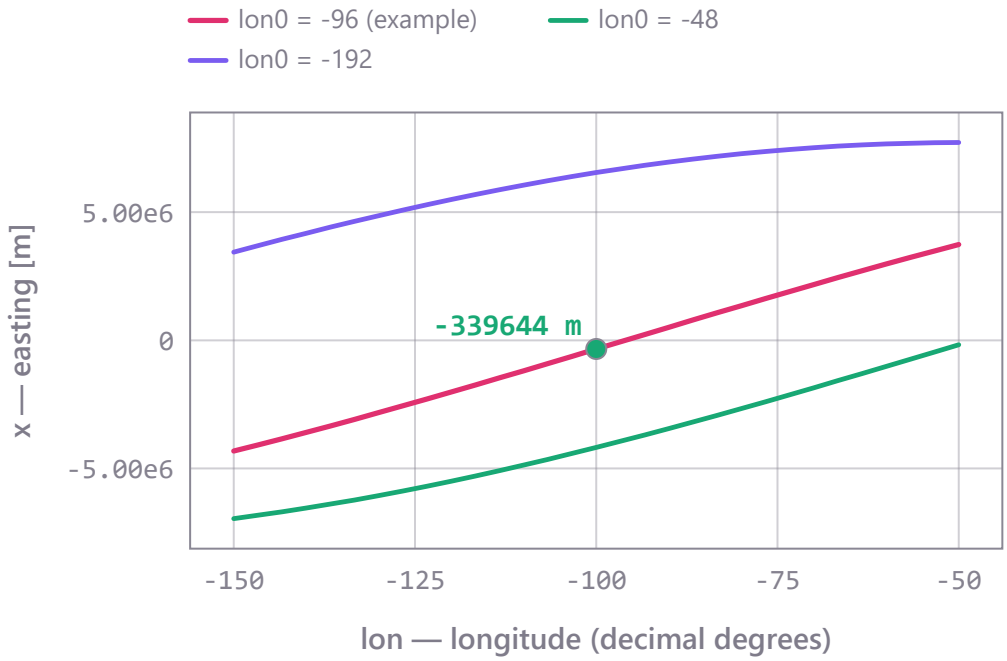
input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lat1`	first standard parallel (decimal degrees)	
`lat2`	second standard parallel (decimal degrees)	
`lat0`	latitude of origin (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`ellipsoid`	reference ellipsoid	
`FE`	false easting (default 0)	
`FN`	false northing (default 0)	

output	meaning
`x`	easting
`y`	northing
`k`	point scale factor
`gamma`	grid convergence (degrees)

- **Method:** n = (ln $m_1 - ln\ m_2)/(ln\ t_1 - ln$ t2), F = m1/(n t1^n), $\rho = a$ F t^n with $t = e^{-\psi}$; $x = FE + \rho$ sin(theta), $y = FN + rho0 - \rho$ cos(theta), $\theta = n\ (\lambda - lambda0)$; $k = \rho$ n/(a m); convergence = theta
- **Assumes:** parallels within 89 degrees of the equator and not symmetric about it
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lambert conformal conic over North America: the graticule (parallels every 5°, meridians every 10°) traced through the package's forward projection with lat1 = 33, lat2 = 45, lat0 = 23, lon0 = -96, ellipsoid = GRS80 (503 projected points). Marked: the worked example, latitude 40°, longitude -100° → x = -339644 m, y = 1.9046e+06 m. Scale factor k = 0.994666, convergence γ = -2.52199°.



x (easting) against lon (longitude (decimal degrees)) from -150 to -50. One curve for each value of lon0 (central meridian (decimal degrees)): -96, -48, -192. Other inputs as in the worked example: lat = 40.0, lat1 = 33, lat2 = 45, lat0 = 23, ellipsoid = GRS80. 31 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, lon = -100, x = -339644 m.

```
geospatial_analytics_ai999x.solve('geo_lambert_conformal_conic_forward', lat=40.0, lon=-100.0,
lat1=33, lat2=45, lat0=23, lon0=-96, ellipsoid='GRS80')
# -> x = -339644 m, y = 1.9046e+06 m, k = 0.994666, gamma = -2.52199   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_lambert_conformal_conic_forward', lat=46.8, lon=2.35,
lat1=44, lat2=49, lat0=46.5, lon0=3, ellipsoid='GRS80', FE=700000.0, FN=6600000.0)
# -> x = 650426 m, y = 6.63352e+06 m, k = 0.999063, gamma = -0.471645   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lambert conformal conic forward: lat = 40, lon = -100, lat1 = 33, lat2 = 45, lat0 = 23, lon0 = -96, ellipsoid = GRS80	SOLVED	x = -339644 m, y = 1.9046e+06 m, k = 0.994666, gamma = -2.52199
Lambert conformal conic forward: lat = 46.8, lon = 2.35, lat1 = 44, lat2 = 49, lat0 = 46.5, lon0 = 3, ellipsoid = GRS80, FE = 700000 m, FN = 6600000 m	SOLVED	x = 650426 m, y = 6.63352e+06 m, k = 0.999063, gamma = -0.471645
Lambert conformal conic forward: lon = -100, lat1 = 33, lat2 = 45, lat0 = 23, lon0 = -96, ellipsoid = GRS80	NEEDS_INPUT	geo_lambert_conformal_conic_forward needs lat (latitude (decimal degrees)). Example: solve('geo_lambert_conformal_conic_
Lambert conformal conic forward: lat = 40, lon = -100, lat1 = 33, lat2 = 45, lat0 = 23, lon0 = -96, ellipsoid = GRS80, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Inverse Lambert conformal conic projection (two standard parallels) — `geo_lambert_conformal_conic_inverse`

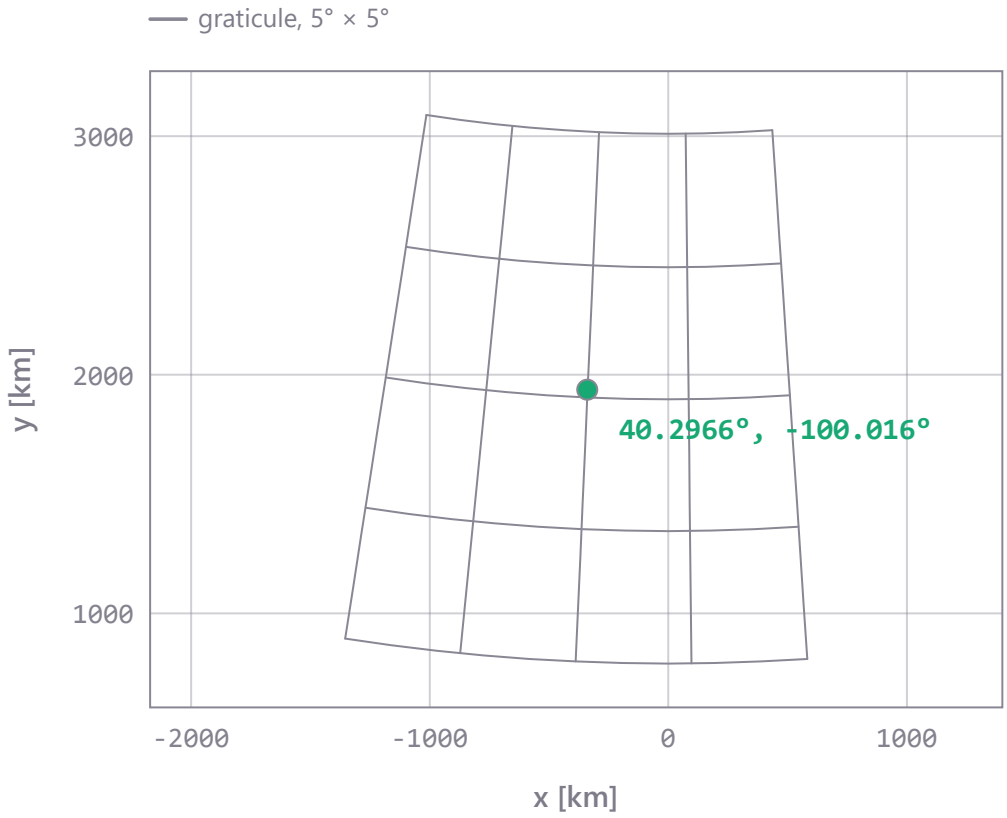
Latitude and longitude of a point given by its easting and northing in the ellipsoidal Lambert conformal conic projection with two standard parallels.

$$\rho = \operatorname{sgn}(n)\sqrt{x'^2 + (\rho_0 - y')^2}, \quad t = \left(\frac{\rho}{aF}\right)^{1/n}, \quad \lambda = \lambda_0 + \frac{\theta}{n}$$

input	meaning	condition
`x`	easting	
`y`	northing	
`lat1`	first standard parallel (decimal degrees)	
`lat2`	second standard parallel (decimal degrees)	
`lat0`	latitude of origin (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`ellipsoid`	reference ellipsoid	
`FE`	false easting (default 0)	
`FN`	false northing (default 0)	

output	meaning
`lat`	latitude (degrees)
`lon`	longitude (degrees)

- **Method:** rho and theta from the offsets, t = (rho/(aF))^$(1/n)$, isometric latitude -ln t, geodetic latitude by Newton iteration; $\lambda = lambda0 + \theta/n$
- **Assumes:** parallels within 89 degrees of the equator and not symmetric about it
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lambert conformal conic inverse: a projected point located on its graticule: the input point x = -339545.0 m, y = 1937380.0 m placed on the graticule that the package's forward projection traces with the same parameters (110 points, parallels every 5°, meridians every 5°). Marked: the returned latitude 40.2966° and longitude -100.016°, which lie where the point falls among the lines.

```
geospatial_analytics_ai999x.solve('geo_lambert_conformal_conic_inverse', x=-339545.0, y=1937380.0,
lat1=33, lat2=45, lat0=23, lon0=-96, ellipsoid='GRS80')
# -> lat = 40.2966, lon = -100.016   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_lambert_conformal_conic_inverse', x=652000.0, y=6862000.0,
lat1=44, lat2=49, lat0=46.5, lon0=3, ellipsoid='GRS80', FE=700000.0, FN=6600000.0)
# -> lat = 48.8562, lon = 2.34581   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lambert conformal conic inverse: x = -339545.0 m, y = 1937380.0 m, lat1 = 33, lat2 = 45, lat0 = 23, lon0 = -96, ellipsoid = GRS80	SOLVED	lat = 40.2966, lon = -100.016
Lambert conformal conic inverse: x = 652000 m, y = 6862000 m, lat1 = 44, lat2 = 49, lat0 = 46.5, lon0 = 3,	SOLVED	lat = 48.8562, lon = 2.34581

question	outcome	result
ellipsoid = GRS80, FE = 700000 m, FN = 6600000 m		
Lambert conformal conic inverse: y = 1937380.0 m, lat1 = 33, lat2 = 45, lat0 = 23, lon0 = -96, ellipsoid = GRS80	NEEDS_INPUT	geo_lambert_conformal_conic_inverse needs x (easting). Example: solve('geo_lambert_conformal_conic_inverse', x='-339545
Lambert conformal conic inverse: x = -339545.0 m, y = 1937380.0 m, lat1 = 33, lat2 = 45, lat0 = 23, lon0 = -96, ellipsoid = GRS80, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Albers equal-area conic projection: coordinates and scale factors —

`geo_albers_equal_area_forward`

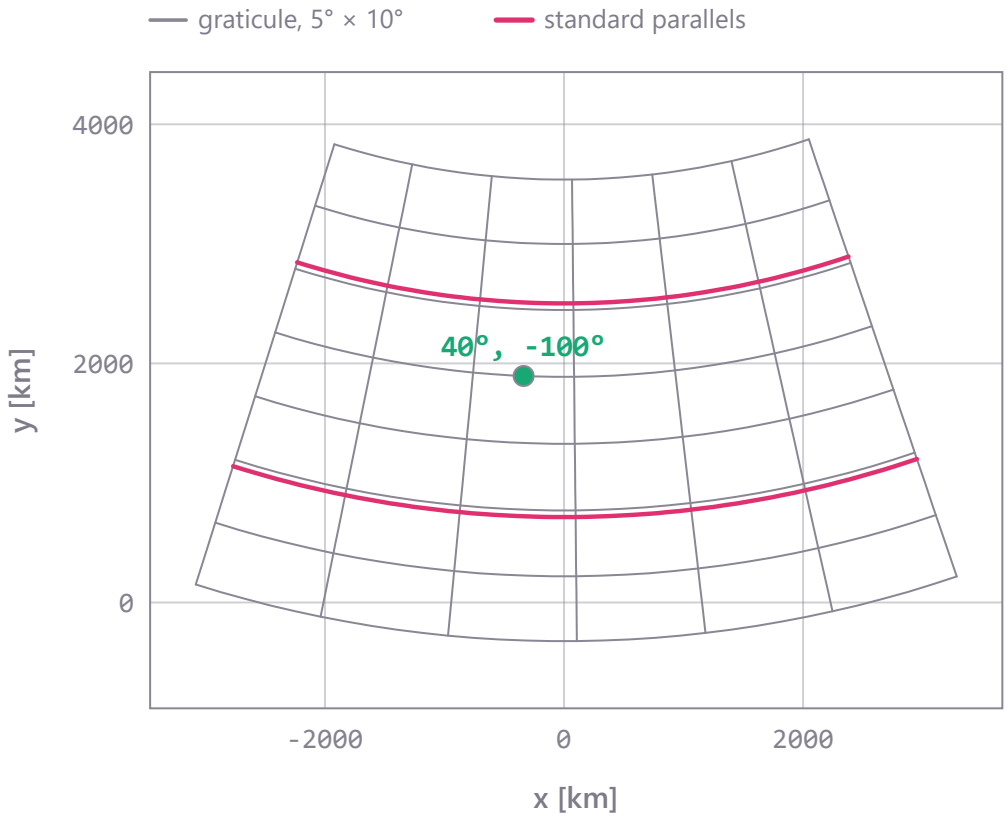
Easting and northing of a point in the ellipsoidal Albers equal-area conic projection with two standard parallels, with the scale factors along the parallel (k) and the meridian $(h = 1/k)$.

$$n = \frac{m_1^2 - m_2^2}{q_2 - q_1}, \quad \rho = \frac{a\sqrt{C - nq}}{n}, \quad x = FE + \rho \sin\theta,\ y = FN + \rho_0 - \rho\cos\theta,\ k = \frac{\rho n}{am} = \frac{1}{h}$$

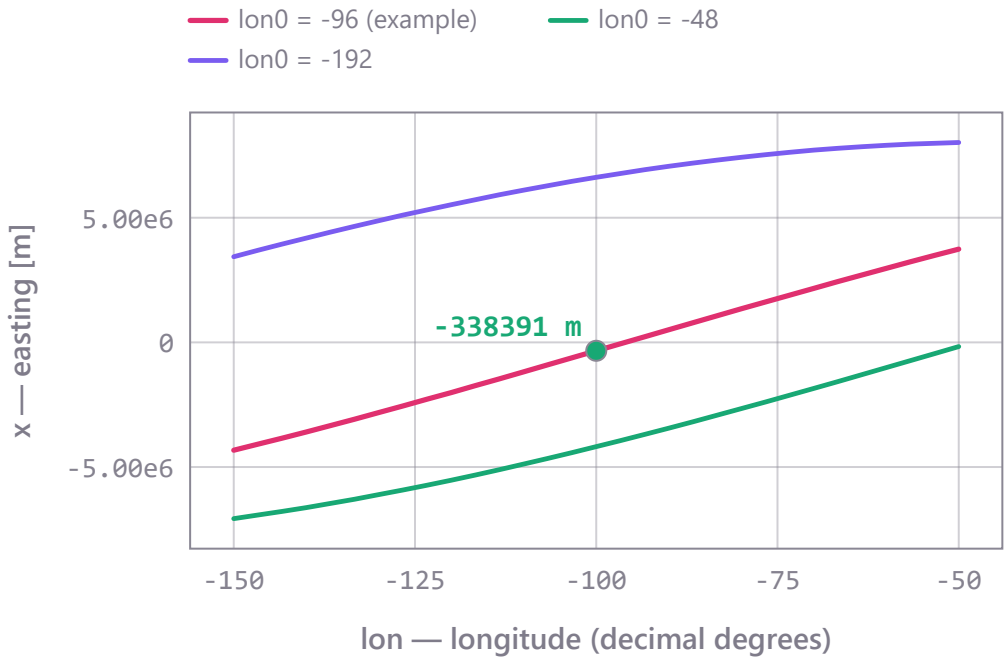
input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lat1`	first standard parallel (decimal degrees)	
`lat2`	second standard parallel (decimal degrees)	
`lat0`	latitude of origin (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`ellipsoid`	reference ellipsoid	
`FE`	false easting (default 0)	
`FN`	false northing (default 0)	

output	meaning
`x`	easting
`y`	northing
`k`	scale factor along the parallel
`h`	scale factor along the meridian

- **Method:** $n = (m_1^2 - m_2^2)/(q_2 - q_1)$, $C = m_1^2 + n$ q1, $\rho = a$ sqrt(C - n q)/n with the authalic function q; $x = FE + \rho$ sin(theta), $y = FN + rho0 - \rho$ cos(theta)
- **Assumes:** parallels within 89 degrees of the equator and not symmetric about it
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Albers equal-area conic over North America: the graticule (parallels every 5°, meridians every 10°) traced through the package's forward projection with lat1 = 29.5, lat2 = 45.5, lat0 = 23, lon0 = -96, ellipsoid = GRS80 (503 projected points). Marked: the worked example, latitude 40°, longitude -100° → x = -338391 m, y = 1.8941e+06 m. Scale factors at the point: k = 0.990969 along the parallel, h = 1.00911 along the meridian.



x (easting) against lon (longitude (decimal degrees)) from -150 to -50. One curve for each value of lon0 (central meridian (decimal degrees)): -96, -48, -192. Other inputs as in the worked example: lat = 40.0, lat1 = 29.5, lat2 = 45.5, lat0 = 23, ellipsoid = GRS80. 31 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, lon = -100, x = -338391 m.

```
geospatial_analytics_ai999x.solve('geo_albers_equal_area_forward', lat=40.0, lon=-100.0, lat1=29.5,
lat2=45.5, lat0=23, lon0=-96, ellipsoid='GRS80')
# -> x = -338391 m, y = 1.8941e+06 m, k = 0.990969, h = 1.00911   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_albers_equal_area_forward', lat=-25.0, lon=135.0, lat1=-18,
lat2=-36, lat0=0, lon0=132, ellipsoid='GRS80')
# -> x = 299362 m, y = -2.70609e+06 m, k = 0.988573, h = 1.01156   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Albers equal-area forward: lat = 40, lon = -100, lat1 = 29.5, lat2 = 45.5, lat0 = 23, lon0 = -96, ellipsoid = GRS80	SOLVED	x = -338391 m, y = 1.8941e+06 m, k = 0.990969, h = 1.00911
Albers equal-area forward: lat = -25, lon = 135, lat1 = -18, lat2 = -36, lat0 = 0, lon0 = 132, ellipsoid = GRS80	SOLVED	x = 299362 m, y = -2.70609e+06 m, k = 0.988573, h = 1.01156
Albers equal-area forward: lon = -100, lat1 = 29.5, lat2 = 45.5, lat0 = 23, lon0 = -96, ellipsoid = GRS80	NEEDS_INPUT	geo_albers_equal_area_forward needs lat (latitude (decimal degrees)). Example: solve('geo_albers_equal_area_forward', la
Albers equal-area forward: lat = 40, lon = -100, lat1 = 29.5, lat2 = 45.5, lat0 = 23, lon0 = -96, ellipsoid = GRS80, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Inverse Albers equal-area conic projection — `geo_albers_equal_area_inverse`

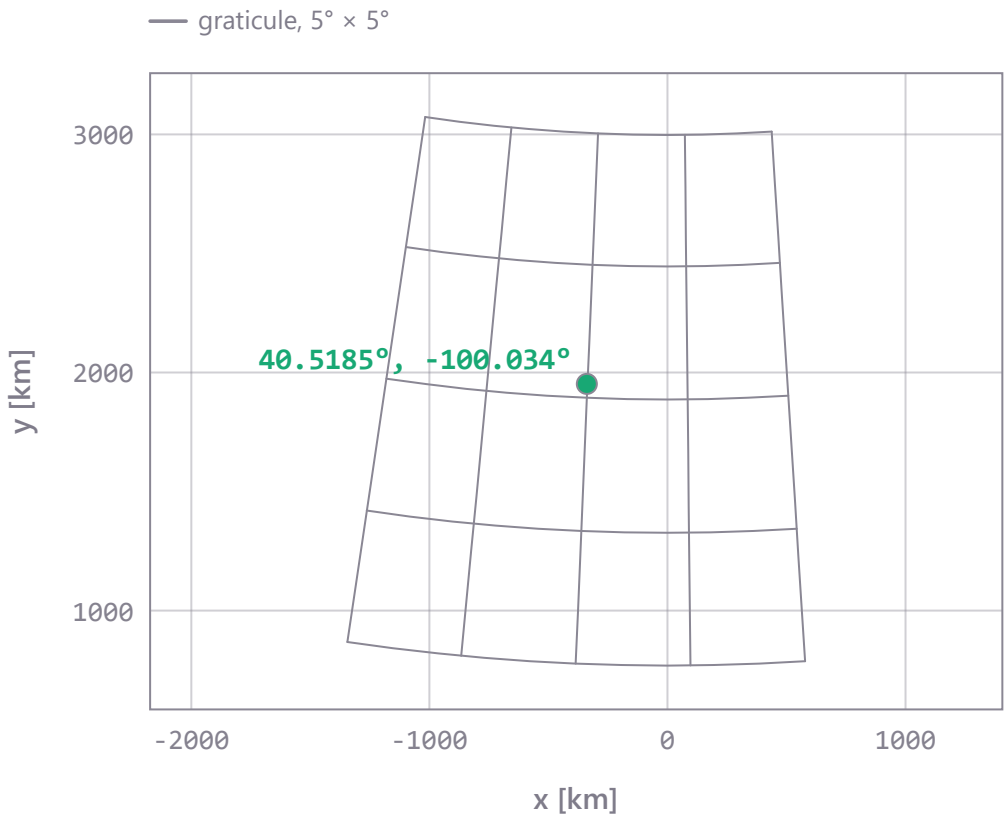
Latitude and longitude of a point given by its easting and northing in the ellipsoidal Albers equal-area conic projection with two standard parallels.

$$q = \frac{C - \rho^2 n^2 / a^2}{n}, \quad \varphi : \; q(\varphi) = q, \quad \lambda = \lambda_0 + \frac{\theta}{n}$$

input	meaning	condition
`x`	easting	
`y`	northing	
`lat1`	first standard parallel (decimal degrees)	
`lat2`	second standard parallel (decimal degrees)	
`lat0`	latitude of origin (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`ellipsoid`	reference ellipsoid	
`FE`	false easting (default 0)	
`FN`	false northing (default 0)	

output	meaning
`lat`	latitude (degrees)
`lon`	longitude (degrees)

- **Method:** rho and theta from the offsets, q = (C - (rho n/a)^2)/n, latitude from q by Newton iteration; $\lambda = lambda0 + \theta/n$
- **Assumes:** parallels within 89 degrees of the equator and not symmetric about it; the point inside the projected area
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Albers inverse: a projected point located on its graticule: the input point x = -338796.0 m, y = 1952260.0 m placed on the graticule that the package's forward projection traces with the same parameters (110 points, parallels every 5°, meridians every 5°). Marked: the returned latitude 40.5185° and longitude -100.034°, which lie where the point falls among the lines.

```
geospatial_analytics_ai999x.solve('geo_albers_equal_area_inverse', x=-338796.0, y=1952260.0,
lat1=29.5, lat2=45.5, lat0=23, lon0=-96, ellipsoid='GRS80')
# -> lat = 40.5185, lon = -100.034   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_albers_equal_area_inverse', x=300000.0, y=-2700000.0,
lat1=-18, lat2=-36, lat0=0, lon0=132, ellipsoid='GRS80')
# -> lat = -24.9456, lon = 135.005   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Albers equal-area inverse: x = -338796.0 m, y = 1952260.0 m, lat1 = 29.5, lat2 = 45.5, lat0 = 23, lon0 = -96, ellipsoid = GRS80	SOLVED	lat = 40.5185, lon = -100.034

question	outcome	result
Albers equal-area inverse: x = 300000 m, y = -2700000 m, lat1 = -18, lat2 = -36, lat0 = 0, lon0 = 132, ellipsoid = GRS80	SOLVED	lat = -24.9456, lon = 135.005
Albers equal-area inverse: y = 1952260.0 m, lat1 = 29.5, lat2 = 45.5, lat0 = 23, lon0 = -96, ellipsoid = GRS80	NEEDS_INPUT	geo_albers_equal_area_inverse needs x (easting). Example: solve('geo_albers_equal_area_inverse', x='-338796 m', y='1.952
Albers equal-area inverse: x = -338796.0 m, y = 1952260.0 m, lat1 = 29.5, lat2 = 45.5, lat0 = 23, lon0 = -96, ellipsoid = GRS80, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Polar stereographic projection (standard parallel): coordinates and scale factor — `geo_polar_stereographic_forward`

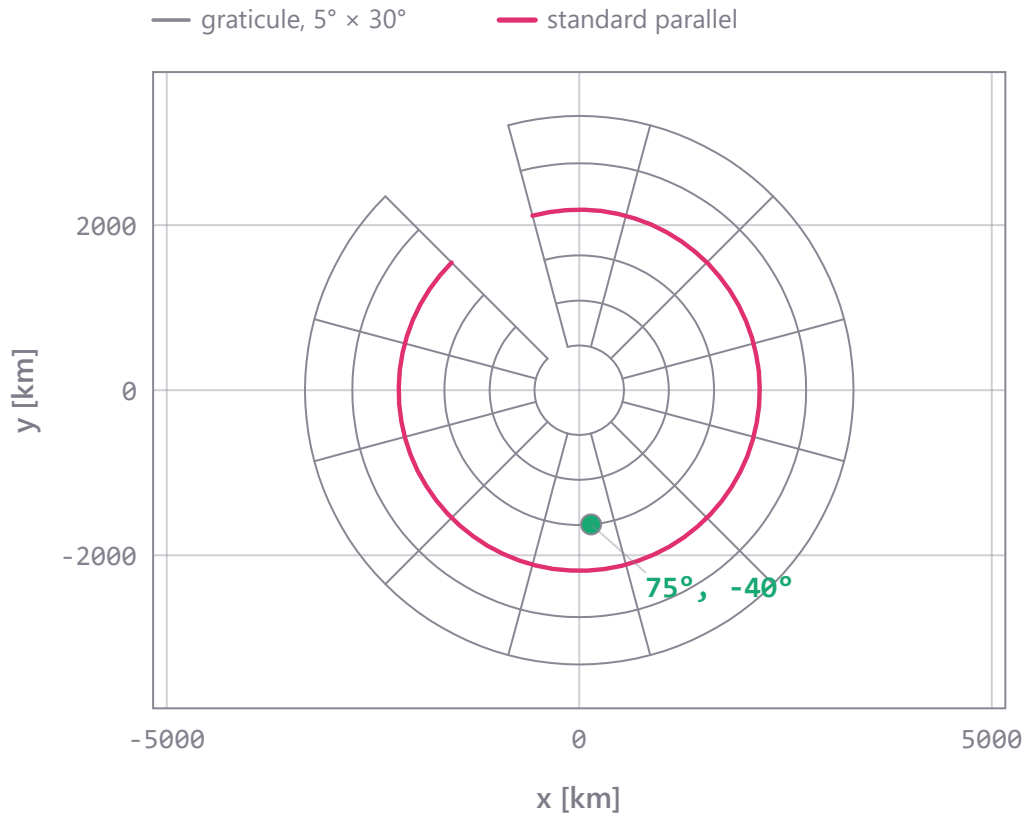
Easting and northing of a point in the ellipsoidal polar stereographic projection defined by a latitude of true scale (EPSG variant B, as in EPSG:3413 and EPSG:3031), with the point scale factor.

$$\rho = a\, m_c \frac{t}{t_c}, \quad t = e^{-\psi}, \quad x = FE + \rho \sin \Delta\lambda,\ y = FN \mp \rho \cos \Delta\lambda, \quad k = \frac{\rho}{am}$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`pole`	pole at the projection centre	
`lat_ts`	latitude of true scale (decimal degrees)	
`lon0`	longitude of the downward (north pole) or upward (south pole) y axis (decimal degrees)	
`ellipsoid`	reference ellipsoid	
`FE`	false easting (default 0)	
`FN`	false northing (default 0)	

output	meaning
`x`	easting
`y`	northing
`k`	point scale factor

- **Method:** $\rho = a$ m_c t/t_c with $t = e^{-\psi}$; $x = FE + \rho$ sin(dlambda), $y = FN$ -/+ rho cos(dlambda); k = rho/(a m)
- **Assumes:** the point in the pole's hemisphere (not at the pole); 30 <= |lat_ts| <= 89.99 in the same hemisphere
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



North polar stereographic graticule: the graticule (parallels every 5°, meridians every 30°) traced through the package's forward projection with pole = north, lat_ts = 70, lon0 = -45, ellipsoid = WGS84 (835 projected points). Marked: the worked example, latitude 75°, longitude -40° → x = 142402 m, y = -1.62766e+06 m. Scale factor k = 0.986664.

```
geospatial_analytics_ai999x.solve('geo_polar_stereographic_forward', lat=75.0, lon=-40.0,
pole='north', lat_ts=70, lon0=-45, ellipsoid='WGS84')
# -> x = 142402 m, y = -1.62766e+06 m, k = 0.986664   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_polar_stereographic_forward', lat=-77.85, lon=166.67,
pole='south', lat_ts=-71, lon0=0, ellipsoid='WGS84')
# -> x = 305467 m, y = -1.2892e+06 m, k = 0.983786   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Polar stereographic forward: lat = 75, lon = -40, pole = north, lat_ts = 70, lon0 = -45, ellipsoid = WGS84	SOLVED	x = 142402 m, y = -1.62766e+06 m, k = 0.986664
Polar stereographic forward: lat = -77.85, lon = 166.67, pole = south, lat_ts = -71, lon0 = 0, ellipsoid = WGS84	SOLVED	x = 305467 m, y = -1.2892e+06 m, k = 0.983786
Polar stereographic forward: lon = -40, pole = north, lat_ts = 70, lon0 = -45, ellipsoid = WGS84	NEEDS_INPUT	geo_polar_stereographic_forward needs lat (latitude (decimal degrees)). Example: solve('geo_polar_stereographic_forward'
Polar stereographic forward: lat = 75, lon = -40, pole = north, lat_ts = 70, lon0 = -45, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Lambert azimuthal equal-area projection (ellipsoidal) —

`geo_lambert_azimuthal_equal_area_forward`

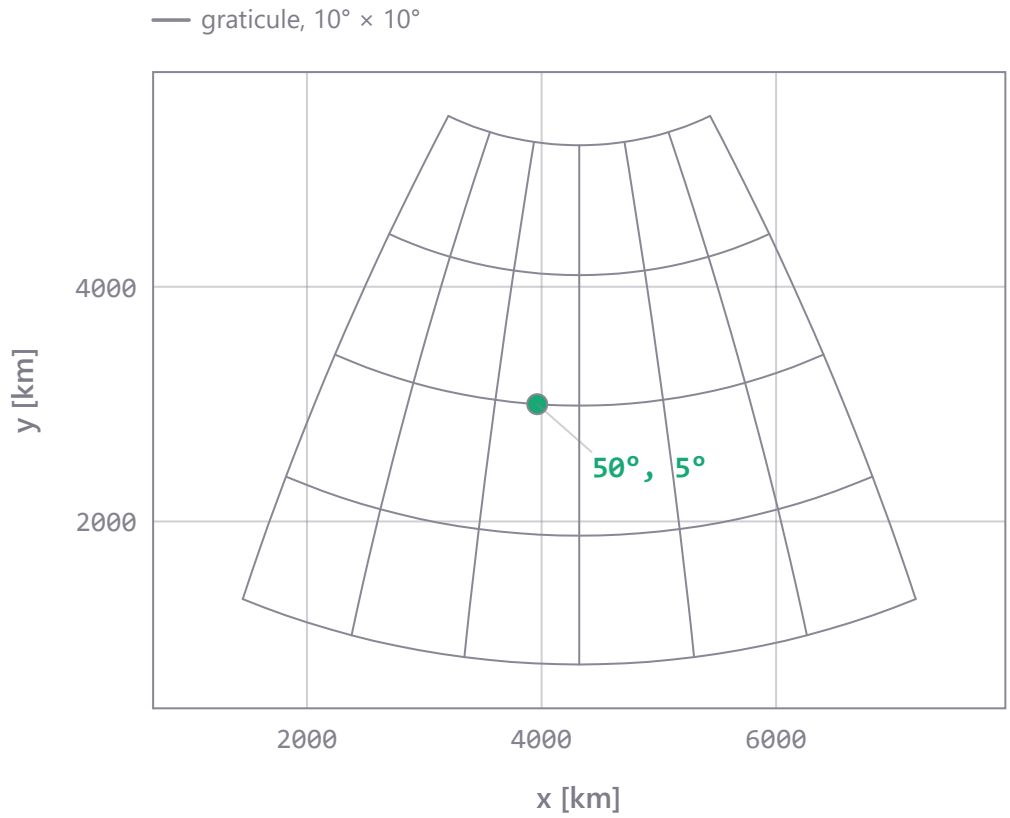
Easting and northing of a point in the ellipsoidal oblique Lambert azimuthal equal-area projection about a centre (as in ETRS89-LAEA, EPSG:3035).

$$B = R_q\sqrt{\frac{2}{1+\sin\beta_1\sin\beta+\cos\beta_1\cos\beta\cos\Delta\lambda}},\ x = BD\cos\beta\sin\Delta\lambda,\ y = \frac{B}{D}(\cos\beta_1\sin\beta-\sin\beta_1\cos$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lat0`	latitude of the centre (decimal degrees)	
`lon0`	longitude of the centre (decimal degrees)	
`ellipsoid`	reference ellipsoid	
`FE`	false easting (default 0)	
`FN`	false northing (default 0)	

output	meaning
`x`	easting
`y`	northing

- **Method:** authalic latitudes beta = asin(q/q_p) on the authalic sphere R_q, then the spherical azimuthal equal-area formulas with Snyder's D factor
- **Assumes:** centre within 89 degrees of the equator; point within 120 degrees of the centre
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lambert azimuthal equal-area over Europe: the graticule (parallels every 10°, meridians every 10°) traced through the package's forward projection with lat0 = 52, lon0 = 10, ellipsoid = GRS80, FE = 4321000.0, FN = 3210000.0 (302 projected points). Marked: the worked example, latitude 50°, longitude 5° → x = 3.9628e+06 m, y = 2.99972e+06 m.

```
geospatial_analytics_ai999x.solve('geo_lambert_azimuthal_equal_area_forward', lat=50.0, lon=5.0,
lat0=52, lon0=10, ellipsoid='GRS80', FE=4321000.0, FN=3210000.0)
# -> x = 3.9628e+06 m, y = 2.99972e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_lambert_azimuthal_equal_area_forward', lat=40.0, lon=-101.0,
lat0=45, lon0=-100, ellipsoid='WGS84')
# -> x = -85464.3 m, y = -554752 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lambert azimuthal equal-area forward: lat = 50, lon = 5, lat0 = 52, lon0 = 10, ellipsoid = GRS80, FE = 4321000 m, FN = 3210000 m	SOLVED	x = 3.9628e+06 m, y = 2.99972e+06 m
Lambert azimuthal equal-area forward: lat = 40, lon = -101, lat0 = 45, lon0 = -100, ellipsoid = WGS84	SOLVED	x = -85464.3 m, y = -554752 m
Lambert azimuthal equal-area forward: lon = 5, lat0 = 52, lon0 = 10, ellipsoid = GRS80, FE = 4321000 m, FN = 3210000 m	NEEDS_INPUT	geo_lambert_azimuthal_equal_area_forward needs lat (latitude (decimal degrees)). Example: solve('geo_lambert_azimuthal_e
Lambert azimuthal equal-area forward: lat = 50, lon = 5, lat0 = 52, lon0 = 10, ellipsoid = GRS80,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

question	outcome	result
FE = 4321000 m, FN = 3210000 m, zz = 1		

Azimuthal equidistant projection (ellipsoidal, geodesic) — `geo_azimuthal_equidistant_forward`

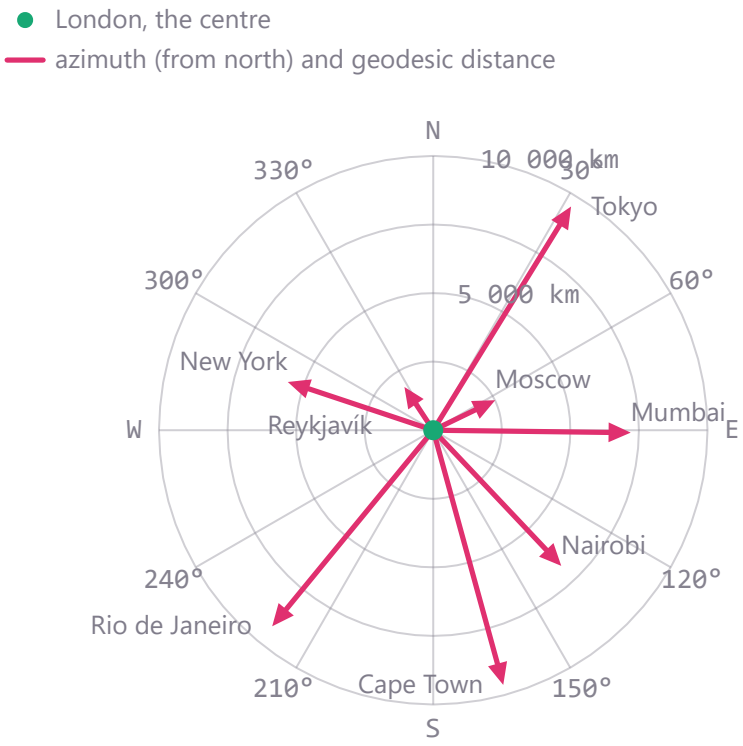
Easting and northing of a point in the ellipsoidal azimuthal equidistant projection about a centre: the geodesic distance from the centre plotted at the geodesic azimuth (used for range rings and airline maps).

$$x = FE + s_{12}\sin\alpha_1, \quad y = FN + s_{12}\cos\alpha_1$$

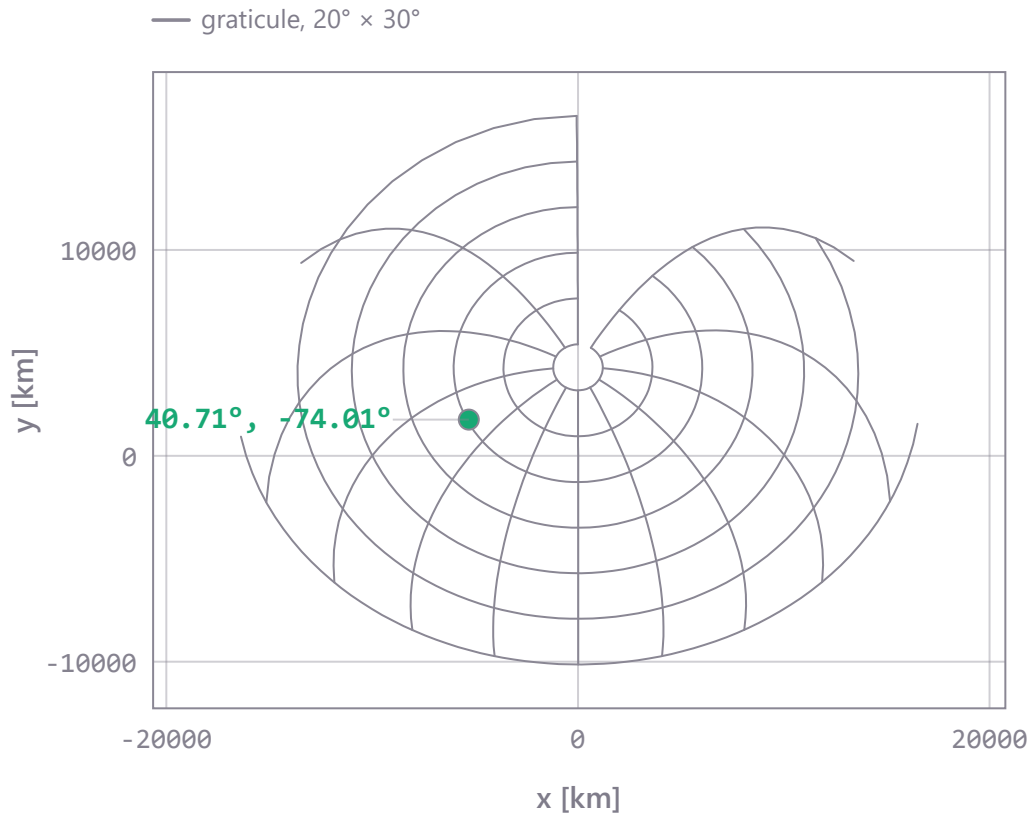
input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lat0`	latitude of the centre (decimal degrees)	
`lon0`	longitude of the centre (decimal degrees)	
`ellipsoid`	reference ellipsoid	
`FE`	false easting (default 0)	
`FN`	false northing (default 0)	

output	meaning
`x`	easting
`y`	northing
`distance`	geodesic distance from the centre
`azimuth`	geodesic azimuth from the centre (degrees)

- **Method:** inverse geodesic problem from the centre (exact integrals), then $x = s$ sin(alpha), $y = s$ cos(alpha)
- **Assumes:** point distinct from the centre and at most 150 degrees from it; centre within 89 degrees of the equator
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Eight cities seen from London as the package computes them on the WGS84 ellipsoid — each arrow points along the initial azimuth, clockwise from north, with length the geodesic distance (an azimuthal equidistant view): New York 5585.81 km at 288.373°; Tokyo 9583.99 km at 31.6564°; Cape Town 9635.07 km at 164.613°; Rio de Janeiro 9251.66 km at 219.388°; Moscow 2509.19 km at 64.2336°; Mumbai 7199.57 km at 90.6929°; Reykjavík 1895.1 km at 326.304°; Nairobi 6805.98 km at 136.642°.



Azimuthal equidistant map centred on London: the graticule (parallels every 20°, meridians every 30°) traced through the package's forward projection with lat0 = 51.5, lon0 = -0.13, ellipsoid = WGS84 (928 projected points). Marked: the worked example, latitude 40.71°, longitude -74.01° → x = -5.30106e+06 m, y = 1.76067e+06 m. Returned geodesic distance 5585.81 km at azimuth 288.373°.

```
geospatial_analytics_ai999x.solve('geo_azimuthal_equidistant_forward', lat=40.71, lon=-74.01,
lat0=51.5, lon0=-0.13, ellipsoid='WGS84')
# -> x = -5.30106e+06 m, y = 1.76067e+06 m, distance = 5.58581e+06 m, azimuth = 288.373   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_azimuthal_equidistant_forward', lat=-33.87, lon=151.21,
lat0=-37.81, lon0=144.96, ellipsoid='GRS80')
# -> x = 578343 m, y = 418382 m, distance = 713810 m, azimuth = 54.1174   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Azimuthal equidistant forward: lat = 40.71, lon = -74.01, lat0 = 51.5, lon0 = -0.13, ellipsoid = WGS84	SOLVED	x = -5.30106e+06 m, y = 1.76067e+06 m, distance = 5.58581e+06 m, azimuth = 288.373
Azimuthal equidistant forward: lat = -33.87, lon = 151.21, lat0 = -37.81, lon0 = 144.96, ellipsoid = GRS80	SOLVED	x = 578343 m, y = 418382 m, distance = 713810 m, azimuth = 54.1174
Azimuthal equidistant forward: lon = -74.01, lat0 = 51.5, lon0 = -0.13, ellipsoid = WGS84	NEEDS_INPUT	geo_azimuthal_equidistant_forward needs lat (latitude (decimal degrees)). Example: solve('geo_azimuthal_equidistant_forw

question	outcome	result
Azimuthal equidistant forward: lat = 40.71, lon = -74.01, lat0 = 51.5, lon0 = -0.13, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Sinusoidal equal-area projection (ellipsoidal) — `geo_sinusoidal_forward`

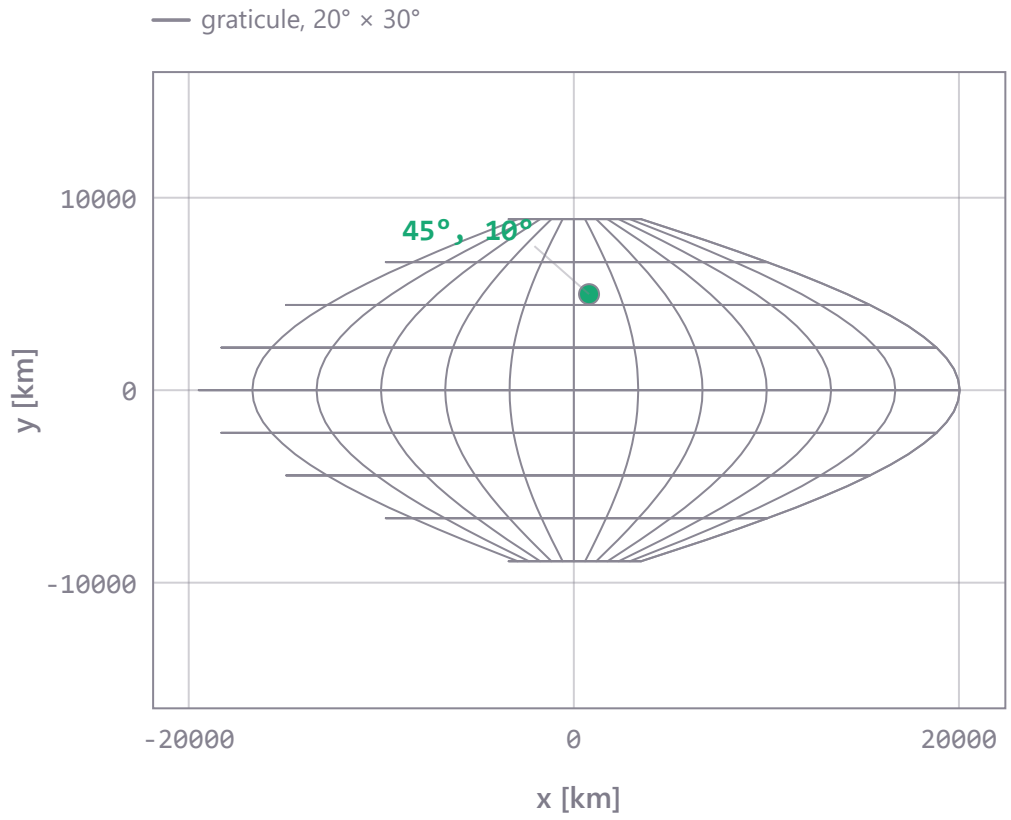
x and y of a point in the ellipsoidal sinusoidal (Sanson-Flamsteed) equal-area projection, as used for MODIS land grids.

$$x = \frac{a\cos\varphi}{\sqrt{1 - e^2\sin^2\varphi}}(\lambda - \lambda_0), \quad y = m(\varphi)$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`ellipsoid`	reference ellipsoid	

output	meaning
`x`	easting
`y`	northing

- **Method:** $x = N$ cos(phi) $(\lambda - lambda0)$, y = meridian distance m(phi) (Helmert's series)
- **Assumes:** |latitude| <= 89.9
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Sinusoidal world graticule: the graticule (parallels every 20°, meridians every 30°) traced through the package's forward projection with lon0 = 0, ellipsoid = WGS84 (1086 projected points). Marked: the worked example, latitude 45°, longitude 10° → x = 788468 m, y = 4.98494e+06 m.

```
geospatial_analytics_ai999x.solve('geo_sinusoidal_forward', lat=45.0, lon=10.0, lon0=0,
ellipsoid='WGS84')
# -> x = 788468 m, y = 4.98494e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_sinusoidal_forward', lat=-12.5, lon=-60.0, lon0=-50,
ellipsoid='GRS80')
# -> x = -1.08698e+06 m, y = -1.3824e+06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sinusoidal forward: lat = 45, lon = 10, lon0 = 0, ellipsoid = WGS84	SOLVED	x = 788468 m, y = 4.98494e+06 m
Sinusoidal forward: lat = -12.5, lon = -60, lon0 = -50, ellipsoid = GRS80	SOLVED	x = -1.08698e+06 m, y = -1.3824e+06 m
Sinusoidal forward: lon = 10, lon0 = 0, ellipsoid = WGS84	NEEDS_INPUT	geo_sinusoidal_forward needs lat (latitude (decimal degrees)). Example: solve('geo_sinusoidal_forward', lat=45, lon=10,
Sinusoidal forward: lat = 45, lon = 10, lon0 = 0, ellipsoid = WGS84, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Mollweide equal-area projection (sphere) — `geo_mollweide_forward`

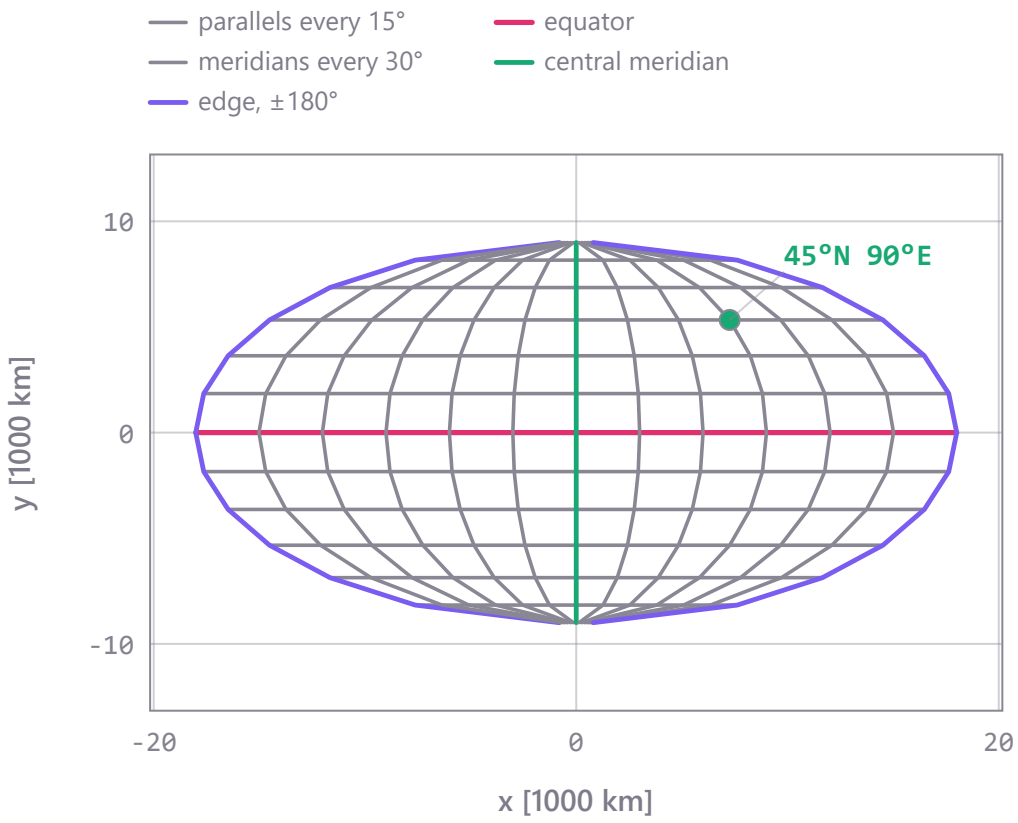
x and y of a point in the Mollweide (homalographic) equal-area projection of a sphere of radius R.

$$2\theta + \sin 2\theta = \pi \sin\varphi, \quad x = \frac{2\sqrt{2}}{\pi} R\,\Delta\lambda \cos\theta, \quad y = \sqrt{2} R \sin\theta$$

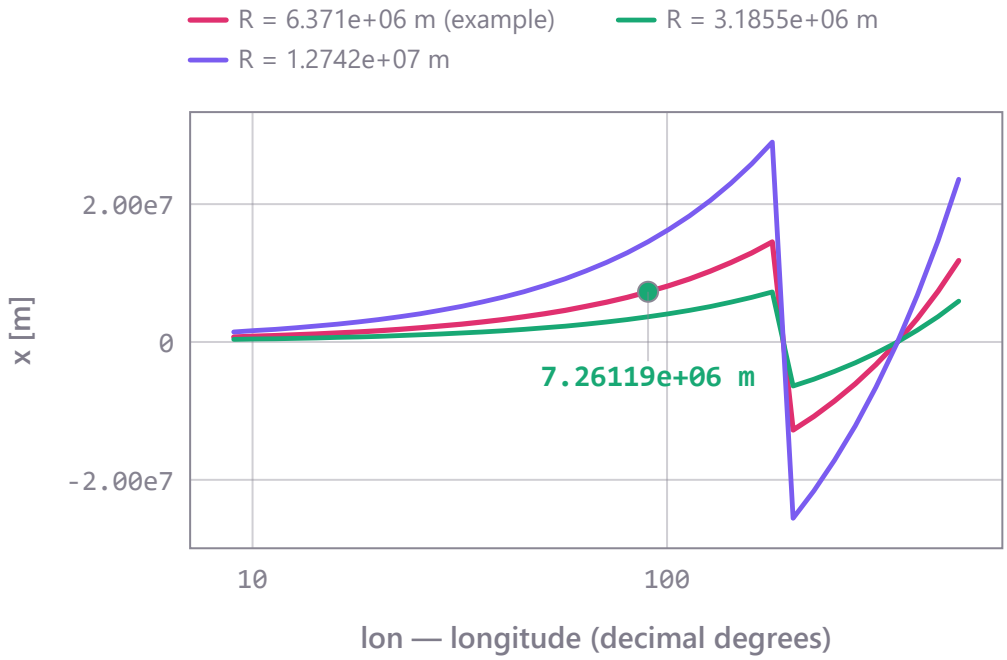
input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`R`	radius of the sphere	

output	meaning
`x`	x
`y`	y

- **Method:** auxiliary angle theta from 2 theta + sin 2 $\theta = pi$ sin(phi) by Newton's method; x = (2 sqrt2/pi) R dlambda cos(theta), $y = sqrt2$ R sin(theta)
- **Assumes:** sphere of radius $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The world graticule in the Mollweide equal-area projection (sphere of radius 6371 km): every parallel and meridian is a chain of points projected by the package (576 points); marked: latitude 45°, longitude 90° → x = 7261.19 km, y = 5334.27 km.



x (x) against lon (longitude (decimal degrees)) from 9 to 900 on a logarithmic axis. One curve for each value of R (radius of the sphere): 6.371e+06 m, 3.1855e+06 m, 1.2742e+07 m. Other inputs as in the worked example: lat = 45.0, lon0 = 0. 36 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, lon = 90, x = 7.26119e+06 m.

```
geospatial_analytics_ai999x.solve('geo_mollweide_forward', lat=45.0, lon=90.0, lon0=0, R=6371000.0)
# -> x = 7.26119e+06 m, y = 5.33427e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_mollweide_forward', lat=-30.0, lon=-60.0, lon0=10, R=1.0)
# -> x = -1.0062 m, y = -0.571304 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mollweide forward: lat = 45, lon = 90, lon0 = 0, R = 6371000 m	SOLVED	x = 7.26119e+06 m, y = 5.33427e+06 m
Mollweide forward: lat = -30, lon = -60, lon0 = 10, R = 1 m	SOLVED	x = -1.0062 m, y = -0.571304 m
Mollweide forward: lon = 90, lon0 = 0, R = 6371000 m	NEEDS_INPUT	geo_mollweide_forward needs lat (latitude (decimal degrees)). Example: solve('geo_mollweide_forward', lat=45, lon=90, lo
Mollweide forward: lat = 45, lon = 90, lon0 = 0, R = 6371000 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Orthographic projection (sphere) — `geo_orthographic_forward`

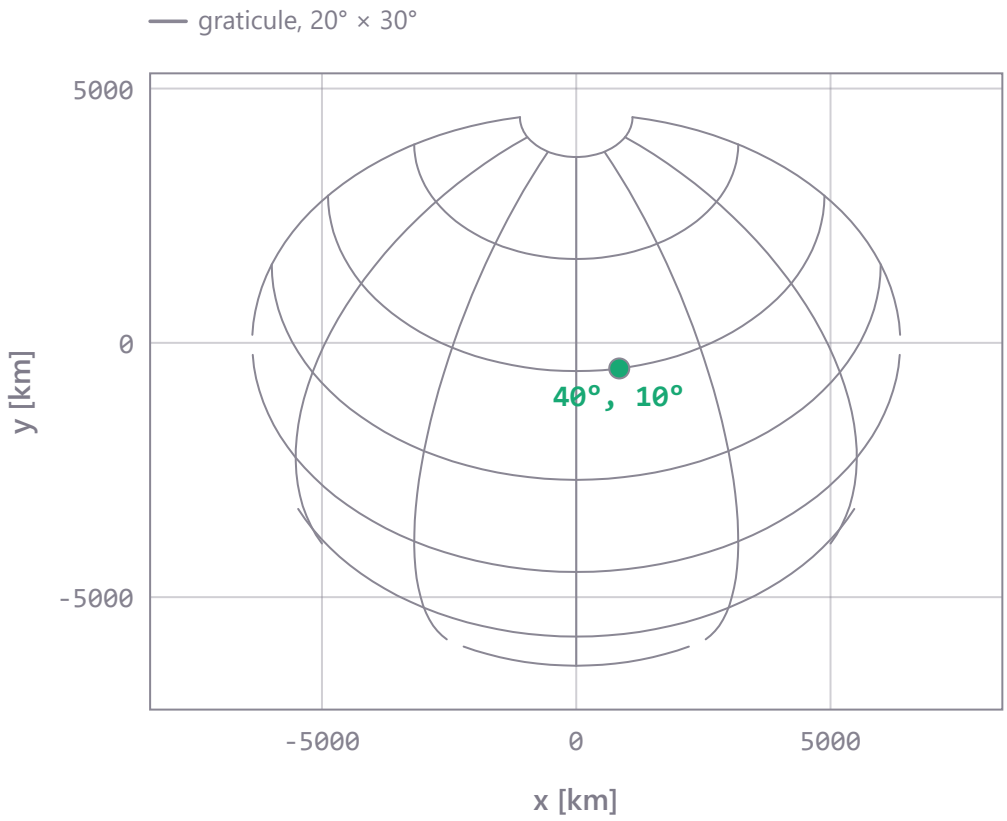
x and y of a point in the orthographic projection (the view of a globe from infinitely far away) of a sphere of radius R centred on a point.

$$x = R\cos\varphi\sin\Delta\lambda, \quad y = R\left(\cos\varphi_0\sin\varphi - \sin\varphi_0\cos\varphi\cos\Delta\lambda\right)$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lat0`	latitude of the centre (decimal degrees)	
`lon0`	longitude of the centre (decimal degrees)	
`R`	radius of the sphere	

output	meaning
`x`	x
`y`	y

- **Method:** $x = R$ cos(phi) sin(dlambda), $y = R$ (cos(phi0) sin(phi) - sin(phi0) cos(phi) cos(dlambda))
- **Assumes:** the point on the visible hemisphere (cos c >= 0)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Orthographic view of the hemisphere centred at 45° N, 0°: the graticule (parallels every 20°, meridians every 30°) traced through the package's forward projection with lat0 = 45, lon0 = 0, R = 6371000.0 (614 projected points) (the visible hemisphere only). Marked: the worked example, latitude 40°, longitude 10° → x = 847485 m, y = -502841 m.

```
geospatial_analytics_ai999x.solve('geo_orthographic_forward', lat=40.0, lon=10.0, lat0=45, lon0=0,
R=6371000.0)
# -> x = 847485 m, y = -502841 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_orthographic_forward', lat=-10.0, lon=100.0, lat0=0,
lon0=80, R=1.0)
# -> x = 0.336824 m, y = -0.173648 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Orthographic forward: lat = 40, lon = 10, lat0 = 45, lon0 = 0, R = 6371000 m	SOLVED	x = 847485 m, y = -502841 m
Orthographic forward: lat = -10, lon = 100, lat0 = 0, lon0 = 80, R = 1 m	SOLVED	x = 0.336824 m, y = -0.173648 m
Orthographic forward: lon = 10, lat0 = 45, lon0 = 0, R = 6371000 m	NEEDS_INPUT	geo_orthographic_forward needs lat (latitude (decimal degrees)). Example: solve('geo_orthographic_forward', lat=40, lon=
Orthographic forward: lat = 40, lon = 10, lat0 = 45, lon0 = 0, R = 6371000 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ
Orthographic forward: lat = -45, lon = 180, lat0 = 45, lon0 = 0, R = 6371000 m	INVALID_INPUT	Orthographic projection (sphere): the point lies on the far side of the globe (not visible in the orthographic view).

Gnomonic projection (sphere) — `geo_gnomonic_forward`

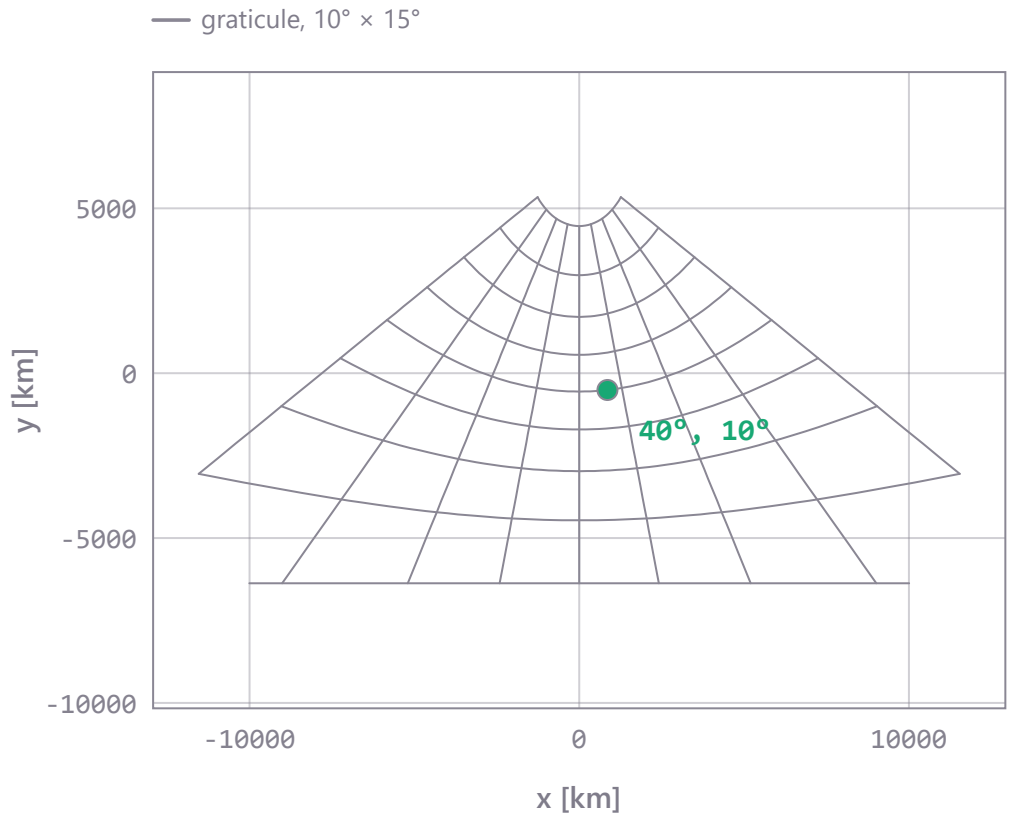
x and y of a point in the gnomonic projection (great circles become straight lines) of a sphere of radius R centred on a point.

$$x = \frac{R\cos\varphi\sin\Delta\lambda}{\cos c}, \quad y = \frac{R\left(\cos\varphi_0\sin\varphi - \sin\varphi_0\cos\varphi\cos\Delta\lambda\right)}{\cos c}$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lat0`	latitude of the centre (decimal degrees)	
`lon0`	longitude of the centre (decimal degrees)	
`R`	radius of the sphere	

output	meaning
`x`	x
`y`	y

- **Method:** the orthographic numerators divided by cos c, the cosine of the angular distance from the centre
- **Assumes:** the point within about 84 degrees of the centre (cos c >= 0.1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Gnomonic projection centred at 45° N, 0°: the graticule (parallels every 10°, meridians every 15°) traced through the package's forward projection with lat0 = 45, lon0 = 0, R = 6371000.0 (896 projected points) (points more than 62° from the centre left out). Marked: the worked example, latitude 40°, longitude 10° → x = 857808 m, y = -508966 m.

```
geospatial_analytics_ai999x.solve('geo_gnomonic_forward', lat=40.0, lon=10.0, lat0=45, lon0=0,
R=6371000.0)
# -> x = 857808 m, y = -508966 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_gnomonic_forward', lat=60.0, lon=-30.0, lat0=90, lon0=0,
R=1.0)
# -> x = -0.288675 m, y = -0.5 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gnomonic forward: lat = 40, lon = 10, lat0 = 45, lon0 = 0, R = 6371000 m	SOLVED	x = 857808 m, y = -508966 m
Gnomonic forward: lat = 60, lon = -30, lat0 = 90, lon0 = 0, R = 1 m	SOLVED	x = -0.288675 m, y = -0.5 m
Gnomonic forward: lon = 10, lat0 = 45, lon0 = 0, R = 6371000 m	NEEDS_INPUT	geo_gnomonic_forward needs lat (latitude (decimal degrees)). Example: solve('geo_gnomonic_forward', lat=40, lon=10, lat0
Gnomonic forward: lat = 40, lon = 10, lat0 = 45, lon0 = 0, R = 6371000 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Equirectangular (equidistant cylindrical / plate carree) projection — `geo_equirectangular_forward`

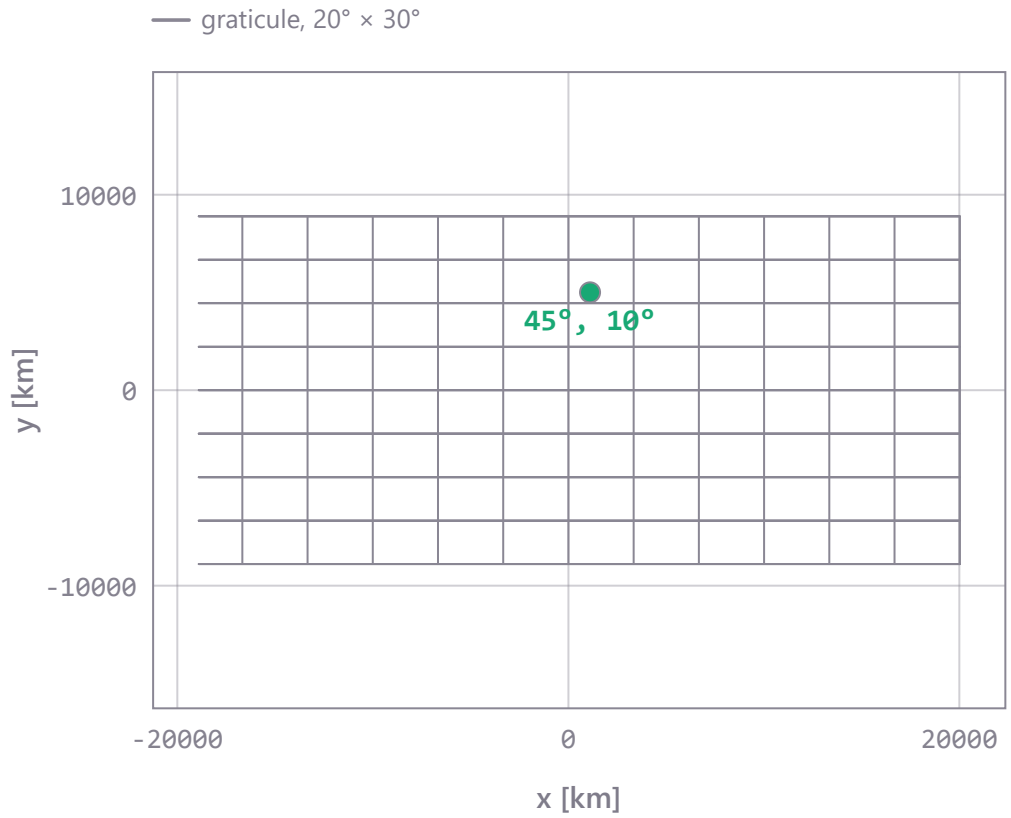
x and y of a point in the equirectangular (equidistant cylindrical) projection of a sphere of radius R with a standard parallel (lat_ts = 0 is the plate carree).

$$x = R(\lambda - \lambda_0)\cos\varphi_{ts}, \quad y = R(\varphi - \varphi_0)$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lat_ts`	standard parallel (true scale) (decimal degrees)	
`lat0`	latitude of origin (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`R`	radius of the sphere	

output	meaning
`x`	x
`y`	y

- **Method:** $x = R\,(\lambda - lambda0)$ cos(lat_ts), $y = R\,(\phi - phi0)$
- **Assumes:** sphere of radius $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Equirectangular world graticule: the graticule (parallels every 20°, meridians every 30°) traced through the package's forward projection with lat_ts = 0, lat0 = 0, lon0 = 0, R = 6371000.0 (554 projected points). Marked: the worked example, latitude 45°, longitude 10° → x = 1.11195e+06 m, y = 5.00377e+06 m.

```
geospatial_analytics_ai999x.solve('geo_equirectangular_forward', lat=45.0, lon=10.0, lat_ts=0,
lat0=0, lon0=0, R=6371000.0)
# -> x = 1.11195e+06 m, y = 5.00377e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_equirectangular_forward', lat=-20.0, lon=30.0, lat_ts=30,
lat0=0, lon0=20, R=6371007.0)
# -> x = 962977 m, y = -2.2239e+06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equirectangular forward: lat = 45, lon = 10, lat_ts = 0, lat0 = 0, lon0 = 0, R = 6371000 m	SOLVED	x = 1.11195e+06 m, y = 5.00377e+06 m
Equirectangular forward: lat = -20, lon = 30, lat_ts = 30, lat0 = 0, lon0 = 20, R = 6371007 m	SOLVED	x = 962977 m, y = -2.2239e+06 m
Equirectangular forward: lon = 10, lat_ts = 0, lat0 = 0, lon0 = 0, R = 6371000 m	NEEDS_INPUT	geo_equirectangular_forward needs lat (latitude (decimal degrees)). Example: solve('geo_equirectangular_forward', lat=45
Equirectangular forward: lat = 45, lon = 10, lat_ts = 0, lat0 = 0, lon0 = 0, R = 6371000 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Lambert cylindrical equal-area projection (sphere) — `geo_cylindrical_equal_area_forward`

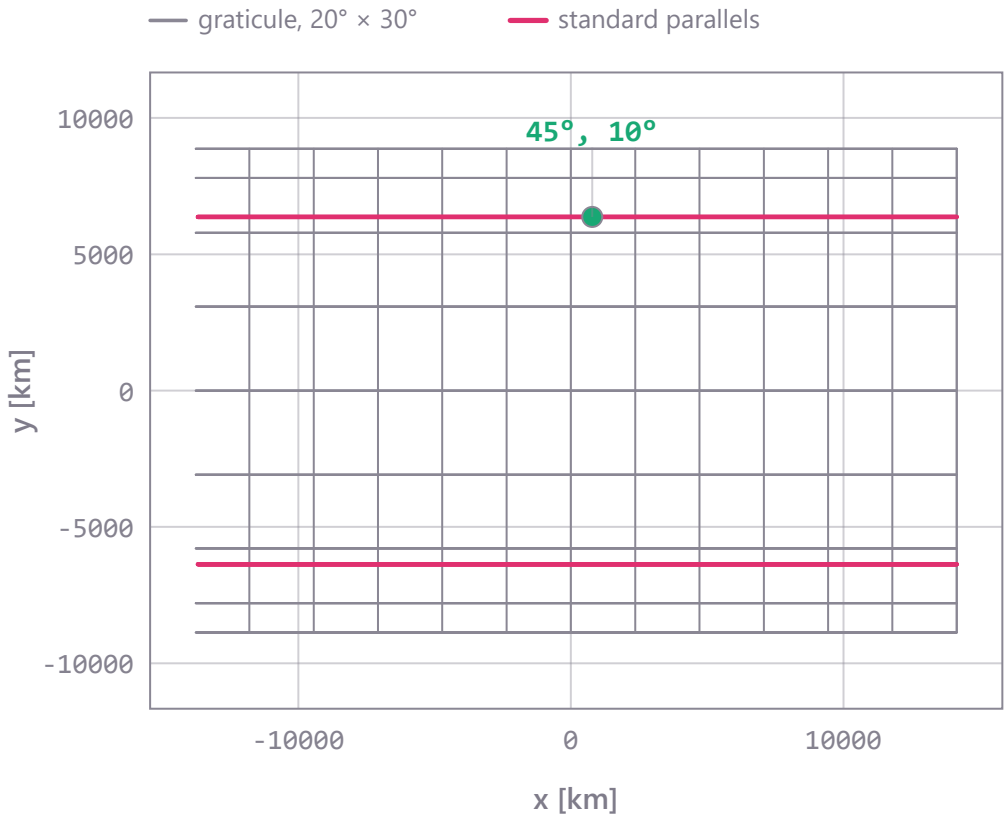
x and y of a point in the cylindrical equal-area projection of a sphere of radius R with a standard parallel (0 Lambert, 30 Behrmann, 45 Gall-Peters).

$$x = R(\lambda - \lambda_0)\cos\varphi_s, \quad y = \frac{R\sin\varphi}{\cos\varphi_s}$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`lat_ts`	standard parallel (decimal degrees)	
`lon0`	central meridian (decimal degrees)	
`R`	radius of the sphere	

output	meaning
`x`	x
`y`	y

- **Method:** $x = R\,(\lambda - lambda0)$ cos(lat_ts), $y = R$ sin(phi)/cos(lat_ts)
- **Assumes:** standard parallel strictly between -90 and 90
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cylindrical equal-area world graticule: the graticule (parallels every 20°, meridians every 30°) traced through the package's forward projection with lat_ts = 45, lon0 = 0, R = 6371000.0 (1208 projected points). Marked: the worked example, latitude 45°, longitude 10° → x = 786267 m, y = 6.371e+06 m.

```
geospatial_analytics_ai999x.solve('geo_cylindrical_equal_area_forward', lat=45.0, lon=10.0,
lat_ts=45, lon0=0, R=6371000.0)
# -> x = 786267 m, y = 6.371e+06 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_cylindrical_equal_area_forward', lat=-30.0, lon=120.0,
lat_ts=30, lon0=0, R=1.0)
# -> x = 1.8138 m, y = -0.57735 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cylindrical equal-area forward: lat = 45, lon = 10, lat_ts = 45, lon0 = 0, R = 6371000 m	SOLVED	x = 786267 m, y = 6.371e+06 m
Cylindrical equal-area forward: lat = -30, lon = 120, lat_ts = 30, lon0 = 0, R = 1 m	SOLVED	x = 1.8138 m, y = -0.57735 m
Cylindrical equal-area forward: lon = 10, lat_ts = 45, lon0 = 0, R = 6371000 m	NEEDS_INPUT	geo_cylindrical_equal_area_forward needs lat (latitude (decimal degrees)). Example: solve('geo_cylindrical_equal_area_fo
Cylindrical equal-area forward: lat = 45, lon = 10, lat_ts = 45, lon0 = 0, R = 6371000 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Tissot's indicatrix: scale factors and distortion of a projection at a point —

`geo_tissot_indicatrix`

The meridian and parallel scale factors h and k, the angle between meridian and parallel, the semi-axes a and b of Tissot's indicatrix, the maximum angular distortion and the areal scale of a world projection of the sphere at a point.

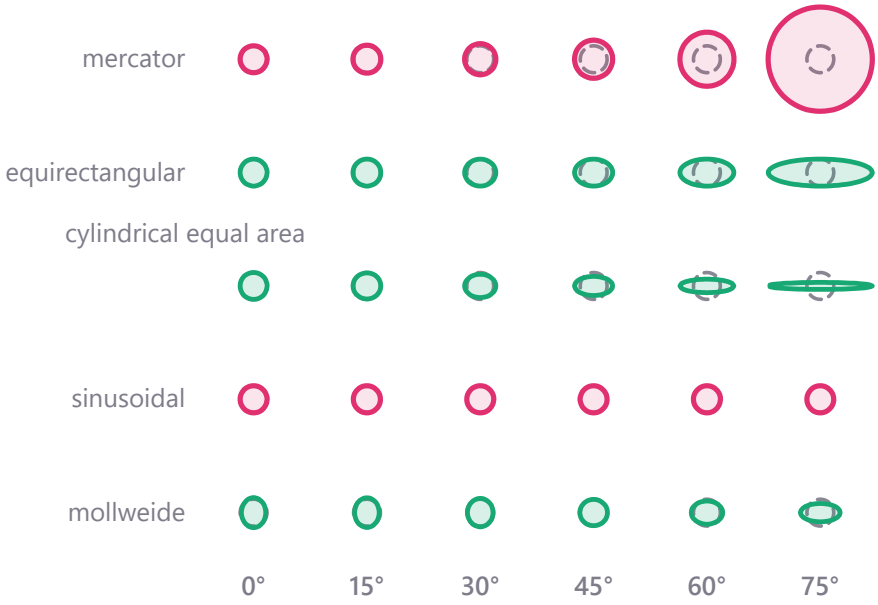
$$a' = \sqrt{h^2 + k^2 + 2hk\sin\theta'},\ b' = \sqrt{h^2 + k^2 - 2hk\sin\theta'},\ a = \tfrac{a'+b'}{2},\ b = \tfrac{a'-b'}{2},\ \omega = 2\arcsin\frac{a-b}{a+b},\ s =$$

input	meaning	condition
`projection`	projection (spherical form, central meridian 0)	
`lat`	latitude (decimal degrees)	
`lon`	longitude measured from the central meridian (decimal degrees)	

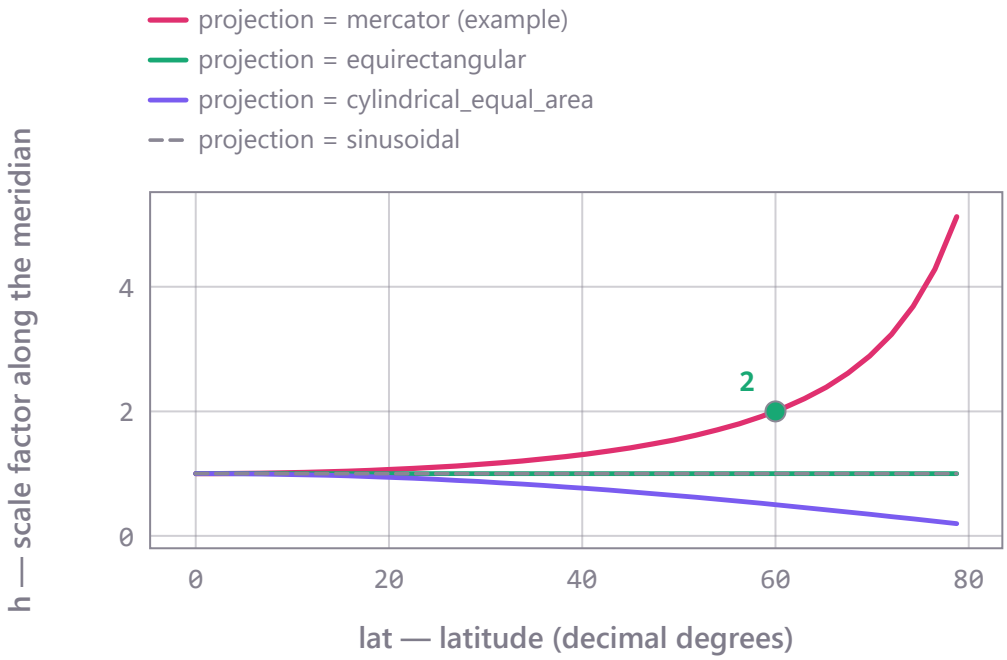
output	meaning
`h`	scale factor along the meridian
`k`	scale factor along the parallel
`theta_prime`	angle between the projected meridian and parallel (degrees, at most 90)
`a`	maximum scale factor (semi-major axis)
`b`	minimum scale factor (semi-minor axis)
`omega`	maximum angular distortion (degrees)

output	meaning
`s`	areal scale factor

- **Method:** partial derivatives of the projection: h = |d(x,y)/dphi|, k = |d(x,y)/dlambda|/cos(phi), sin(theta') from their cross product; a' = sqrt($h^2 + k^2 + 2hk$ sin theta'), b' = sqrt($h^2 + k^2 - 2hk$ sin theta'), a = (a'+b')/2, b = (a'-b')/2, $\omega = 2$ asin((a-b)/(a+b)), $s = hk$ sin theta'
- **Assumes:** |latitude| <= 80
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tissot's indicatrix along the central meridian (longitude 0) at latitudes 0° to 75° for five world projections: each ellipse has the returned parallel scale k as its horizontal semi-axis and the meridian scale h as its vertical one (on the central meridian the two directions are perpendicular); the dashed circle is scale 1, all drawn at one scale. Marked by size: Mercator at 60° is h = k = 2 (the worked example, areal scale 4); the equal-area projections keep h·k = 1 while their shape distorts.



h (scale factor along the meridian) against lat (latitude (decimal degrees)) from 0 to 90. One curve for each value of projection (projection (spherical form, central meridian 0)): mercator, equirectangular, cylindrical_equal_area, sinusoidal. Other inputs as in the worked example: lon = 0. 36 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, lat = 60, h = 2.

```
geospatial_analytics_ai999x.solve('geo_tissot_indicatrix', projection='mercator', lat=60, lon=0)
# -> h = 2, k = 2, theta_prime = 90, a = 2, b = 2, omega = 0, s = 4   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_tissot_indicatrix', projection='sinusoidal', lat=40, lon=30)
# -> h = 1.05512, k = 1, theta_prime = 71.3987, a = 1.18234, b = 0.845779, omega = 19.1046, s = 1
(SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_tissot_indicatrix', projection='mollweide', lat=50, lon=100)
# -> h = 1.2997, k = 1.06301, theta_prime = 46.3693, a = 1.55019, b = 0.645084, omega = 48.6984, s
= 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Tissot indicatrix: projection = mercator, lat = 60, lon = 0	SOLVED	h = 2, k = 2, theta_prime = 90, a = 2, b = 2, omega = 0, s = 4
Tissot indicatrix: projection = sinusoidal, lat = 40, lon = 30	SOLVED	h = 1.05512, k = 1, theta_prime = 71.3987, a = 1.18234, b = 0.845779, omega = 19.1046, s = 1
Tissot indicatrix: projection = mollweide, lat = 50, lon = 100	SOLVED	h = 1.2997, k = 1.06301, theta_prime = 46.3693, a = 1.55019, b = 0.645084, omega = 48.6984, s = 1
Tissot indicatrix: lat = 60, lon = 0	NEEDS_INPUT	geo_tissot_indicatrix needs projection (projection (spherical form, central meridian 0)). Example: solve('geo_tissot_ind
Tissot indicatrix: projection = mercator, lat = 60, lon = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Degrees, minutes and seconds to decimal degrees — `geo_dms_to_decimal`

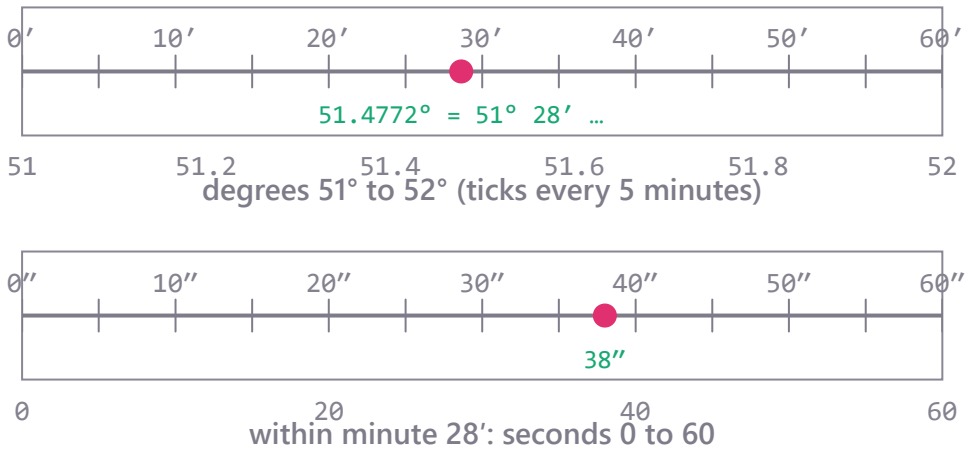
A latitude or longitude written in degrees, minutes and seconds with a hemisphere letter, converted to signed decimal degrees and radians.

$$\varphi = \pm\left(d + \frac{m}{60} + \frac{s}{3600}\right)$$

input	meaning	condition
`degrees`	whole degrees (>= 0)	
`minutes`	whole minutes $(0 - 59)$	
`seconds`	seconds $(0 \le s < 60)$	
`hemisphere`	hemisphere letter (S and W are negative)	

output	meaning
`decimal`	signed decimal degrees
`radians`	signed angle in radians

- **Method:** decimal = sign (degrees + minutes/60 + seconds/3600), exact rational arithmetic
- **Assumes:** 0 <= minutes <= 59; 0 <= seconds < 60; at most 90 (N/S) or 180 (E/W) degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The angle 51° 28′ 38.0″ N (inputs) on nested scales — seconds within the minute (bottom), the position within the degree (top) — and the returned decimal value 51.4772° (0.898447 rad).

```
geospatial_analytics_ai999x.solve('geo_dms_to_decimal', degrees=51, minutes=28, seconds=38.0,
hemisphere='N')
# -> decimal = 51.4772, radians = 0.898447   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_dms_to_decimal', degrees=0, minutes=7, seconds=39.0,
hemisphere='W')
# -> decimal = -0.1275, radians = -0.00222529   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
DMS to decimal degrees: degrees = 51, minutes = 28, seconds = 38, hemisphere = N	SOLVED	decimal = 51.4772, radians = 0.898447
DMS to decimal degrees: degrees = 0, minutes = 7, seconds = 39, hemisphere = W	SOLVED	decimal = -0.1275, radians = -0.00222529
DMS to decimal degrees: minutes = 28, seconds = 38, hemisphere = N	NEEDS_INPUT	geo_dms_to_decimal needs degrees (whole degrees (>= 0)). Example: solve('geo_dms_to_decimal', degrees=51, minutes=28, se
DMS to decimal degrees: degrees = 51, minutes = 28, seconds = 38, hemisphere = N, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ
DMS to decimal degrees: degrees = 51, minutes = 75, seconds = 38, hemisphere = N	INVALID_INPUT	minutes: must be at most 59, got 75.

Decimal degrees to degrees, minutes and seconds — `geo_decimal_to_dms`

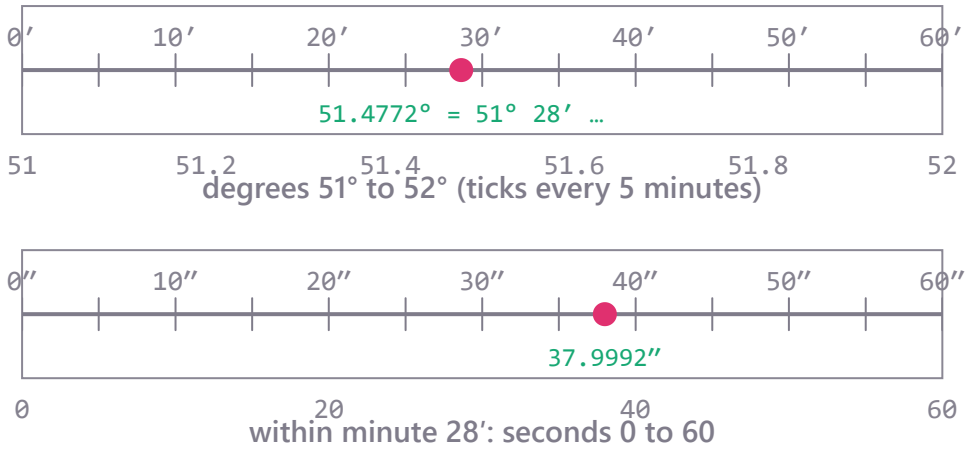
A signed latitude or longitude in decimal degrees written as whole degrees, whole minutes, seconds and a hemisphere letter.

$$d = \lfloor |x| \rfloor, \quad m = \lfloor 60(|x| - d) \rfloor, \quad s = 3600\left(|x| - d - \tfrac{m}{60}\right)$$

input	meaning	condition
`decimal`	signed angle (decimal degrees)	
`axis`	whether the angle is a latitude (N/S) or a longitude (E/W)	

output	meaning
`degrees`	whole degrees
`minutes`	whole minutes
`seconds`	seconds
`hemisphere`	N, S, E or W

- **Method:** exact decimal arithmetic: degrees = floor(|x|), minutes = floor(60 frac), seconds = the remainder times 3600
- **Assumes:** |latitude| <= 90, |longitude| <= 180
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The angle 51.477222° (latitude) on two nested scales: the whole degree 51° divided into minutes, then minute 28′ divided into seconds. Returned: 51° 28′ 37.9992″ N.

```
geospatial_analytics_ai999x.solve('geo_decimal_to_dms', decimal=51.477222, axis='latitude')
# -> degrees = 51, minutes = 28, seconds = 37.9992, hemisphere = N   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_decimal_to_dms', decimal=-0.1275, axis='longitude')
# -> degrees = 0, minutes = 7, seconds = 39, hemisphere = W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Decimal degrees to DMS: decimal = 51.477222, axis = latitude	SOLVED	degrees = 51, minutes = 28, seconds = 37.9992, hemisphere = N
Decimal degrees to DMS: decimal = -0.1275, axis = longitude	SOLVED	degrees = 0, minutes = 7, seconds = 39, hemisphere = W
Decimal degrees to DMS: axis = latitude	NEEDS_INPUT	geo_decimal_to_dms needs decimal (signed angle (decimal degrees)). Example: solve('geo_decimal_to_dms', decimal=51.4772,
Decimal degrees to DMS: decimal = 51.477222, axis = latitude, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Ground distance from a map distance and a representative fraction —

`geo_map_scale_ground_distance`

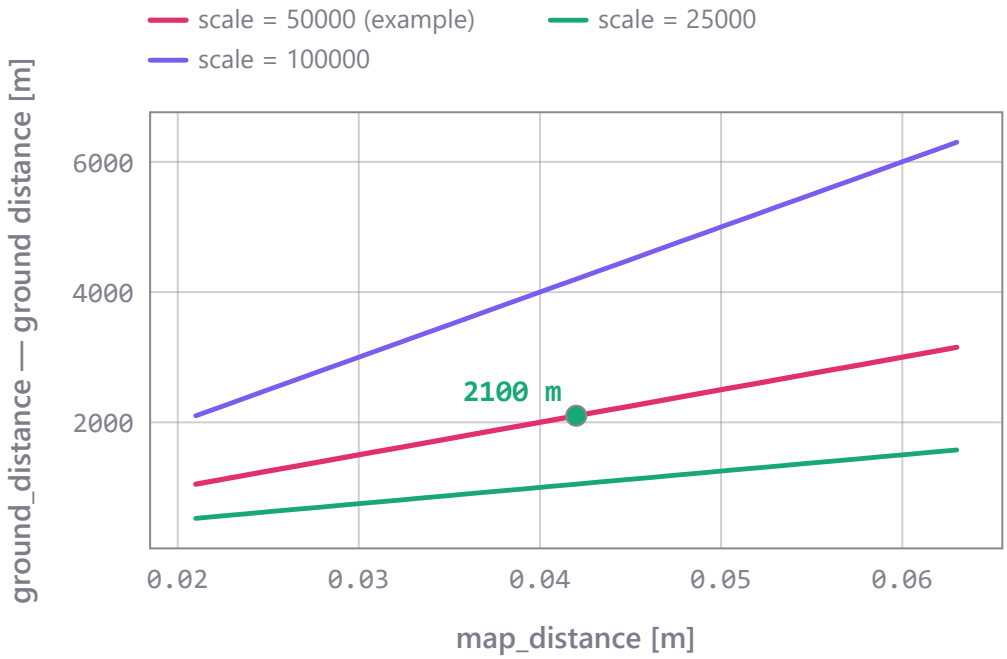
The real-world distance represented by a distance measured on a map of scale 1 : scale.

$$D = d \times S$$

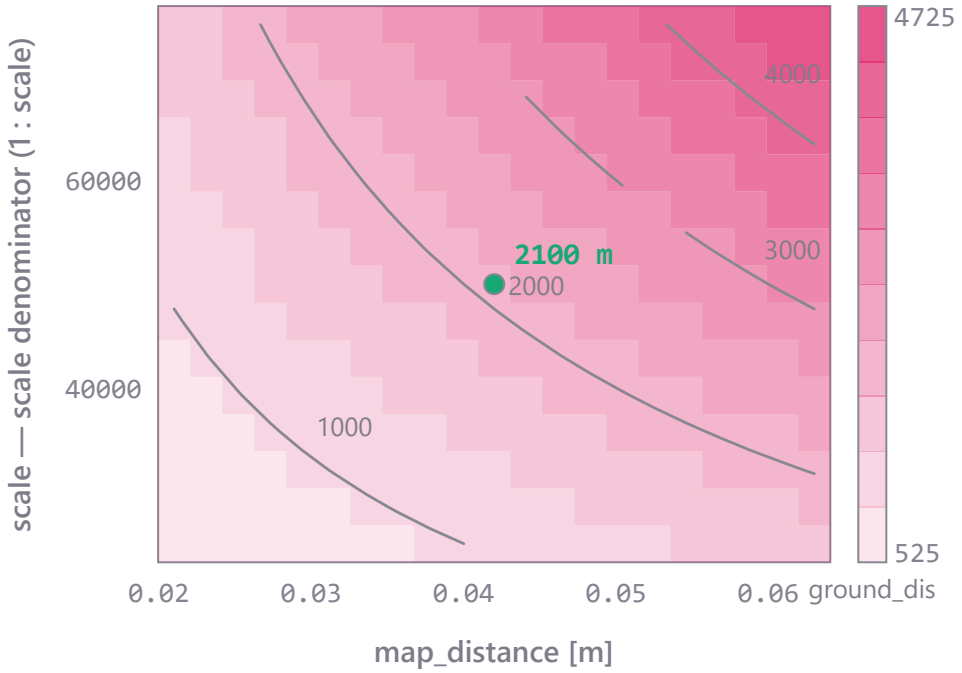
input	meaning	condition
`map_distance`	distance measured on the map	
`scale`	scale denominator (1 : scale)	

output	meaning
`ground_distance`	ground distance
`ground_km`	ground distance in kilometres

- **Method:** ground distance = map distance x scale denominator
- **Assumes:** positive map distance and scale
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ground_distance (ground distance) against map_distance (distance measured on the map) from 0.021 m to 0.063 m. One curve for each value of scale (scale denominator (1 : scale)): 50000, 25000, 100000. 31 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, map_distance = 0.042 m, ground_distance = 2100 m.



ground_distance (ground distance) over map_distance (distance measured on the map) from 0.021 m to 0.063 m and scale (scale denominator (1 : scale)) from 25000 to 75000: each of the 21 × 15 cells is one run of geospatial-analytics-ai999x, shaded by its value (light 525 m to dark 4725 m); iso-lines at ground_distance = 1000, 2000, 3000, 4000 m. Marked: the worked example, map_distance = 0.042 m, scale = 50000, ground_distance = 2100 m.

```
geospatial_analytics_ai999x.solve('geo_map_scale_ground_distance', map_distance=0.042, scale=50000)
# -> ground_distance = 2100 m, ground_km = 2.1   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_map_scale_ground_distance', map_distance=0.012, scale=25000)
# -> ground_distance = 300 m, ground_km = 0.3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Map scale ground distance: map_distance = 4.2 cm, scale = 50000	SOLVED	ground_distance = 2100 m, ground_km = 2.1
Map scale ground distance: map_distance = 12 mm, scale = 25000	SOLVED	ground_distance = 300 m, ground_km = 0.3
Map scale ground distance: scale = 50000	NEEDS_INPUT	geo_map_scale_ground_distance needs map_distance (distance measured on the map). Example: solve('geo_map_scale_ground_di
Map scale ground distance: map_distance = 4.2 cm, scale = 50000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Map scale (representative fraction) from a map and a ground distance — `geo_map_scale_from_distances`

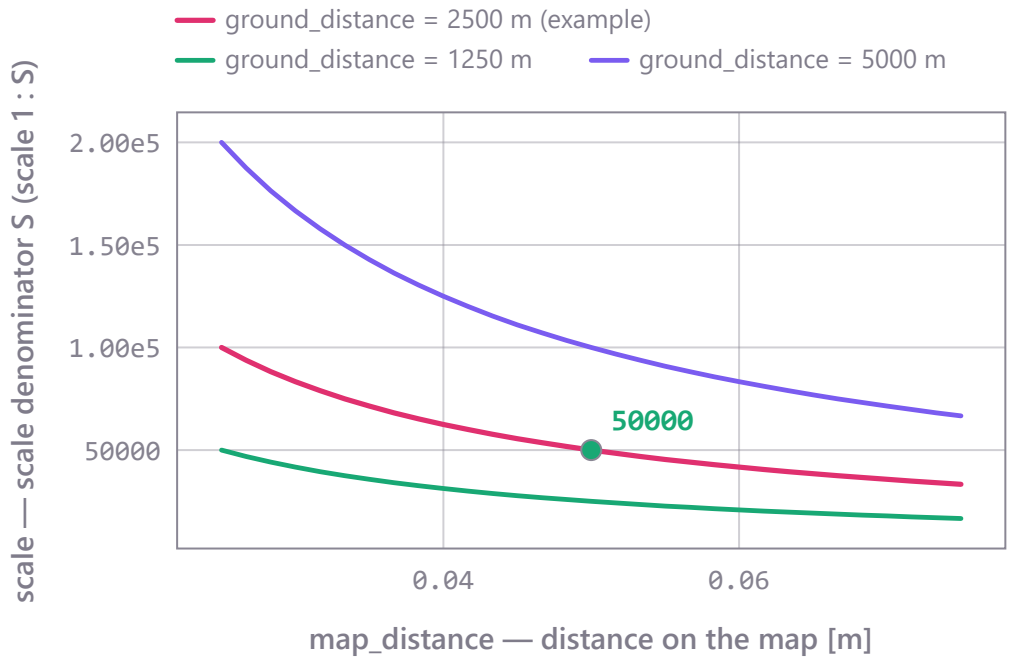
The scale denominator and representative fraction of a map on which a known ground distance measures a given map distance.

$$S = \frac{D}{d}, \quad RF = \frac{1}{S}$$

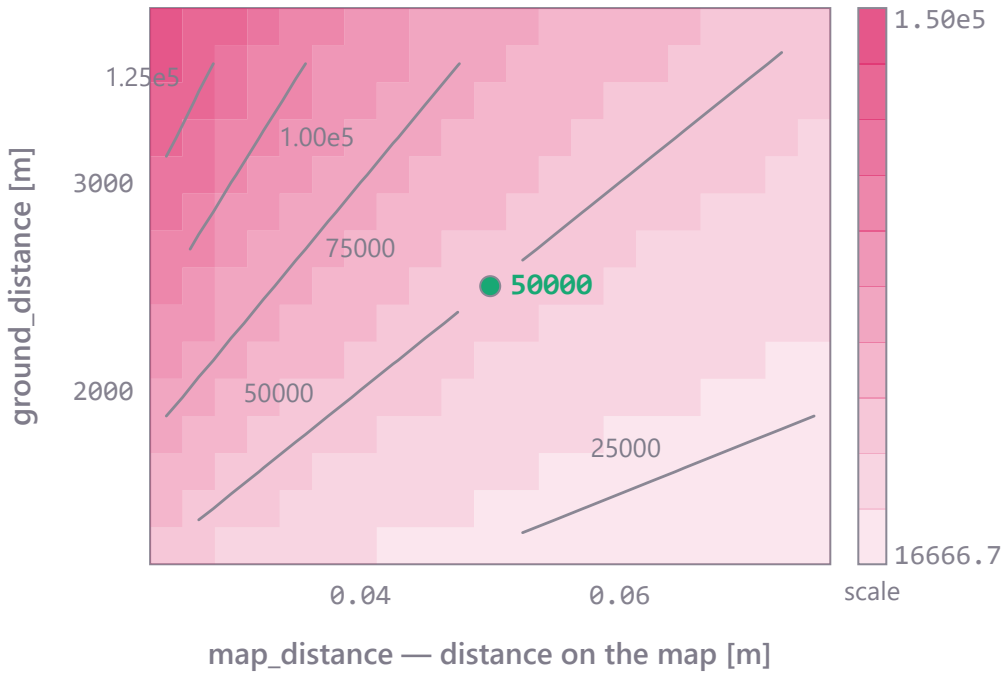
input	meaning	condition
`map_distance`	distance on the map	
`ground_distance`	corresponding ground distance	

output	meaning
`scale`	scale denominator S (scale 1 : S)
`rf`	representative fraction $1/S$

- **Method:** S = ground distance / map distance in the same unit
- **Assumes:** ground distance > map distance > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



scale (scale denominator S (scale 1 : S)) against map_distance (distance on the map) from 0.025 m to 0.075 m. One curve for each value of ground_distance (corresponding ground distance): 2500 m, 1250 m, 5000 m. 31 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, map_distance = 0.05 m, scale = 50000.



scale (scale denominator S (scale 1 : S)) over map_distance (distance on the map) from 0.025 m to 0.075 m and ground_distance (corresponding ground distance) from 1250 m to 3750 m: each of the 21 × 15 cells is one run of geospatial-analytics-ai999x, shaded by its value (light 16666.7 to dark 150000); iso-lines at scale = 25000, 50000, 75000, 1.00e5, 1.25e5. Marked: the worked example, map_distance = 0.05 m, ground_distance = 2500 m, scale = 50000.

```
geospatial_analytics_ai999x.solve('geo_map_scale_from_distances', map_distance=0.05,
ground_distance=2500.0)
# -> scale = 50000, rf = 2e-05   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_map_scale_from_distances', map_distance=0.032,
ground_distance=800.0)
# -> scale = 25000, rf = 4e-05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Map scale from distances: map_distance = 5 cm, ground_distance = 2.5 km	SOLVED	scale = 50000, rf = 2e-05
Map scale from distances: map_distance = 3.2 cm, ground_distance = 800 m	SOLVED	scale = 25000, rf = 4e-05
Map scale from distances: ground_distance = 2.5 km	NEEDS_INPUT	geo_map_scale_from_distances needs map_distance (distance on the map). Example: solve('geo_map_scale_from_distances', ma
Map scale from distances: map_distance = 5 cm, ground_distance = 2.5 km, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Geohash of a point and its cell — `geo_geohash_encode`

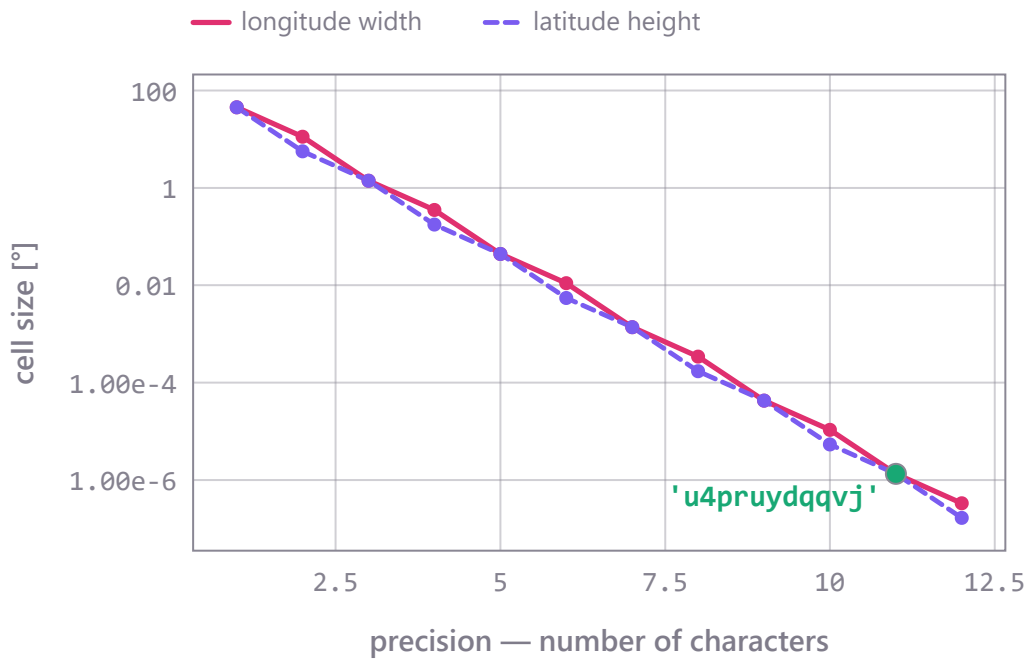
The geohash of a latitude and longitude at a given number of characters, with the centre and half-sizes of the geohash cell.

$$\text{bits} = \lambda_1\varphi_1\lambda_2\varphi_2\ldots, \quad \text{character} = \text{base32}(5\text{ bits})$$

input	meaning	condition
`lat`	latitude (decimal degrees)	
`lon`	longitude (decimal degrees)	
`precision`	number of geohash characters $(1-12)$	

output	meaning
`geohash`	geohash
`lat_center`	latitude of the cell centre
`lon_center`	longitude of the cell centre
`lat_error`	half the cell height (degrees)
`lon_error`	half the cell width (degrees)

- **Method:** interleaved binary bisection of longitude and latitude (longitude first), five bits per base-32 character
- **Assumes:** 1 to 12 characters
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The cell a geohash of each length covers at latitude 57.64911°, longitude 10.40744°: width and height are twice the longitude and latitude errors the package returns (doubled here), for 1 to 12 characters. Each character adds five bits, alternately splitting longitude and latitude, so the sizes fall in a zig-zag. Marked: the worked example, precision 11 → 'u4pruydqqvj', cell 1.3411e-06° × 1.3411e-06°.

```
geospatial_analytics_ai999x.solve('geo_geohash_encode', lat=57.64911, lon=10.40744, precision=11)
# -> geohash = u4pruydqqvj, lat_center = 57.6491, lon_center = 10.4074, lat_error = 6.70552e-07,
lon_error = 6.70552e-07   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_geohash_encode', lat=-33.8688, lon=151.2093, precision=6)
# -> geohash = r3gx2f, lat_center = -33.8681, lon_center = 151.21, lat_error = 0.00274658,
lon_error = 0.00549316   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geohash encode: lat = 57.64911, lon = 10.40744, precision = 11	SOLVED	geohash = u4pruydqqvj, lat_center = 57.6491, lon_center = 10.4074, lat_error = 6.70552e-07, lon_error = 6.70552e-07
Geohash encode: lat = -33.8688, lon = 151.2093, precision = 6	SOLVED	geohash = r3gx2f, lat_center = -33.8681, lon_center = 151.21, lat_error = 0.00274658, lon_error = 0.00549316
Geohash encode: lon = 10.40744, precision = 11	NEEDS_INPUT	geo_geohash_encode needs lat (latitude (decimal degrees)). Example: solve('geo_geohash_encode', lat=57.6491, lon=10.4074
Geohash encode: lat = 57.64911, lon = 10.40744, precision = 11, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Decode a geohash: cell centre and size — `geo_geohash_decode`

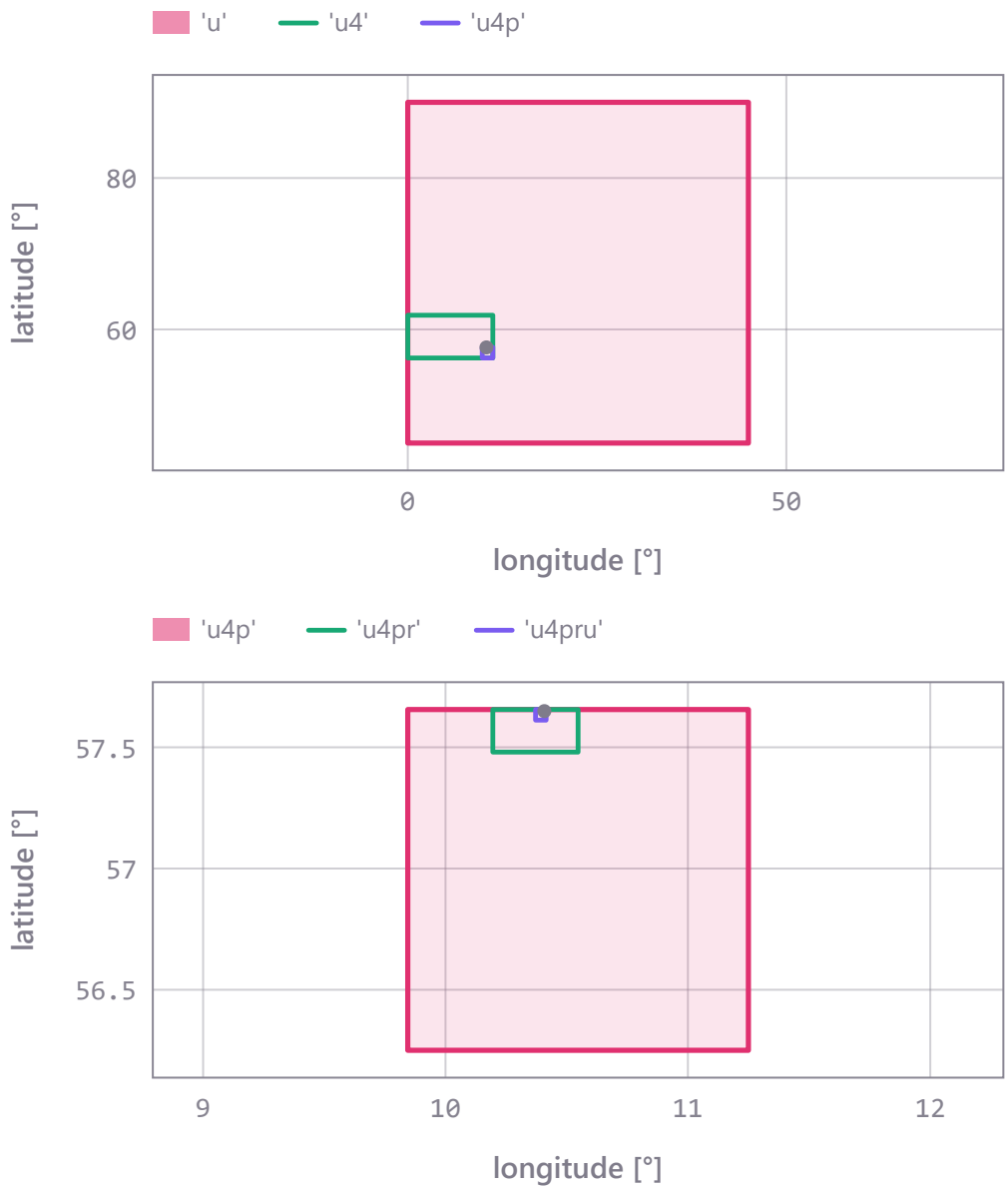
The centre latitude and longitude of a geohash cell and its half-height and half-width in degrees.

$$[\varphi_{min}, \varphi_{max}] \times [\lambda_{min}, \lambda_{max}] \text{ by alternate bisection}$$

input	meaning	condition
`geohash`	geohash string (characters $\mathbf{0} - \mathbf{9}$, b-z without a, i, l, o)	

output	meaning
`lat_center`	latitude of the cell centre
`lon_center`	longitude of the cell centre
`lat_error`	half the cell height (degrees)
`lon_error`	half the cell width (degrees)

- **Method:** each character gives five bits that halve the longitude and latitude intervals alternately (longitude first)
- **Assumes:** 1 to 12 valid characters
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The geohash 'u4pruydqqvj' read character by character: each prefix decoded by the package is a cell (centre ± returned error), and every added character subdivides the previous cell. Top: prefixes of 1–3 characters ('u': 67.5° ± 22.5°, 22.5° ± 22.5°; 'u4': 59.0625° ± 2.8125°, 5.625° ± 5.625°; 'u4p': 56.9531° ± 0.703125°, 10.5469° ± 0.703125°); bottom: 3–5 characters, zoomed into the third cell. The dot is the full hash's centre, 57.6491°, 10.4074° (error 6.7055e-07°).

```
geospatial_analytics_ai999x.solve('geo_geohash_decode', geohash='u4pruydqqvj')
# -> lat_center = 57.6491, lon_center = 10.4074, lat_error = 6.70552e-07, lon_error = 6.70552e-07
(SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_geohash_decode', geohash='9q8yy')
# -> lat_center = 37.771, lon_center = -122.41, lat_error = 0.0219727, lon_error = 0.0219727
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geohash decode: geohash = u4pruydqqvj	SOLVED	lat_center = 57.6491, lon_center = 10.4074, lat_error = 6.70552e-07, lon_error = 6.70552e-07
Geohash decode: geohash = 9q8yy	SOLVED	lat_center = 37.771, lon_center = -122.41, lat_error = 0.0219727, lon_error = 0.0219727

question	outcome	result
Geohash decode:	NEEDS_INPUT	geo_geohash_decode needs geohash (geohash string (characters 0-9, b-z without a, i, l, o)). Example: solve('geo_geohash_
Geohash decode: geohash = u4pruydqqvj, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ
Geohash decode: geohash = u4pruaqqvj	INVALID_INPUT	Decode a geohash: cell centre and size: a geohash uses the characters 0-9 and b-z without a, i, l, o.

MGRS grid reference of a point (1 m) — `geo_mgrs_encode`

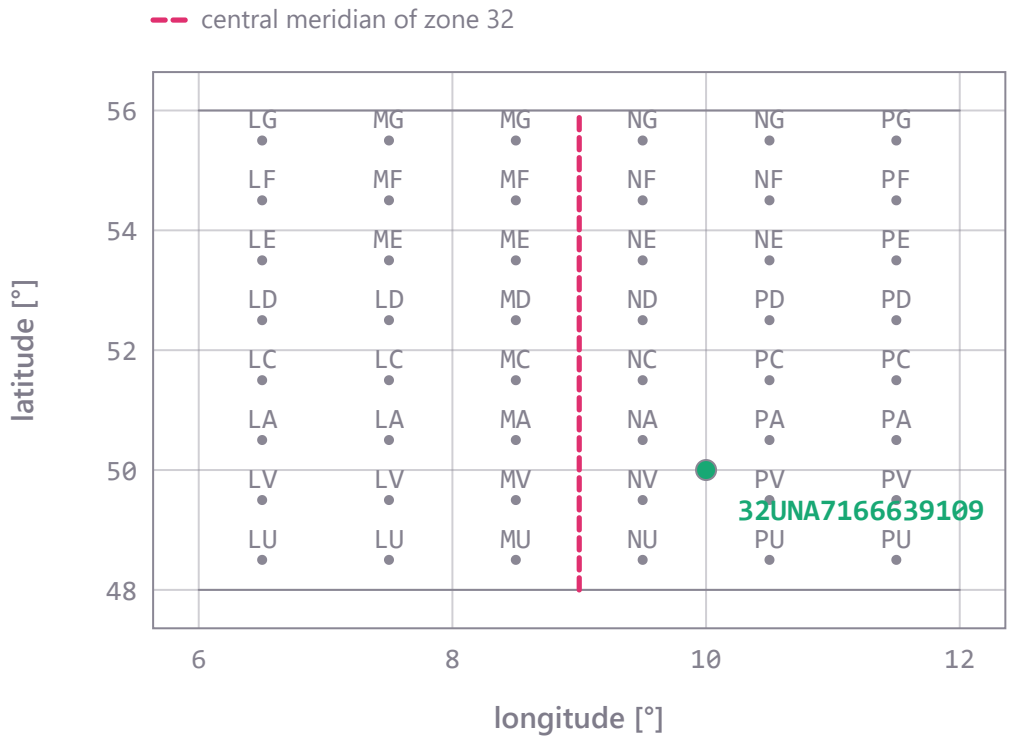
The Military Grid Reference System (MGRS / USNG) reference of a WGS84 latitude and longitude to 1 m: zone, latitude band, 100 km square letters and 5-digit easting and northing.

$$\mathrm{MGRS} = z\,B\,C_{100k}R_{100k}\,\lfloor E \rfloor \bmod 10^5\,\lfloor N \rfloor \bmod 10^5$$

input	meaning	condition
`lat`	latitude (WGS84) (decimal degrees)	
`lon`	longitude (WGS84) (decimal degrees)	

output	meaning
`mgrs`	MGRS reference (no spaces)

- **Method:** UTM coordinates (Krueger series), 100 km column letter from the zone's set (A-H, J-R, S-Z), row letter cycling A-V (offset by 5 in even zones), digits truncated to 1 m; the zone is written with two digits
- **Assumes:** 80 S <= latitude <= 84 N (polar regions use UPS)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Inside grid zone 32U (6° E to 12° E, 48° N to 56° N): at points every 1° the package's MGRS code, of which the two letters of the 100 km square are printed (48 runs); the letters change along easting and northing, not along the graticule. Marked: the worked example 50.0°, 10.0° → 32UNA7166639109 (zone 32U, square NA, then 5-digit easting and northing within it).

```
geospatial_analytics_ai999x.solve('geo_mgrs_encode', lat=50.0, lon=10.0)
# -> mgrs = 32UNA7166639109   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_mgrs_encode', lat=38.8977, lon=-77.0365)
# -> mgrs = 18SUJ2339407395   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_mgrs_encode', lat=-33.8568, lon=151.2153)
# -> mgrs = 56HLH3490052288   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MGRS grid reference: lat = 50, lon = 10	SOLVED	mgrs = 32UNA7166639109
MGRS grid reference: lat = 38.8977, lon = -77.0365	SOLVED	mgrs = 18SUJ2339407395
MGRS grid reference: lat = -33.8568, lon = 151.2153	SOLVED	mgrs = 56HLH3490052288
MGRS grid reference: lon = 10	NEEDS_INPUT	geo_mgrs_encode needs lat (latitude (WGS84) (decimal degrees)). Example: solve('geo_mgrs_encode', lat=50, lon=10).
MGRS grid reference: lat = 50, lon = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Douglas-Peucker line simplification — `geo_douglas_peucker_simplification`

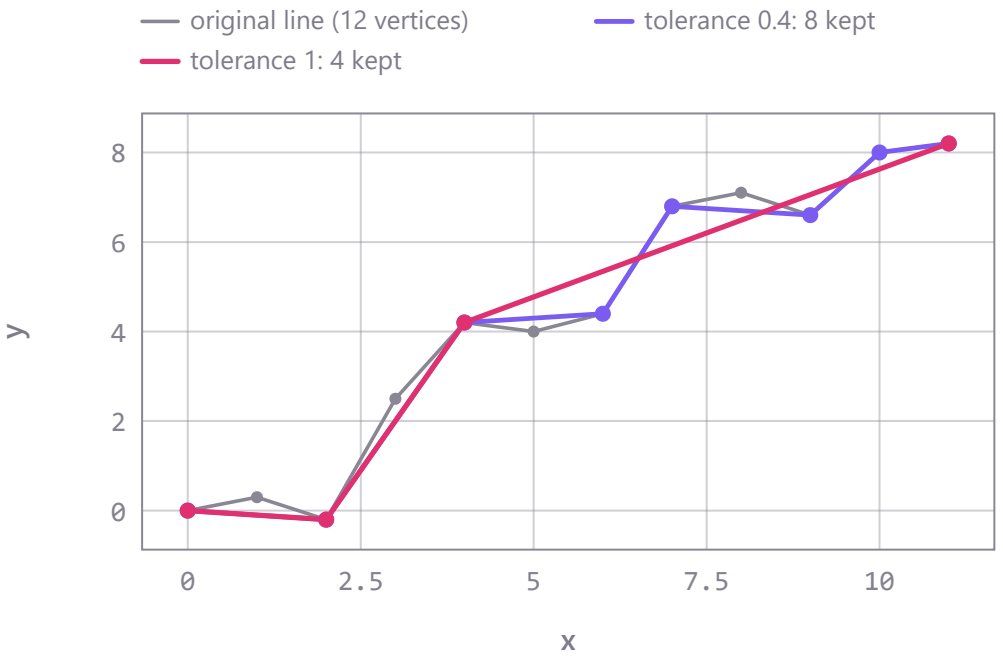
The vertices of a polyline kept by the Ramer-Douglas-Peucker simplification at a distance tolerance (indices from 0).

$$d_{max} = \max_k \operatorname{dist}(P_k, \overline{P_i P_j}) > \varepsilon \Rightarrow \text{keep } P_k$$

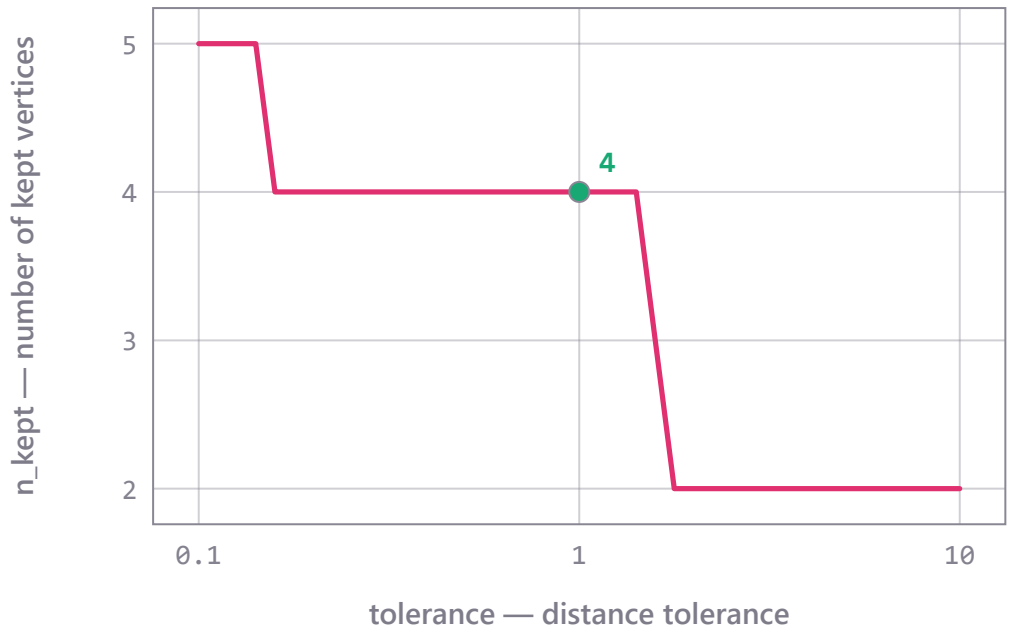
input	meaning	condition
`x`	x coordinates of the vertices in order	
`y`	y coordinates of the vertices	
`tolerance`	distance tolerance (same unit as the coordinates)	

output	meaning
`kept`	indices of the kept vertices (from 0, in order)
`n_kept`	number of kept vertices

- **Method:** recursive: keep the vertex farthest from the segment joining the ends of a run if its distance exceeds the tolerance, then split there
- **Assumes:** distinct vertices; tolerance > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A 12-vertex line simplified by the Douglas–Peucker algorithm at two tolerances, the kept vertices as returned by the package: tolerance 0.4 keeps 8 vertices (0, 2, 4, 6, 7, 9, 10, 11); tolerance 1 keeps 4 vertices (0, 2, 4, 11).



n_kept (number of kept vertices) against tolerance (distance tolerance (same unit as the coordinates)) from 0.1 to 10 on a logarithmic axis. Other inputs as in the worked example: x = [0, 1, 2, 3, 4, 5, 6, 7], y = [0, 0.1, -0.1, 5, 6, 7, 8.1, 9]. 41 points computed by geospatial-analytics-ai999x; marked: the worked example, tolerance = 1, n_kept = 4.

```
geospatial_analytics_ai999x.solve('geo_douglas_peucker_simplification', x=[0, 1, 2, 3, 4, 5, 6, 7],
y=[0, 0.1, -0.1, 5, 6, 7, 8.1, 9], tolerance=1.0)
# -> kept = [0.0, 2.0, 3.0, 7.0], n_kept = 4   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_douglas_peucker_simplification', x=[0, 2, 4, 6, 8, 10], y=
[0, 1.5, 0.2, 2.4, 0.1, 0], tolerance=0.5)
# -> kept = [0.0, 1.0, 2.0, 3.0, 4.0, 5.0], n_kept = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Douglas-Peucker simplification: x = [0, 1, 2, 3, 4, 5, 6, 7], y = [0, 0.1, -0.1, 5, 6, 7, 8.1, 9], tolerance = 1	SOLVED	kept = [0.0, 2.0, 3.0, 7.0], n_kept = 4
Douglas-Peucker simplification: x = [0, 2, 4, 6, 8, 10], y = [0, 1.5, 0.2, 2.4, 0.1, 0], tolerance = 0.5	SOLVED	kept = [0.0, 1.0, 2.0, 3.0, 4.0, 5.0], n_kept = 6
Douglas-Peucker simplification: y = [0, 0.1, -0.1, 5, 6, 7, 8.1, 9], tolerance = 1	NEEDS_INPUT	geo_douglas_peucker_simplification needs x (x coordinates of the vertices in order). Example: solve('geo_douglas_peucker
Douglas-Peucker simplification: x = [0, 1, 2, 3, 4, 5, 6, 7], y = [0, 0.1, -0.1, 5, 6, 7, 8.1, 9], tolerance = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Discrete Frechet distance between two polylines — `geo_discrete_frechet_distance`

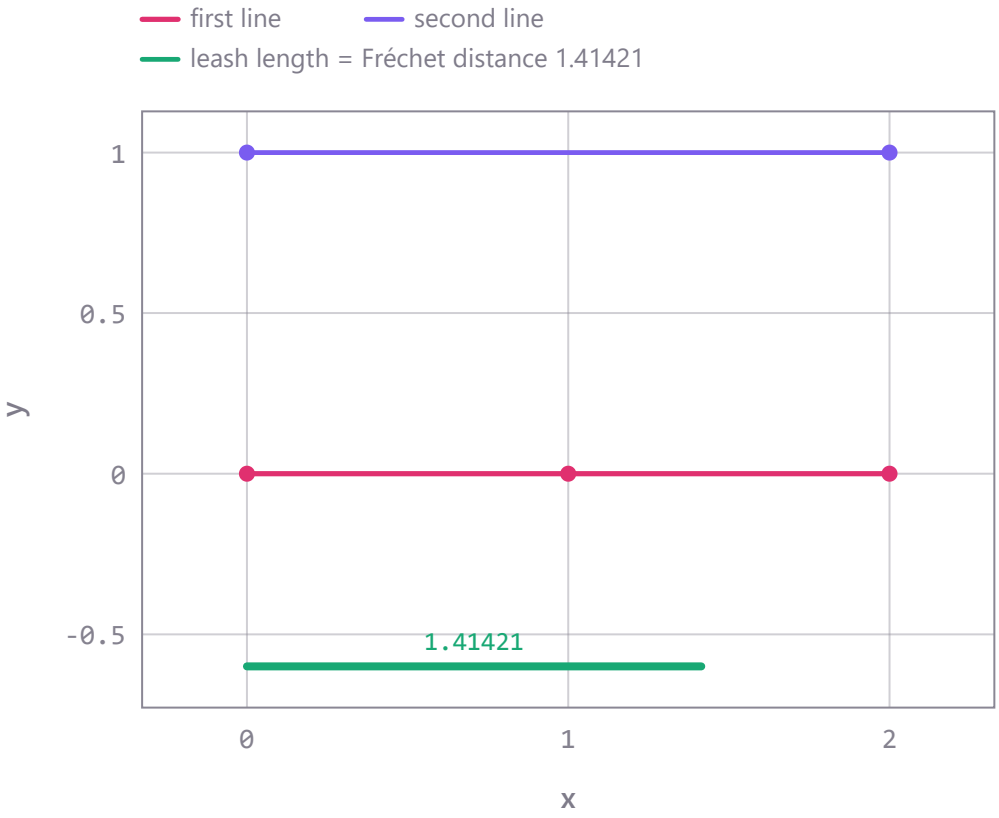
The discrete Frechet ('dog-walking') distance between two polylines (GPS tracks, routes) measured over their vertices.

$$c_{i,j} = \max\left(d(P_i, Q_j),\ \min(c_{i-1,j},\ c_{i-1,j-1},\ c_{i,j-1})\right)$$

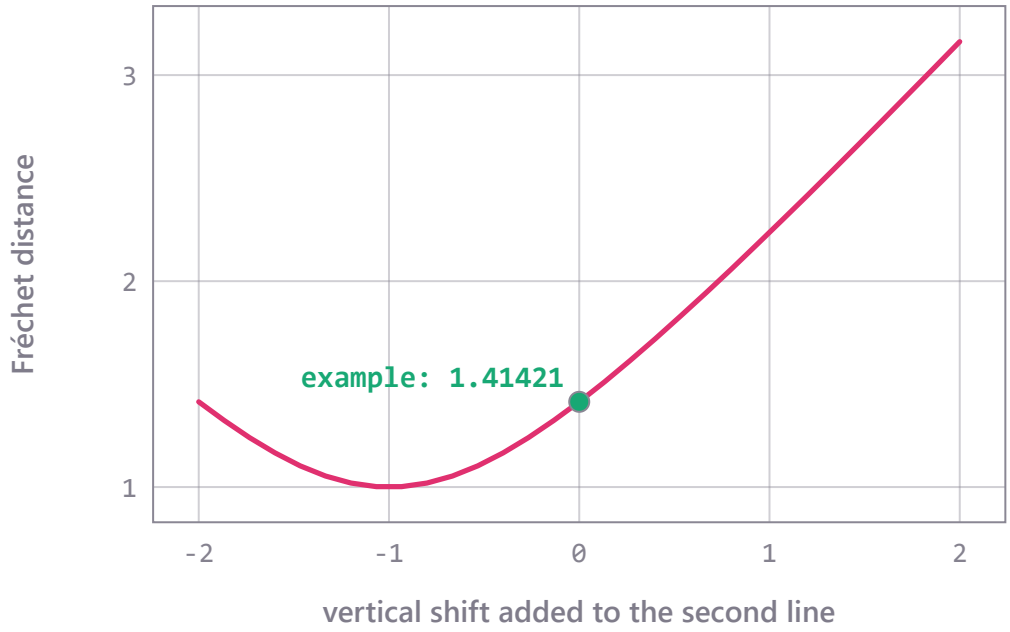
input	meaning	condition
`x1`	x coordinates of the first line	
`y1`	y coordinates of the first line	
`x2`	x coordinates of the second line	
`y2`	y coordinates of the second line	

output	meaning
`distance`	discrete Frechet distance (coordinate units)

- **Method:** dynamic programme (Eiter-Mannila): c(i,j) = max(d(i,j), min(c($i-1$,j), c($i-1$,j-1), c(i,j-1)))
- **Assumes:** at least 2 vertices per line
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Two polylines, ([0, 1, 2], [0, 0, 0]) and ([0, 2], [1, 1]) as x and y lists, and the discrete Fréchet distance the package returns, 1.41421: the shortest leash that lets a walker on each line advance vertex by vertex, never backwards, while staying connected (drawn as a bar of that length).



The discrete Fréchet distance returned by the package as the second line of the worked example (y2 = [1, 1]) is shifted vertically by −2 to 2 (31 runs): it is smallest when the second line runs along the first and grows with the separation. Marked: no shift, 1.41421.

```
geospatial_analytics_ai999x.solve('geo_discrete_frechet_distance', x1=[0, 1, 2], y1=[0, 0, 0], x2=
[0, 2], y2=[1, 1])
# -> distance = 1.41421   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_discrete_frechet_distance', x1=[0, 2, 4, 6], y1=[0, 2, 0,
2], x2=[0, 3, 6], y2=[1, 1, 1])
# -> distance = 1.41421   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Frechet distance: x1 = [0, 1, 2], y1 = [0, 0, 0], x2 = [0, 2], y2 = [1, 1]	SOLVED	distance = 1.41421
Frechet distance: x1 = [0, 2, 4, 6], y1 = [0, 2, 0, 2], x2 = [0, 3, 6], y2 = [1, 1, 1]	SOLVED	distance = 1.41421
Frechet distance: y1 = [0, 0, 0], x2 = [0, 2], y2 = [1, 1]	NEEDS_INPUT	geo_discrete_frechet_distance needs x1 (x coordinates of the first line). Example: solve('geo_discrete_frechet_distance'
Frechet distance: x1 = [0, 1, 2], y1 = [0, 0, 0], x2 = [0, 2], y2 = [1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Shape compactness indices of a polygon (Polsby-Popper, Schwartzberg, convex hull, Reock) — `geo_polygon_compactness`

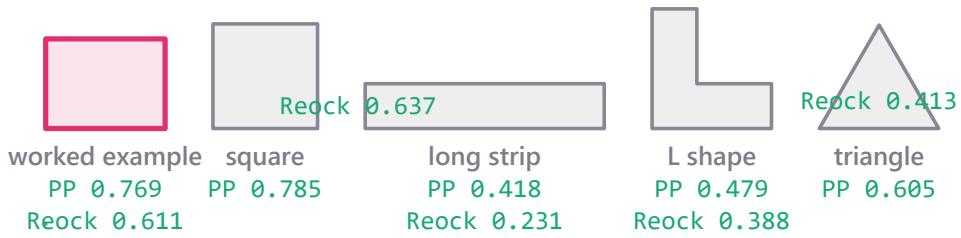
Area, perimeter and the standard compactness measures of a polygon in projected coordinates (used for districts and land parcels): Polsby-Popper, Schwartzberg, convex-hull ratio and Reock.

$$PP = \frac{4\pi A}{P^2}, \quad S = \frac{P}{2\sqrt{\pi A}}, \quad C_H = \frac{A}{A_{hull}}, \quad R = \frac{A}{\pi r_{mec}^2}$$

input	meaning	condition
`x`	x coordinates of the vertices in order (not repeating the first)	
`y`	y coordinates of the vertices	

output	meaning
`area`	area (square coordinate units)
`perimeter`	perimeter (coordinate units)
`polsby_popper`	4 pi A/P^2
`schwartzberg`	P / (2 sqrt(pi A))
`convex_hull_ratio`	area / convex hull area
`reock`	area / area of the minimum enclosing circle

- **Method:** shoelace area, perimeter, monotone-chain convex hull, Welzl minimum enclosing circle
- **Assumes:** a simple polygon with distinct vertices
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Five polygons and the compactness scores the package returns for each (1 = a circle): worked example: Polsby–Popper 0.76937, Reock 0.611155; square: Polsby–Popper 0.785398, Reock 0.63662; long strip: Polsby–Popper 0.417719, Reock 0.230625; L shape: Polsby–Popper 0.478602, Reock 0.38794; triangle: Polsby–Popper 0.604599, Reock 0.413496. The worked example ([0, 4, 4, 0]; [0, 0, 3, 3]) has area 12, perimeter 14, Schwartzberg 1.14007, convex-hull ratio 1.

```
geospatial_analytics_ai999x.solve('geo_polygon_compactness', x=[0, 4, 4, 0], y=[0, 0, 3, 3])
# -> area = 12, perimeter = 14, polsby_popper = 0.76937, schwartzberg = 1.14007, convex_hull_ratio
= 1, reock = 0.611155   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_polygon_compactness', x=[0, 10, 10, 5, 0], y=[0, 0, 10, 2,
10])
# -> area = 60, perimeter = 48.868, polsby_popper = 0.315728, schwartzberg = 1.77969,
convex_hull_ratio = 0.6, reock = 0.381972   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Polygon compactness: x = [0, 4, 4, 0], y = [0, 0, 3, 3]	SOLVED	area = 12, perimeter = 14, polsby_popper = 0.76937, schwartzberg = 1.14007, convex_hull_ratio = 1, reock = 0.611155
Polygon compactness: x = [0, 10, 10, 5, 0], y = [0, 0, 10, 2, 10]	SOLVED	area = 60, perimeter = 48.868, polsby_popper = 0.315728, schwartzberg = 1.77969, convex_hull_ratio = 0.6, reock = 0.381972
Polygon compactness: y = [0, 0, 3, 3]	NEEDS_INPUT	geo_polygon_compactness needs x (x coordinates of the vertices in order (not repeating the first)). Example: solve('geo_
Polygon compactness: x = [0, 4, 4, 0], y = [0, 0, 3, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ
Polygon compactness: x = [0, 4, 0, 4], y = [0, 0, 3, 3]	INVALID_INPUT	Shape compactness indices of a polygon (Polsby-Popper, Schwartzberg, convex hull, Reock): the polygon must be simple (ed

Global Moran's I spatial autocorrelation with normal and randomization inference — `geo_morans_i`

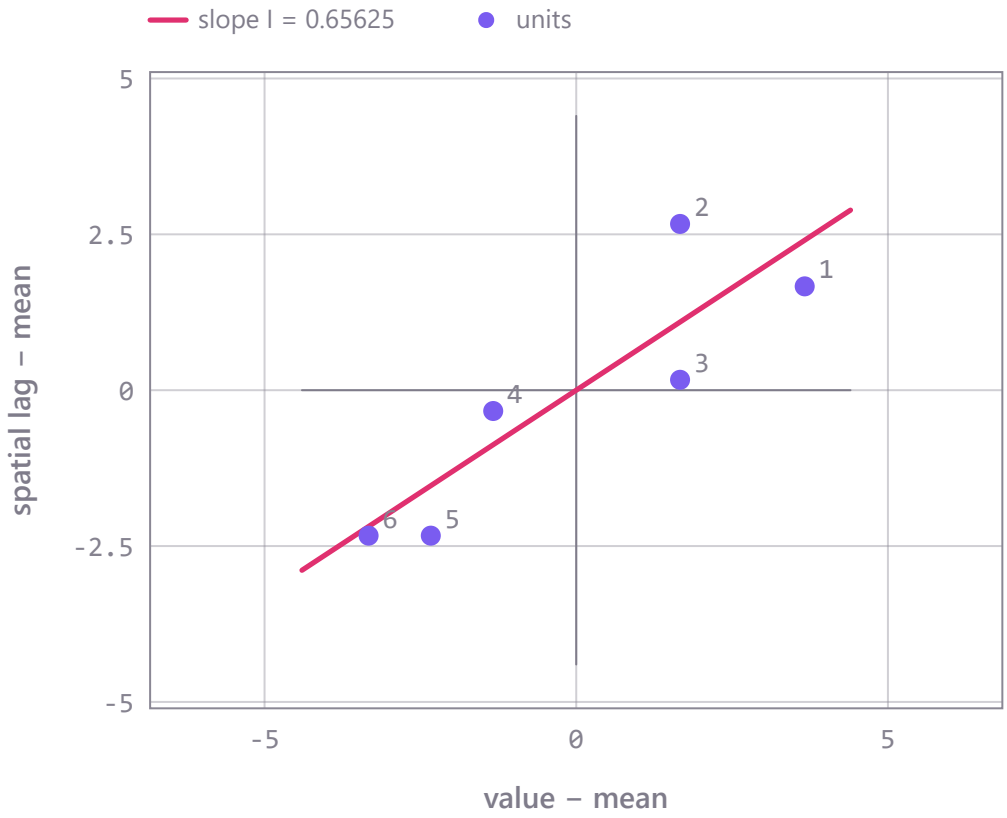
Global Moran's I of a variable observed at n locations with a given spatial weights matrix, its expectation and variance under normality and under randomization, z-scores and two-sided p-values.

$$I = \frac{n}{S_0}\frac{\sum_i \sum_j w_{ij} z_i z_j}{\sum_i z_i^2}, \quad E[I] = -\frac{1}{n-1}, \quad V_N[I] = \frac{n^2 S_1 - n S_2 + 3 S_0^2}{(n^2-1) S_0^2} - E[I]^2$$

input	meaning	condition
`y`	values at the n locations	
`W`	spatial weights matrix (n x n, zero diagonal, row i lists the weights of i's neighbours)	
`transform`	row-standardize the weights (row) or use them as given (none)	

output	meaning
`I`	Moran's I
`EI`	expected $I = -1/(n-1)$
`VI_norm`	variance under normality
`z_norm`	z-score under normality
`p_norm`	two-sided p-value under normality
`VI_rand`	variance under randomization
`z_rand`	z-score under randomization
`p_rand`	two-sided p-value under randomization

- **Method:** $I = (n/S_0)$ sum w_ij z_i z_j / sum z_i^2 with z = y - mean; Cliff-Ord moments with S0, S1, S2 and the sample kurtosis
- **Assumes:** $n \geq 4$, values not all equal; no observation without neighbours
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Moran scatter plot for y = [8, 6, 6, 3, 2, 1] with the contiguity matrix of the worked example: each unit's deviation from the mean against the deviation of its row-standardised spatial lag (lags returned by the package's spatial-lag capability; deviations computed here). The line through the origin has the returned Moran's I = 0.65625 as its slope (expected -0.2 without autocorrelation; z = 2.28068, p = 0.0225674 under normality). Points in the upper-right and lower-left quadrants are like-valued neighbours.

```
geospatial_analytics_ai999x.solve('geo_morans_i', y=[8, 6, 6, 3, 2, 1], W=[[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], transform='row')
# -> I = 0.65625, EI = -0.2, VI_norm = 0.140952, z_norm = 2.28068, p_norm = 0.0225674, VI_rand = 0.181793, z_rand = 2.00823, p_rand = 0.0446193   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_morans_i', y=[1, 2, 1, 2, 1, 2], W=[[0, 1, 0, 0, 0, 1], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [1, 0, 0, 0, 1, 0]], transform='none')
# -> I = -1, EI = -0.2, VI_norm = 0.102857, z_norm = -2.49444, p_norm = 0.0126157, VI_rand = 0.16, z_rand = -2, p_rand = 0.0455003   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Moran's I: y = [8, 6, 6, 3, 2, 1], W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], transform = row	SOLVED	I = 0.65625, EI = -0.2, VI_norm = 0.140952, z_norm = 2.28068, p_norm = 0.0225674, VI_rand = 0.181793, z_rand = 2.00823, p_rand = 0.0446193
Moran's I: y = [1, 2, 1, 2, 1, 2], W = [[0, 1, 0, 0, 0, 1], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [1, 0, 0, 0, 1, 0]], transform = none	SOLVED	I = -1, EI = -0.2, VI_norm = 0.102857, z_norm = -2.49444, p_norm = 0.0126157, VI_rand = 0.16, z_rand = -2, p_rand = 0.0455003

question	outcome	result
Moran's I: W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], transform = row	NEEDS_INPUT	geo_morans_i needs y (values at the n locations). Example: solve('geo_morans_i', y=[8, 6, 6, 3, 2, 1], W= [[0, 1, 0, 0, 0
Moran's I: y = [8, 6, 6, 3, 2, 1], W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], transform = row, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Geary's C spatial autocorrelation with normal and randomization inference — `geo_gearys_c`

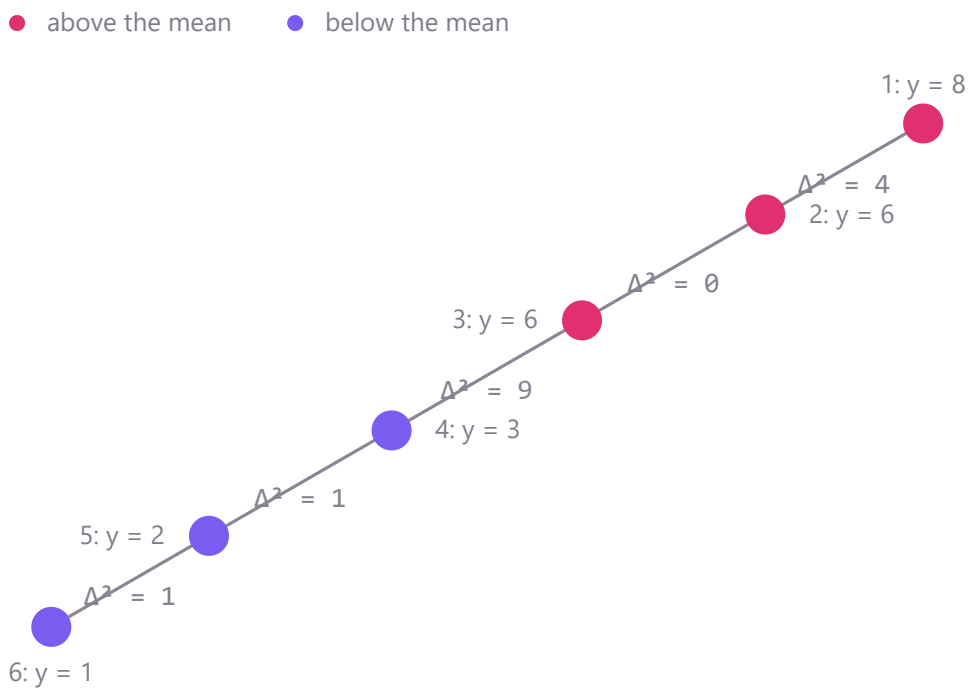
Geary's contiguity ratio C of a variable with a given spatial weights matrix, its variance under normality and randomization, and the z-scores (C < 1 positive, $C > 1$ negative autocorrelation).

$$C = \frac{(n-1)\sum_i \sum_j w_{ij}(y_i - y_j)^2}{2S_0 \sum_i (y_i - \bar{y})^2}, \quad V_N[C] = \frac{(2S_1 + S_2)(n-1) - 4S_0^2}{2(n+1)S_0^2}$$

input	meaning	condition
`y`	values at the n locations	
`W`	spatial weights matrix (n x n, zero diagonal, row i lists the weights of i's neighbours)	
`transform`	row-standardize the weights (row) or use them as given (none)	

output	meaning
`C`	Geary's C
`VC_norm`	variance under normality
`z_norm`	z-score under normality
`VC_rand`	variance under randomization
`z_rand`	z-score under randomization

- **Method:** $C = (n-1)$ sum w_ij (y_i - y_j)^2 / (2 S0 sum z^2); Cliff-Ord moments (E[C] = 1)
- **Assumes:** $n \geq 4$, values not all equal; no observation without neighbours
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 6 units of the worked example as a network of their neighbour links (W; layout without meaning), each with its value y; on each link the squared difference of the two values (computed here from the inputs). Geary's C compares these with the overall variance; the package returns C = 0.195312 (below 1: neighbours are alike), z = -2.41406 under normality, -2.21289 under randomisation.

```
geospatial_analytics_ai999x.solve('geo_gearys_c', y=[8, 6, 6, 3, 2, 1], W=[[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], transform='row')
# -> C = 0.195312, VC_norm = 0.111111, z_norm = -2.41406, VC_rand = 0.132231, z_rand = -2.21289 (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_gearys_c', y=[1, 2, 1, 2, 1, 2], W=[[0, 1, 0, 0, 0, 1], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [1, 0, 0, 0, 1, 0]], transform='none')
# -> C = 1.66667, VC_norm = 0.0714286, z_norm = 2.49444, VC_rand = 0.111111, z_rand = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geary's C: y = [8, 6, 6, 3, 2, 1], W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], transform = row	SOLVED	C = 0.195312, VC_norm = 0.111111, z_norm = -2.41406, VC_rand = 0.132231, z_rand = -2.21289
Geary's C: y = [1, 2, 1, 2, 1, 2], W = [[0, 1, 0, 0, 0, 1], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [1, 0, 0, 0, 1, 0]], transform = none	SOLVED	C = 1.66667, VC_norm = 0.0714286, z_norm = 2.49444, VC_rand = 0.111111, z_rand = 2
Geary's C: W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], transform = row	NEEDS_INPUT	geo_gearys_c needs y (values at the n locations). Example: solve('geo_gearys_c', y=[8, 6, 6, 3, 2, 1], W=[[0, 1, 0, 0, 0
Geary's C: y = [8, 6, 6, 3, 2, 1], W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0],	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
[0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], transform = row, zz = 1		geospatial_analytics_ai999x.templ

Getis-Ord general G (global high/low clustering) — `geo_getis_ord_general_g`

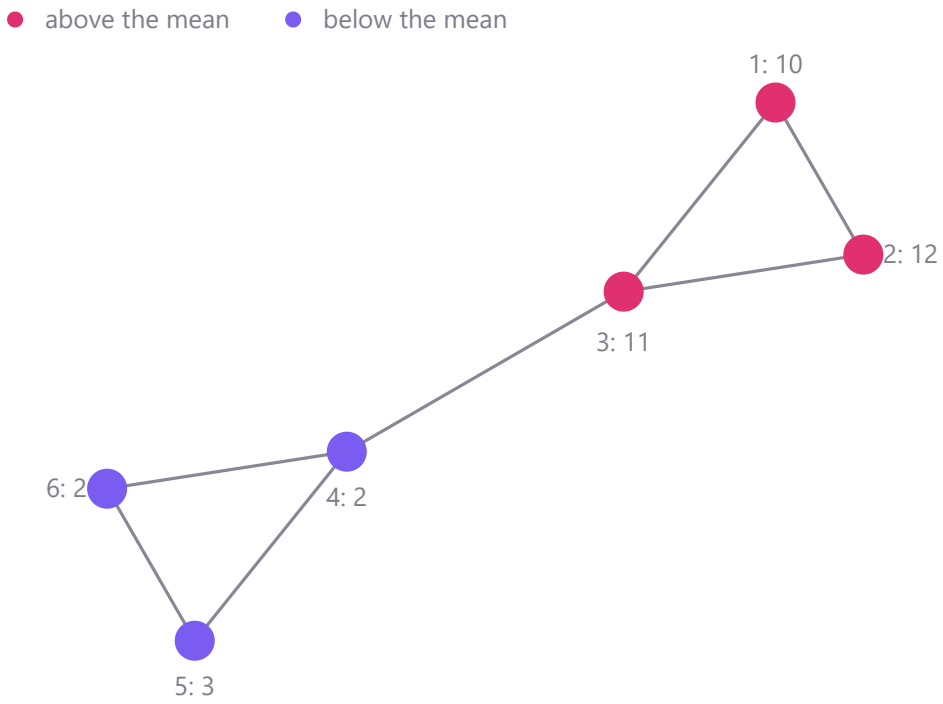
The Getis-Ord general G statistic of a positive variable with binary spatial weights, its expectation and variance under randomization and the z-score (G above its expectation: high values cluster).

$$G = \frac{\sum_i \sum_{j \neq i} w_{ij} y_i y_j}{\sum_i \sum_{j \neq i} y_i y_j}, \quad E[G] = \frac{S_0}{n(n-1)}$$

input	meaning	condition
`y`	positive values at the n locations	
`W`	binary spatial weights matrix (0/1, zero diagonal)	

output	meaning
`G`	general G
`EG`	expected G = S0/(n(n-1))
`VG`	variance of G under randomization
`z_norm`	z-score

- **Method:** G = sum_{$i \neq j$} w_ij y_i y_j / sum_{$i \neq j$} y_i y_j; Getis-Ord (1992) moments
- **Assumes:** positive values, $n \geq 4$; binary weights, no observation without neighbours
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Units with values y = [10, 12, 11, 2, 3, 2] linked by the worked example's weights (layout without meaning): the high values form one connected group. The package returns General G = 0.656814 against its expectation 0.466667 (variance 0.00892117, z = 2.01317): high values cluster.

```
geospatial_analytics_ai999x.solve('geo_getis_ord_general_g', y=[10, 12, 11, 2, 3, 2], W=[[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]])
# -> G = 0.656814, EG = 0.466667, VG = 0.00892117, z_norm = 2.01317   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_getis_ord_general_g', y=[4, 8, 2, 6, 5, 7, 3], W=[[0, 1, 0, 0, 0, 0, 1], [1, 0, 1, 0, 0, 0, 0], [0, 1, 0, 1, 0, 0, 0], [0, 0, 1, 0, 1, 0, 0], [0, 0, 0, 1, 0, 1, 0], [0, 0, 0, 0, 1, 0, 1], [1, 0, 0, 0, 0, 1, 0]])
# -> G = 0.309198, EG = 0.333333, VG = 0.000341944, z_norm = -1.30521   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Getis-Ord general G: y = [10, 12, 11, 2, 3, 2], W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]]	SOLVED	G = 0.656814, EG = 0.466667, VG = 0.00892117, z_norm = 2.01317
Getis-Ord general G: y = [4, 8, 2, 6, 5, 7, 3], W = [[0, 1, 0, 0, 0, 0, 1], [1, 0, 1, 0, 0, 0, 0], [0, 1, 0, 1, 0, 0, 0], [0, 0, 1, 0, 1, 0, 0], [0, 0, 0, 1, 0, 1, 0], [0, 0, 0, 0, 1, 0, 1], [1, 0, 0, 0, 0, 1, 0]]	SOLVED	G = 0.309198, EG = 0.333333, VG = 0.000341944, z_norm = -1.30521
Getis-Ord general G: W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]]	NEEDS_INPUT	geo_getis_ord_general_g needs y (positive values at the n locations). Example: solve('geo_getis_ord_general_g', y=[10, 1
Getis-Ord general G: y = [10, 12, 11, 2, 3, 2], W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Local Getis-Ord Gi and Gi* hot-spot statistics — `geo_local_getis_ord_gi`

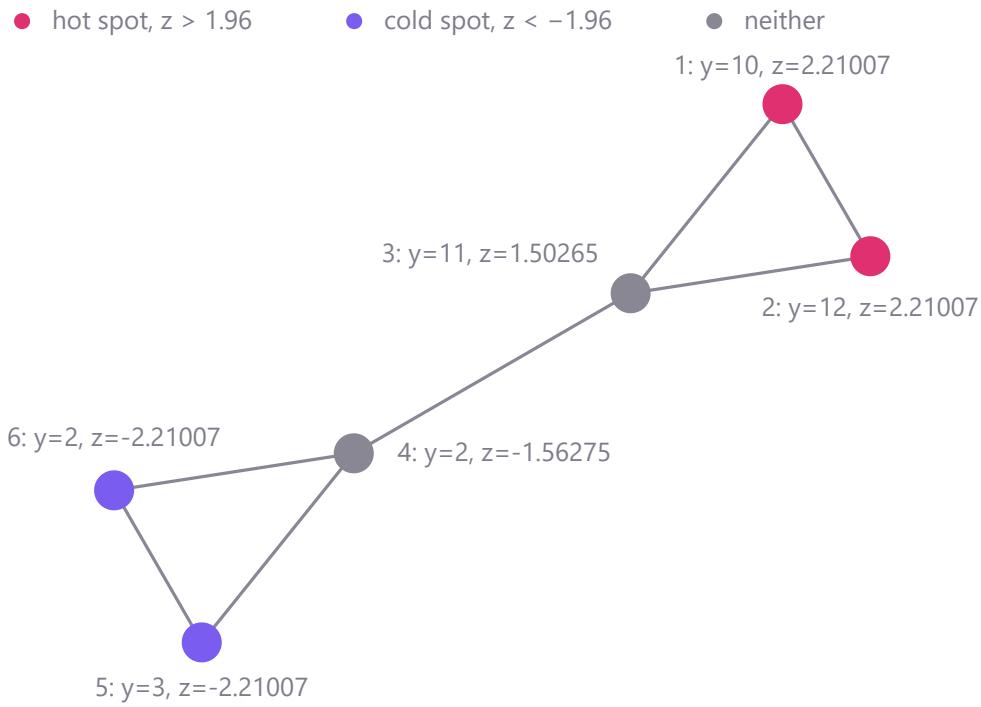
The local Getis-Ord statistic of every location (Gi* includes the location itself, Gi excludes it) with binary weights, and its z-score for hot-spot (positive) and cold-spot (negative) mapping.

$$G_i^* = \frac{\sum_j w_{ij} y_j}{\sum_j y_j}, \quad Z_i = \frac{G_i - W_i/N}{\sqrt{\frac{W_i(N-W_i)}{(N-1)N^2}\,\frac{s^2}{\bar{y}^2}}}$$

input	meaning	condition
`y`	positive values at the n locations	
`W`	binary spatial weights matrix (0/1, zero diagonal)	
`statistic`	Gi* (the location counts as its own neighbour) or Gi	

output	meaning
`Gs`	local G of each location
`Zs`	z-score of each location

- **Method:** G_i = sum_j w_ij y_j / sum_j y_j (j != i for Gi); $E = W_i/N$, $V = W_i$ (N - W_i)/(N-1)/N^2 * s^2/ybar^2 with the values other than i for Gi
- **Assumes:** positive values, $n \geq 4$; binary weights, no observation without neighbours
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Local Getis–Ord statistics (gi_star) on the units of the worked example (values y = [10, 12, 11, 2, 3, 2], links of W, layout without meaning): each node shows its value and the z-score the package returns, coloured hot (z > 1.96) or cold (z < −1.96); returned G values: 0.825, 0.825, 0.875, 0.45, 0.175, 0.175.

```
geospatial_analytics_ai999x.solve('geo_local_getis_ord_gi', y=[10, 12, 11, 2, 3, 2], W=[[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]], statistic='gi_star')
# -> Gs = [0.825, 0.825, 0.875, 0.45, 0.175, 0.175], Zs = [2.21006630298, 2.21006630298, 1.50264699011, -1.56275286971, -2.21006630298, -2.21006630298]   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_local_getis_ord_gi', y=[10, 12, 11, 2, 3, 2], W=[[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]], statistic='gi')
# -> Gs = [0.766666666667, 0.75, 0.827586206897, 0.421052631579, 0.108108108108, 0.131578947368], Zs = [1.98852919667, 1.98558020126, 1.25086177191, -1.31451594415, -1.97973605279, -1.97177391622]  (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Local Getis-Ord Gi: y = [10, 12, 11, 2, 3, 2], W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]], statistic = gi_star	SOLVED	Gs = [0.825, 0.825, 0.875, 0.45, 0.175, 0.175], Zs = [2.21006630298, 2.21006630298, 1.50264699011, -1.56275286971, -2.21006630298, -2.21006630298]
Local Getis-Ord Gi: y = [10, 12, 11, 2, 3, 2], W = [[0, 1, 1, 0, 0, 0], [1, 0,	SOLVED	Gs = [0.766666666667, 0.75, 0.827586206897, 0.421052631579, 0.108108108108, 0.131578947368], Zs =

question	outcome	result
1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]], statistic = gi		[1.98852919667, 1.98558020126, 1.25086177191, -1.31451594415, -1.97973605279, -1.97177391622]
Local Getis-Ord Gi: W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]], statistic = gi_star	NEEDS_INPUT	geo_local_getis_ord_gi needs y (positive values at the n locations). Example: solve('geo_local_getis_ord_gi', y=[10, 12,
Local Getis-Ord Gi: y = [10, 12, 11, 2, 3, 2], W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]], statistic = gi_star, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Local Moran's I (LISA) and Moran scatterplot quadrants — `geo_local_morans_i`

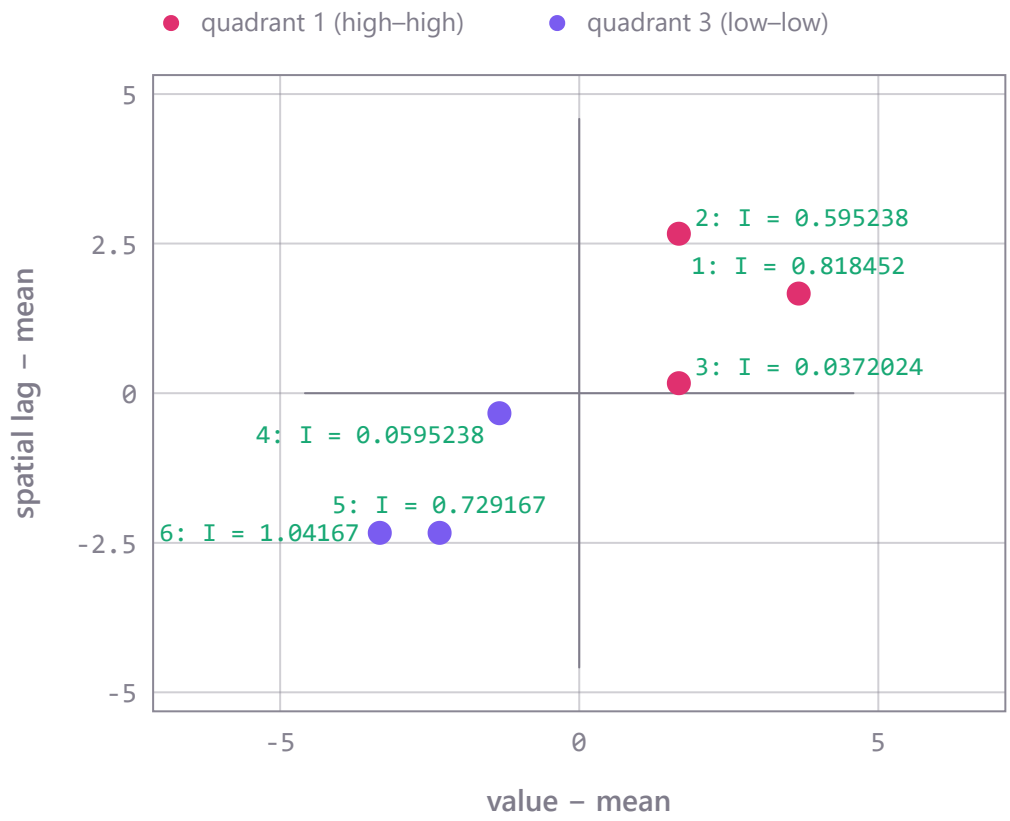
The local Moran statistic of every location with row-standardized weights, and its Moran scatterplot quadrant: 1 high-high, 2 low-high, 3 low-low, 4 high-low.

$$I_i = (n-1)\frac{z_i \sum_j w_{ij} z_j}{\sum_k z_k^2}$$

input	meaning	condition
`y`	values at the n locations	
`W`	spatial weights matrix (row-standardized internally)	

output	meaning
`Is`	local Moran's I of each location
`quadrant`	quadrant of each location (1 HH, 2 LH, 3 LL, 4 HL)

- **Method:** $I_i = (n-1)$ z_i sum_j w_ij z_j / sum_k z_k^2 with z = y - mean and row-standardized w (as esda)
- **Assumes:** $n \geq 4$, values not all equal; no observation without neighbours
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Local Moran's I for y = [8, 6, 6, 3, 2, 1] with the worked example's weights: each unit in the Moran scatter (deviation from the mean against its row-standardised lag, lags from the package's spatial-lag capability), coloured by the quadrant and labelled with the local I the package returns: unit 1 I = 0.818452 (quadrant 1), unit 2 I = 0.595238 (quadrant 1), unit 3 I = 0.0372024 (quadrant 1), unit 4 I = 0.0595238 (quadrant 3), unit 5 I = 0.729167 (quadrant 3), unit 6 I = 1.04167 (quadrant 3).

```
geospatial_analytics_ai999x.solve('geo_local_morans_i', y=[8, 6, 6, 3, 2, 1], W=[[0, 1, 0, 0, 0,
0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1,
0]])
# -> Is = [0.818452380952, 0.595238095238, 0.0372023809524, 0.0595238095238, 0.729166666667,
1.04166666667], quadrant = [1.0, 1.0, 1.0, 3.0, 3.0, 3.0]   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_local_morans_i', y=[10, 12, 11, 2, 3, 1], W=[[0, 1, 1, 0, 0,
0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1,
0]])
# -> Is = [0.697211155378, 0.876494023904, 0.268924302789, 0.268924302789, 0.697211155378,
0.876494023904], quadrant = [1.0, 1.0, 1.0, 3.0, 3.0, 3.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Local Moran's I: y = [8, 6, 6, 3, 2, 1], W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]]	SOLVED	Is = [0.818452380952, 0.595238095238, 0.0372023809524, 0.0595238095238, 0.729166666667, 1.04166666667], quadrant = [1.0, 1.0, 1.0, 3.0, 3.0, 3.0]
Local Moran's I: y = [10, 12, 11, 2, 3, 1], W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]]	SOLVED	Is = [0.697211155378, 0.876494023904, 0.268924302789, 0.268924302789, 0.697211155378, 0.876494023904], quadrant = [1.0, 1.0, 1.0, 3.0, 3.0, 3.0]
Local Moran's I: W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0,	NEEDS_INPUT	geo_local_morans_i needs y (values at the n locations). Example: solve('geo_local_morans_i', y=[8, 6, 6, 3, 2, 1],

question	outcome	result
1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]]		W=[[0
Local Moran's I: y = [8, 6, 6, 3, 2, 1], W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Bivariate Moran's I (spatial cross-correlation) — `geo_bivariate_morans_i`

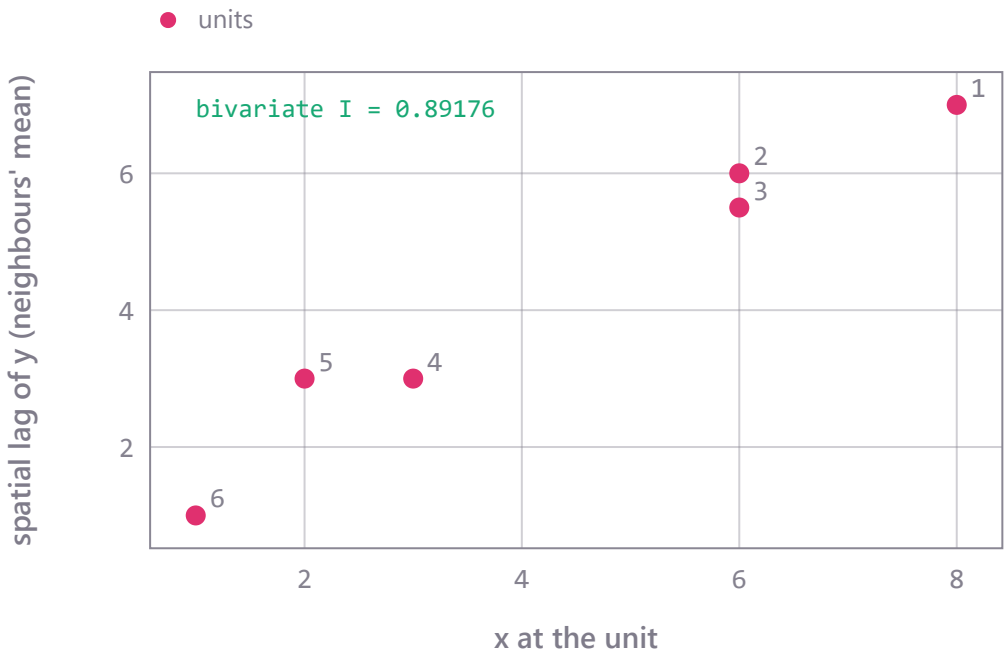
The bivariate Moran's I between a variable x and the spatial lag of a second variable y with row-standardized weights (how x at a location relates to y around it).

$$I_{xy} = \frac{\sum_i z_{x,i} \sum_j w_{ij} z_{y,j}}{n-1}$$

input	meaning	condition
`x`	first variable at the n locations	
`y`	second variable at the n locations	
`W`	spatial weights matrix (row-standardized internally)	

output	meaning
`I_xy`	bivariate Moran's I

- **Method:** $I_xy = zx$' W $zy/(n-1)$ with both variables standardized (sample standard deviation) and row-standardized W
- **Assumes:** $n \geq 4$, neither variable constant; no observation without neighbours
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Each unit's x = [8, 6, 6, 3, 2, 1] against the mean of y = [7, 7, 5, 4, 1, 2] over its neighbours (the row-standardised lag returned by the package's spatial-lag capability, with the worked example's weights): high x sits beside high y. The package returns the bivariate Moran's I = 0.89176.

```
geospatial_analytics_ai999x.solve('geo_bivariate_morans_i', x=[8, 6, 6, 3, 2, 1], y=[7, 7, 5, 4, 1, 2], W=[[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]])
# -> I_xy = 0.89176   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_bivariate_morans_i', x=[1, 2, 3, 4, 5, 6], y=[6, 5, 4, 3, 2, 1], W=[[0, 1, 0, 0, 0, 1], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [1, 0, 0, 0, 1, 0]])
# -> I_xy = -0.142857   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bivariate Moran's I: x = [8, 6, 6, 3, 2, 1], y = [7, 7, 5, 4, 1, 2], W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]]	SOLVED	I_xy = 0.89176
Bivariate Moran's I: x = [1, 2, 3, 4, 5, 6], y = [6, 5, 4, 3, 2, 1], W = [[0, 1, 0, 0, 0, 1], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [1, 0, 0, 0, 1, 0]]	SOLVED	I_xy = -0.142857
Bivariate Moran's I: y = [7, 7, 5, 4, 1, 2], W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]]	NEEDS_INPUT	geo_bivariate_morans_i needs x (first variable at the n locations). Example: solve('geo_bivariate_morans_i', x=[8, 6, 6,
Bivariate Moran's I: x = [8, 6, 6, 3, 2, 1], y = [7, 7, 5, 4, 1, 2], W = [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Join count statistics of a binary map (BB, BW, WW) — `geo_join_counts`

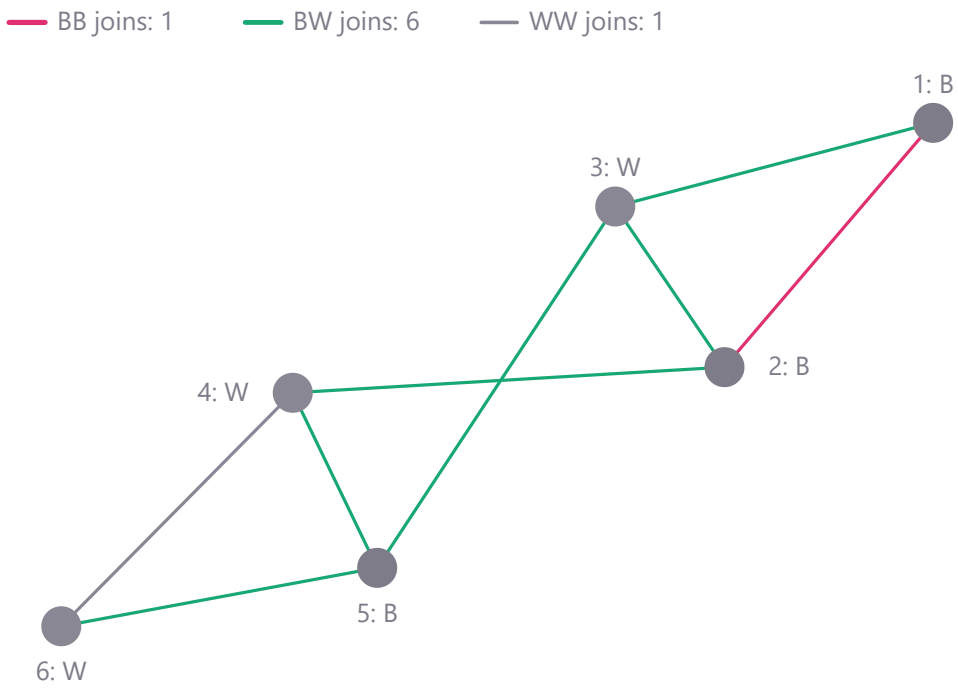
The numbers of black-black, black-white and white-white joins between neighbouring areas of a binary (presence/absence) map, and the total number of joins.

$$BB = \tfrac{1}{2}\sum_{i,j} w_{ij} y_i y_j,\; BW = \tfrac{1}{2}\sum_{i,j} w_{ij}(y_i - y_j)^2,\; WW = \tfrac{1}{2}\sum_{i,j} w_{ij}(1-y_i)(1-y_j)$$

input	meaning	condition
`y`	binary variable at the n areas (1 black, 0 white)	
`W`	binary symmetric contiguity matrix (0/1, zero diagonal)	

output	meaning
`BB`	black-black joins
`BW`	black-white joins
`WW`	white-white joins
`J`	total joins

- **Method:** count each neighbouring pair once by the colours of its two areas
- **Assumes:** binary symmetric weights
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Binary map y = [1, 1, 0, 0, 1, 0] (1 = black B, 0 = white W) on the links of the worked example's weights (layout without meaning); each link is coloured by the kind of join it is (classified here from the inputs). The package counts BB = 1, BW = 6, WW = 1 of J = 8 joins: many black–white joins, a dispersed pattern.

```
geospatial_analytics_ai999x.solve('geo_join_counts', y=[1, 1, 0, 0, 1, 0], W=[[0, 1, 1, 0, 0, 0],
[1, 0, 1, 1, 0, 0], [1, 1, 0, 0, 1, 0], [0, 1, 0, 0, 1, 1], [0, 0, 1, 1, 0, 1], [0, 0, 0, 1, 1,
0]])
# -> BB = 1, BW = 6, WW = 1, J = 8   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_join_counts', y=[1, 0, 1, 0], W=[[0, 1, 0, 1], [1, 0, 1, 0],
[0, 1, 0, 1], [1, 0, 1, 0]])
# -> BB = 0, BW = 4, WW = 0, J = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Join count statistics: y = [1, 1, 0, 0, 1, 0], W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 1, 0, 0], [1, 1, 0, 0, 1, 0], [0, 1, 0, 0, 1, 1], [0, 0, 1, 1, 0, 1], [0, 0, 0, 1, 1, 0]]	SOLVED	BB = 1, BW = 6, WW = 1, J = 8
Join count statistics: y = [1, 0, 1, 0], W = [[0, 1, 0, 1], [1, 0, 1, 0], [0, 1, 0, 1], [1, 0, 1, 0]]	SOLVED	BB = 0, BW = 4, WW = 0, J = 4
Join count statistics: W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 1, 0, 0], [1, 1, 0, 0, 1, 0], [0, 1, 0, 0, 1, 1], [0, 0, 1, 1, 0, 1], [0, 0, 0, 1, 1, 0]]	NEEDS_INPUT	geo_join_counts needs y (binary variable at the n areas (1 black, 0 white)). Example: solve('geo_join_counts', y=[1, 1,
Join count statistics: y = [1, 1, 0, 0, 1, 0], W = [[0, 1, 1, 0, 0, 0], [1, 0, 1, 1, 0, 0], [1, 1, 0, 0, 1, 0], [0, 1, 0, 0, 1, 1], [0, 0, 1, 1, 0, 1], [0, 0, 0, 1, 1, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Spatial lag of a variable — `geo_spatial_lag`

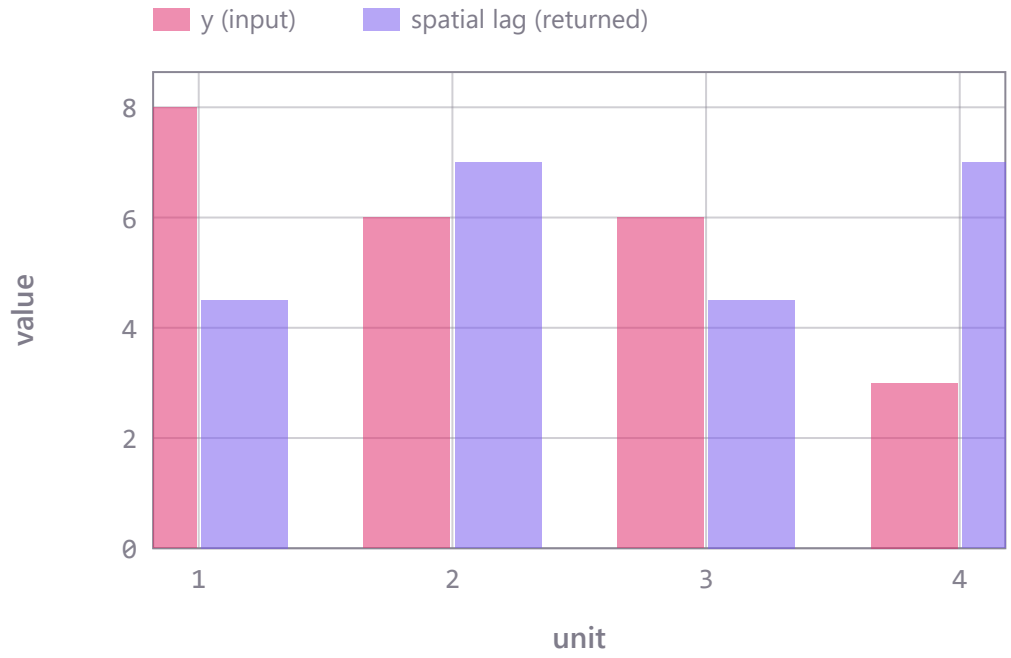
The spatial lag W y of a variable: for row-standardized weights, the average of each location's neighbours; otherwise the weighted sum.

$$[Wy]_i = \sum_j w_{ij} y_j$$

input	meaning	condition
`y`	values at the n locations	
`W`	spatial weights matrix (n x n, zero diagonal, row i lists the weights of i's neighbours)	
`transform`	row-standardize the weights (row) or use them as given (none)	

output	meaning
`lag`	spatial lag at each location

- **Method:** lag_i = sum_j w_ij y_j (weights row-standardized first if asked)
- **Assumes:** no observation without neighbours
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



For each unit, its value y = [8, 6, 6, 3] beside the spatial lag the package returns with the worked example's weights (row-standardised): the lag is the neighbours' mean, 4.5, 7, 4.5, 7, which smooths the map.

```
geospatial_analytics_ai999x.solve('geo_spatial_lag', y=[8, 6, 6, 3], W=[[0, 1, 0, 1], [1, 0, 1, 0],
[0, 1, 0, 1], [1, 0, 1, 0]], transform='row')
# -> lag = [4.5, 7.0, 4.5, 7.0]   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_spatial_lag', y=[1, 2, 3], W=[[0, 1, 1], [1, 0, 0], [1, 0,
0]], transform='none')
# -> lag = [5.0, 1.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Spatial lag: y = [8, 6, 6, 3], W = [[0, 1, 0, 1], [1, 0, 1, 0], [0, 1, 0, 1], [1, 0, 1, 0]], transform = row	SOLVED	lag = [4.5, 7.0, 4.5, 7.0]
Spatial lag: y = [1, 2, 3], W = [[0, 1, 1], [1, 0, 0], [1, 0, 0]], transform = none	SOLVED	lag = [5.0, 1.0, 1.0]
Spatial lag: W = [[0, 1, 0, 1], [1, 0, 1, 0], [0, 1, 0, 1], [1, 0, 1, 0]], transform = row	NEEDS_INPUT	geo_spatial_lag needs y (values at the n locations). Example: solve('geo_spatial_lag', y=[8, 6, 6, 3], W=[[0, 1, 0, 1],
Spatial lag: y = [8, 6, 6, 3], W = [[0, 1, 0, 1], [1, 0, 1, 0], [0, 1, 0, 1], [1, 0, 1, 0]], transform = row, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

k-nearest-neighbour spatial weights — `geo_knn_weights`

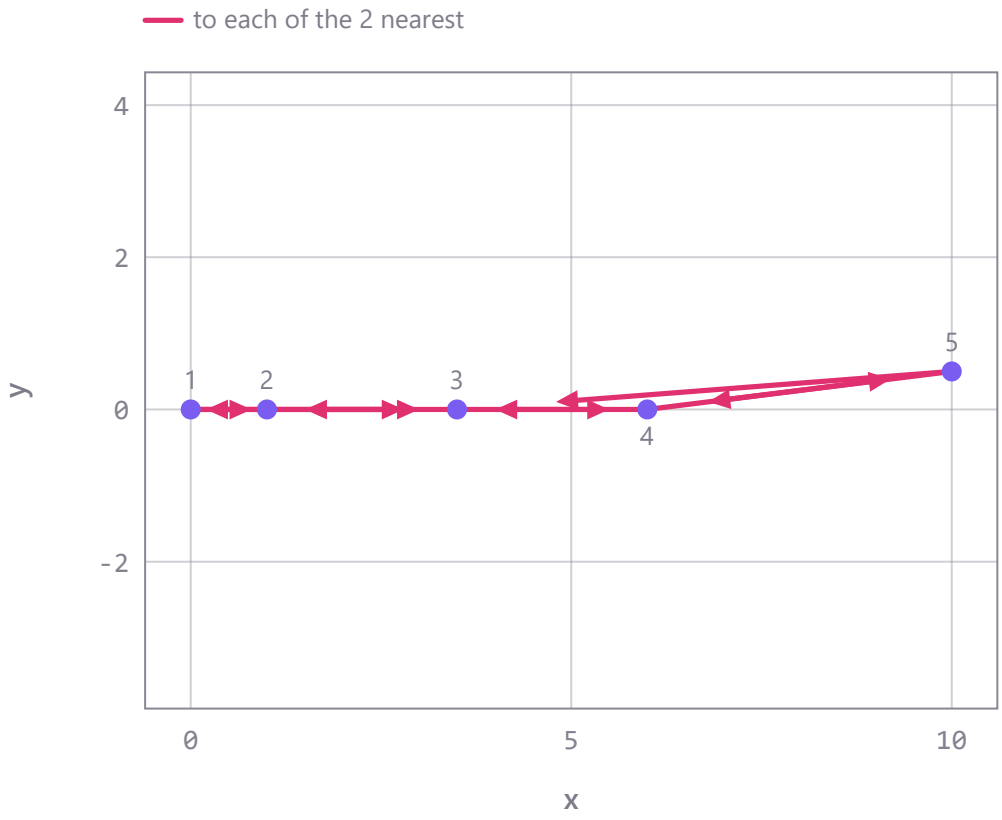
The binary k-nearest-neighbour weights matrix of a set of points (row i marks the k nearest points to point i), its total weight and whether it is symmetric.

$$w_{ij} = 1 \iff j \in N_k(i)$$

input	meaning	condition
`x`	x coordinates	
`y`	y coordinates	
`k`	number of neighbours	

output	meaning
`W`	weights matrix (n x n, 0/1)
`s0`	sum of all weights (n k)
`symmetric`	whether W is symmetric

- **Method:** sort each point's distances to the others and mark the k smallest
- **Assumes:** distinct points; $1 \leq k < n$, no tie at the k-th neighbour
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



k-nearest-neighbour weights (k = 2) for points x = [0, 1, 3.5, 6, 10], y = [0, 0, 0, 0, 0.5]: an arrow from each point to every neighbour the package's matrix gives it. The relation is not mutual (returned symmetric: False) — the far point 5 points to 3 and 4, but only 4 points back. S0 = 10.

```
geospatial_analytics_ai999x.solve('geo_knn_weights', x=[0, 1, 3.5, 6, 10], y=[0, 0, 0, 0, 0.5],
k=2)
# -> W = [[0.0, 1.0, 1.0, 0.0, 0.0], [1.0, 0.0, 1.0, 0.0, 0.0], [0.0, 1.0, 0.0, 1.0, 0.0], [0.0,
0.0, 1.0, 0.0, 1.0], [0.0, 0.0, 1.0, 1.0, 0.0]], s0 = 10, symmetric = False   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_knn_weights', x=[0, 2, 0, 2, 5], y=[0, 0, 2.2, 2.5, 1], k=1)
# -> W = [[0.0, 1.0, 0.0, 0.0, 0.0], [1.0, 0.0, 0.0, 0.0, 0.0], [0.0, 0.0, 0.0, 1.0, 0.0], [0.0,
0.0, 1.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0, 0.0]], s0 = 5, symmetric = False   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
k nearest neighbour weights: x = [0, 1, 3.5, 6, 10], y = [0, 0, 0, 0, 0.5], k = 2	SOLVED	W = [[0.0, 1.0, 1.0, 0.0, 0.0], [1.0, 0.0, 1.0, 0.0, 0.0], [0.0, 1.0, 0.0, 1.0, 0.0], [0.0, 0.0, 1.0, 0.0, 1.0], [0.0, 0.0, 1.0, 1.0, 0.0]], s0 = 10, symmetric = False
k nearest neighbour weights: x = [0, 2, 0, 2, 5], y = [0, 0, 2.2, 2.5, 1], k = 1	SOLVED	W = [[0.0, 1.0, 0.0, 0.0, 0.0], [1.0, 0.0, 0.0, 0.0, 0.0], [0.0, 0.0, 0.0, 1.0, 0.0], [0.0, 0.0, 1.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0, 0.0]], s0 = 5, symmetric = False
k nearest neighbour weights: y = [0, 0, 0, 0, 0.5], k = 2	NEEDS_INPUT	geo_knn_weights needs x (x coordinates). Example: solve('geo_knn_weights', x=[0, 1, 3.5, 6, 10], y=[0, 0, 0, 0, 0.5], k=
k nearest neighbour weights: x = [0, 1, 3.5, 6, 10], y = [0, 0, 0, 0, 0.5], k = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Distance-band spatial weights — `geo_distance_band_weights`

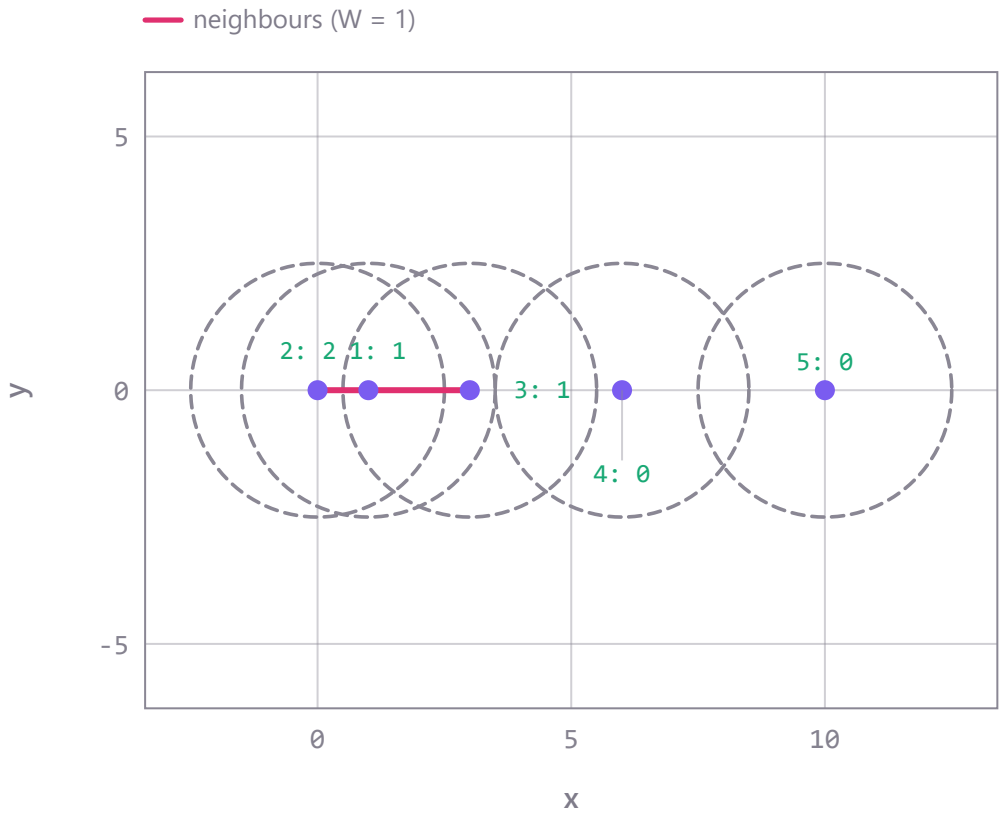
The binary distance-band weights matrix of a set of points (neighbours within a threshold distance), the number of neighbours of each point, the total weight and the number of islands.

$$w_{ij} = \begin{cases} 1 & d_{ij} \le \delta,\ i \ne j \\ 0 & \text{otherwise} \end{cases}$$

input	meaning	condition
`x`	x coordinates	
`y`	y coordinates	
`threshold`	distance threshold (coordinate units)	

output	meaning
`W`	weights matrix (n x n, 0/1)
`neighbours`	number of neighbours of each point
`s0`	sum of all weights
`islands`	number of points without neighbours

- **Method:** $w_ij = 1$ when 0 < d_ij <= threshold
- **Assumes:** threshold > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Points at x = [0, 1, 3, 6, 10], y = [0, 0, 0, 0, 0] with a distance band of 2.5 (dashed circles of that radius around each point): the links are the pairs the package's weights matrix joins; labels give each point's number of neighbours as returned. Returned: 2 islands without neighbours, S0 = 4.

```
geospatial_analytics_ai999x.solve('geo_distance_band_weights', x=[0, 1, 3, 6, 10], y=[0, 0, 0, 0, 0], threshold=2.5)
# -> W = [[0.0, 1.0, 0.0, 0.0, 0.0], [1.0, 0.0, 1.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0, 0.0], [0.0, 0.0, 0.0, 0.0, 0.0], [0.0, 0.0, 0.0, 0.0, 0.0]], neighbours = [1.0, 2.0, 1.0, 0.0, 0.0], s0 = 4, islands = 2   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_distance_band_weights', x=[0, 2, 0, 2, 5], y=[0, 0, 2, 2, 1], threshold=2.0001)
# -> W = [[0.0, 1.0, 1.0, 0.0, 0.0], [1.0, 0.0, 0.0, 1.0, 0.0], [1.0, 0.0, 0.0, 1.0, 0.0], [0.0, 1.0, 1.0, 0.0, 0.0], [0.0, 0.0, 0.0, 0.0, 0.0]], neighbours = [2.0, 2.0, 2.0, 2.0, 0.0], s0 = 8, islands = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Distance band weights: x = [0, 1, 3, 6, 10], y = [0, 0, 0, 0, 0], threshold = 2.5	SOLVED	W = [[0.0, 1.0, 0.0, 0.0, 0.0], [1.0, 0.0, 1.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0, 0.0], [0.0, 0.0, 0.0, 0.0, 0.0], [0.0, 0.0, 0.0, 0.0, 0.0]], neighbours = [1.0, 2.0, 1.0, 0.0, 0.0], s0 = 4, islands = 2
Distance band weights: x = [0, 2, 0, 2, 5], y = [0, 0, 2, 2, 1], threshold = 2.0001	SOLVED	W = [[0.0, 1.0, 1.0, 0.0, 0.0], [1.0, 0.0, 0.0, 1.0, 0.0], [1.0, 0.0, 0.0, 1.0, 0.0], [0.0, 1.0, 1.0, 0.0, 0.0], [0.0, 0.0, 0.0, 0.0, 0.0]], neighbours = [2.0, 2.0, 2.0, 2.0, 0.0], s0 = 8, islands = 1
Distance band weights: y = [0, 0, 0, 0, 0], threshold = 2.5	NEEDS_INPUT	geo_distance_band_weights needs x (x coordinates). Example: solve('geo_distance_band_weights', x=[0, 1, 3, 6, 10], y=[0,

question	outcome	result
Distance band weights: x = [0, 1, 3, 6, 10], y = [0, 0, 0, 0, 0], threshold = 2.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Nearest-neighbour index (Clark-Evans ratio) of a point pattern — `geo_nearest_neighbour_index`

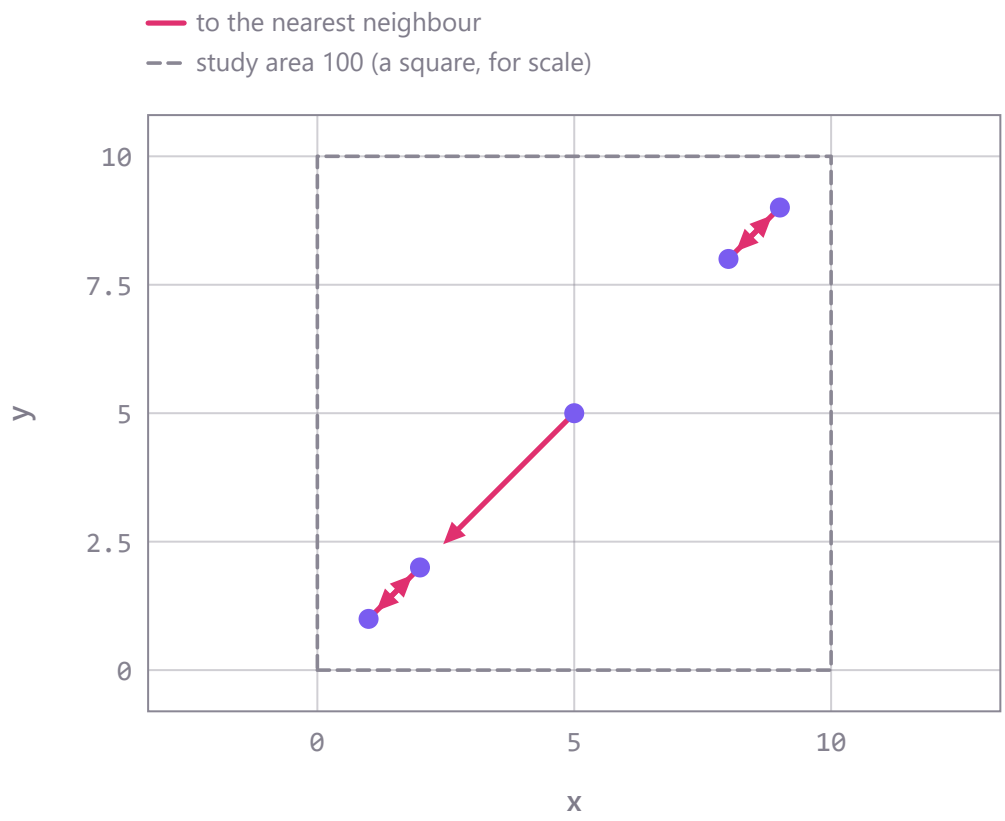
The mean observed nearest-neighbour distance of a point pattern, the mean expected under complete spatial randomness in a study area, their ratio R (R < 1 clustered, $R > 1$ dispersed), the z-score and two-sided p-value.

$$R = \frac{\bar{d}_{obs}}{0.5/\sqrt{n/A}}, \quad z = \frac{\bar{d}_{obs} - \bar{d}_{exp}}{0.26136/\sqrt{n^2/A}}$$

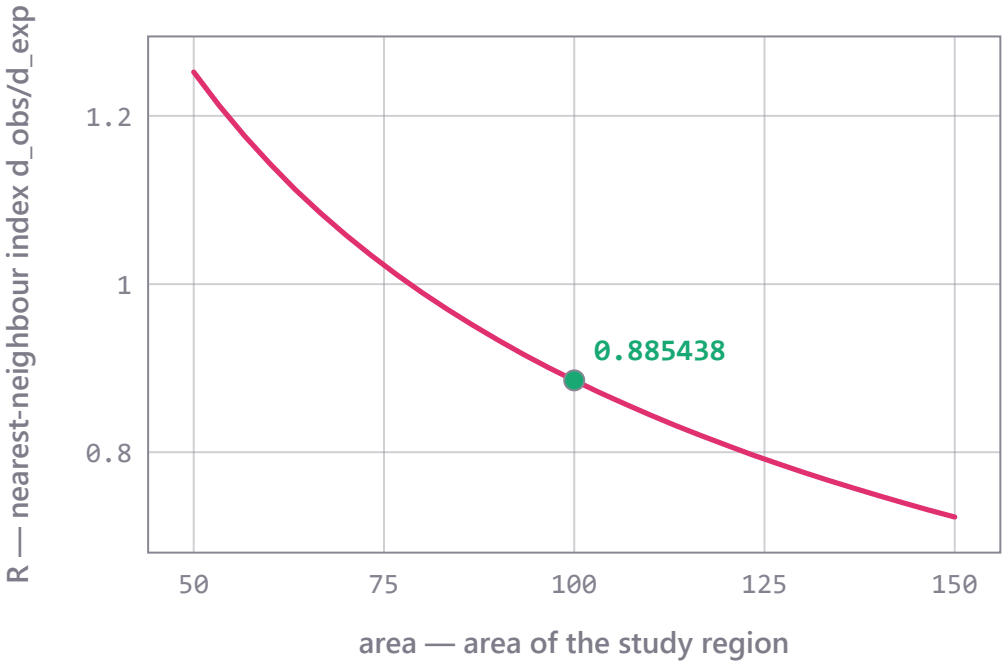
input	meaning	condition
`x`	x coordinates of the points	
`y`	y coordinates of the points	
`area`	area of the study region (square coordinate units)	

output	meaning
`d_obs`	mean observed nearest-neighbour distance
`d_exp`	expected mean distance $0.5/sqrt$(n/A)
`R`	nearest-neighbour index d_obs/d_exp
`z`	z-score
`p_value`	two-sided p-value

- **Method:** mean of each point's distance to its nearest neighbour; Clark-Evans expectation $0.5/sqrt$ (density) and standard error $0.26136/sqrt$(n^2/A)
- **Assumes:** distinct points; no edge correction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 5 points x = [1, 2, 8, 9, 5], y = [1, 2, 8, 9, 5] with an arrow to each one's nearest neighbour (found here from the inputs). The package returns the mean nearest-neighbour distance 1.9799 against 2.23607 expected for a random pattern in area 100: R = 0.885438 (below 1 leans to clustering), z = -0.490069, p = 0.624085.



R (nearest-neighbour index d_obs/d_exp) against area (area of the study region (square coordinate units)) from 50 to 150. Other inputs as in the worked example: x = [1, 2, 8, 9, 5], y = [1, 2, 8, 9, 5]. 31 points computed by geospatial-analytics-ai999x; marked: the worked example, area = 100, R = 0.885438.

```
geospatial_analytics_ai999x.solve('geo_nearest_neighbour_index', x=[1, 2, 8, 9, 5], y=[1, 2, 8, 9, 5], area=100)
# -> d_obs = 1.9799, d_exp = 2.23607, R = 0.885438, z = -0.490069, p_value = 0.624085   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_nearest_neighbour_index', x=[0, 10, 0, 10, 5], y=[0, 0, 10,
10, 5], area=100)
# -> d_obs = 7.07107, d_exp = 2.23607, R = 3.16228, z = 9.24969, p_value = 2.25137e-20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nearest neighbour index: x = [1, 2, 8, 9, 5], y = [1, 2, 8, 9, 5], area = 100	SOLVED	d_obs = 1.9799, d_exp = 2.23607, R = 0.885438, z = -0.490069, p_value = 0.624085
Nearest neighbour index: x = [0, 10, 0, 10, 5], y = [0, 0, 10, 10, 5], area = 100	SOLVED	d_obs = 7.07107, d_exp = 2.23607, R = 3.16228, z = 9.24969, p_value = 2.25137e-20
Nearest neighbour index: y = [1, 2, 8, 9, 5], area = 100	NEEDS_INPUT	geo_nearest_neighbour_index needs x (x coordinates of the points). Example: solve('geo_nearest_neighbour_index', x=[1, 2
Nearest neighbour index: x = [1, 2, 8, 9, 5], y = [1, 2, 8, 9, 5], area = 100, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Quadrat analysis: chi-square test of complete spatial randomness — `geo_quadrat_test`

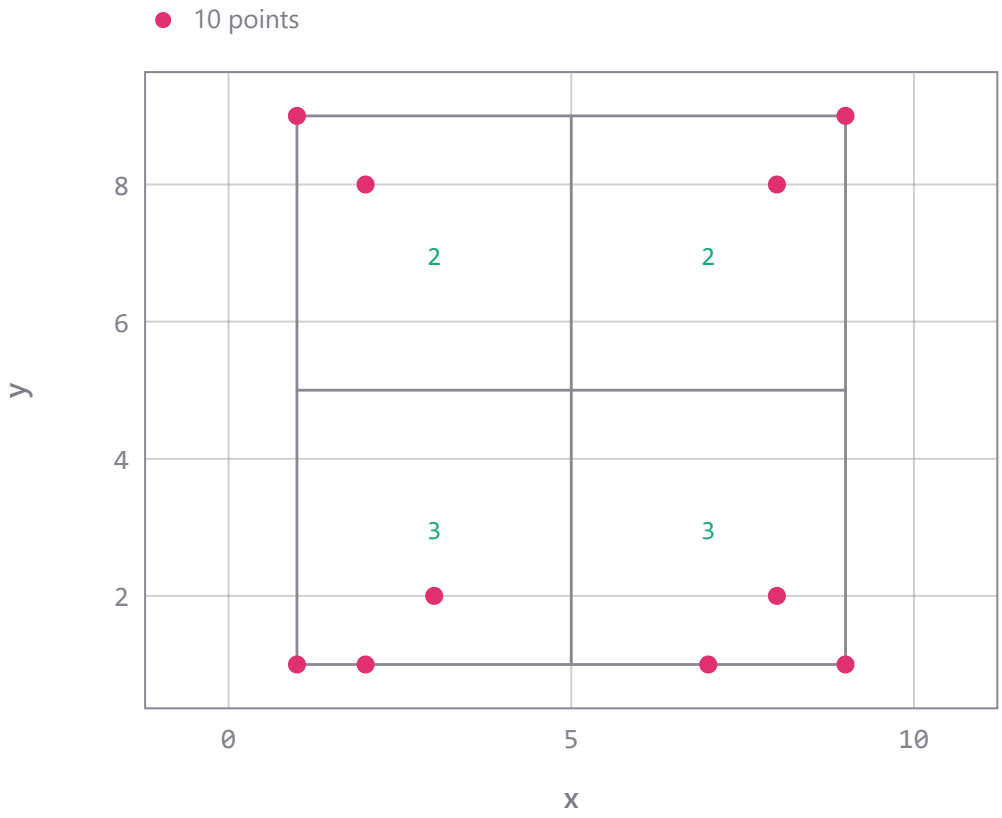
Counts of points in an nx x ny grid of quadrats over the bounding rectangle of a point pattern, the chi-square statistic against complete spatial randomness, its p-value and the variance-to-mean ratio.

$$\chi^2 = \sum_c \frac{(O_c - E)^2}{E}, \quad E = \frac{n}{n_x n_y}, \quad VMR = \frac{\chi^2}{n_x n_y - 1}$$

input	meaning	condition
`x`	x coordinates of the points	
`y`	y coordinates of the points	
`nx`	number of quadrat columns	
`ny`	number of quadrat rows	

output	meaning
`chi2`	chi-square statistic
`df`	degrees of freedom (cells - 1)
`p_value`	upper-tail p-value
`vmr`	variance-to-mean ratio of the counts $(chi2/df)$
`counts`	count of each quadrat, row by row from the south-west

- **Method:** equal-size cells over the bounding box; chi2 = sum $(O-E)^2/E$ with E = n/cells; p from the chi-square upper tail (regularized incomplete gamma)
- **Assumes:** the window is the bounding rectangle of the points; cells include their west and south edges; the last column and row include the east and north edges
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 10 points of the worked example in a 2 × 2 grid of quadrats over their extent, each quadrat labelled with the count the package returns (counts listed row by row from the bottom left: 3, 3, 2, 2). χ^2 = 0.4 on 3 degrees of freedom, p = 0.940242; variance-to-mean ratio 0.133333 (below 1: more even than random).

```
geospatial_analytics_ai999x.solve('geo_quadrat_test', x=[1, 2, 3, 8, 9, 2, 1, 7, 8, 9], y=[1, 1, 2, 8, 9, 8, 9, 1, 2, 1], nx=2, ny=2)
# -> chi2 = 0.4, df = 3, p_value = 0.940242, vmr = 0.133333, counts = [3.0, 3.0, 2.0, 2.0]  (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_quadrat_test', x=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9], y=[0, 5, 1, 6, 2, 7, 3, 8, 4, 9], nx=3, ny=1)
# -> chi2 = 0.2, df = 2, p_value = 0.904837, vmr = 0.1, counts = [3.0, 3.0, 4.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Quadrat test: x = [1, 2, 3, 8, 9, 2, 1, 7, 8, 9], y = [1, 1, 2, 8, 9, 8, 9, 1, 2, 1], nx = 2, ny = 2	SOLVED	chi2 = 0.4, df = 3, p_value = 0.940242, vmr = 0.133333, counts = [3.0, 3.0, 2.0, 2.0]
Quadrat test: x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], y = [0, 5, 1, 6, 2, 7, 3, 8, 4, 9], nx = 3, ny = 1	SOLVED	chi2 = 0.2, df = 2, p_value = 0.904837, vmr = 0.1, counts = [3.0, 3.0, 4.0]
Quadrat test: y = [1, 1, 2, 8, 9, 8, 9, 1, 2, 1], nx = 2, ny = 2	NEEDS_INPUT	geo_quadrat_test needs x (x coordinates of the points). Example: solve('geo_quadrat_test', x=[1, 2, 3, 8, 9, 2, 1, 7, 8,
Quadrat test: x = [1, 2, 3, 8, 9, 2, 1, 7, 8, 9], y = [1, 1, 2, 8, 9, 8, 9, 1, 2, 1], nx = 2, ny = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Ripley's K and L functions of a point pattern — `geo_ripley_k`

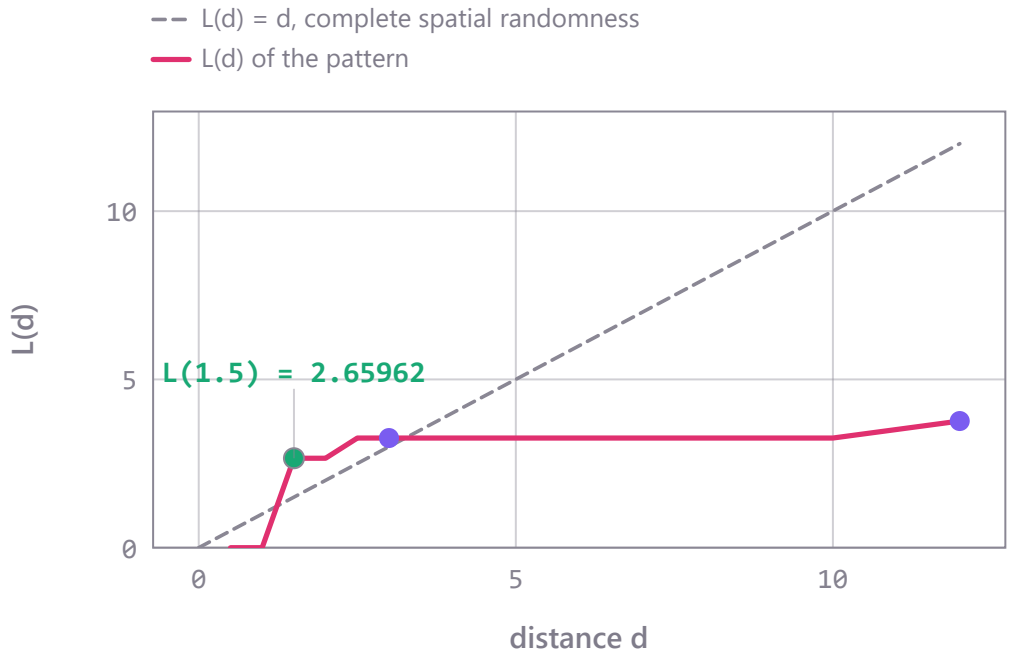
Ripley's K function and its variance-stabilized L function at given distances for a point pattern, without edge correction, in the bounding rectangle of the points.

$$\hat{K}(d) = \frac{A}{n^2} \sum_{i \neq j} \mathbf{1}(d_{ij} < d), \quad L(d) = \sqrt{K(d)/\pi}$$

input	meaning	condition
`x`	x coordinates of the points	
`y`	y coordinates of the points	
`d`	distances at which to evaluate K	

output	meaning
`K`	K(d) at each distance
`L`	L(d) = sqrt(K/pi) at each distance
`area`	area of the bounding rectangle

- **Method:** K(d) = A * (number of ordered pairs closer than d) / n^2 with A the bounding-rectangle area (as pointpats)
- **Assumes:** window = bounding rectangle of the points; pairs closer than d (strictly) are counted; no edge correction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ripley's L function of the six points x = [0, 1, 2, 8, 9, 10], y = [0, 1, 0, 9, 10, 9] (study area 100), returned by the package at distances up to 12; above the diagonal L = d the points are closer together than random (two tight groups). Dots: the worked example's distances L(1.5) = 2.65962, L(3) = 3.25735, L(12) = 3.76126.

```
geospatial_analytics_ai999x.solve('geo_ripley_k', x=[0, 1, 2, 8, 9, 10], y=[0, 1, 0, 9, 10, 9], d=
[1.5, 3, 12])
# -> K = [22.2222222222, 33.3333333333, 44.4444444444], L = [2.65961520268, 3.25735007935,
3.76126389032], area = 100   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_ripley_k', x=[0, 10, 0, 10, 5], y=[0, 0, 10, 10, 5], d=[5,
8])
# -> K = [0.0, 32.0], L = [0.0, 3.19153824321], area = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ripley's K: x = [0, 1, 2, 8, 9, 10], y = [0, 1, 0, 9, 10, 9], d = [1.5, 3, 12]	SOLVED	K = [22.2222222222, 33.3333333333, 44.4444444444], L = [2.65961520268, 3.25735007935, 3.76126389032], area = 100
Ripley's K: x = [0, 10, 0, 10, 5], y = [0, 0, 10, 10, 5], d = [5, 8]	SOLVED	K = [0.0, 32.0], L = [0.0, 3.19153824321], area = 100
Ripley's K: y = [0, 1, 0, 9, 10, 9], d = [1.5, 3, 12]	NEEDS_INPUT	geo_ripley_k needs x (x coordinates of the points). Example: solve('geo_ripley_k', x=[0, 1, 2, 8, 9, 10], y=[0, 1, 0, 9,
Ripley's K: x = [0, 1, 2, 8, 9, 10], y = [0, 1, 0, 9, 10, 9], d = [1.5, 3, 12], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Mean centre and standard distance of a point pattern — `geo_mean_center_standard_distance`

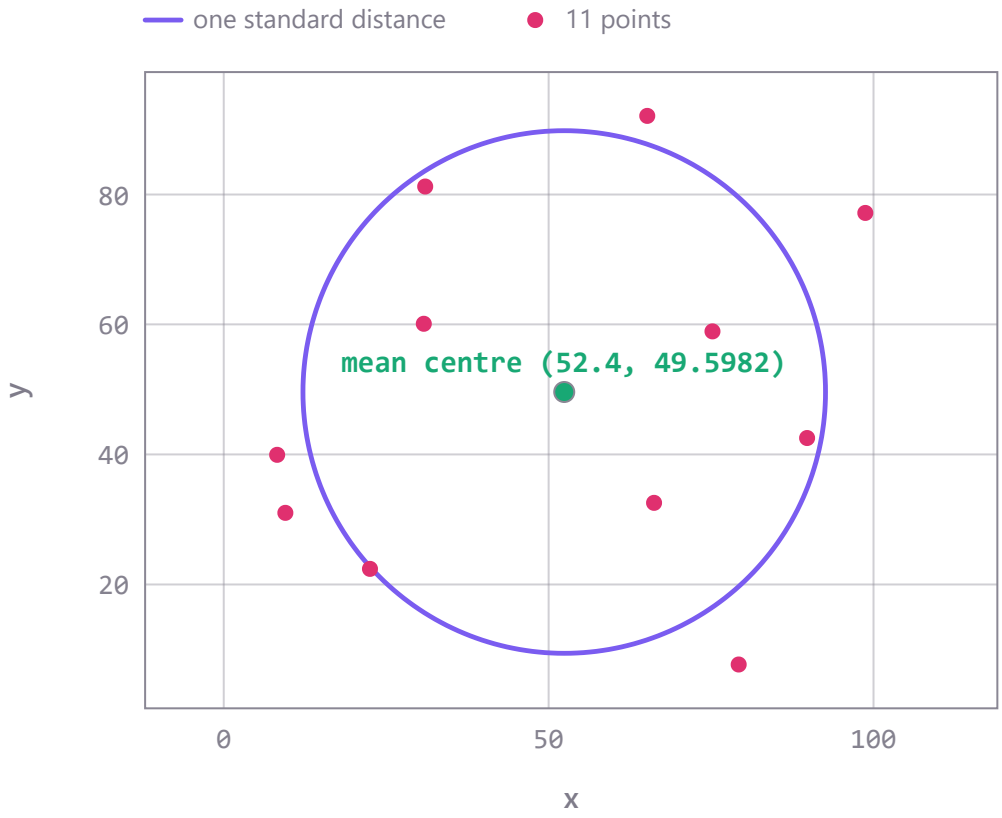
The mean centre of a set of points and the standard distance (root-mean-square distance from the mean centre).

$$SD = \sqrt{\frac{\sum_i (x_i - \bar{x})^2 + (y_i - \bar{y})^2}{n}}$$

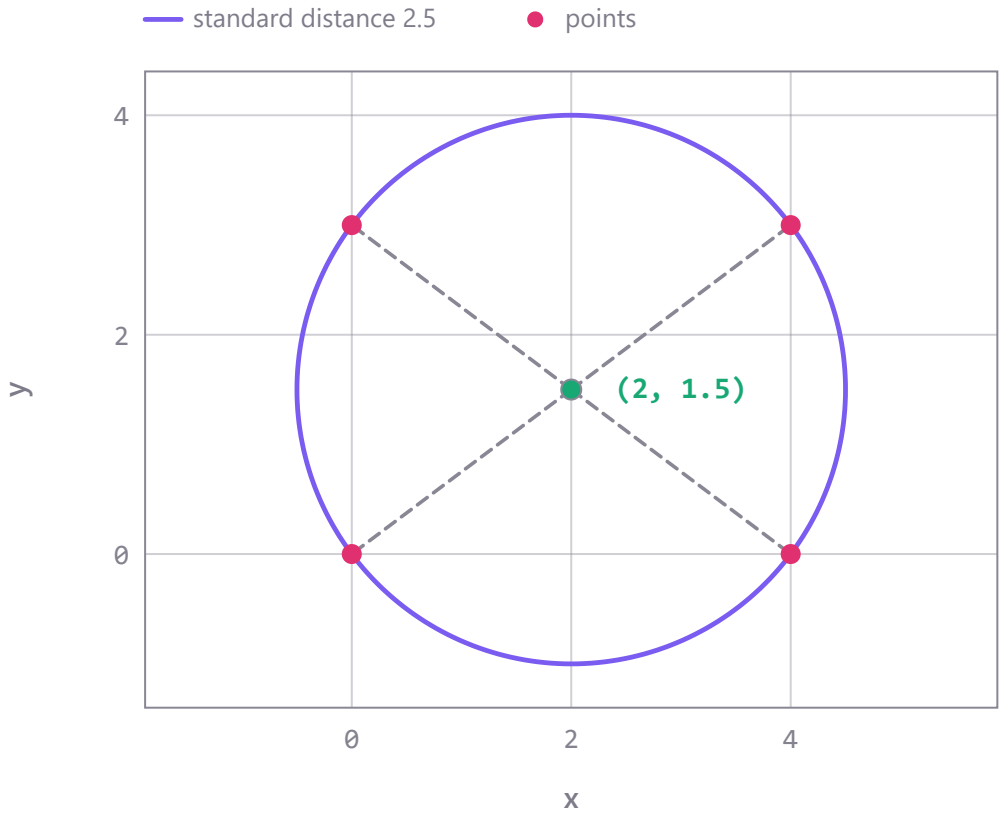
input	meaning	condition
`x`	x coordinates	
`y`	y coordinates	

output	meaning
`mean_x`	x of the mean centre
`mean_y`	y of the mean centre
`std_distance`	standard distance

- **Method:** mean of the coordinates; standard distance sqrt(sum ((x - xbar)^2 + (y - ybar)^2)/n)
- **Assumes:** population form (divide by n)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A pattern of 11 points with the mean centre and standard distance computed by the package: marked: x = 52.4, y = 49.5982; the circle has radius 40.2158, the standard distance (root-mean-square distance of the points from the centre).



The worked example's points x = [0, 4, 4, 0], y = [0, 0, 3, 3], their mean centre (2, 1.5) and a circle of the returned standard distance 2.5 — here every corner of the rectangle lies on it, since all four are equally far from the centre.

```
geospatial_analytics_ai999x.solve('geo_mean_center_standard_distance', x=[0, 4, 4, 0], y=[0, 0, 3, 3])
# -> mean_x = 2, mean_y = 1.5, std_distance = 2.5   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_mean_center_standard_distance', x=[66.22, 22.52, 31.01, 9.47, 30.78, 75.21, 79.26, 8.23, 98.73, 89.78, 65.19], y=[32.54, 22.39, 81.21, 31.02, 60.1, 58.93, 7.68, 39.93, 77.17, 42.53, 92.08])
# -> mean_x = 52.4, mean_y = 49.5982, std_distance = 40.2158   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mean center and standard distance: x = [0, 4, 4, 0], y = [0, 0, 3, 3]	SOLVED	mean_x = 2, mean_y = 1.5, std_distance = 2.5
Mean center and standard distance: x = [66.22, 22.52, 31.01, 9.47, 30.78, 75.21, 79.26, 8.23, 98.73, 89.78, 65.19], y = [32.54, 22.39, 81.21, 31.02, 60.1, 58.93, 7.68, 39.93, 77.17, 42.53, 92.08]	SOLVED	mean_x = 52.4, mean_y = 49.5982, std_distance = 40.2158
Mean center and standard distance: y = [0, 0, 3, 3]	NEEDS_INPUT	geo_mean_center_standard_distance needs x (x coordinates). Example: solve('geo_mean_center_standard_distance', x=[0, 4,
Mean center and standard distance: x = [0, 4, 4, 0], y = [0, 0, 3, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Weighted mean centre and weighted standard distance — `geo_weighted_mean_center`

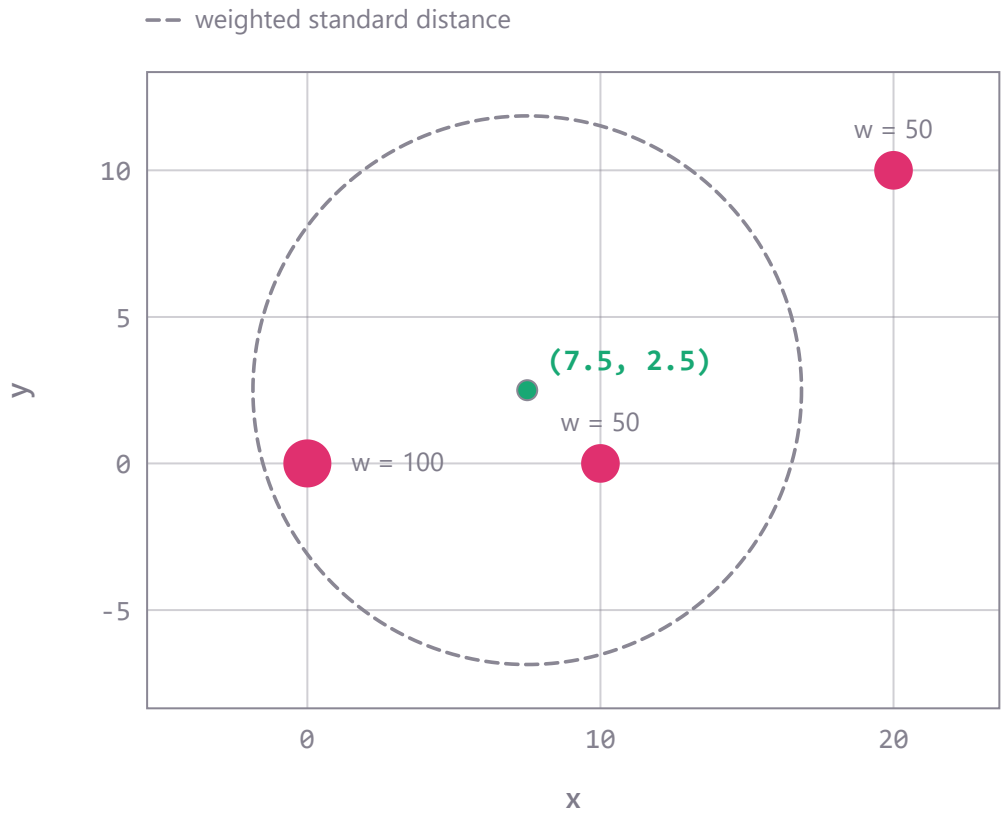
The weighted mean centre of a set of points (e.g. population-weighted centroid) and the weighted standard distance about it.

$$\bar{x}_w = \frac{\sum w_i x_i}{\sum w_i}, \quad SD_w = \sqrt{\frac{\sum w_i \left[(x_i - \bar{x}_w)^2 + (y_i - \bar{y}_w)^2\right]}{\sum w_i}}$$

input	meaning	condition
`x`	x coordinates	
`y`	y coordinates	
`w`	non-negative weights	

output	meaning
`wmean_x`	x of the weighted mean centre
`wmean_y`	y of the weighted mean centre
`weighted_std_distance`	weighted standard distance

- **Method:** weighted means of the coordinates; sqrt(sum w d^2 / sum w)
- **Assumes:** non-negative weights, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Points ([0, 10, 20]; [0, 0, 10]) with weights [100, 50, 50] (dot area grows with weight) and the weighted mean centre the package returns, (7.5, 2.5), pulled toward the heavy point; the circle has the returned weighted standard distance 9.35414 as radius.

```
geospatial_analytics_ai999x.solve('geo_weighted_mean_center', x=[0, 10, 20], y=[0, 0, 10], w=[100, 50, 50])
# -> wmean_x = 7.5, wmean_y = 2.5, weighted_std_distance = 9.35414   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_weighted_mean_center', x=[1, 2, 3, 4], y=[4, 3, 2, 1], w=[1, 2, 3, 4])
# -> wmean_x = 3, wmean_y = 2, weighted_std_distance = 1.41421   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weighted mean center: x = [0, 10, 20], y = [0, 0, 10], w = [100, 50, 50]	SOLVED	wmean_x = 7.5, wmean_y = 2.5, weighted_std_distance = 9.35414
Weighted mean center: x = [1, 2, 3, 4], y = [4, 3, 2, 1], w = [1, 2, 3, 4]	SOLVED	wmean_x = 3, wmean_y = 2, weighted_std_distance = 1.41421
Weighted mean center: y = [0, 0, 10], w = [100, 50, 50]	NEEDS_INPUT	geo_weighted_mean_center needs x (x coordinates). Example: solve('geo_weighted_mean_center', x=[0, 10, 20], y=[0, 0, 10]
Weighted mean center: x = [0, 10, 20], y = [0, 0, 10], w = [100, 50, 50], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Standard deviational ellipse of a point pattern — `geo_standard_deviational_ellipse`

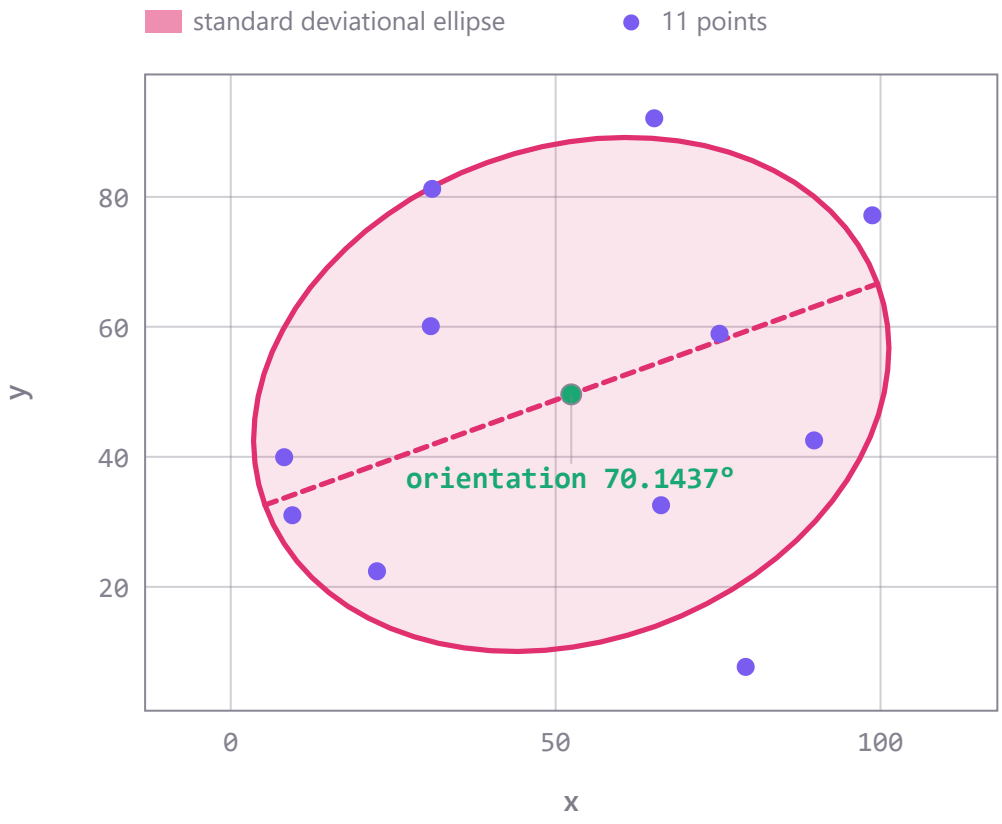
The semi-axes of the standard deviational ellipse of a point pattern (CrimeStat correction sqrt(2) sqrt($n/(n-2)$)) and the orientation of its major axis.

$$s_{x,y} = \sqrt{\frac{2n}{n-2}\lambda_{1,2}}, \quad \lambda_{1,2} = \frac{s_{xx}+s_{yy}}{2} \pm \sqrt{\frac{(s_{xx}-s_{yy})^2}{4} + s_{xy}^2}$$

input	meaning	condition
`x`	x coordinates	
`y`	y coordinates	

output	meaning
`sx`	semi-major axis
`sy`	semi-minor axis
`orientation`	bearing of the major axis, degrees clockwise from north (y axis), in [0, 180)

- **Method:** eigenvalues of the $(1/n)$ covariance of the coordinates give the variances along the principal axes; axes = sqrt(2 $n/(n-2)$ lambda) (CrimeStat); orientation from the major eigenvector
- **Assumes:** $n \geq 3$, points neither collinear nor circularly symmetric
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The standard deviational ellipse of 11 points, centred on their mean centre (52.4, 49.5982) from the package's mean-centre capability, with the returned semi-axes sx = 50.1303 (along the dashed axis, at 70.1437° clockwise from north) and sy = 37.9522.

```
geospatial_analytics_ai999x.solve('geo_standard_deviational_ellipse', x=[66.22, 22.52, 31.01, 9.47, 30.78, 75.21, 79.26, 8.23, 98.73, 89.78, 65.19], y=[32.54, 22.39, 81.21, 31.02, 60.1, 58.93, 7.68, 39.93, 77.17, 42.53, 92.08])
# -> sx = 50.1303, sy = 37.9522, orientation = 70.1437   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_standard_deviational_ellipse', x=[0, 1, 2, 3, 4, 5], y=[0, 1.5, 1.8, 3.2, 3.9, 5.5])
# -> sx = 4.26288, sy = 0.324625, orientation = 43.7668   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Standard deviational ellipse: x = [66.22, 22.52, 31.01, 9.47, 30.78, 75.21, 79.26, 8.23, 98.73, 89.78, 65.19], y = [32.54, 22.39, 81.21, 31.02, 60.1, 58.93, 7.68, 39.93, 77.17, 42.53, 92.08]	SOLVED	sx = 50.1303, sy = 37.9522, orientation = 70.1437
Standard deviational ellipse: x = [0, 1, 2, 3, 4, 5], y = [0, 1.5, 1.8, 3.2, 3.9, 5.5]	SOLVED	sx = 4.26288, sy = 0.324625, orientation = 43.7668
Standard deviational ellipse: y = [32.54, 22.39, 81.21, 31.02, 60.1, 58.93, 7.68, 39.93, 77.17, 42.53, 92.08]	NEEDS_INPUT	geo_standard_deviational_ellipse needs x (x coordinates). Example: solve('geo_standard_deviational_ellipse', x=[66.22, 2
Standard deviational ellipse: x = [66.22, 22.52, 31.01, 9.47, 30.78, 75.21, 79.26, 8.23, 98.73, 89.78, 65.19], y = [32.54, 22.39, 81.21, 31.02, 60.1, 58.93, 7.68, 39.93, 77.17, 42.53, 92.08], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Kernel density estimate of a point pattern at a location — `geo_kernel_density`

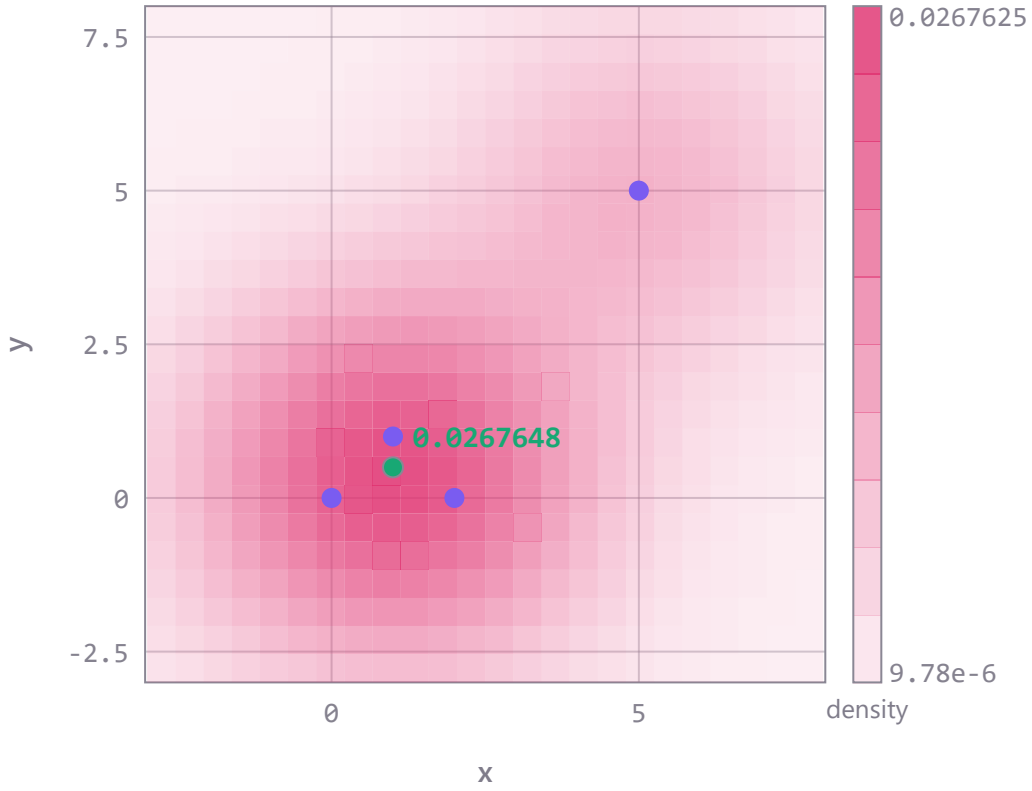
The two-dimensional kernel density (per unit area, integrating to 1) and the intensity (events per unit area) of a point pattern at a location, with a stated kernel and bandwidth.

$$\hat{f}(\mathbf{p}) = \frac{1}{n}\sum_i K_h(\|\mathbf{p}-\mathbf{x}_i\|), \quad K_h^{gauss} = \frac{e^{-u^2/2}}{2\pi h^2},\ K_h^{epa} = \frac{2(1-u^2)}{\pi h^2},\ K_h^{unif} = \frac{1}{\pi h^2}$$

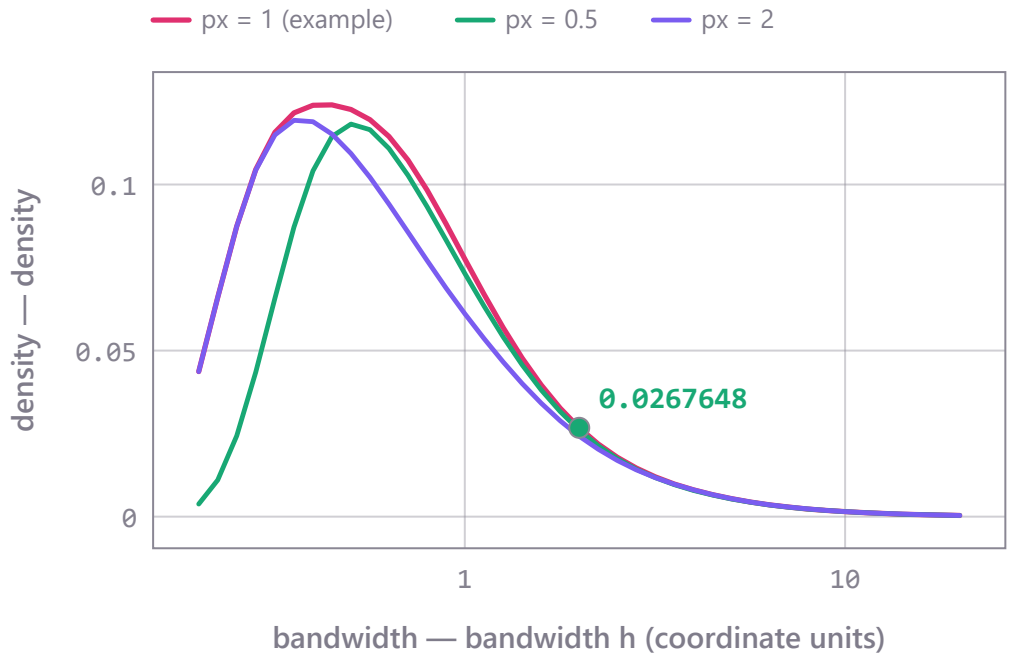
input	meaning	condition
`x`	x coordinates of the events	
`y`	y coordinates of the events	
`px`	x of the location	
`py`	y of the location	
`bandwidth`	bandwidth h (coordinate units)	
`kernel`	kernel (gaussian with sd h; epanechnikov and uniform with radius h)	

output	meaning
`density`	density (per square coordinate unit, integrates to 1)
`intensity`	intensity = n x density (events per square unit)

- **Method:** f(p) = (1/n) sum K_h(|p - x_i|) with $2 - D$ normalized kernels: gaussian exp($-u^2/2$)/(2 pi h^2), epanechnikov 2($1 - u^2$)/(pi h^2), uniform 1/(pi h^2) for $u = d/h < 1$
- **Assumes:** bandwidth > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Kernel density of four events (x = [0, 1, 2, 5], y = [0, 1, 0, 5]) with a gaussian kernel of bandwidth 2: each of 576 cells is shaded by the density the package returns at its centre. Marked: the worked example at (1, 0.5), density 0.0267648 (intensity 0.107059 events per unit area).



density (density (per square coordinate unit, integrates to 1)) against bandwidth (bandwidth h (coordinate units)) from 0.2 to 20 on a logarithmic axis. One curve for each value of px (x of the location): 1, 0.5, 2. Other inputs as in the worked example: x = [0, 1, 2, 5], y = [0, 1, 0, 5], py = 0.5, kernel = gaussian. 41 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, bandwidth = 2, density = 0.0267648.

```
geospatial_analytics_ai999x.solve('geo_kernel_density', x=[0, 1, 2, 5], y=[0, 1, 0, 5], px=1,
py=0.5, bandwidth=2, kernel='gaussian')
# -> density = 0.0267648, intensity = 0.107059   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_kernel_density', x=[0, 1, 2, 5], y=[0, 1, 0, 5], px=1,
py=0.5, bandwidth=2, kernel='epanechnikov')
# -> density = 0.0920115, intensity = 0.368046   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kernel density: x = [0, 1, 2, 5], y = [0, 1, 0, 5], px = 1, py = 0.5, bandwidth = 2, kernel = gaussian	SOLVED	density = 0.0267648, intensity = 0.107059
Kernel density: x = [0, 1, 2, 5], y = [0, 1, 0, 5], px = 1, py = 0.5, bandwidth = 2, kernel = epanechnikov	SOLVED	density = 0.0920115, intensity = 0.368046
Kernel density: y = [0, 1, 0, 5], px = 1, py = 0.5, bandwidth = 2, kernel = gaussian	NEEDS_INPUT	geo_kernel_density needs x (x coordinates of the events). Example: solve('geo_kernel_density', x=[0, 1, 2, 5], y=[0, 1,
Kernel density: x = [0, 1, 2, 5], y = [0, 1, 0, 5], px = 1, py = 0.5, bandwidth = 2, kernel = gaussian, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Inverse distance weighted (IDW) interpolation at a point — `geo_idw_interpolation`

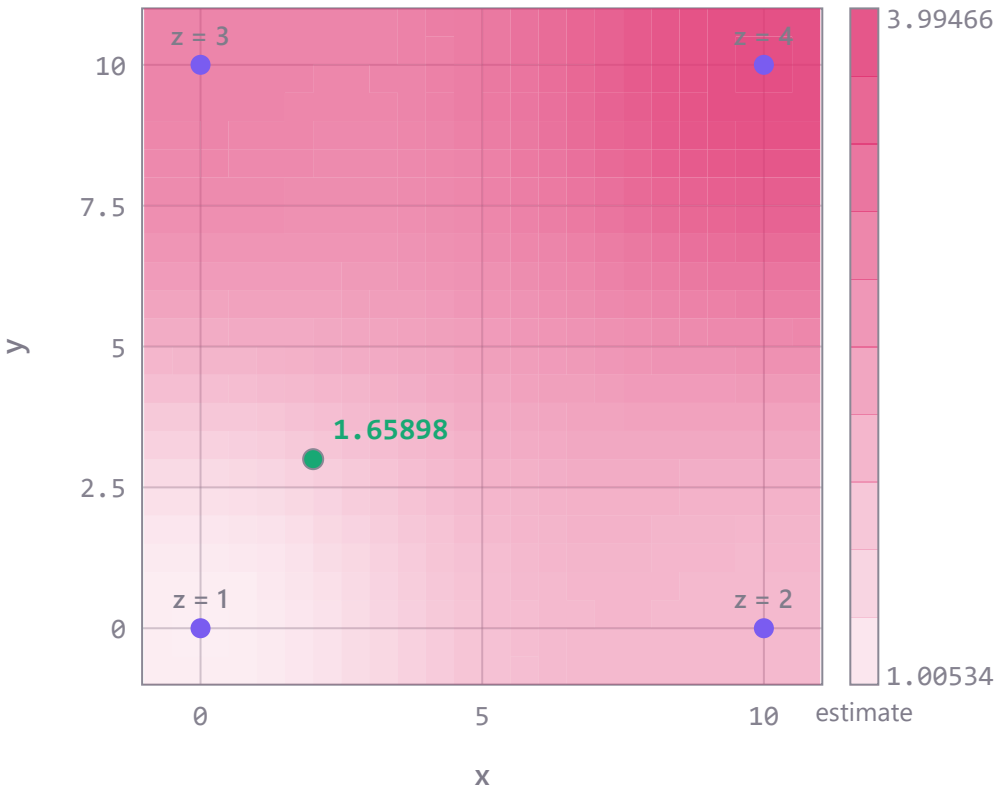
The inverse-distance-weighted (Shepard) estimate of a variable at a location from sample points, with a stated power (2 is the usual choice).

$$\hat{z}(\mathbf{p}) = \frac{\sum_i d_i^{-p} z_i}{\sum_i d_i^{-p}}$$

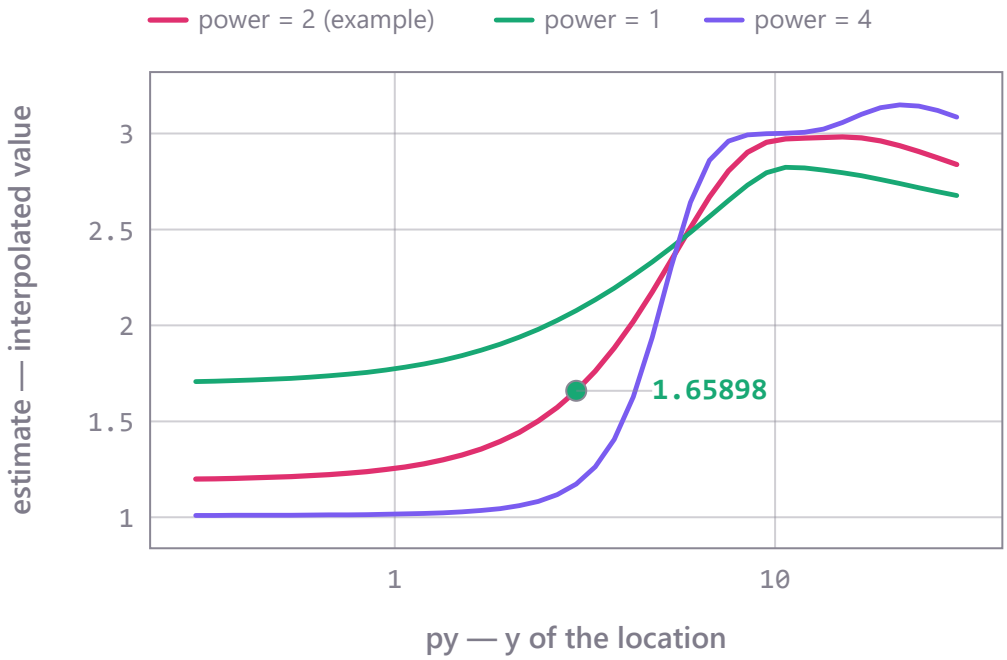
input	meaning	condition
`x`	x coordinates of the samples	
`y`	y coordinates of the samples	
`z`	sample values	
`px`	x of the location	
`py`	y of the location	
`power`	distance power p (> 0)	

output	meaning
`estimate`	interpolated value

- **Method:** z = sum d_i^{-p} z_i / sum d_i^{-p} over all samples
- **Assumes:** power > 0 (at most 10); a location on a sample returns that sample's value
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Inverse-distance-weighted surface (power 2) from four samples z = [1, 2, 3, 4] at the corners (x = [0, 10, 0, 10], y = [0, 0, 10, 10]): each of the 576 cells is shaded by the estimate the package returns at its centre. Marked: the worked example at (2, 3), estimate 1.65898.



estimate (interpolated value) against py (y of the location) from 0.3 to 30 on a logarithmic axis. One curve for each value of power (distance power p (> 0)): 2, 1, 4. Other inputs as in the worked example: x = [0, 10, 0, 10], y = [0, 0, 10, 10], z = [1, 2, 3, 4], px = 2. 41 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, py = 3, estimate = 1.65898.

```
geospatial_analytics_ai999x.solve('geo_idw_interpolation', x=[0, 10, 0, 10], y=[0, 0, 10, 10], z=
[1, 2, 3, 4], px=2, py=3, power=2)
# -> estimate = 1.65898   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_idw_interpolation', x=[0, 4, 8], y=[0, 3, 0], z=[10, 20,
15], px=4, py=1, power=1)
# -> estimate = 16.3068   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
IDW interpolation: x = [0, 10, 0, 10], y = [0, 0, 10, 10], z = [1, 2, 3, 4], px = 2, py = 3, power = 2	SOLVED	estimate = 1.65898
IDW interpolation: x = [0, 4, 8], y = [0, 3, 0], z = [10, 20, 15], px = 4, py = 1, power = 1	SOLVED	estimate = 16.3068
IDW interpolation: y = [0, 0, 10, 10], z = [1, 2, 3, 4], px = 2, py = 3, power = 2	NEEDS_INPUT	geo_idw_interpolation needs x (x coordinates of the samples). Example: solve('geo_idw_interpolation', x=[0, 10, 0, 10],
IDW interpolation: x = [0, 10, 0, 10], y = [0, 0, 10, 10], z = [1, 2, 3, 4], px = 2, py = 3, power = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Empirical (Matheron) semivariogram in distance bins — `geo_empirical_semivariogram`

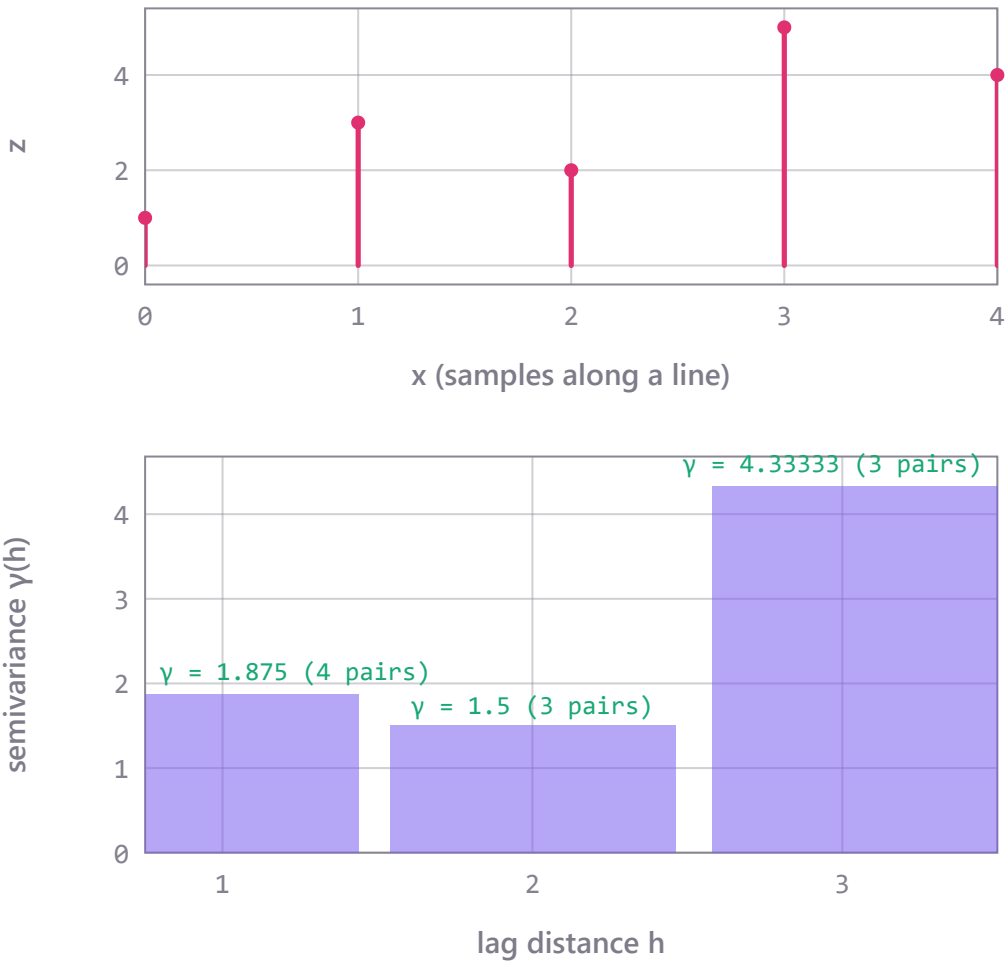
The classical (Matheron) empirical semivariogram of sampled values: half the mean squared difference of all pairs whose separation falls in each distance bin, and the pair counts.

$$\hat{\gamma}(h_k) = \frac{1}{2N_k} \sum_{(i,j)\in k} (z_i - z_j)^2$$

input	meaning	condition
`x`	x coordinates of the samples	
`y`	y coordinates of the samples	
`z`	sample values	
`bin_edges`	increasing distance bin edges (a bin includes its lower edge)	

output	meaning
`gamma`	semivariance of each bin
`counts`	number of pairs in each bin

- **Method:** gamma(h_k) = 1/(2 N_k) sum over pairs in bin k of $(z_i - z_j)^2$
- **Assumes:** every bin contains at least one pair; bins [e_k, e_k+1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: the samples z = [1, 3, 2, 5, 4] at x = [0, 1, 2, 3, 4] (y = [0, 0, 0, 0, 0]). Bottom: the empirical semivariogram the package returns for lag bins [0, 1.5, 2.5, 4.5] — each bar spans its bin and has the returned semivariance as height, labelled with the number of pairs.

```
geospatial_analytics_ai999x.solve('geo_empirical_semivariogram', x=[0, 1, 2, 3, 4], y=[0, 0, 0, 0, 0], z=[1, 3, 2, 5, 4], bin_edges=[0, 1.5, 2.5, 4.5])
# -> gamma = [1.875, 1.5, 4.33333333333], counts = [4.0, 3.0, 3.0]   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_empirical_semivariogram', x=[0, 10, 0, 10, 5], y=[0, 0, 10, 10, 5], z=[1, 2, 3, 4, 2.5], bin_edges=[0, 8, 12, 15])
# -> gamma = [0.625, 1.25, 2.5], counts = [4.0, 4.0, 2.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Empirical semivariogram: x = [0, 1, 2, 3, 4], y = [0, 0, 0, 0, 0], z = [1, 3, 2, 5, 4], bin_edges = [0, 1.5, 2.5, 4.5]	SOLVED	gamma = [1.875, 1.5, 4.33333333333], counts = [4.0, 3.0, 3.0]
Empirical semivariogram: x = [0, 10, 0, 10, 5], y = [0, 0, 10, 10, 5], z = [1, 2, 3, 4, 2.5], bin_edges = [0, 8, 12, 15]	SOLVED	gamma = [0.625, 1.25, 2.5], counts = [4.0, 4.0, 2.0]
Empirical semivariogram: y = [0, 0, 0, 0, 0], z = [1, 3, 2, 5, 4], bin_edges = [0, 1.5, 2.5, 4.5]	NEEDS_INPUT	geo_empirical_semivariogram needs x (x coordinates of the samples). Example: solve('geo_empirical_semivariogram', x=[0,
Empirical semivariogram: x = [0, 1, 2, 3, 4], y = [0, 0, 0, 0, 0], z = [1, 3, 2, 5, 4], bin_edges = [0, 1.5, 2.5, 4.5], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Fit an exponential or Gaussian variogram model (nugget, partial sill, range) — `geo_variogram_model_fit`

The nugget, partial sill, range and sill of an exponential or Gaussian variogram model fitted by ordinary least squares to an empirical semivariogram (lags and semivariances).

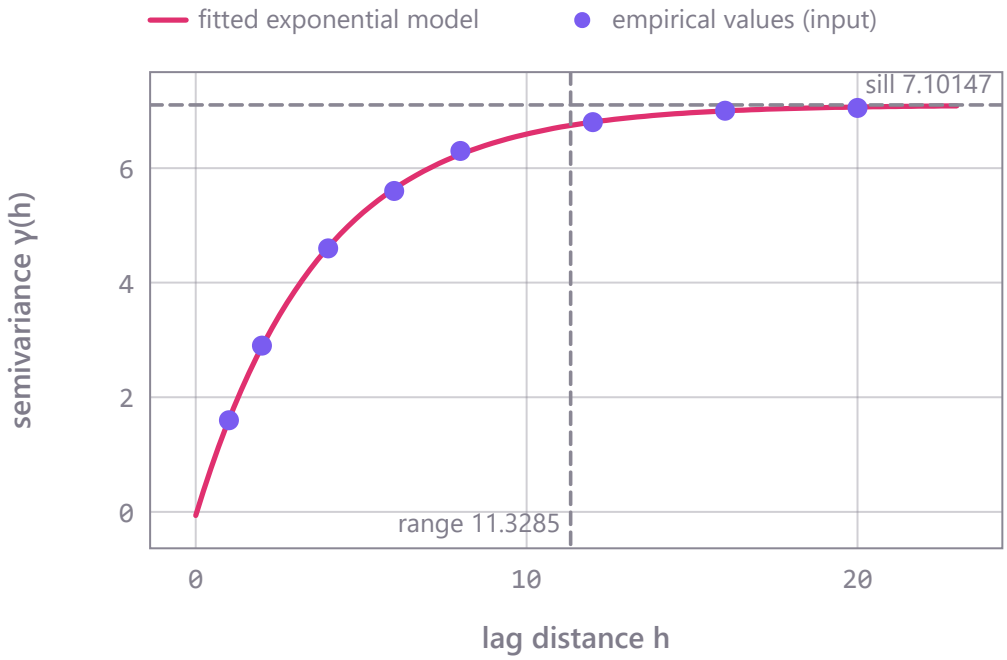
$$\gamma(h) = c_0 + c\left(1 - e^{-3h/a}\right) \text{ or } c_0 + c\left(1 - e^{-(7h/4a)^2}\right), \quad \min_{c_0,c,a} \sum_k \left(\hat{\gamma}_k - \gamma(h_k)\right)^2$$

input	meaning	condition
`h`	lag distances	
`gamma`	semivariances at the lags	
`model`	variogram model (pykrige parameterization, practical range)	

output	meaning
`nugget`	nugget c0
`psill`	partial sill c
`range`	practical range a
`sill`	sill $c_0 + c$

- **Method:** variable projection: for each range the nugget and partial sill are linear least squares; the range solves d(SSE)/da = 0 (grid search, then bisection, polished in 30-digit arithmetic with mpmath)
- **Assumes:** the least-squares range has an interior minimum (between min(h)/20 and 20 max(h))

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The empirical semivariances γ = [1.6, 2.9, 4.6, 5.6, 6.3, 6.8, 7.0, 7.05] at lags h = [1, 2, 4, 6, 8, 12, 16, 20] and the exponential model with the parameters the package fits: nugget -0.0628821, partial sill 7.16435, sill 7.10147, practical range 11.3285 (where the model reaches 95 % of its partial sill); the curve γ(h) = nugget + partial sill·(1 − e^(−3h/range)) is evaluated here from those returned values.

```
geospatial_analytics_ai999x.solve('geo_variogram_model_fit', h=[1, 2, 4, 6, 8, 12, 16, 20], gamma=
[1.6, 2.9, 4.6, 5.6, 6.3, 6.8, 7.0, 7.05], model='exponential')
# -> nugget = -0.0628821, psill = 7.16435, range = 11.3285, sill = 7.10147   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_variogram_model_fit', h=[1, 2, 4, 6, 8, 12, 16, 20], gamma=
[0.6, 1.1, 2.8, 4.6, 5.8, 6.9, 7.0, 7.0], model='gaussian')
# -> nugget = 0.428937, psill = 6.57148, range = 10.5442, sill = 7.00042   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Variogram model fit: h = [1, 2, 4, 6, 8, 12, 16, 20], gamma = [1.6, 2.9, 4.6, 5.6, 6.3, 6.8, 7, 7.05], model = exponential	SOLVED	nugget = -0.0628821, psill = 7.16435, range = 11.3285, sill = 7.10147
Variogram model fit: h = [1, 2, 4, 6, 8, 12, 16, 20], gamma = [0.6, 1.1, 2.8, 4.6, 5.8, 6.9, 7, 7], model = gaussian	SOLVED	nugget = 0.428937, psill = 6.57148, range = 10.5442, sill = 7.00042
Variogram model fit: gamma = [1.6, 2.9, 4.6, 5.6, 6.3, 6.8, 7, 7.05], model = exponential	NEEDS_INPUT	geo_variogram_model_fit needs h (lag distances). Example: solve('geo_variogram_model_fit', h=[1, 2, 4, 6, 8, 12, 16, 20]
Variogram model fit: h = [1, 2, 4, 6, 8, 12, 16, 20], gamma = [1.6, 2.9, 4.6, 5.6, 6.3, 6.8, 7, 7.05], model = exponential, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Ordinary kriging estimate and kriging variance at a point — `geo_ordinary_kriging`

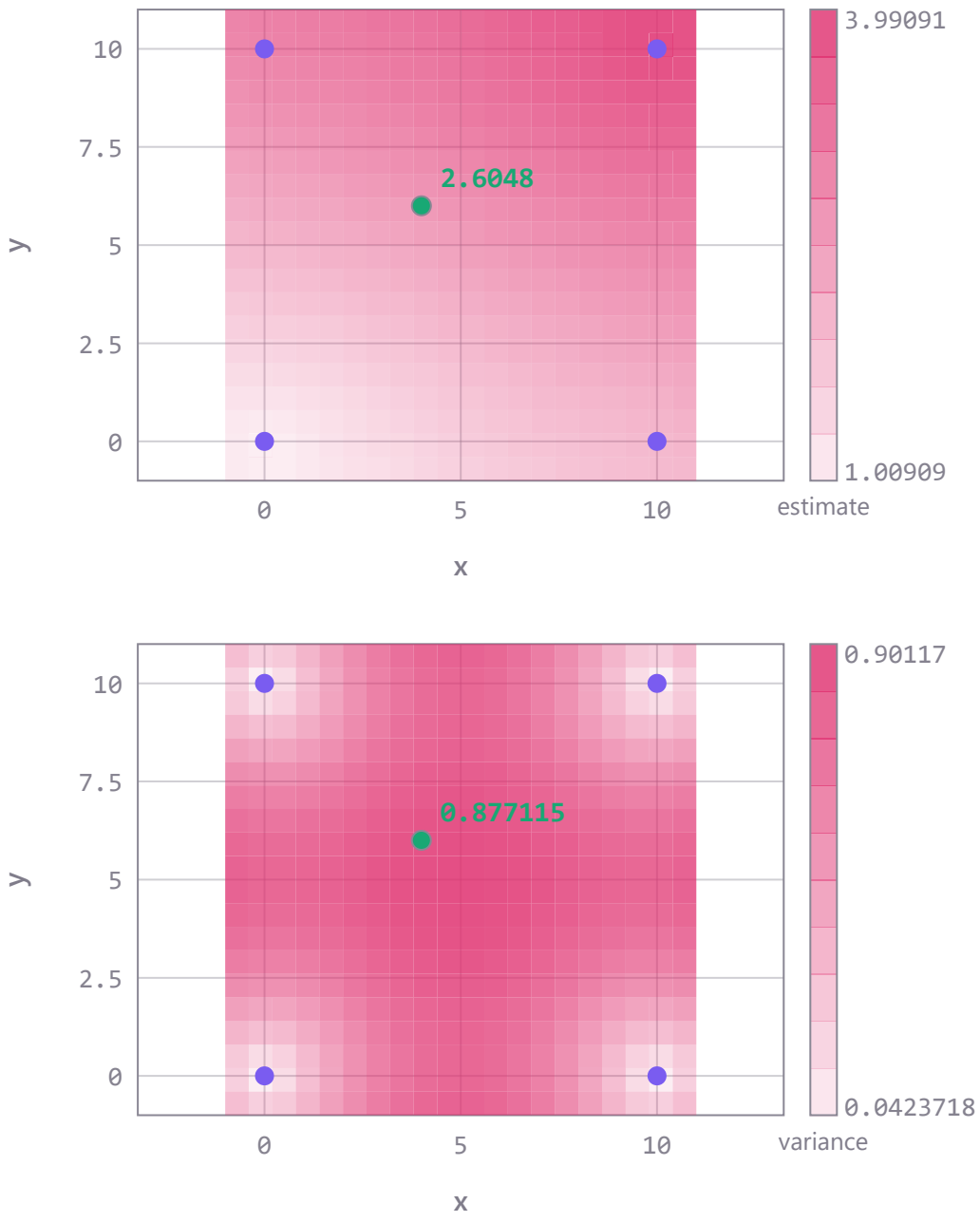
The ordinary kriging estimate of a variable at a location from samples, with a given variogram model (nugget, sill, range), and the kriging variance.

$$\begin{bmatrix} \Gamma & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{\lambda} \\ \mu \end{bmatrix} = \begin{bmatrix} \boldsymbol{\gamma}_0 \\ 1 \end{bmatrix}, \quad \hat{z} = \boldsymbol{\lambda}^T \mathbf{z}, \quad \sigma^2_{OK} = \boldsymbol{\lambda}^T \boldsymbol{\gamma}_0 + \mu$$

input	meaning	condition
`x`	x coordinates of the samples	
`y`	y coordinates of the samples	
`z`	sample values	
`px`	x of the location	
`py`	y of the location	
`model`	variogram model (pykrige parameterization)	
`nugget`	nugget	
`sill`	sill (nugget + partial sill)	
`range`	range (practical range for exponential and gaussian)	

output	meaning
`estimate`	kriging estimate
`variance`	kriging variance

- **Method:** solve the ordinary kriging system [Gamma 1; 1' 0][lambda; mu] = [gamma0; 1]; estimate = lambda'z, variance = lambda'$gamma0 + mu$
- **Assumes:** distinct sample locations; sill > nugget >= 0, range > 0; the variogram is 0 at zero distance (exact interpolation)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ordinary kriging from four samples z = [1, 2, 3, 4] at the corners of a 10 × 10 square (spherical variogram, nugget 0, sill 2, range 20): top, the estimate the package returns at each of 400 cell centres; bottom, the kriging variance, zero at the samples and largest away from them. Marked: the worked example at (4, 6), estimate 2.6048, variance 0.877115.

```
geospatial_analytics_ai999x.solve('geo_ordinary_kriging', x=[0, 10, 0, 10], y=[0, 0, 10, 10], z=[1, 2, 3, 4], px=4, py=6, model='spherical', nugget=0, sill=2, range=20)
# -> estimate = 2.6048, variance = 0.877115   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_ordinary_kriging', x=[0, 50, 20, 80, 60], y=[0, 10, 70, 40, 90], z=[12, 15, 9, 20, 14], px=40, py=40, model='exponential', nugget=0.5, sill=10, range=100)
# -> estimate = 13.8779, variance = 7.57644   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ordinary kriging: x = [0, 10, 0, 10], y = [0, 0, 10, 10], z = [1, 2, 3, 4], px = 4, py = 6,	SOLVED	estimate = 2.6048, variance = 0.877115

question	outcome	result
model = spherical, nugget = 0, sill = 2, range = 20		
Ordinary kriging: x = [0, 50, 20, 80, 60], y = [0, 10, 70, 40, 90], z = [12, 15, 9, 20, 14], px = 40, py = 40, model = exponential, nugget = 0.5, sill = 10, range = 100	SOLVED	estimate = 13.8779, variance = 7.57644
Ordinary kriging: y = [0, 0, 10, 10], z = [1, 2, 3, 4], px = 4, py = 6, model = spherical, nugget = 0, sill = 2, range = 20	NEEDS_INPUT	geo_ordinary_kriging needs x (x coordinates of the samples). Example: solve('geo_ordinary_kriging', x= [0, 10, 0, 10], y=
Ordinary kriging: x = [0, 10, 0, 10], y = [0, 0, 10, 10], z = [1, 2, 3, 4], px = 4, py = 6, model = spherical, nugget = 0, sill = 2, range = 20, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Polynomial trend surface (first or second order) by least squares — `geo_trend_surface`

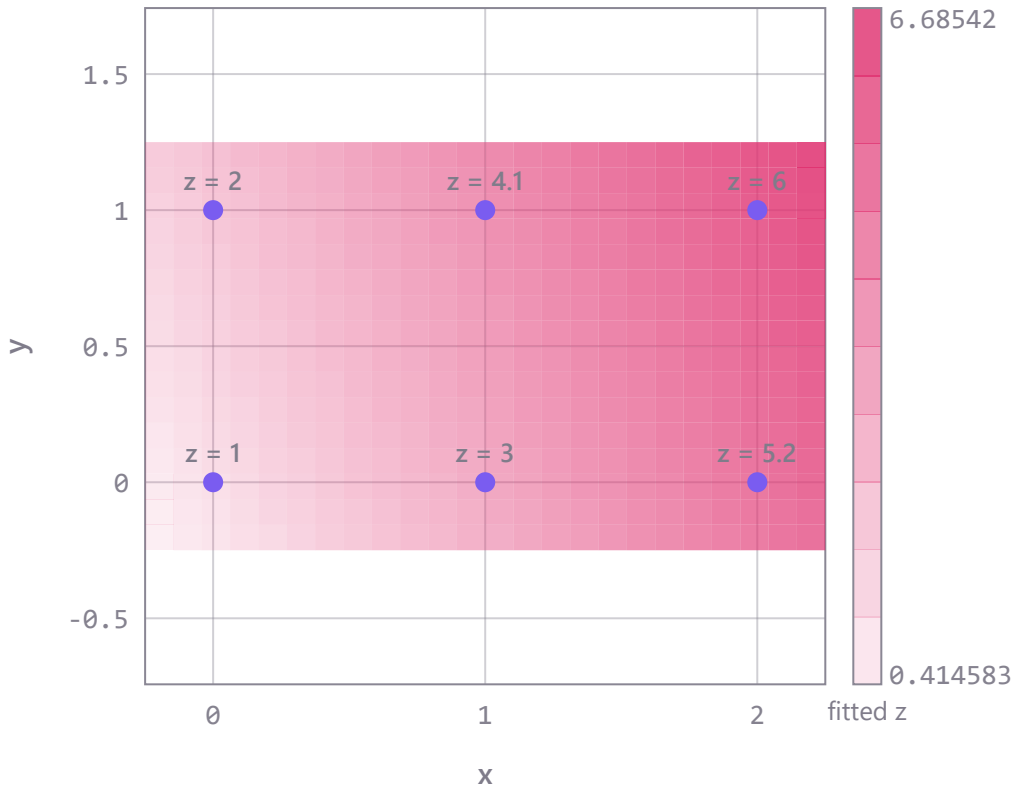
The coefficients and R^2 of a first-order (plane) or second-order (quadratic) trend surface fitted to sampled values by least squares.

$$z = b_0 + b_1 x + b_2 y \ \left[+ b_3 x^2 + b_4 xy + b_5 y^2 \right]$$

input	meaning	condition
`x`	x coordinates	
`y`	y coordinates	
`z`	values	
`order`	order of the surface (1 or 2)	

output	meaning
`coefficients`	b0, b1 (x), b2 (y)[, b3 (x^2), b4 (xy), b5 (y^2)]
`r_squared`	coefficient of determination

- **Method:** ordinary least squares by the normal equations
- **Assumes:** more samples than coefficients; values not all equal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A first-order trend surface fitted to six samples (z = [1, 3, 5.2, 2, 4.1, 6] at the drawn points): the shading is the plane z = 1.01667 + 2.05·x + 0.966667·y with the coefficients the package returns (evaluated here on a grid); R^2 = 0.99872.

```
geospatial_analytics_ai999x.solve('geo_trend_surface', x=[0, 1, 2, 0, 1, 2], y=[0, 0, 0, 1, 1, 1],
z=[1, 3, 5.2, 2, 4.1, 6], order=1)
# -> coefficients = [1.01666666667, 2.05, 0.966666666667], r_squared = 0.99872   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_trend_surface', x=[0, 1, 2, 0, 1, 2, 0, 1, 2], y=[0, 0, 0,
1, 1, 1, 2, 2, 2], z=[0.3, 1.1, 3.9, 1, 2.4, 5.2, 4.1, 5, 8.6], order=2)
# -> coefficients = [0.280555555556, -0.208333333333, -0.0916666666667, 1.01666666667, 0.225,
0.966666666667], r_squared = 0.997304   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Trend surface: x = [0, 1, 2, 0, 1, 2], y = [0, 0, 0, 1, 1, 1], z = [1, 3, 5.2, 2, 4.1, 6], order = 1	SOLVED	coefficients = [1.01666666667, 2.05, 0.966666666667], r_squared = 0.99872
Trend surface: x = [0, 1, 2, 0, 1, 2, 0, 1, 2], y = [0, 0, 0, 1, 1, 1, 2, 2, 2], z = [0.3, 1.1, 3.9, 1, 2.4, 5.2, 4.1, 5, 8.6], order = 2	SOLVED	coefficients = [0.280555555556, -0.208333333333, -0.0916666666667, 1.01666666667, 0.225, 0.966666666667], r_squared = 0.997304
Trend surface: y = [0, 0, 0, 1, 1, 1], z = [1, 3, 5.2, 2, 4.1, 6], order = 1	NEEDS_INPUT	geo_trend_surface needs x (x coordinates). Example: solve('geo_trend_surface', x=[0, 1, 2, 0, 1, 2], y=[0, 0, 0, 1, 1, 1
Trend surface: x = [0, 1, 2, 0, 1, 2], y = [0, 0, 0, 1, 1, 1], z = [1, 3, 5.2, 2, 4.1, 6], order = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Rank-size rule (Zipf) fit of a settlement system — `geo_rank_size_rule`

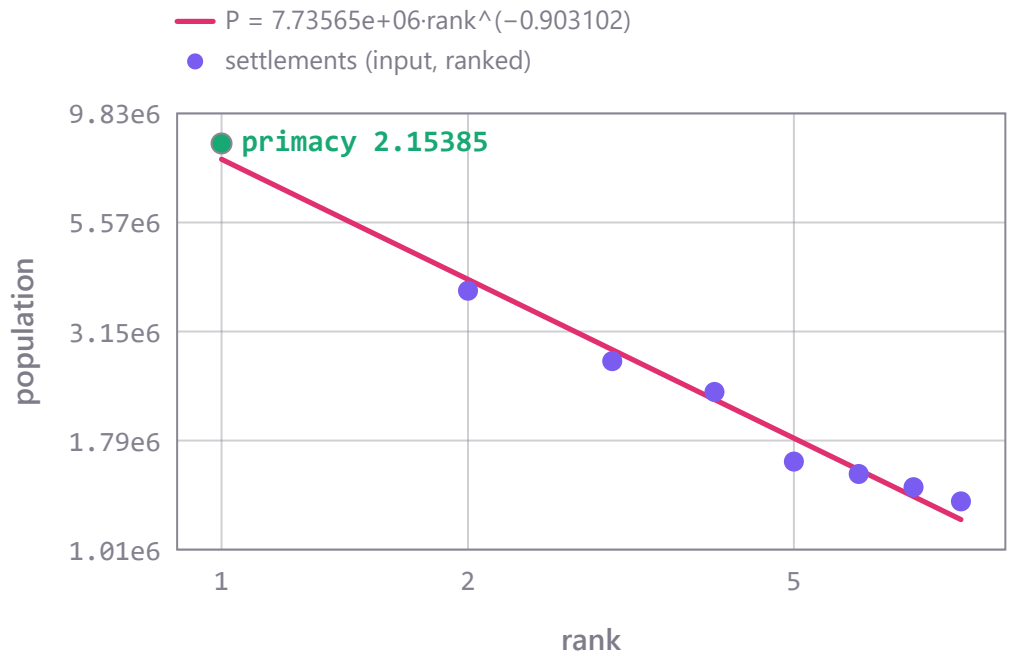
The Zipf exponent q and fitted size of the largest city from a log-log regression of population on rank (P_r = P_1 r^-q), the R^2 of the fit and the two-city primacy ratio.

$$\ln P_r = \ln P_1 - q \ln r$$

input	meaning	condition
`populations`	populations of the settlements (any order)	

output	meaning
`q`	rank-size exponent (1 is Zipf's law)
`P1_fit`	fitted population of rank 1
`r_squared`	R^2 of the log-log fit
`primacy`	largest / second largest population

- **Method:** sort descending; least squares of ln P on ln rank
- **Assumes:** positive populations
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Settlement populations ranked from largest to smallest on logarithmic axes, with the rank–size line of the returned exponent q = 0.903102 and fitted largest size 7.73565e+06 (R^2 = 0.985307; the line is evaluated here from them). Marked: the largest city, primacy (largest over second) 2.15385.

```
geospatial_analytics_ai999x.solve('geo_rank_size_rule', populations=[8400000, 3900000, 2700000,
2300000, 1600000, 1500000, 1400000, 1300000])
# -> q = 0.903102, P1_fit = 7.73565e+06, r_squared = 0.985307, primacy = 2.15385   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_rank_size_rule', populations=[1000, 480, 350, 260, 190,
170])
# -> q = 0.992989, P1_fit = 994.942, r_squared = 0.996108, primacy = 2.08333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rank-size rule: populations = [8400000, 3900000, 2700000, 2300000, 1600000, 1500000, 1400000, 1300000]	SOLVED	q = 0.903102, P1_fit = 7.73565e+06, r_squared = 0.985307, primacy = 2.15385
Rank-size rule: populations = [1000, 480, 350, 260, 190, 170]	SOLVED	q = 0.992989, P1_fit = 994.942, r_squared = 0.996108, primacy = 2.08333
Rank-size rule:	NEEDS_INPUT	geo_rank_size_rule needs populations (populations of the settlements (any order)). Example: solve('geo_rank_size_rule',
Rank-size rule: populations = [8400000, 3900000, 2700000, 2300000, 1600000, 1500000, 1400000, 1300000], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Urban population density gradient (Clark's negative exponential model) — `geo_density_gradient`

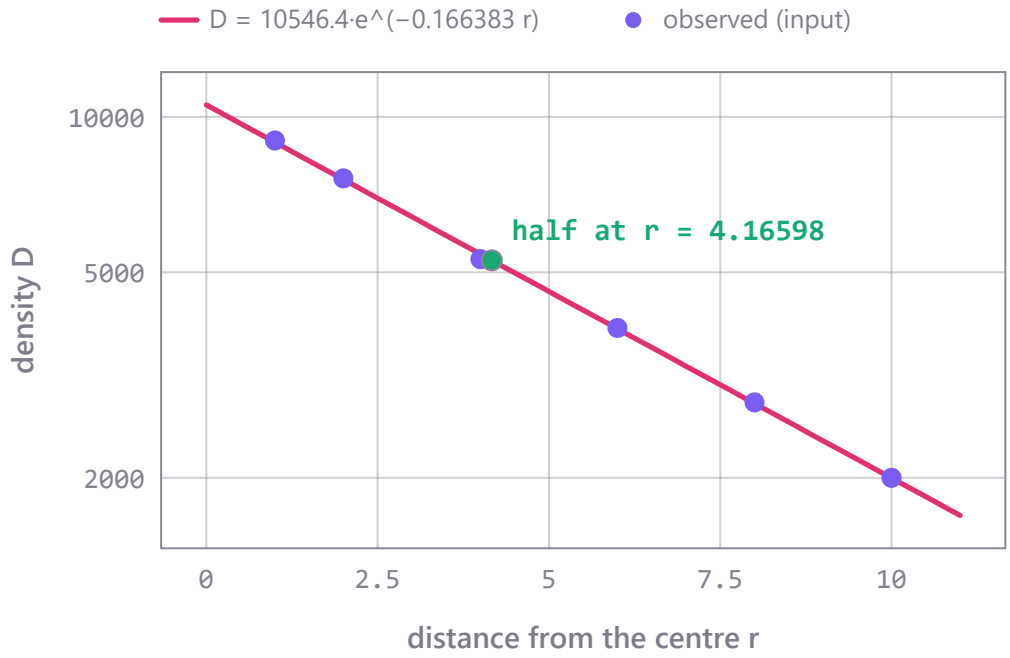
The central density D0, density gradient b and half-distance of Clark's model D(r) = D0 exp(-b r) fitted by log-linear least squares to densities at distances from the centre, with the R^2.

$$D(r) = D_0 e^{-br}, \quad \ln D = \ln D_0 - br$$

input	meaning	condition
`r`	distances from the centre (any unit)	
`D`	population densities at those distances	

output	meaning
`D0`	fitted central density
`b`	density gradient (per distance unit)
`r_squared`	R^2 of the log-linear fit
`half_distance`	distance over which density halves (ln 2 / b), when $b > 0$

- **Method:** least squares of ln D on r
- **Assumes:** positive densities; distances not all equal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Population densities D = [9000, 7600, 5300, 3900, 2800, 2000] at distances r = [1, 2, 4, 6, 8, 10] on a logarithmic density axis, and the negative-exponential gradient with the parameters the package returns (D0 = 10546.4, b = 0.166383, R^2 = 0.999624; the line is evaluated here from them). Marked: the returned half distance 4.16598, where density falls to D0/2.

```
geospatial_analytics_ai999x.solve('geo_density_gradient', r=[1, 2, 4, 6, 8, 10], D=[9000, 7600,
5300, 3900, 2800, 2000])
# -> D0 = 10546.4, b = 0.166383, r_squared = 0.999624, half_distance = 4.16598   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_density_gradient', r=[0.5, 1.5, 2.5, 3.5], D=[12000, 8500,
6100, 4300])
# -> D0 = 14226.4, b = 0.341065, r_squared = 0.999908, half_distance = 2.0323   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Density gradient: r = [1, 2, 4, 6, 8, 10], D = [9000, 7600, 5300, 3900, 2800, 2000]	SOLVED	D0 = 10546.4, b = 0.166383, r_squared = 0.999624, half_distance = 4.16598
Density gradient: r = [0.5, 1.5, 2.5, 3.5], D = [12000, 8500, 6100, 4300]	SOLVED	D0 = 14226.4, b = 0.341065, r_squared = 0.999908, half_distance = 2.0323
Density gradient: D = [9000, 7600, 5300, 3900, 2800, 2000]	NEEDS_INPUT	geo_density_gradient needs r (distances from the centre (any unit)). Example: solve('geo_density_gradient', r=[1, 2, 4,
Density gradient: r = [1, 2, 4, 6, 8, 10], D = [9000, 7600, 5300, 3900, 2800, 2000], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Spatial interaction (gravity) model fitted by Poisson regression — `geo_gravity_model_poisson_fit`

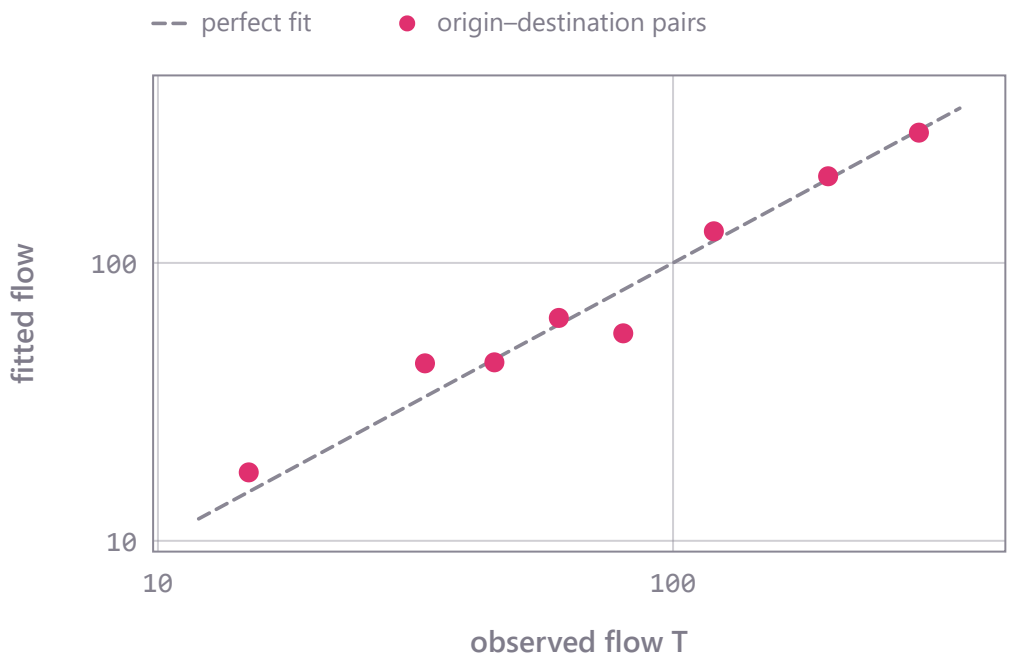
The constant k, the origin and destination mass exponents and the distance-decay exponent of the unconstrained gravity model $T = k\,O^{\alpha}\,D^{\beta}\,d^{-\gamma}$, fitted to observed flows by Poisson maximum likelihood.

$$T_{ij} \sim \text{Poisson}(\mu_{ij}), \quad \ln \mu_{ij} = \ln k + \alpha \ln O_i + \beta \ln D_j - \gamma \ln d_{ij}$$

input	meaning	condition
`T`	observed flows between the pairs	
`O`	origin masses (e.g. populations)	
`D`	destination masses	
`d`	distances between the pairs	

output	meaning
`k`	constant
`alpha`	origin mass exponent
`beta`	destination mass exponent
`gamma`	distance-decay exponent

- **Method:** Poisson GLM with log link, ln $mu = ln$ k + alpha ln O + beta ln D - gamma ln d, by iteratively reweighted least squares
- **Assumes:** non-negative flows, positive masses and distances that vary
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Observed flows T = [120, 45, 300, 80, 15, 60, 200, 33] against the flows of the gravity model fitted by the package, T̂ = k·O^α·D^β·d^(−γ) with k = 0.000370957, α = 0.802892, β = 0.33475, γ = 0.509079 (the fitted values are evaluated here from these returned parameters and the inputs O, D, d); points near the diagonal are well explained.

```
geospatial_analytics_ai999x.solve('geo_gravity_model_poisson_fit', T=[120, 45, 300, 80, 15, 60,
200, 33], O=[500000.0, 200000.0, 900000.0, 300000.0, 100000.0, 400000.0, 800000.0, 250000.0], D=
[300000.0, 400000.0, 600000.0, 200000.0, 300000.0, 100000.0, 500000.0, 200000.0], d=[50, 120, 40,
90, 200, 70, 60, 110])
# -> k = 0.000370957, alpha = 0.802892, beta = 0.33475, gamma = 0.509079   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_gravity_model_poisson_fit', T=[10, 25, 5, 40, 18, 7, 30], O=
[10000.0, 30000.0, 10000.0, 50000.0, 20000.0, 10000.0, 40000.0], D=[20000.0, 20000.0, 10000.0,
30000.0, 30000.0, 20000.0, 20000.0], d=[10, 15, 30, 12, 20, 40, 14])
# -> k = 0.000287153, alpha = 0.830605, beta = 0.345863, gamma = 0.261763   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gravity model fit: T = [120, 45, 300, 80, 15, 60, 200, 33], O = [500000, 200000, 900000, 300000, 100000, 400000, 800000, 250000], D = [300000, 400000, 600000, 200000, 300000, 100000, 500000, 200000], d = [50, 120, 40, 90, 200, 70, 60, 110]	SOLVED	k = 0.000370957, alpha = 0.802892, beta = 0.33475, gamma = 0.509079
Gravity model fit: T = [10, 25, 5, 40, 18, 7, 30], O = [10000, 30000, 10000, 50000, 20000, 10000, 40000], D = [20000, 20000, 10000, 30000, 30000, 20000, 20000], d = [10, 15, 30, 12, 20, 40, 14]	SOLVED	k = 0.000287153, alpha = 0.830605, beta = 0.345863, gamma = 0.261763
Gravity model fit: O = [500000, 200000, 900000, 300000, 100000, 400000, 800000, 250000], D = [300000, 400000, 600000, 200000, 300000, 100000, 500000, 200000], d = [50, 120, 40, 90, 200, 70, 60, 110]	NEEDS_INPUT	geo_gravity_model_poisson_fit needs T (observed flows between the pairs). Example: solve('geo_gravity_model_poisson_fit'
Gravity model fit: T = [120, 45, 300, 80, 15, 60, 200, 33], O = [500000, 200000, 900000, 300000, 100000, 400000, 800000, 250000], D = [300000, 400000, 600000, 200000, 300000, 100000, 500000, 200000], d = [50, 120, 40, 90, 200, 70, 60, 110], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Vegetation indices: NDVI, EVI, SAVI, OSAVI, MSAVI and EVI2 from band reflectances — `geo_vegetation_indices`

The common spectral vegetation indices of a pixel from its red, near-infrared and blue surface reflectances, with a stated soil adjustment factor L for SAVI (EVI uses its standard coefficients $G = 2.5$, $C_1 = 6$, $C_2 = 7.5$, L = 1).

$$NDVI = \frac{\rho_N - \rho_R}{\rho_N + \rho_R},\ EVI = 2.5\frac{\rho_N - \rho_R}{\rho_N + 6\rho_R - 7.5\rho_B + 1},\ SAVI = (1+L)\frac{\rho_N - \rho_R}{\rho_N + \rho_R + L}$$

input	meaning	condition
`red`	red reflectance $(0-1)$	
`nir`	near-infrared reflectance $(0-1)$	
`blue`	blue reflectance $(0-1)$	
`L`	SAVI soil adjustment factor (0.5 is usual)	

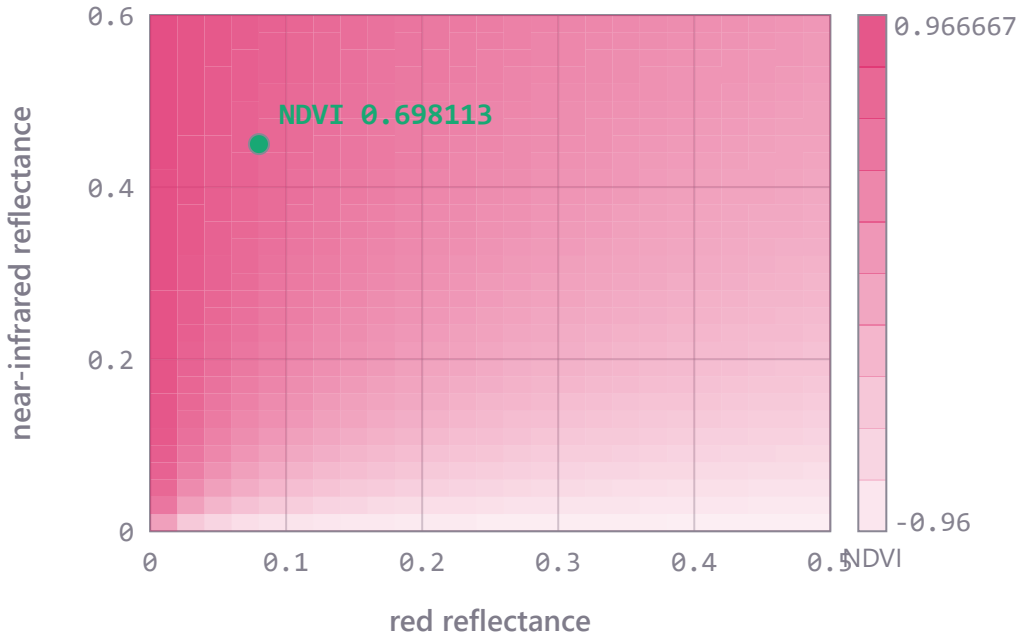
output	meaning
`NDVI`	(NIR - R)/(NIR + R)

output	meaning
`EVI`	2.5 (NIR - R)/(NIR + 6R - 7.5B + 1)
`SAVI`	$(1 + L)$(NIR - R)/(NIR + R + L)
`OSAVI`	(NIR - R)/(NIR + R + 0.16)
`MSAVI`	(2NIR + 1 - sqrt((2NIR + 1)^2 - 8(NIR - R)))/2
`EVI2`	2.5 (NIR - R)/(NIR + 2.4R + 1)

- **Method:** the published formulas (Rouse 1974; Huete 1988, 2002; Rondeaux 1996; Qi 1994; Jiang 2008)
- **Assumes:** reflectances between 0 and 1; positive denominators
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



All 6 vegetation indices the package returns for the worked example's reflectances (red = 0.08, nir = 0.45, blue = 0.04, L = 0.5): NDVI = 0.698113, EVI = 0.567485, SAVI = 0.538835, OSAVI = 0.536232, MSAVI = 0.546887, EVI2 = 0.563337.



NDVI returned by the package over red reflectance 0–0.5 and near-infrared 0–0.6 (750 runs, blue 0.04): vegetation (bright near-infrared, dark red) is high, bare soil and water low. Marked: the worked example, red 0.08, near-infrared 0.45, NDVI 0.698113.

```
geospatial_analytics_ai999x.solve('geo_vegetation_indices', red=0.08, nir=0.45, blue=0.04, L=0.5)
# -> NDVI = 0.698113, EVI = 0.567485, SAVI = 0.538835, OSAVI = 0.536232, MSAVI = 0.546887, EVI2 =
0.563337   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_vegetation_indices', red=0.2, nir=0.25, blue=0.1, L=0.5)
# -> NDVI = 0.111111, EVI = 0.0735294, SAVI = 0.0789474, OSAVI = 0.0819672, MSAVI = 0.0699265, EVI2
= 0.0722543   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Vegetation indices: red = 0.08, nir = 0.45, blue = 0.04, L = 0.5	SOLVED	NDVI = 0.698113, EVI = 0.567485, SAVI = 0.538835, OSAVI = 0.536232, MSAVI = 0.546887, EVI2 = 0.563337
Vegetation indices: red = 0.2, nir = 0.25, blue = 0.1, L = 0.5	SOLVED	NDVI = 0.111111, EVI = 0.0735294, SAVI = 0.0789474, OSAVI = 0.0819672, MSAVI = 0.0699265, EVI2 = 0.0722543
Vegetation indices: nir = 0.45, blue = 0.04, L = 0.5	NEEDS_INPUT	geo_vegetation_indices needs red (red reflectance (0-1)). Example: solve('geo_vegetation_indices', red=0.08, nir=0.45, b
Vegetation indices: red = 0.08, nir = 0.45, blue = 0.04, L = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Water, moisture, burn and built-up indices: NDWI, MNDWI, NDMI, NBR and NDBI — `geo_water_burn_built_indices`

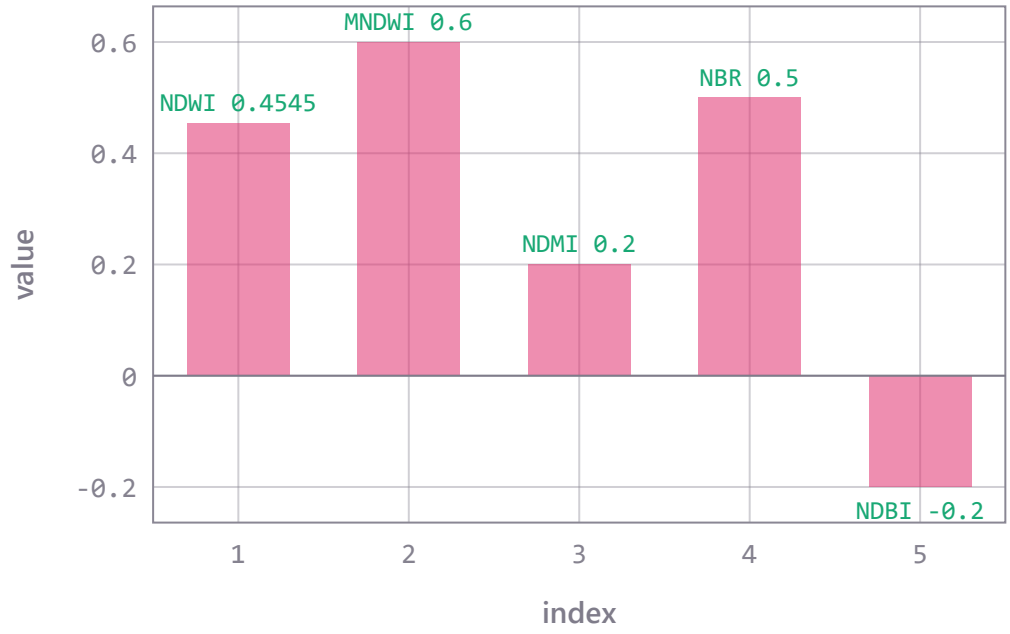
Normalized-difference indices for open water (NDWI, MNDWI), vegetation moisture (NDMI), burn severity (NBR) and built-up land (NDBI) from green, near-infrared and two shortwave-infrared reflectances.

$$ND(a,b) = \frac{\rho_a - \rho_b}{\rho_a + \rho_b}$$

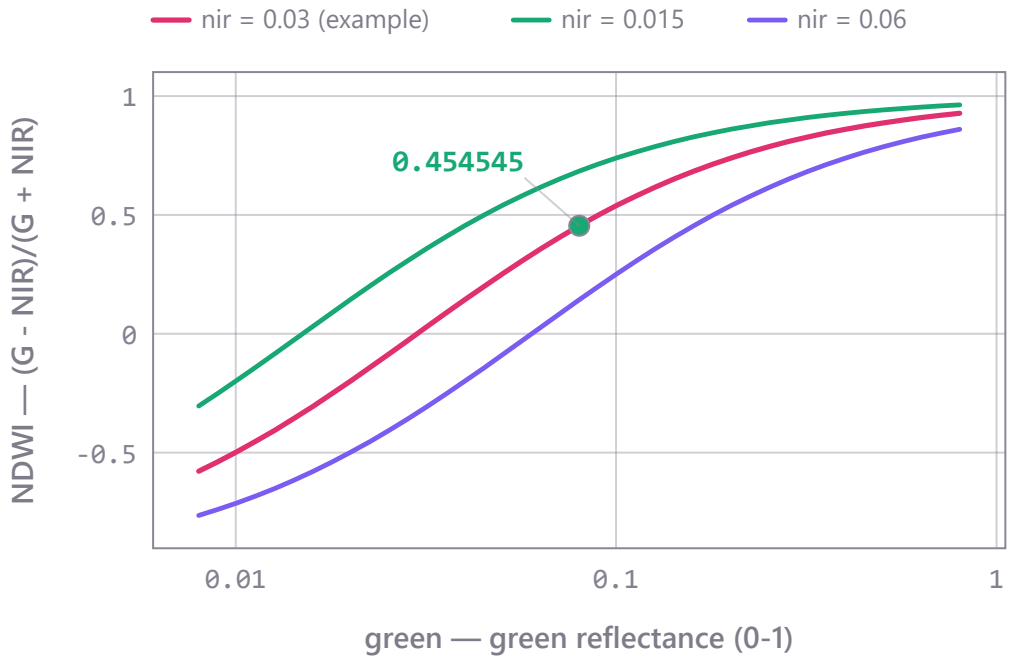
input	meaning	condition
`green`	green reflectance $(0 - 1)$	
`nir`	near-infrared reflectance $(0 - 1)$	
`swir1`	shortwave infrared 1 (about 1.6 um) reflectance	
`swir2`	shortwave infrared 2 (about 2.2 um) reflectance	

output	meaning
`NDWI`	(G - NIR)/(G + NIR)
`MNDWI`	(G - SWIR1)/(G + SWIR1)
`NDMI`	(NIR - SWIR1)/(NIR + SWIR1)
`NBR`	(NIR - SWIR2)/(NIR + SWIR2)
`NDBI`	(SWIR1 - NIR)/(SWIR1 + NIR)

- **Method:** normalized differences of the band pairs (McFeeters 1996; Xu 2006; Gao 1996; Key and Benson 2006; Zha 2003)
- **Assumes:** reflectances between 0 and 1, band sums non-zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



All 5 normalised-difference indices the package returns for the worked example's reflectances (green = 0.08, nir = 0.03, swir1 = 0.02, swir2 = 0.01): NDWI = 0.454545, MNDWI = 0.6, NDMI = 0.2, NBR = 0.5, NDBI = -0.2.



NDWI ((G - NIR)/(G + NIR)) against green (green reflectance (0-1)) from 0.008 to 0.8 on a logarithmic axis. One curve for each value of nir (near-infrared reflectance (0-1)): 0.03, 0.015, 0.06. Other inputs as in the worked example: swir1 = 0.02, swir2 = 0.01. 41 points per curve computed by geospatial-analytics-ai999x; marked: the worked example, green = 0.08, NDWI = 0.454545.

```
geospatial_analytics_ai999x.solve('geo_water_burn_built_indices', green=0.08, nir=0.03, swir1=0.02, swir2=0.01)
# -> NDWI = 0.454545, MNDWI = 0.6, NDMI = 0.2, NBR = 0.5, NDBI = -0.2   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_water_burn_built_indices', green=0.07, nir=0.35, swir1=0.2, swir2=0.1)
# -> NDWI = -0.666667, MNDWI = -0.481481, NDMI = 0.272727, NBR = 0.555556, NDBI = -0.272727
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Water indices: green = 0.08, nir = 0.03, swir1 = 0.02, swir2 = 0.01	SOLVED	NDWI = 0.454545, MNDWI = 0.6, NDMI = 0.2, NBR = 0.5, NDBI = -0.2
Water indices: green = 0.07, nir = 0.35, swir1 = 0.2, swir2 = 0.1	SOLVED	NDWI = -0.666667, MNDWI = -0.481481, NDMI = 0.272727, NBR = 0.555556, NDBI = -0.272727
Water indices: nir = 0.03, swir1 = 0.02, swir2 = 0.01	NEEDS_INPUT	geo_water_burn_built_indices needs green (green reflectance (0-1)). Example: solve('geo_water_burn_built_indices', green
Water indices: green = 0.08, nir = 0.03, swir1 = 0.02, swir2 = 0.01, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Top-of-atmosphere reflectance from at-sensor radiance — `geo_toa_reflectance`

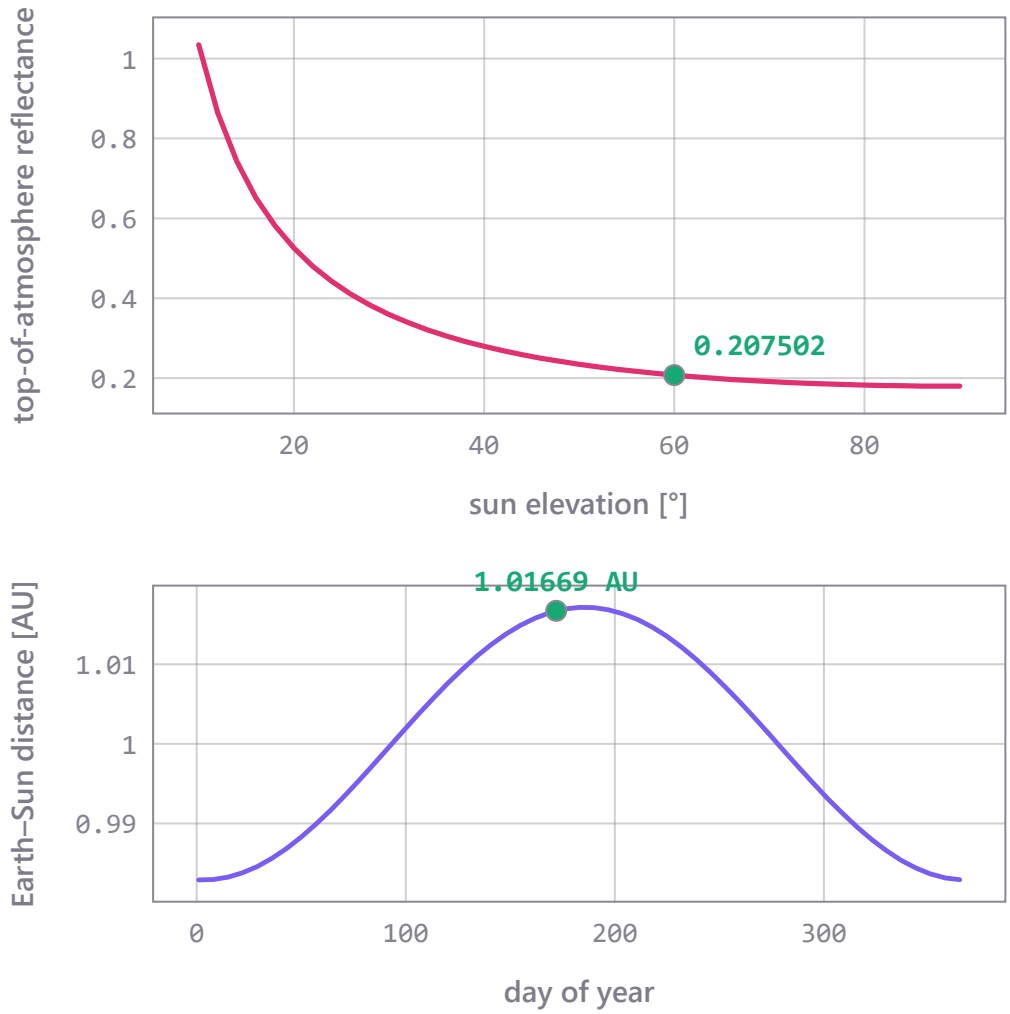
The top-of-atmosphere (planetary) reflectance of a pixel from its at-sensor spectral radiance, the band's mean exoatmospheric solar irradiance ESUN, the day of year and the sun elevation, with the Earth-Sun distance.

$$\rho_{TOA} = \frac{\pi L_\lambda d^2}{ESUN_\lambda \cos\theta_s}, \quad \frac{1}{d^2} = 1.00011 + 0.034221\cos\Gamma + 0.00128\sin\Gamma + 0.000719\cos 2\Gamma + 0.000($$

input	meaning	condition
`radiance`	at-sensor spectral radiance (W/(m^2 sr um))	
`esun`	band mean exoatmospheric solar irradiance ESUN (W/(m^2 um))	
`doy`	day of year $(1-366)$	
`sun_elevation`	sun elevation angle (decimal degrees)	

output	meaning
`earth_sun_distance`	Earth-Sun distance (astronomical units)
`reflectance`	top-of-atmosphere reflectance

- **Method:** $\rho = pi$ L d^2 / (ESUN sin(sun elevation)); d^2 from Spencer's (1971) Fourier series for $(r_0/r)^2$
- **Assumes:** sun above the horizon
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top-of-atmosphere reflectance returned by the package for radiance 85.0 W/(m² sr µm) and solar irradiance 1536: top, against the sun's elevation (day 172); bottom, the Earth–Sun distance it uses through the year (farthest in early July). Marked: the worked example, elevation 60°, day 172 → reflectance 0.207502, distance 1.01669 AU.

```
geospatial_analytics_ai999x.solve('geo_toa_reflectance', radiance=85.0, esun=1536, doy=172,
sun_elevation=60)
# -> earth_sun_distance = 1.01669, reflectance = 0.207502   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_toa_reflectance', radiance=40.5, esun=1031, doy=15,
sun_elevation=25)
# -> earth_sun_distance = 0.983269, reflectance = 0.282321   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
TOA reflectance: radiance = 85, esun = 1536, doy = 172, sun_elevation = 60	SOLVED	earth_sun_distance = 1.01669, reflectance = 0.207502
TOA reflectance: radiance = 40.5, esun = 1031, doy = 15, sun_elevation = 25	SOLVED	earth_sun_distance = 0.983269, reflectance = 0.282321
TOA reflectance: esun = 1536, doy = 172, sun_elevation = 60	NEEDS_INPUT	geo_toa_reflectance needs radiance (at-sensor spectral radiance (W/(m^2 sr um))). Example: solve('geo_toa_reflectance',
TOA reflectance: radiance = 85, esun = 1536, doy = 172, sun_elevation = 60, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

At-sensor brightness temperature from thermal radiance — `geo_brightness_temperature`

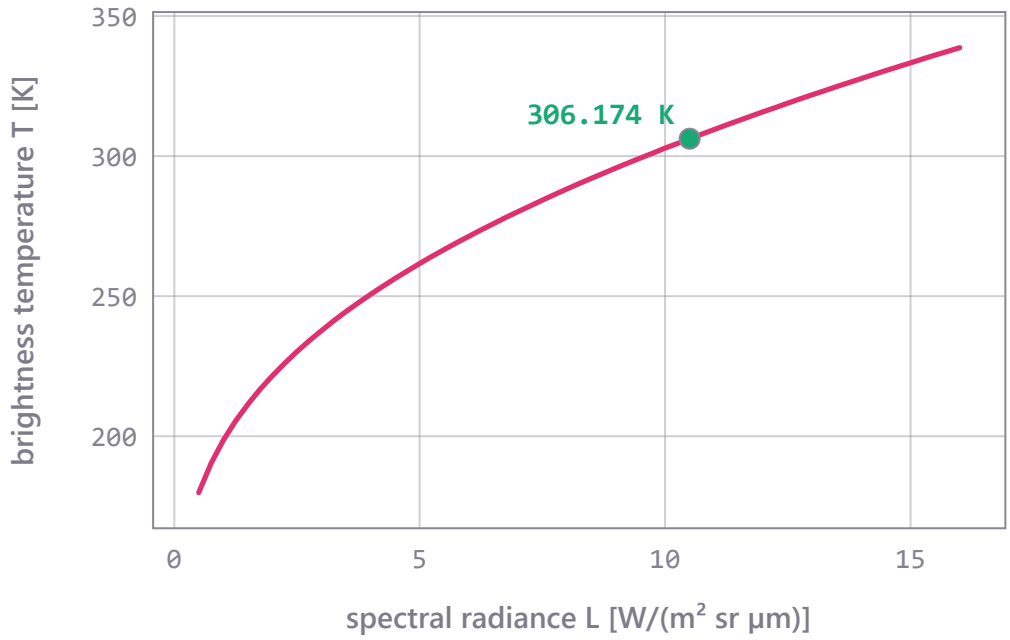
The at-sensor brightness temperature of a thermal-infrared band from its spectral radiance and the band's calibration constants K1 and K2 (as in Landsat metadata).

$$T_B = \frac{K_2}{\ln\left(\frac{K_1}{L_\lambda} + 1\right)}$$

input	meaning	condition
`radiance`	spectral radiance (W/(m^2 sr um))	
`K1`	band constant K1 (W/(m^2 sr um))	
`K2`	band constant K2 (kelvin)	

output	meaning
`T`	brightness temperature

- **Method:** inverted Planck relation $T = K_2/ln$(K1/L + 1)
- **Assumes:** positive radiance and constants
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Brightness temperature returned by the package for thermal radiances 0.5 to 16 W/(m² sr µm) with the band constants K1 = 774.8853, K2 = 1321.0789 (63 runs). Marked: the worked example, radiance 10.5 → T = 306.174 K (33.0239 °C).

```
geospatial_analytics_ai999x.solve('geo_brightness_temperature', radiance=10.5, K1=774.8853,
K2=1321.0789)
# -> T = 306.174 K   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_brightness_temperature', radiance=8.2, K1=480.8883,
K2=1201.1442)
# -> T = 293.793 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Brightness temperature: radiance = 10.5, K1 = 774.8853, K2 = 1321.0789	SOLVED	T = 306.174 K
Brightness temperature: radiance = 8.2, K1 = 480.8883, K2 = 1201.1442	SOLVED	T = 293.793 K
Brightness temperature: K1 = 774.8853, K2 = 1321.0789	NEEDS_INPUT	geo_brightness_temperature needs radiance (spectral radiance (W/(m^2 sr um))). Example: solve('geo_brightness_temperatur
Brightness temperature: radiance = 10.5, K1 = 774.8853, K2 = 1321.0789, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Ground sample distance and swath of a nadir-looking camera — `geo_ground_sample_distance`

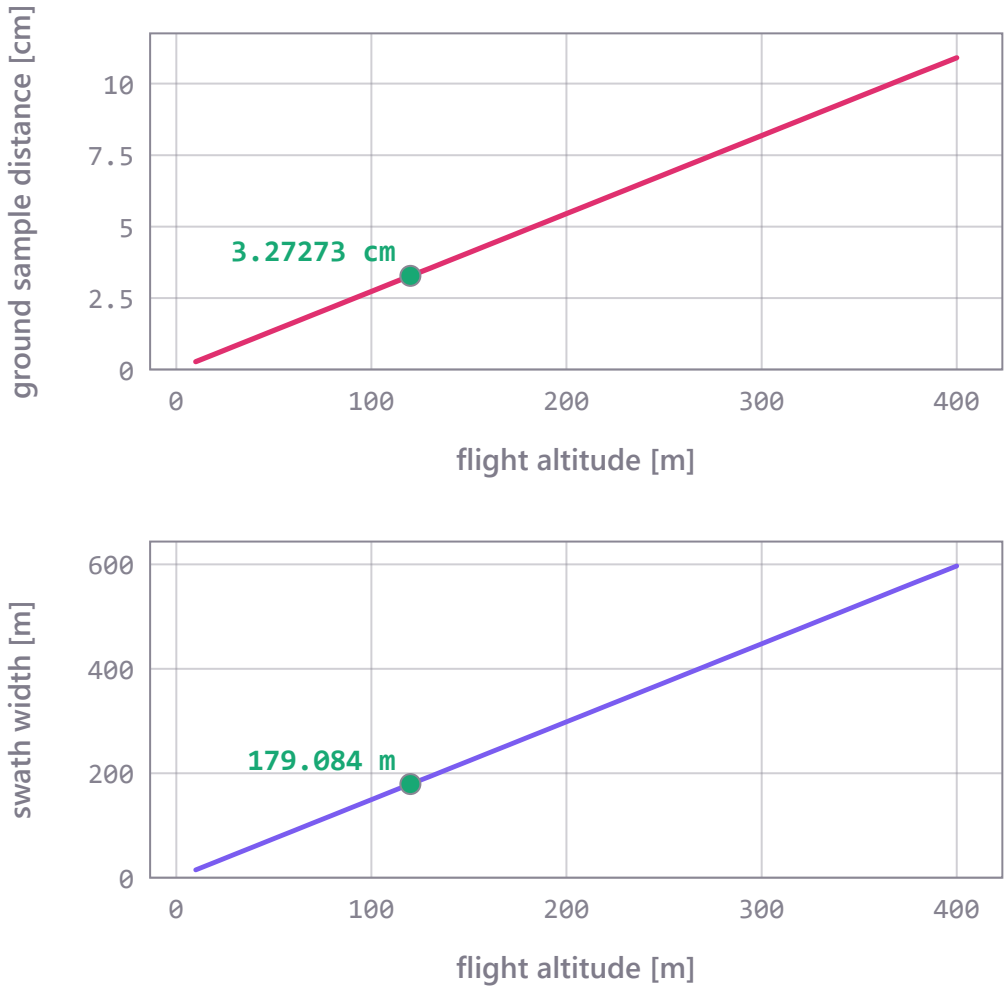
The ground sample distance (pixel footprint) and swath width of a nadir-looking frame camera or scanner from its pixel pitch, focal length, flying height above ground and number of pixels across track.

$$GSD = \frac{p\,H}{f}, \quad W = GSD \times N$$

input	meaning	condition
`pixel_pitch`	detector pixel pitch	
`focal_length`	focal length	
`altitude`	height above the ground	
`pixels`	number of pixels across track	

output	meaning
`gsd`	ground sample distance
`swath`	swath width

- **Method:** similar triangles: GSD = pixel pitch x height / focal length; swath = GSD x pixels
- **Assumes:** flat ground, nadir view
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ground sample distance (top, in cm) and swath width (bottom) returned by the package for a camera with pixel pitch 2.4e-06 m, focal length 0.0088 m and 5472 pixels across, at altitudes 10–400 m: both grow in proportion to altitude. Marked: the worked example at 120.0 m, GSD 0.0327273 m, swath 179.084 m.

```
geospatial_analytics_ai999x.solve('geo_ground_sample_distance', pixel_pitch=2.4e-06,
focal_length=0.0088, altitude=120.0, pixels=5472)
# -> gsd = 0.0327273 m, swath = 179.084 m   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_ground_sample_distance', pixel_pitch=1.2e-05,
focal_length=1.2, altitude=700000.0, pixels=12000)
# -> gsd = 7 m, swath = 84000 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ground sample distance: pixel_pitch = 0.0024 mm, focal_length = 8.8 mm, altitude = 120 m, pixels = 5472	SOLVED	gsd = 0.0327273 m, swath = 179.084 m
Ground sample distance: pixel_pitch = 0.012 mm, focal_length = 1200 mm, altitude = 700 km, pixels = 12000	SOLVED	gsd = 7 m, swath = 84000 m
Ground sample distance: focal_length = 8.8 mm, altitude = 120 m, pixels = 5472	NEEDS_INPUT	geo_ground_sample_distance needs pixel_pitch (detector pixel pitch). Example: solve('geo_ground_sample_distance', pixel_
Ground sample distance: pixel_pitch = 0.0024 mm, focal_length = 8.8 mm, altitude = 120 m, pixels = 5472, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Thematic map accuracy: overall accuracy, kappa, producer's and user's accuracy — `geo_classification_accuracy`

The accuracy assessment of a classified map from its error (confusion) matrix: overall accuracy, Cohen's kappa, and the producer's and user's accuracy of each class.

$$OA = \frac{\sum_i n_{ii}}{N}, \quad \kappa = \frac{p_o - p_e}{1 - p_e}, \quad PA_j = \frac{n_{jj}}{n_{+j}}, \quad UA_i = \frac{n_{ii}}{n_{i+}}$$

input	meaning	condition
`matrix`	confusion matrix of sample counts: rows = map (classified) classes, columns = reference classes	

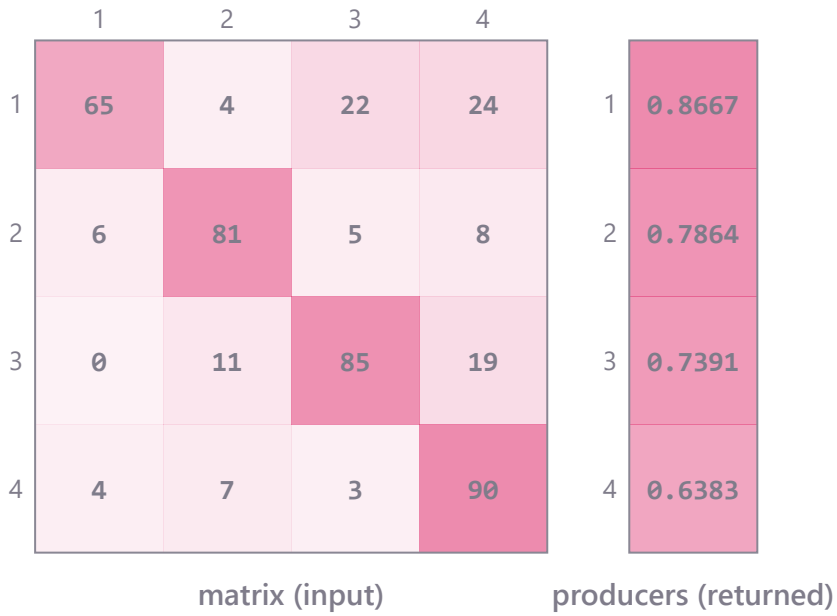
output	meaning
`overall`	overall accuracy
`kappa`	Cohen's kappa
`producers`	producer's accuracy of each class (1 - omission error)
`users`	user's accuracy of each class (1 - commission error)

- **Method:** overall = trace/N; $\kappa = (po - pe)/(1 - pe)$ with pe from the row and column totals; producer's = diagonal/column total, user's = diagonal/row total
- **Assumes:** square matrix of whole counts; every class sampled in the map and the reference

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Per-class accuracies the package returns for the worked example's 4 × 4 error matrix (rows: classified, columns: reference): producer's accuracy 0.866667, 0.786408, 0.73913, 0.638298; user's accuracy 0.565217, 0.81, 0.73913, 0.865385; overall 0.739631, κ = 0.653516.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): matrix (input); producers (returned). Returned with them by geospatial-analytics-ai999x: overall = 0.739631, kappa = 0.653516.

```
geospatial_analytics_ai999x.solve('geo_classification_accuracy', matrix=[[65, 4, 22, 24], [6, 81,
5, 8], [0, 11, 85, 19], [4, 7, 3, 90]])
# -> overall = 0.739631, kappa = 0.653516, producers = [0.866666666667, 0.78640776699,
0.739130434783, 0.63829787234], users = [0.565217391304, 0.81, 0.739130434783, 0.865384615385]
(SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_classification_accuracy', matrix=[[45, 5], [10, 40]])
# -> overall = 0.85, kappa = 0.7, producers = [0.818181818182, 0.888888888889], users = [0.9, 0.8]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Classification accuracy: matrix = [[65, 4, 22, 24], [6, 81, 5, 8], [0, 11, 85, 19], [4, 7, 3, 90]]	SOLVED	overall = 0.739631, kappa = 0.653516, producers = [0.866666666667, 0.78640776699, 0.739130434783, 0.63829787234], users = [0.565217391304, 0.81, 0.739130434783, 0.865384615385]
Classification accuracy: matrix = [[45, 5], [10, 40]]	SOLVED	overall = 0.85, kappa = 0.7, producers = [0.818181818182, 0.888888888889], users = [0.9, 0.8]
Classification accuracy:	NEEDS_INPUT	geo_classification_accuracy needs matrix (confusion matrix of sample counts: rows = map (classified) classes, columns =
Classification accuracy: matrix = [[65, 4, 22, 24], [6, 81, 5, 8], [0, 11, 85, 19], [4, 7, 3, 90]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Slope and aspect of a 3x3 elevation window (Horn's method) — `geo_slope_aspect_horn`

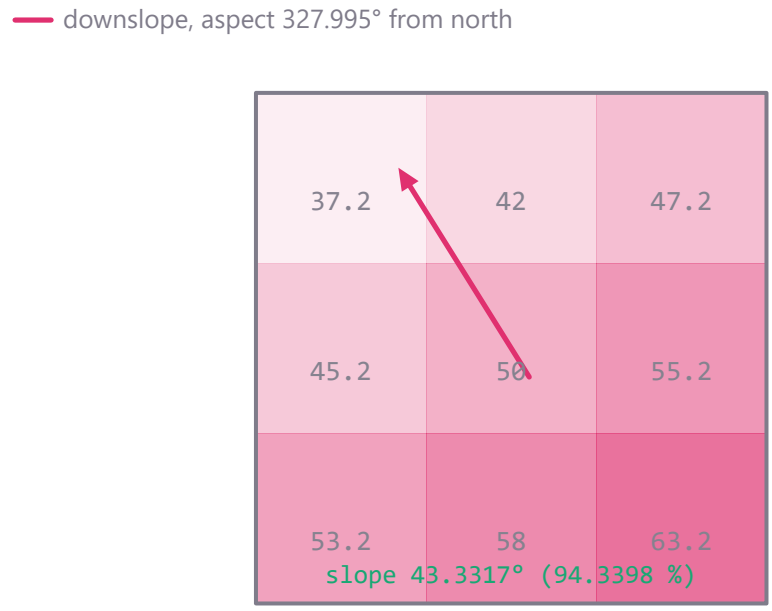
The slope (degrees and per cent) and aspect (downslope direction, degrees clockwise from north) at the centre cell of a 3 x 3 window of a digital elevation model, by Horn's third-order finite differences.

$$\frac{\partial z}{\partial x} = \frac{(z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)}{8\Delta},\ \frac{\partial z}{\partial y} = \frac{(z_1 + 2z_2 + z_3) - (z_7 + 2z_8 + z_9)}{8\Delta},\ S = \arctan\|\nabla z\|$$

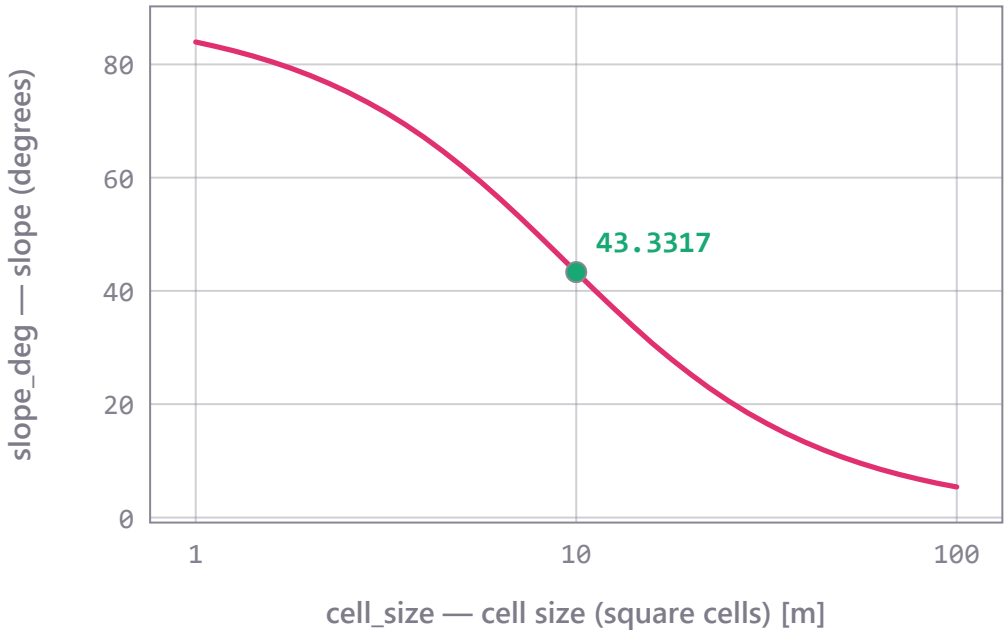
input	meaning	condition
`window`	3 x 3 elevations in metres, row 0 to the north, column 0 to the west	
`cell_size`	cell size (square cells)	

output	meaning
`slope_deg`	slope (degrees)
`slope_percent`	slope (per cent)
`aspect_deg`	aspect: downslope direction clockwise from north (none for a flat window)

- **Method:** Horn (1981): dz/dx = (($c + 2f + i$) − ($a + 2d + g$))/(8 dx), dz/dy = (($a + 2b + c$) − ($g + 2h + i$))/(8 dy) with y north; slope = atan(|grad|), aspect = atan2(-dz/dx, -dz/dy)
- **Assumes:** square cells; row 0 is the northern row
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 3 × 3 elevation window of the worked example (cell size 10.0 m, north at the top; values in m) and the slope and aspect the package returns for its centre: slope 43.3317° (94.3398 %), aspect 327.995° clockwise from north, drawn as an arrow pointing downslope toward the low corner.



slope_deg (slope (degrees)) against cell_size (cell size (square cells)) from 1 m to 100 m on a logarithmic axis. Other inputs as in the worked example: window = [[37.2, 42, 47.2], [45.2, 50, 55.2], [53.2, 5…. 41 points computed by geospatial-analytics-ai999x; marked: the worked example, cell_size = 10 m, slope_deg = 43.3317.

```
geospatial_analytics_ai999x.solve('geo_slope_aspect_horn', window=[[37.2, 42, 47.2], [45.2, 50,
55.2], [53.2, 58, 63.2]], cell_size=10.0)
# -> slope_deg = 43.3317, slope_percent = 94.3398, aspect_deg = 327.995   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_slope_aspect_horn', window=[[100, 101, 102], [100, 101,
102], [100, 101, 102]], cell_size=10.0)
# -> slope_deg = 5.71059, slope_percent = 10, aspect_deg = 270   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Slope and aspect: window = [[37.2, 42, 47.2], [45.2, 50, 55.2], [53.2, 58, 63.2]], cell_size = 10 m	SOLVED	slope_deg = 43.3317, slope_percent = 94.3398, aspect_deg = 327.995
Slope and aspect: window = [[100, 101, 102], [100, 101, 102], [100, 101, 102]], cell_size = 10 m	SOLVED	slope_deg = 5.71059, slope_percent = 10, aspect_deg = 270
Slope and aspect: cell_size = 10 m	NEEDS_INPUT	geo_slope_aspect_horn needs window (3 x 3 elevations in metres, row 0 to the north, column 0 to the west). Example: solv
Slope and aspect: window = [[37.2, 42, 47.2], [45.2, 50, 55.2], [53.2, 58, 63.2]], cell_size = 10 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Hillshade (analytical relief shading) of a 3x3 elevation window — `geo_hillshade`

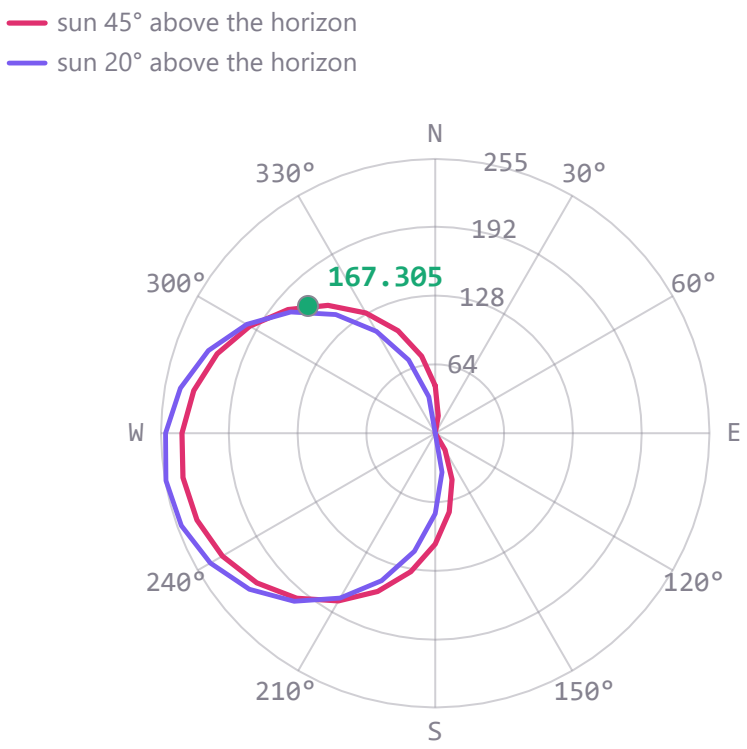
The illumination (cosine of the solar incidence angle on the terrain) and the $0-255$ hillshade value at the centre of a 3 x 3 elevation window for a sun azimuth and altitude.

$$\cos i = \cos Z \cos S + \sin Z \sin S \cos(A - \alpha), \quad H = 255 \max(0, \cos i)$$

input	meaning	condition
`window`	3 x 3 elevations in metres, row 0 to the north	
`cell_size`	cell size	
`sun_azimuth`	sun azimuth, clockwise from north (decimal degrees)	
`sun_altitude`	sun altitude above the horizon (decimal degrees)	

output	meaning
`illumination`	cos(incidence angle) (negative in self-shadow)
`hillshade`	255 x max(0, illumination)

- **Method:** Horn slope and aspect, then cos $i = cos$(Zenith) cos(Slope) + sin(Zenith) sin(Slope) cos(Azimuth - Aspect)
- **Assumes:** row 0 is the northern row; no cast shadows
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hillshade of the worked example's 3 × 3 window (cell size 5.0 m) as the sun goes round: distance from the centre is the hillshade value (0–255) the package returns for a sun at that azimuth (clockwise from north, every 10°), at altitudes 45° and 20°; the slope faces roughly west, so it is brightest under a western sun. Marked: the worked example, sun at 315°, hillshade 167.305 (illumination 0.656099).

```
geospatial_analytics_ai999x.solve('geo_hillshade', window=[[2452.25, 2463, 2473.75], [2450, 2461,
2472], [2447.75, 2459, 2470.25]], cell_size=5.0, sun_azimuth=315, sun_altitude=45)
# -> illumination = 0.656099, hillshade = 167.305   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_hillshade', window=[[100, 101, 102], [100, 101, 102], [100,
101, 102]], cell_size=10.0, sun_azimuth=135, sun_altitude=30)
# -> illumination = 0.436585, hillshade = 111.329   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hillshade: window = [[2452.25, 2463, 2473.75], [2450, 2461, 2472], [2447.75, 2459, 2470.25]], cell_size = 5 m, sun_azimuth = 315, sun_altitude = 45	SOLVED	illumination = 0.656099, hillshade = 167.305
Hillshade: window = [[100, 101, 102], [100, 101, 102], [100, 101, 102]], cell_size = 10 m, sun_azimuth = 135, sun_altitude = 30	SOLVED	illumination = 0.436585, hillshade = 111.329
Hillshade: cell_size = 5 m, sun_azimuth = 315, sun_altitude = 45	NEEDS_INPUT	geo_hillshade needs window (3 x 3 elevations in metres, row 0 to the north). Example: solve('geo_hillshade', window=[[24
Hillshade: window = [[2452.25, 2463, 2473.75], [2450, 2461, 2472], [2447.75,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
2459, 2470.25]], cell_size = 5 m, sun_azimuth = 315, sun_altitude = 45, zz = 1		geospatial_analytics_ai999x.templ

Terrain curvature: total, profile and plan curvature (Zevenbergen-Thorne) — `geo_terrain_curvature`

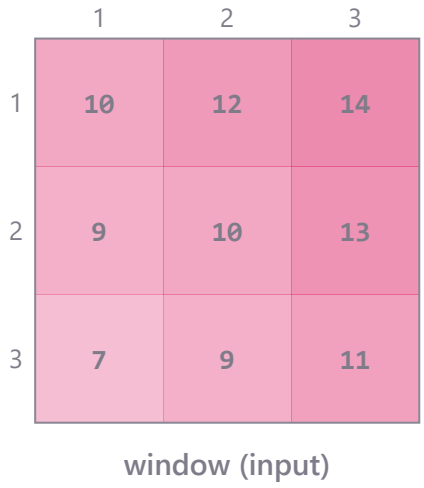
The total, profile (along the slope) and plan (across the slope) curvature at the centre of a 3 x 3 elevation window from the Zevenbergen-Thorne partial quartic surface.

$$K = -2(D+E), \quad K_{prof} = \frac{-2(DG^2 + EH^2 + FGH)}{G^2 + H^2}, \quad K_{plan} = \frac{2(DH^2 + EG^2 - FGH)}{G^2 + H^2}$$

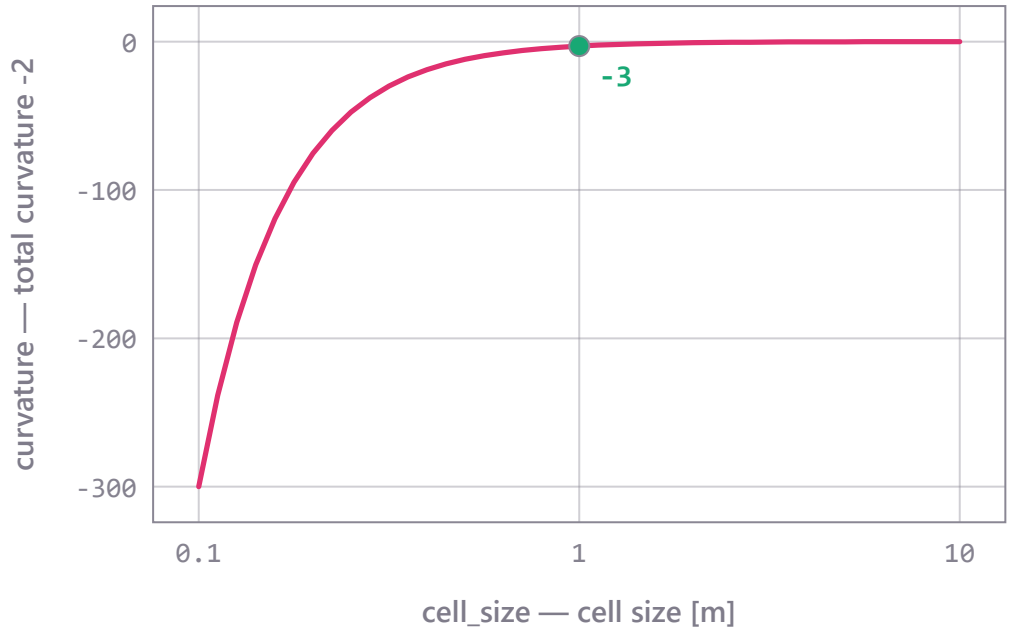
input	meaning	condition
`window`	3 x 3 elevations in metres, row 0 to the north	
`cell_size`	cell size	

output	meaning
`curvature`	total curvature -2(D + E) (1/m; positive upwardly convex)
`profile`	profile curvature $(1/m)$
`plan`	plan curvature $(1/m)$

- **Method:** Zevenbergen and Thorne (1987): D, E, F, G, H from the window; profile = -2(DG^2 + EH^2 + FGH)/(G^2 + H^2), plan = 2(DH^2 + EG^2 - FGH)/(G^2 + H^2) (sign convention of Moore et al. 1991 and ArcGIS, without the factor 100)
- **Assumes:** non-zero gradient at the centre for profile and plan curvature
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): window (input). Returned with them by geospatial-analytics-ai999x: curvature = -3, profile = -1.64, plan = 1.36. Other inputs: cell_size = 1.0 m.



curvature (total curvature -2(D + E) (1/m; positive upwardly convex)) against cell_size (cell size) from 0.1 m to 10 m on a logarithmic axis. Other inputs as in the worked example: window = [[10, 12, 14], [9, 10, 13], [7, 9, 11]]. 41 points computed by geospatial-analytics-ai999x; marked: the worked example, cell_size = 1 m, curvature = -3.

```
geospatial_analytics_ai999x.solve('geo_terrain_curvature', window=[[10, 12, 14], [9, 10, 13], [7,
9, 11]], cell_size=1.0)
# -> curvature = -3, profile = -1.64, plan = 1.36   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_terrain_curvature', window=[[105, 100, 98], [104, 101, 97],
[106, 103, 99]], cell_size=10.0)
# -> curvature = 0, profile = 0.00689655, plan = 0.00689655   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Terrain curvature: window = [[10, 12, 14], [9, 10, 13], [7, 9, 11]], cell_size = 1 m	SOLVED	curvature = -3, profile = -1.64, plan = 1.36
Terrain curvature: window = [[105, 100, 98], [104, 101, 97], [106, 103, 99]], cell_size = 10 m	SOLVED	curvature = 0, profile = 0.00689655, plan = 0.00689655
Terrain curvature: cell_size = 1 m	NEEDS_INPUT	geo_terrain_curvature needs window (3 x 3 elevations in metres, row 0 to the north). Example: solve('geo_terrain_curvatu
Terrain curvature: window = [[10, 12, 14], [9, 10, 13], [7, 9, 11]], cell_size = 1 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Terrain ruggedness index (Riley), topographic position index and local relief — `geo_terrain_ruggedness`

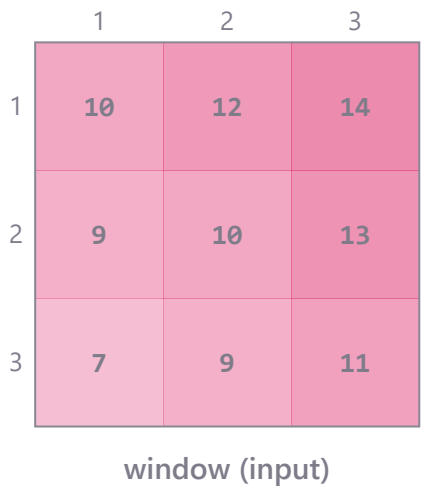
Riley's terrain ruggedness index (root of the summed squared elevation differences to the 8 neighbours), the topographic position index (centre minus neighbour mean) and the local relief of a 3 x 3 window.

$$TRI = \sqrt{\sum_{i=1}^{8}(z_i - z_0)^2}, \quad TPI = z_0 - \frac{1}{8}\sum_{i=1}^{8} z_i$$

input	meaning	condition
`window`	3 x 3 elevations in metres	

output	meaning
`TRI`	terrain ruggedness index (m)
`TPI`	topographic position index (m)
`relief`	maximum minus minimum elevation (m)

- **Method:** TRI = sqrt(sum (z_i - z_0)^2); TPI = z_0 - mean(z_i); relief = max - min
- **Assumes:** Riley et al. (1999) form of the TRI
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



window (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): window (input). Returned with them by geospatial-analytics-ai999x: TRI = 6.40312, TPI = -0.625, relief = 7.

```
geospatial_analytics_ai999x.solve('geo_terrain_ruggedness', window=[[10, 12, 14], [9, 10, 13], [7, 9, 11]])
# -> TRI = 6.40312, TPI = -0.625, relief = 7   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_terrain_ruggedness', window=[[105, 100, 98], [104, 101, 97], [106, 103, 99]])
# -> TRI = 9.16515, TPI = -0.5, relief = 9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Terrain ruggedness index: window = [[10, 12, 14], [9, 10, 13], [7, 9, 11]]	SOLVED	TRI = 6.40312, TPI = -0.625, relief = 7

question	outcome	result
Terrain ruggedness index: window = [[105, 100, 98], [104, 101, 97], [106, 103, 99]]	SOLVED	TRI = 9.16515, TPI = -0.5, relief = 9
Terrain ruggedness index:	NEEDS_INPUT	geo_terrain_ruggedness needs window (3 x 3 elevations in metres). Example: solve('geo_terrain_ruggedness', window= [[10,
Terrain ruggedness index: window = [[10, 12, 14], [9, 10, 13], [7, 9, 11]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

D8 flow direction of every cell of a small DEM — `geo_d8_flow_direction`

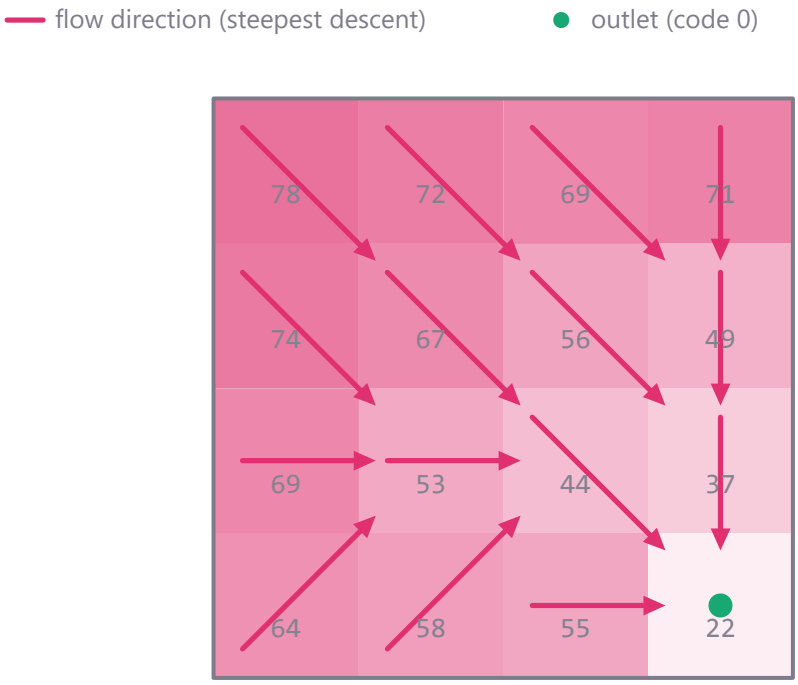
The D8 (steepest-descent) flow direction of every cell of a digital elevation model, as ESRI codes (1 E, 2 SE, 4 S, 8 SW, 16 W, 32 NW, 64 N, 128 NE; 0 where no neighbour inside the raster is lower).

$$d^{*} = \arg\max_{k} \frac{z_0 - z_k}{\ell_k}, \quad \ell_k \in \{\Delta, \sqrt{2}\Delta\}$$

input	meaning	condition
`dem`	elevations (row 0 to the north), at most 900 cells	
`cell_size`	cell size	

output	meaning
`direction`	D8 code of each cell (matrix)

- **Method:** for each cell the neighbour with the largest drop per unit distance (positive drops only)
- **Assumes:** square cells; diagonal distance sqrt(2) cells; ties go to the first direction in the order E, SE, S, SW, W, NW, N, NE
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example's 4 × 4 elevation grid (cell size 10.0 m; shading and printed values: elevation in m, north at the top) with an arrow in each cell for the D8 direction the package returns (codes 1 = east, 2 = south-east, 4 = south, 8 = south-west, 16 = west, 32 = north-west, 64 = north, 128 = north-east): 2 2 2 4; 2 2 2 4; 1 1 2 4; 128 128 1 0. Every path drains to the lowest corner, the outlet (code 0).

```
geospatial_analytics_ai999x.solve('geo_d8_flow_direction', dem=[[78, 72, 69, 71], [74, 67, 56, 49], [69, 53, 44, 37], [64, 58, 55, 22]], cell_size=10.0)
# -> direction = [[2.0, 2.0, 2.0, 4.0], [2.0, 2.0, 2.0, 4.0], [1.0, 1.0, 2.0, 4.0], [128.0, 128.0, 1.0, 0.0]]   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_d8_flow_direction', dem=[[10, 9, 8], [9, 7, 5], [8, 5, 1]], cell_size=1.0)
# -> direction = [[2.0, 2.0, 4.0], [2.0, 2.0, 4.0], [1.0, 1.0, 0.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
D8 flow direction: dem = [[78, 72, 69, 71], [74, 67, 56, 49], [69, 53, 44, 37], [64, 58, 55, 22]], cell_size = 10 m	SOLVED	direction = [[2.0, 2.0, 2.0, 4.0], [2.0, 2.0, 2.0, 4.0], [1.0, 1.0, 2.0, 4.0], [128.0, 128.0, 1.0, 0.0]]
D8 flow direction: dem = [[10, 9, 8], [9, 7, 5], [8, 5, 1]], cell_size = 1 m	SOLVED	direction = [[2.0, 2.0, 4.0], [2.0, 2.0, 4.0], [1.0, 1.0, 0.0]]
D8 flow direction: cell_size = 10 m	NEEDS_INPUT	geo_d8_flow_direction needs dem (elevations (row 0 to the north), at most 900 cells). Example: solve('geo_d8_flow_direct
D8 flow direction: dem = [[78, 72, 69, 71], [74, 67, 56, 49], [69, 53, 44, 37], [64, 58, 55, 22]], cell_size = 10 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

D8 flow accumulation of a small DEM — `geo_d8_flow_accumulation`

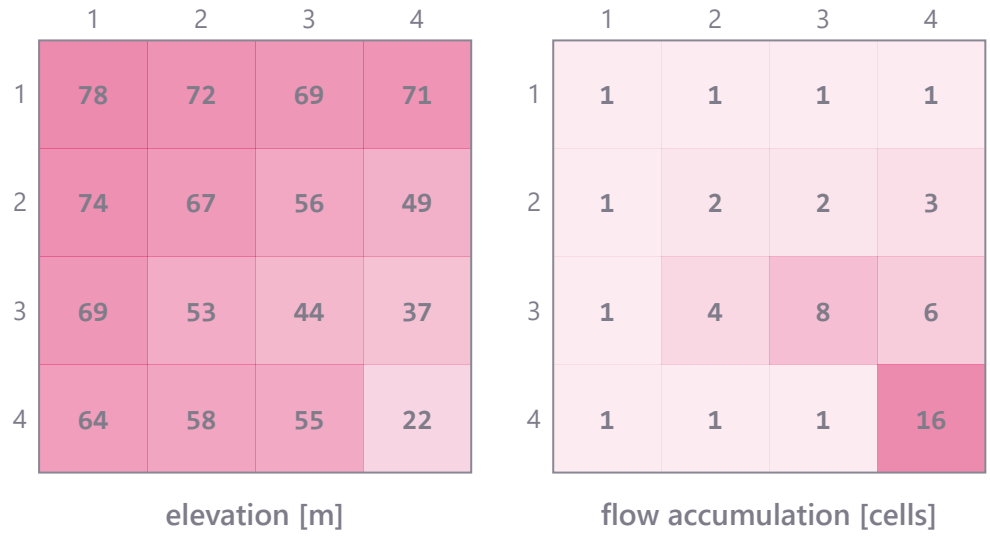
The number of cells draining through each cell (the cell itself included) under D8 flow directions, and the largest accumulation (the main outlet).

$$A(c) = 1 + \sum_{u \to c} A(u)$$

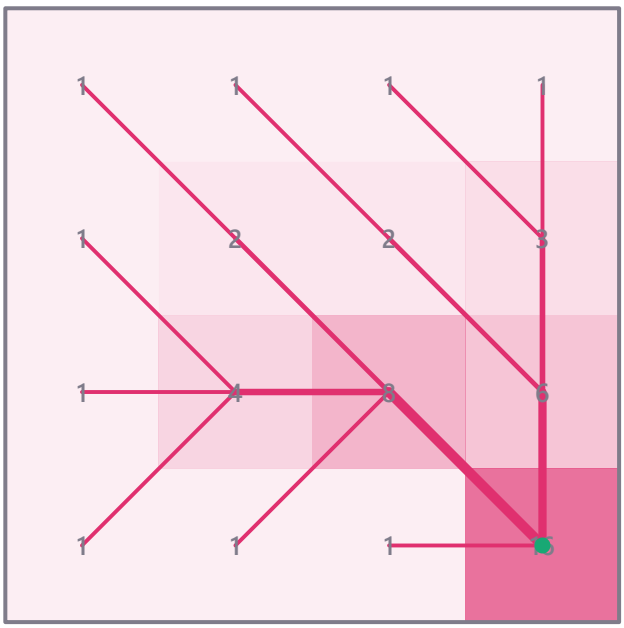
input	meaning	condition
`dem`	elevations (row 0 to the north), at most 900 cells	
`cell_size`	cell size	

output	meaning
`accumulation`	contributing cells of each cell, itself included (matrix)
`max_accumulation`	largest accumulation

- **Method:** D8 directions, then accumulation in topological order of the flow graph (Kahn's algorithm)
- **Assumes:** D8 directions as in geo_d8_flow_direction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A 4 × 4 elevation grid (left, metres, cell size 10 m) and the D8 flow accumulation the package computes from it (right): each cell shows how many cells drain through it, itself included; the maximum, 16, is the outlet in the lowest corner. Each panel is shaded by its own maximum.



Flow accumulation on the worked example's elevation grid: each cell shows the number of cells draining through it (itself included) returned by the package, and a stream line to its downstream neighbour (direction from the package's D8 capability) whose width grows with that number; the outlet collects all 16 cells.

```
geospatial_analytics_ai999x.solve('geo_d8_flow_accumulation', dem=[[78, 72, 69, 71], [74, 67, 56, 49], [69, 53, 44, 37], [64, 58, 55, 22]], cell_size=10.0)
# -> accumulation = [[1.0, 1.0, 1.0, 1.0], [1.0, 2.0, 2.0, 3.0], [1.0, 4.0, 8.0, 6.0], [1.0, 1.0, 1.0, 16.0]], max_accumulation = 16   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_d8_flow_accumulation', dem=[[10, 9, 8], [9, 7, 5], [8, 5, 1]], cell_size=1.0)
# -> accumulation = [[1.0, 1.0, 1.0], [1.0, 2.0, 3.0], [1.0, 3.0, 9.0]], max_accumulation = 9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flow accumulation: dem = [[78, 72, 69, 71], [74, 67, 56, 49], [69, 53, 44, 37], [64, 58, 55, 22]], cell_size = 10 m	SOLVED	accumulation = [[1.0, 1.0, 1.0, 1.0], [1.0, 2.0, 2.0, 3.0], [1.0, 4.0, 8.0, 6.0], [1.0, 1.0, 1.0, 16.0]], max_accumulation = 16
Flow accumulation: dem = [[10, 9, 8], [9, 7, 5], [8, 5, 1]], cell_size = 1 m	SOLVED	accumulation = [[1.0, 1.0, 1.0], [1.0, 2.0, 3.0], [1.0, 3.0, 9.0]], max_accumulation = 9
Flow accumulation: cell_size = 10 m	NEEDS_INPUT	geo_d8_flow_accumulation needs dem (elevations (row 0 to the north), at most 900 cells). Example: solve('geo_d8_flow_acc
Flow accumulation: dem = [[78, 72, 69, 71], [74, 67, 56, 49], [69, 53, 44, 37], [64, 58, 55, 22]], cell_size = 10 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Watershed (catchment) of an outlet cell on a small DEM — `geo_watershed_delineation`

The cells that drain to a chosen outlet cell under D8 flow directions: their number, the catchment area and the catchment mask.

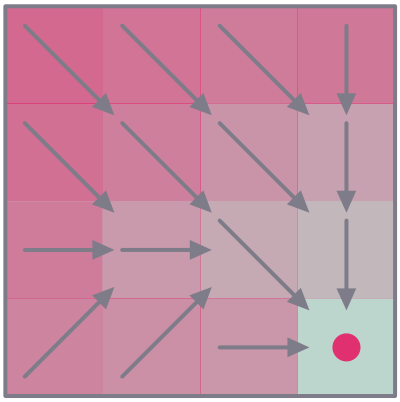
$$W(o) = \{c : c \to \cdots \to o\}$$

input	meaning	condition
`dem`	elevations (row 0 to the north), at most 900 cells	
`cell_size`	cell size	
`row`	outlet row (from 0, north)	
`col`	outlet column (from 0, west)	

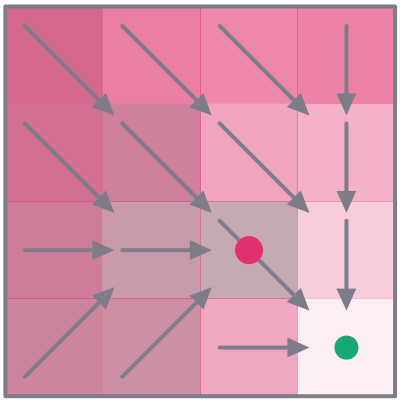
output	meaning
`cells`	number of cells in the catchment (outlet included)
`area`	catchment area
`mask`	1 for cells in the catchment, 0 elsewhere (matrix)

- **Method:** D8 directions, then a search upstream from the outlet through the cells that flow into it
- **Assumes:** D8 directions as in geo_d8_flow_direction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

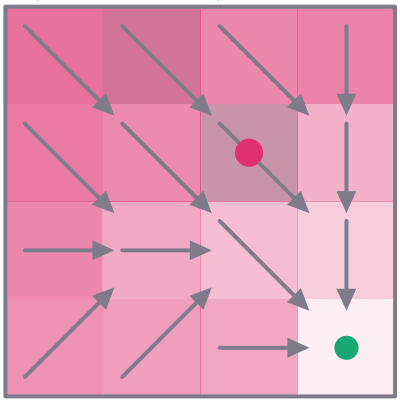
outlet (row 3, col 3): 16 cells, 1600 m²



outlet (row 2, col 2): 8 cells, 800 m²



outlet (row 1, col 2): 2 cells, 200 m²



Watersheds on the worked example's elevation grid (cell size 10.0 m) for three outlets (dot): the cells of the mask the package returns are shaded, the arrows are the D8 directions from its flow-direction capability. Rows and columns count from 0 at the top left. row 3, column 3: 16 cells, 1600 m^2; row 2, column 2: 8 cells, 800 m^2; row 1, column 2: 2 cells, 200 m^2 — the first is the worked example, the grid's outlet, which drains everything.

```
geospatial_analytics_ai999x.solve('geo_watershed_delineation', dem=[[78, 72, 69, 71], [74, 67, 56, 49], [69, 53, 44, 37], [64, 58, 55, 22]], cell_size=10.0, row=3, col=3)
# -> cells = 16, area = 1600 m^2, mask = [[1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0]]   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_watershed_delineation', dem=[[10, 9, 8], [9, 7, 5], [8, 5,
1]], cell_size=1.0, row=2, col=2)
# -> cells = 9, area = 9 m^2, mask = [[1.0, 1.0, 1.0], [1.0, 1.0, 1.0], [1.0, 1.0, 1.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Watershed delineation: dem = [[78, 72, 69, 71], [74, 67, 56, 49], [69, 53, 44, 37], [64, 58, 55, 22]], cell_size = 10 m, row = 3, col = 3	SOLVED	cells = 16, area = 1600 m^2, mask = [[1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0]]
Watershed delineation: dem = [[10, 9, 8], [9, 7, 5], [8, 5, 1]], cell_size = 1 m, row = 2, col = 2	SOLVED	cells = 9, area = 9 m^2, mask = [[1.0, 1.0, 1.0], [1.0, 1.0, 1.0], [1.0, 1.0, 1.0]]
Watershed delineation: cell_size = 10 m, row = 3, col = 3	NEEDS_INPUT	geo_watershed_delineation needs dem (elevations (row 0 to the north), at most 900 cells). Example: solve('geo_watershed_
Watershed delineation: dem = [[78, 72, 69, 71], [74, 67, 56, 49], [69, 53, 44, 37], [64, 58, 55, 22]], cell_size = 10 m, row = 3, col = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Line-of-sight visibility along a terrain profile (with Earth curvature and refraction) — `geo_line_of_sight_profile`

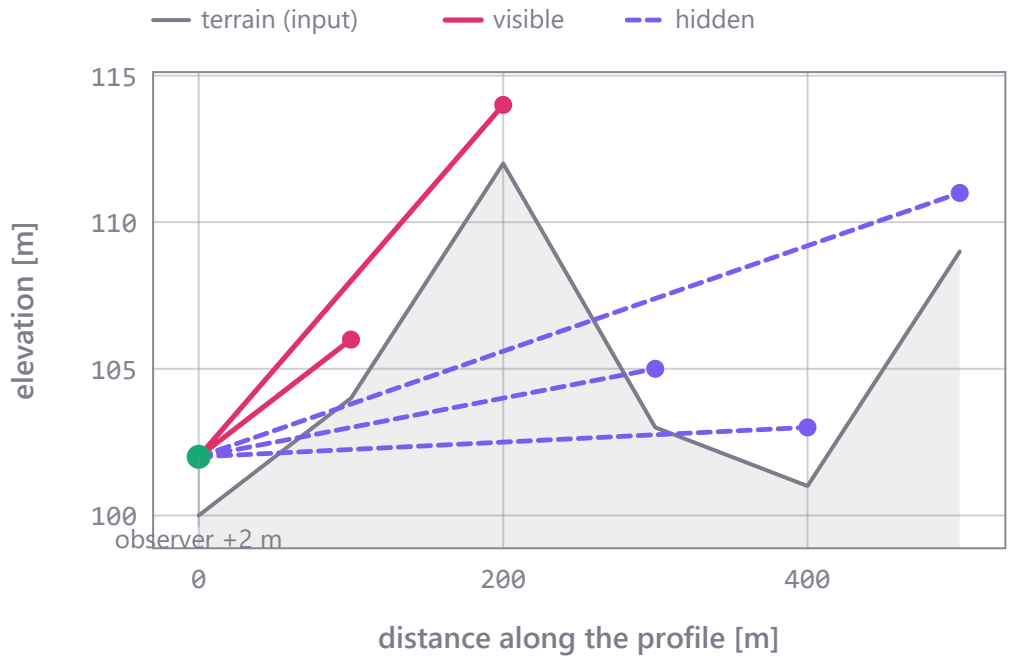
Which points along a terrain profile are visible from an observer at its start, for given observer and target heights above the ground, optionally with the Earth-curvature and refraction correction.

$$\text{visible}(i) \iff z_j < z_{eye} + (z_i + h_t - z_{eye})\frac{d_j}{d_i}\ \forall\, 0 < j < i, \quad \Delta z_{curv} = \frac{d^2(1-k)}{2R}$$

input	meaning	condition
`d`	distances along the profile in metres, starting at 0 (the observer)	
`z`	ground elevations in metres	
`observer_height`	eye height above the ground	
`target_height`	target height above the ground	
`curvature`	none, or the drop $d^2\,(1-k)/(2R)$ with $R = 6371$ km and $k = 0.13$	

output	meaning
`visible`	1 if the target over each point is visible, else 0 (the observer's own point is 1)
`n_visible`	number of visible points beyond the observer

- **Method:** a target is visible when every intermediate ground point lies strictly below the sight line from the eye to the target
- **Assumes:** distances increase from 0; positive observer and target heights
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A terrain profile (elevations [100, 104, 112, 103, 101, 109] m at [0, 100, 200, 300, 400, 500] m) seen by an observer 2 m above the first point, looking at targets 2 m above each point (curvature: none): sight lines are drawn solid where the package returns the point as visible and dashed where hidden — 2 of the 5 other points are visible; the ridge at 200 m hides those beyond.

```
geospatial_analytics_ai999x.solve('geo_line_of_sight_profile', d=[0, 100, 200, 300, 400, 500], z=
[100, 104, 112, 103, 101, 109], observer_height=2.0, target_height=2.0, curvature='none')
# -> visible = [1.0, 1.0, 1.0, 0.0, 0.0, 0.0], n_visible = 2   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_line_of_sight_profile', d=[0, 5000, 10000, 15000, 20000], z=
[50, 52, 49, 51, 55], observer_height=10.0, target_height=10.0, curvature='earth_refraction')
# -> visible = [1.0, 1.0, 1.0, 1.0, 1.0], n_visible = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Line of sight: d = [0, 100, 200, 300, 400, 500], z = [100, 104, 112, 103, 101, 109], observer_height = 2 m, target_height = 2 m, curvature = none	SOLVED	visible = [1.0, 1.0, 1.0, 0.0, 0.0, 0.0], n_visible = 2
Line of sight: d = [0, 5000, 10000, 15000, 20000], z = [50, 52, 49, 51, 55], observer_height = 10 m, target_height = 10 m, curvature = earth_refraction	SOLVED	visible = [1.0, 1.0, 1.0, 1.0, 1.0], n_visible = 4
Line of sight: z = [100, 104, 112, 103, 101, 109], observer_height = 2 m, target_height = 2 m, curvature = none	NEEDS_INPUT	geo_line_of_sight_profile needs d (distances along the profile in metres, starting at 0 (the observer)). Example: solve(
Line of sight: d = [0, 100, 200, 300, 400, 500], z = [100, 104, 112, 103, 101, 109], observer_height = 2 m, target_height = 2 m, curvature = none, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Gravity-based (Hansen) accessibility on a road network — `geo_gravity_accessibility`

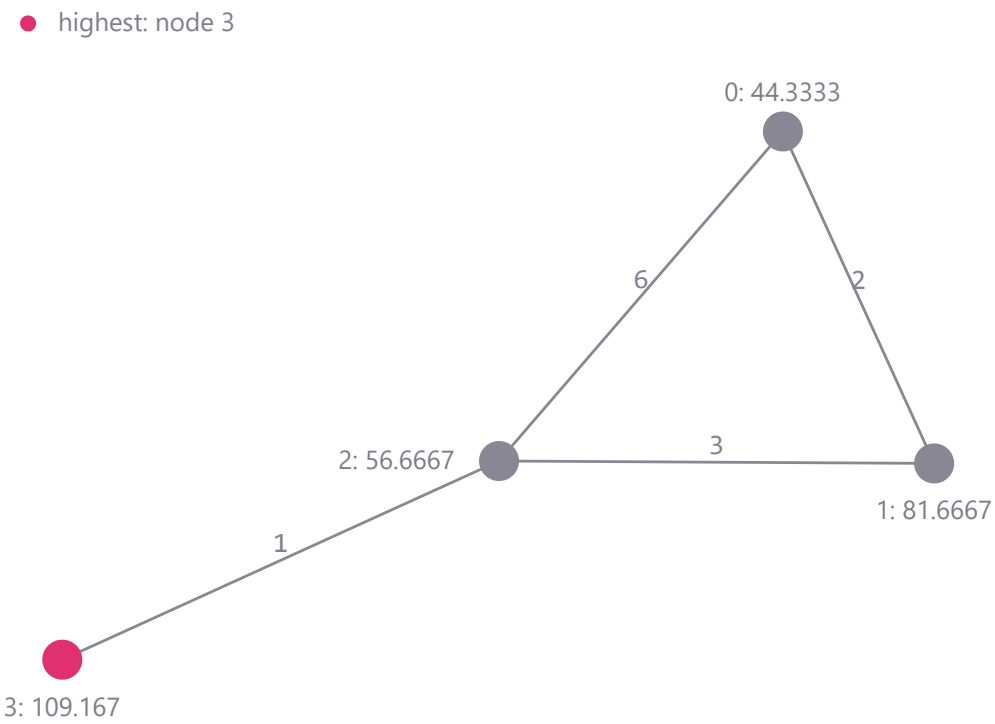
The Hansen gravity accessibility of every node of a network: the sum of the opportunities (jobs, services) at the other nodes discounted by a power or negative-exponential function of the shortest-path distance.

$$A_i = \sum_{j \neq i} O_j \, f(d_{ij}), \quad f(d) = d^{-\beta} \text{ or } e^{-\beta d}$$

input	meaning	condition
`edges`	undirected network as weighted edges between nodes numbered 0 .. $n-1$	
`opportunities`	opportunities at nodes 0 .. $n-1$	
`beta`	distance-decay parameter	
`impedance`	impedance function $d^{-\beta}$ or exp(-beta d)	

output	meaning
`accessibility`	accessibility of each node

- **Method:** Dijkstra shortest paths from every node; A_i = sum_{$j \neq i$} O_j f(d_ij)
- **Assumes:** positive edge lengths; a node's own opportunities are excluded; unreachable nodes contribute 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example's network (edge lengths printed; layout without meaning) with each node labelled by the gravity accessibility the package returns: node 0 44.3333, node 1 81.6667, node 2 56.6667, node 3 109.167.

```
geospatial_analytics_ai999x.solve('geo_gravity_accessibility', edges='0-1 2; 1-2 3; 0-2 6; 2-3 1',
opportunities=[100, 50, 80, 20], beta=1, impedance='power')
# -> accessibility = [44.3333333333, 81.6666666667, 56.6666666667, 109.166666667]   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_gravity_accessibility', edges='0-1 5; 1-2 5; 2-3 5; 3-0 5',
opportunities=[10, 20, 30, 40], beta=0.1, impedance='exponential')
# -> accessibility = [47.4282228179, 38.9764040354, 40.0706339945, 31.6188152119]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gravity accessibility: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1], opportunities = [100, 50, 80, 20], beta = 1, impedance = power	SOLVED	accessibility = [44.3333333333, 81.6666666667, 56.6666666667, 109.166666667]
Gravity accessibility: edges = [0-1 5; 1-2 5; 2-3 5; 3-0 5], opportunities = [10, 20, 30, 40], beta = 0.1, impedance = exponential	SOLVED	accessibility = [47.4282228179, 38.9764040354, 40.0706339945, 31.6188152119]
Gravity accessibility: opportunities = [100, 50, 80, 20], beta = 1, impedance = power	NEEDS_INPUT	geo_gravity_accessibility needs edges (undirected network as weighted edges between nodes numbered 0 .. n-1). Example: s
Gravity accessibility: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1], opportunities = [100, 50, 80, 20], beta = 1, impedance = power, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Huff model: probabilities of shopping at each store from an origin (network distances) — `geo_huff_model`

The Huff probabilities that a consumer at an origin node patronizes each store, from store attractiveness (floor space) and shortest-path distances on a road network.

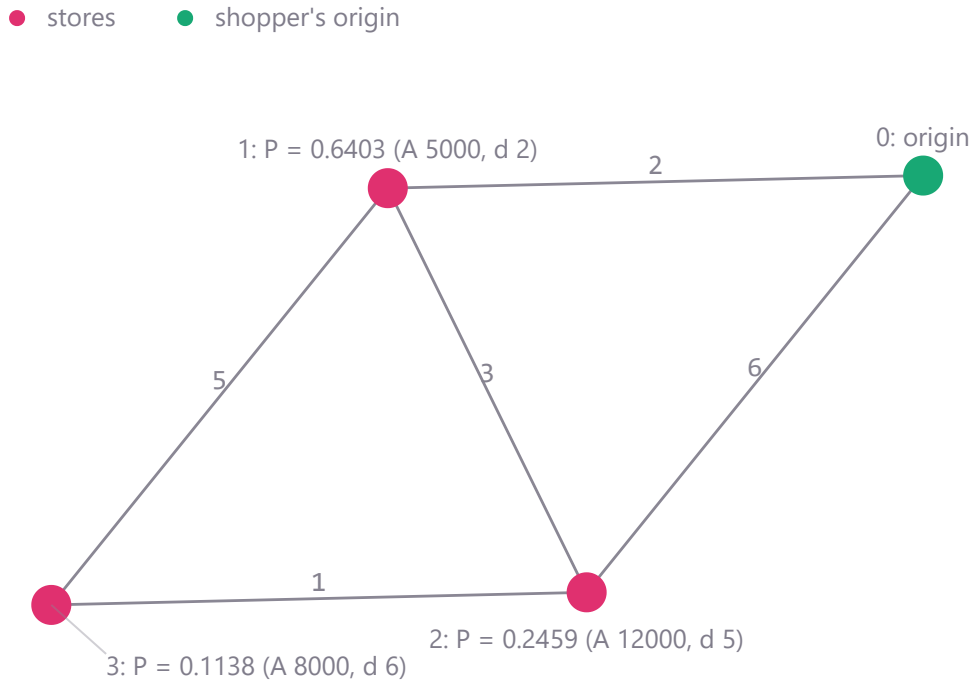
$$P_{ij} = \frac{S_j^{\alpha} d_{ij}^{-\beta}}{\sum_k S_k^{\alpha} d_{ik}^{-\beta}}$$

input	meaning	condition
`edges`	undirected network as weighted edges between nodes numbered 0 .. $n-1$	
`origin`	origin node	
`stores`	store nodes	
`attractiveness`	attractiveness of each store (e.g. floor area)	
`alpha`	attractiveness exponent	
`beta`	distance-decay exponent	

output	meaning
`probabilities`	probability of each store
`distances`	shortest-path distance to each store

- **Method:** Dijkstra distances from the origin; $P_j = S_j^{\alpha}$ d_j^-beta / sum_k $S_k^{\alpha}\, d_k^{-\beta}$
- **Assumes:** every store reachable and not at the origin

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Huff model on the worked example's network (edge lengths printed; layout without meaning): a shopper at node 0 chooses among stores 1, 2, 3 with attractiveness [5000, 12000, 8000] (α = 1, distance decay β = 2). The package returns shortest-path distances 2, 5, 6 and patronage probabilities 0.640296, 0.245874, 0.11383: the nearest store wins despite being the least attractive.

```
geospatial_analytics_ai999x.solve('geo_huff_model', edges='0-1 2; 1-2 3; 0-2 6; 2-3 1; 1-3 5',
origin=0, stores=[1, 2, 3], attractiveness=[5000, 12000, 8000], alpha=1, beta=2)
# -> probabilities = [0.640295959021, 0.245873648264, 0.113830392715], distances = [2.0, 5.0, 6.0]
(SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_huff_model', edges='0-1 1; 1-2 1; 2-3 1; 3-4 1', origin=2,
stores=[0, 4], attractiveness=[100, 300], alpha=1, beta=1.5)
# -> probabilities = [0.25, 0.75], distances = [2.0, 2.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Huff model: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 1-3 5], origin = 0, stores = [1, 2, 3], attractiveness = [5000, 12000, 8000], alpha = 1, beta = 2	SOLVED	probabilities = [0.640295959021, 0.245873648264, 0.113830392715], distances = [2.0, 5.0, 6.0]
Huff model: edges = [0-1 1; 1-2 1; 2-3 1; 3-4 1], origin = 2, stores = [0, 4], attractiveness = [100, 300], alpha = 1, beta = 1.5	SOLVED	probabilities = [0.25, 0.75], distances = [2.0, 2.0]
Huff model: origin = 0, stores = [1, 2, 3], attractiveness = [5000, 12000, 8000], alpha = 1, beta = 2	NEEDS_INPUT	geo_huff_model needs edges (undirected network as weighted edges between nodes numbered 0 .. n-1). Example: solve('geo_h

question	outcome	result
Huff model: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 1-3 5], origin = 0, stores = [1, 2, 3], attractiveness = [5000, 12000, 8000], alpha = 1, beta = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Two-step floating catchment area (2SFCA) accessibility — `geo_two_step_floating_catchment`

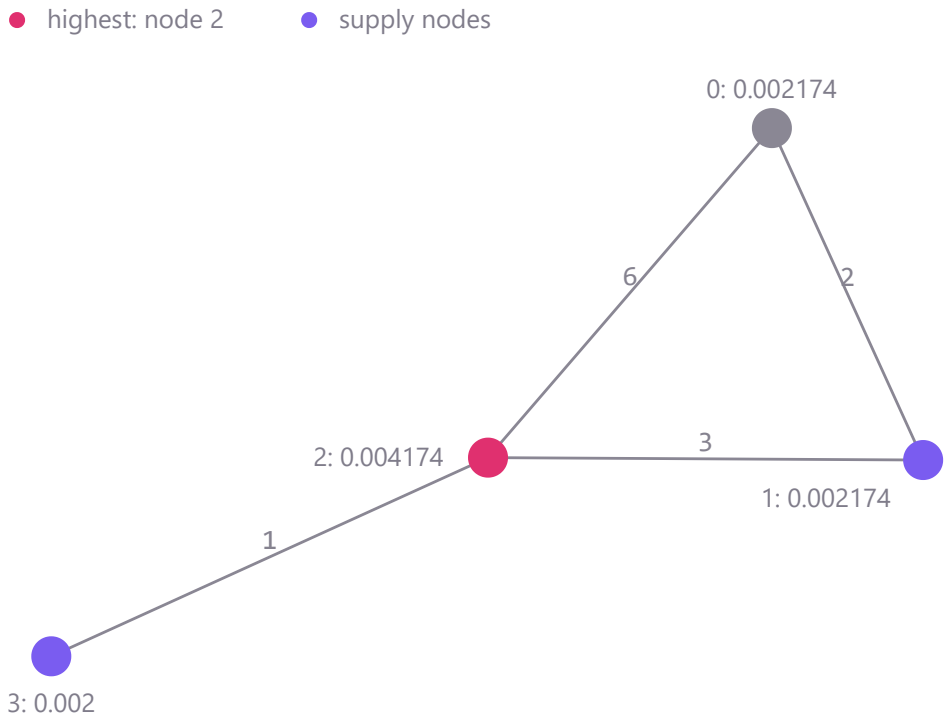
The 2SFCA spatial accessibility of every node to services (doctors, schools): supply-to-population ratios within each site's catchment, summed over the sites within reach of each node.

$$R_j = \frac{S_j}{\sum_{k:d_{kj}\le d_0} P_k}, \quad A_i = \sum_{j:d_{ij}\le d_0} R_j$$

input	meaning	condition
`edges`	undirected network as weighted edges between nodes numbered 0 .. $n-1$	
`population`	population at nodes 0 .. $n-1$	
`supply_nodes`	nodes with a service site	
`capacity`	capacity of each site (e.g. physicians)	
`threshold`	catchment threshold (network distance)	

output	meaning
`accessibility`	accessibility of each node (supply per person)
`ratios`	supply-to-population ratio of each site

- **Method:** step 1: R_j = S_j / sum of population within d0 of j; step 2: A_i = sum of R_j over sites within d0 of i (Luo and Wang 2003)
- **Assumes:** a catchment includes nodes at network distance <= threshold; a site with no population in reach has ratio 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example's network (edge lengths printed; layout without meaning) with each node labelled by the accessibility (supply per person) the package returns: node 0 0.00217391, node 1 0.00217391, node 2 0.00417391, node 3 0.002.

```
geospatial_analytics_ai999x.solve('geo_two_step_floating_catchment', edges='0-1 2; 1-2 3; 0-2 6; 2-3 1', population=[1000, 500, 800, 200], supply_nodes=[1, 3], capacity=[5, 2], threshold=3)
# -> accessibility = [0.00217391304348, 0.00217391304348, 0.00417391304348, 0.002], ratios = [0.00217391304348, 0.002]   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_two_step_floating_catchment', edges='0-1 5; 1-2 5; 2-3 5; 3-4 5', population=[100, 200, 300, 400, 500], supply_nodes=[0, 4], capacity=[3, 6], threshold=10)
# -> accessibility = [0.005, 0.005, 0.01, 0.005, 0.005], ratios = [0.005, 0.005]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-step floating catchment: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1], population = [1000, 500, 800, 200], supply_nodes = [1, 3], capacity = [5, 2], threshold = 3	SOLVED	accessibility = [0.00217391304348, 0.00217391304348, 0.00417391304348, 0.002], ratios = [0.00217391304348, 0.002]
Two-step floating catchment: edges = [0-1 5; 1-2 5; 2-3 5; 3-4 5], population = [100, 200, 300, 400, 500], supply_nodes = [0, 4], capacity = [3, 6], threshold = 10	SOLVED	accessibility = [0.005, 0.005, 0.01, 0.005, 0.005], ratios = [0.005, 0.005]
Two-step floating catchment: population = [1000, 500, 800, 200], supply_nodes = [1, 3], capacity = [5, 2], threshold = 3	NEEDS_INPUT	geo_two_step_floating_catchment needs edges (undirected network as weighted edges between nodes numbered 0 .. n-1). Exam
Two-step floating catchment: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1], population = [1000, 500, 800, 200], supply_nodes = [1, 3], capacity = [5, 2], threshold = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Network service area (isochrone) of a facility — `geo_service_area`

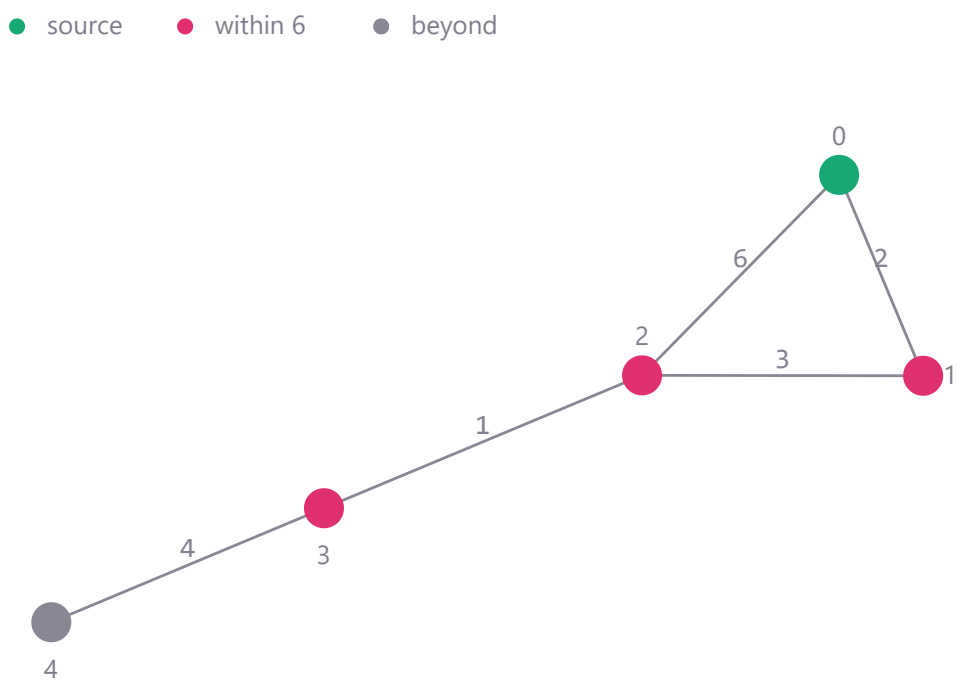
The nodes of a road network within a travel-cost cutoff of a facility (the service area or isochrone), their number and the largest distance reached.

$$SA(s, c) = \{v : d(s, v) \le c\}$$

input	meaning	condition
`edges`	undirected network as weighted edges between nodes numbered 0 .. $n-1$	
`source`	facility node	
`cutoff`	maximum network distance or travel time	

output	meaning
`reachable`	nodes within the cutoff (ascending)
`count`	number of nodes reached (the facility included)
`farthest`	largest network distance within the cutoff

- **Method:** Dijkstra from the facility, keeping the nodes with distance <= cutoff
- **Assumes:** positive edge costs; distance <= cutoff counts as reached
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Service area from node 0 with cutoff 6 on the worked example's network (edge lengths printed; layout without meaning): the nodes the package returns as reachable (0, 1, 2, 3; 4 nodes, farthest at 6) are highlighted.

```
geospatial_analytics_ai999x.solve('geo_service_area', edges='0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4',
source=0, cutoff=6)
# -> reachable = [0.0, 1.0, 2.0, 3.0], count = 4, farthest = 6   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_service_area', edges='0-1 1; 1-2 1; 2-3 1; 3-4 1', source=2,
cutoff=1.5)
# -> reachable = [1.0, 2.0, 3.0], count = 3, farthest = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Service area: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4], source = 0, cutoff = 6	SOLVED	reachable = [0.0, 1.0, 2.0, 3.0], count = 4, farthest = 6
Service area: edges = [0-1 1; 1-2 1; 2-3 1; 3-4 1], source = 2, cutoff = 1.5	SOLVED	reachable = [1.0, 2.0, 3.0], count = 3, farthest = 1
Service area: source = 0, cutoff = 6	NEEDS_INPUT	geo_service_area needs edges (undirected network as weighted edges between nodes numbered 0 .. n-1). Example: solve('geo
Service area: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4], source = 0, cutoff = 6, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Closest facility for every node of a network — `geo_closest_facility`

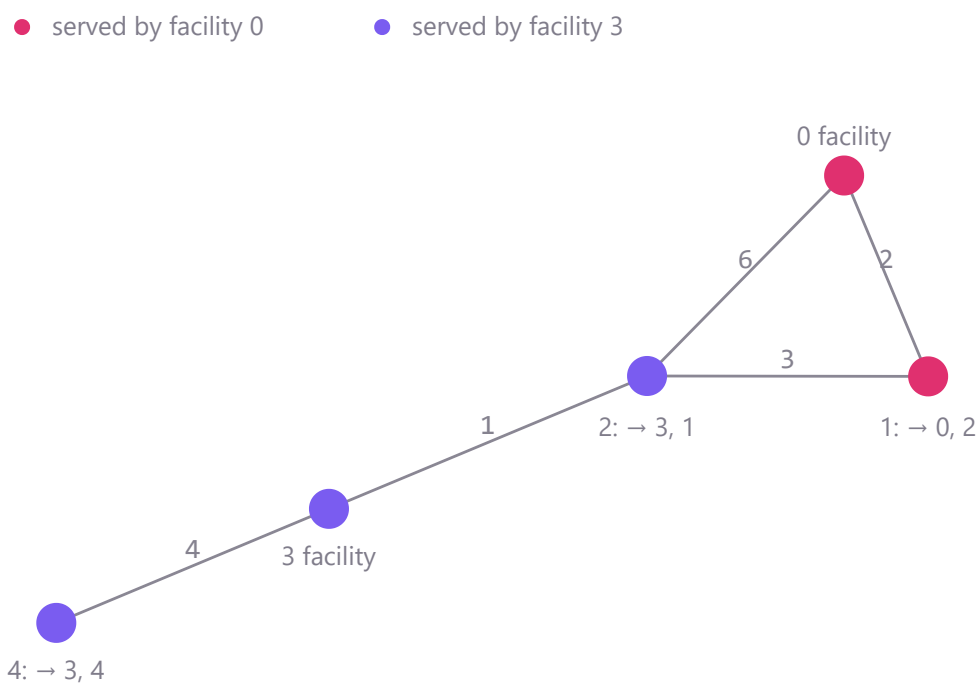
For every node of a road network, the nearest facility by shortest-path distance and that distance, with the mean distance to the nearest facility.

$$f^*(i) = \arg\min_{f \in F} d(i, f)$$

input	meaning	condition
`edges`	undirected network as weighted edges between nodes numbered 0 .. $n-1$	
`facilities`	facility nodes	

output	meaning
`nearest`	nearest facility of each node
`distance`	distance to it
`mean_distance`	mean distance to the nearest facility

- **Method:** Dijkstra from every facility; each node takes the minimum
- **Assumes:** every node reaches a facility; no node equally near two facilities
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example's network (edge lengths printed; layout without meaning) with facilities at nodes 0, 3: each node is coloured by the nearest facility the package returns and labelled with it and the shortest-path distance; mean distance 1.4.

```
geospatial_analytics_ai999x.solve('geo_closest_facility', edges='0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4
4', facilities=[0, 3])
# -> nearest = [0.0, 0.0, 3.0, 3.0, 3.0], distance = [0.0, 2.0, 1.0, 0.0, 4.0], mean_distance = 1.4
(SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_closest_facility', edges='0-1 1; 1-2 1.5; 2-3 1; 3-4 1',
facilities=[4])
# -> nearest = [4.0, 4.0, 4.0, 4.0, 4.0], distance = [4.5, 3.5, 2.0, 1.0, 0.0], mean_distance = 2.2
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Closest facility: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4], facilities = [0, 3]	SOLVED	nearest = [0.0, 0.0, 3.0, 3.0, 3.0], distance = [0.0, 2.0, 1.0, 0.0, 4.0], mean_distance = 1.4
Closest facility: edges = [0-1 1; 1-2 1.5; 2-3 1; 3-4 1], facilities = [4]	SOLVED	nearest = [4.0, 4.0, 4.0, 4.0, 4.0], distance = [4.5, 3.5, 2.0, 1.0, 0.0], mean_distance = 2.2
Closest facility: facilities = [0, 3]	NEEDS_INPUT	geo_closest_facility needs edges (undirected network as weighted edges between nodes numbered 0 .. n-1). Example: solve(
Closest facility: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4], facilities = [0, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

p-median facility location on a network — `geo_p_median`

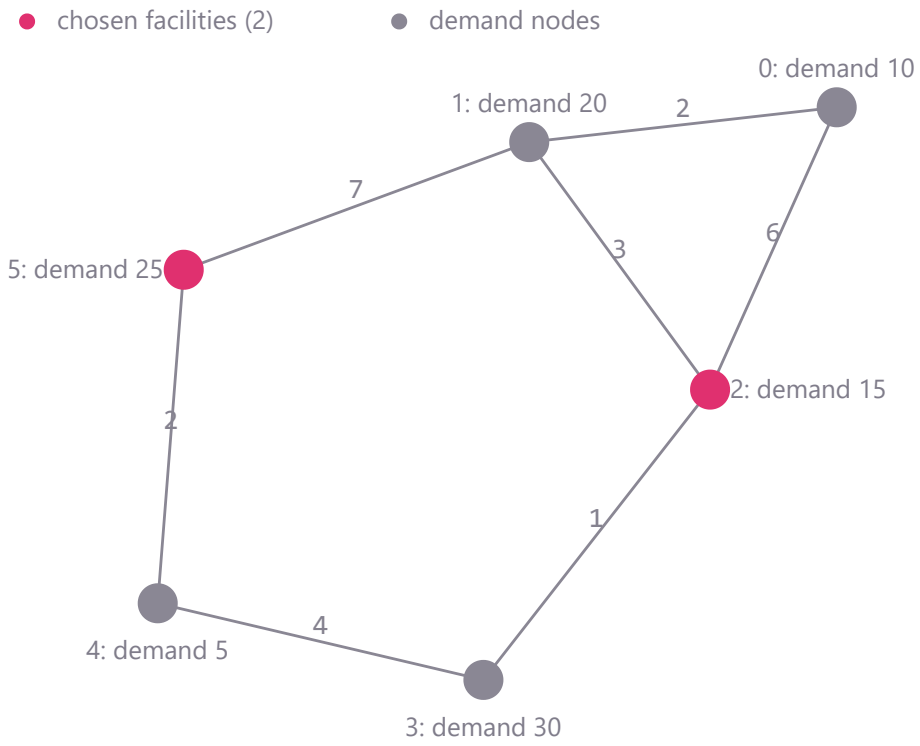
The p facility sites, chosen from candidate nodes, that minimize the total demand-weighted shortest-path distance from every node to its nearest open facility.

$$\min_{S \subseteq C, |S|=p} \sum_i w_i \min_{j \in S} d_{ij}$$

input	meaning	condition
`edges`	undirected network as weighted edges between nodes numbered 0 .. $n-1$	
`demand`	demand at nodes 0 .. $n-1$	
`candidates`	candidate facility nodes	
`p`	number of facilities to open	

output	meaning
`objective`	minimum total weighted distance
`facilities`	chosen facility nodes (ascending)
`mean_distance`	demand-weighted mean distance

- **Method:** exact enumeration of the candidate combinations with all-pairs Dijkstra distances
- **Assumes:** connected network; at most 20000 candidate combinations; a unique optimal set
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The p-median solution (p = 2) on the worked example's network (edge lengths printed; layout without meaning; demand at each node): the package places facilities at nodes 2, 5, with total demand-weighted distance 150 (mean 1.42857).

```
geospatial_analytics_ai999x.solve('geo_p_median', edges='0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4; 4-5 2;
1-5 7', demand=[10, 20, 15, 30, 5, 25], candidates=[0, 1, 2, 3, 4, 5], p=2)
# -> objective = 150, facilities = [2.0, 5.0], mean_distance = 1.42857   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_p_median', edges='0-1 1; 1-2 1; 2-3 1; 3-4 1; 4-5 1',
demand=[5, 1, 1, 1, 1, 7], candidates=[0, 2, 3, 5], p=1)
# -> objective = 33, facilities = [3.0], mean_distance = 2.0625   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
p-median: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4; 4-5 2; 1-5 7], demand = [10, 20, 15, 30, 5, 25], candidates = [0, 1, 2, 3, 4, 5], p = 2	SOLVED	objective = 150, facilities = [2.0, 5.0], mean_distance = 1.42857
p-median: edges = [0-1 1; 1-2 1; 2-3 1; 3-4 1; 4-5 1], demand = [5, 1, 1, 1, 1, 7], candidates = [0, 2, 3, 5], p = 1	SOLVED	objective = 33, facilities = [3.0], mean_distance = 2.0625
p-median: demand = [10, 20, 15, 30, 5, 25], candidates = [0, 1, 2, 3, 4, 5], p = 2	NEEDS_INPUT	geo_p_median needs edges (undirected network as weighted edges between nodes numbered 0 .. n-1). Example: solve('geo_p_m
p-median: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4; 4-5 2; 1-5 7], demand = [10, 20, 15, 30, 5, 25], candidates = [0, 1, 2, 3, 4, 5], p = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Maximal covering location problem (MCLP) on a network — `geo_maximal_covering_location`

The largest demand that p facilities, chosen from candidate nodes, can cover within a coverage distance on a road network, and the covered share.

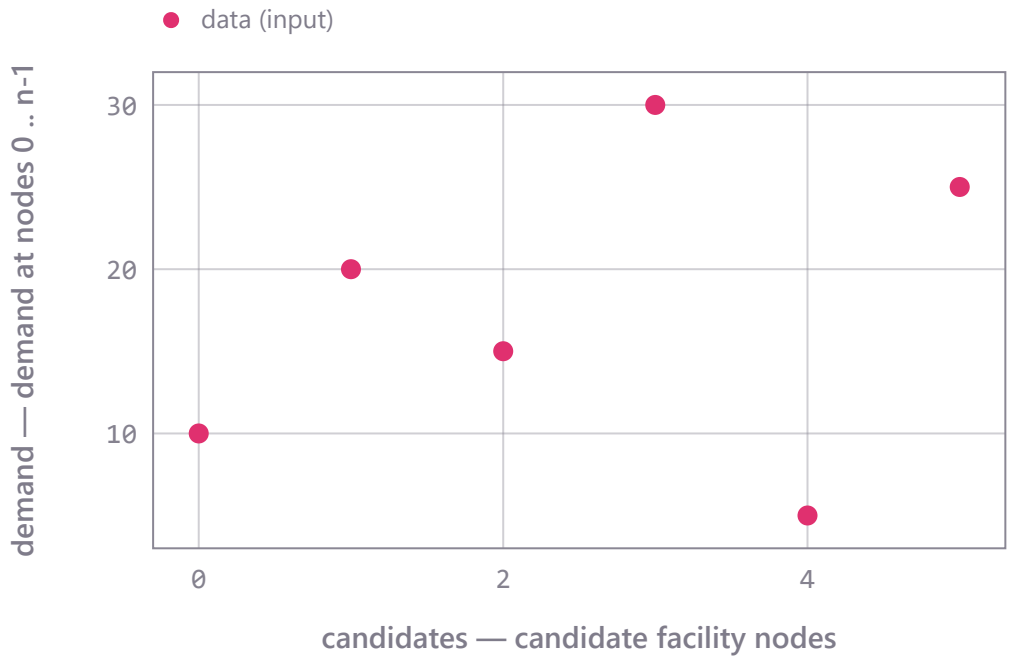
$$\max_{S \subseteq C, |S|=p} \sum_i w_i \, \mathbf{1}\left[\min_{j \in S} d_{ij} \le s\right]$$

input	meaning	condition
`edges`	undirected network as weighted edges between nodes numbered 0 .. $n-1$	
`demand`	demand at nodes 0 .. $n-1$	
`candidates`	candidate facility nodes	
`p`	number of facilities	
`coverage`	coverage distance (network distance <= coverage counts)	

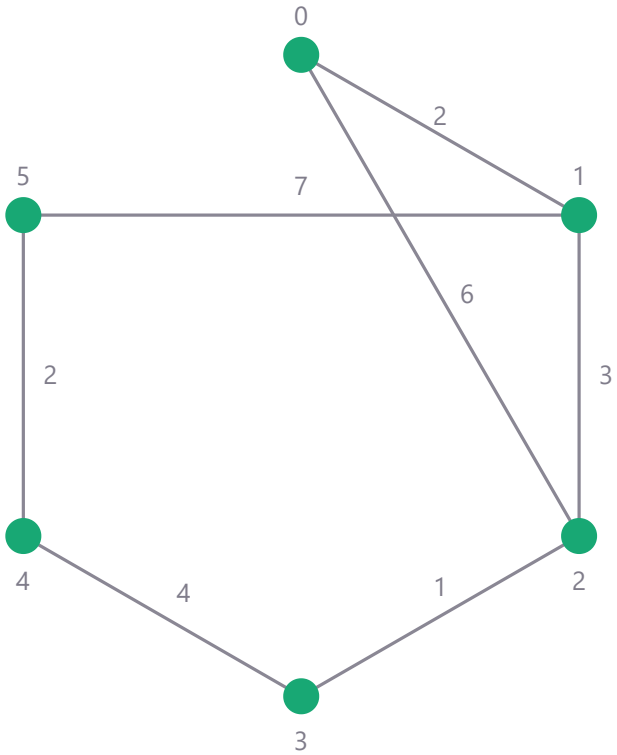
output	meaning
`covered`	maximum covered demand
`fraction`	covered share of the total demand

- **Method:** exact enumeration of the candidate combinations (Church and ReVelle 1974 objective)
- **Assumes:** at most 20000 candidate combinations

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data demand (demand at nodes 0 .. n-1) against candidates (candidate facility nodes) as given in the worked example (6 points). Also returned: covered = 95, fraction = 0.904762. Other inputs: edges = 0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4; 4-5 2; 1-5 7, p = 2, coverage = 3.



The network edges (undirected network as weighted edges between nodes numbered 0 .. n-1) of the worked example (6 nodes, 7 edges, laid out on a circle); on each edge its stated weight. Returned: covered = 95, fraction = 0.904762. Other inputs: demand = [10, 20, 15, 30, 5, 25], candidates = [0, 1, 2, 3, 4, 5], p = 2, coverage = 3.

```
geospatial_analytics_ai999x.solve('geo_maximal_covering_location', edges='0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4; 4-5 2; 1-5 7', demand=[10, 20, 15, 30, 5, 25], candidates=[0, 1, 2, 3, 4, 5], p=2, coverage=3)
# -> covered = 95, fraction = 0.904762   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_maximal_covering_location', edges='0-1 1; 1-2 1; 2-3 1; 3-4 1; 4-5 1', demand=[5, 1, 1, 1, 1, 7], candidates=[0, 2, 3, 5], p=1, coverage=2)
# -> covered = 11, fraction = 0.6875   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximal covering location: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4; 4-5 2; 1-5 7], demand = [10, 20, 15, 30, 5, 25], candidates = [0, 1, 2, 3, 4, 5], p = 2, coverage = 3	SOLVED	covered = 95, fraction = 0.904762
Maximal covering location: edges = [0-1 1; 1-2 1; 2-3 1; 3-4 1; 4-5 1], demand = [5, 1, 1, 1, 1, 7], candidates = [0, 2, 3, 5], p = 1, coverage = 2	SOLVED	covered = 11, fraction = 0.6875
Maximal covering location: demand = [10, 20, 15, 30, 5, 25], candidates = [0, 1, 2, 3, 4, 5], p = 2, coverage = 3	NEEDS_INPUT	geo_maximal_covering_location needs edges (undirected network as weighted edges between nodes numbered 0 .. n-1). Exampl
Maximal covering location: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1; 3-4 4; 4-5 2; 1-5 7], demand = [10, 20, 15, 30, 5, 25], candidates = [0, 1, 2, 3, 4, 5], p = 2, coverage = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Network detour index (circuity) between two nodes — `geo_detour_index`

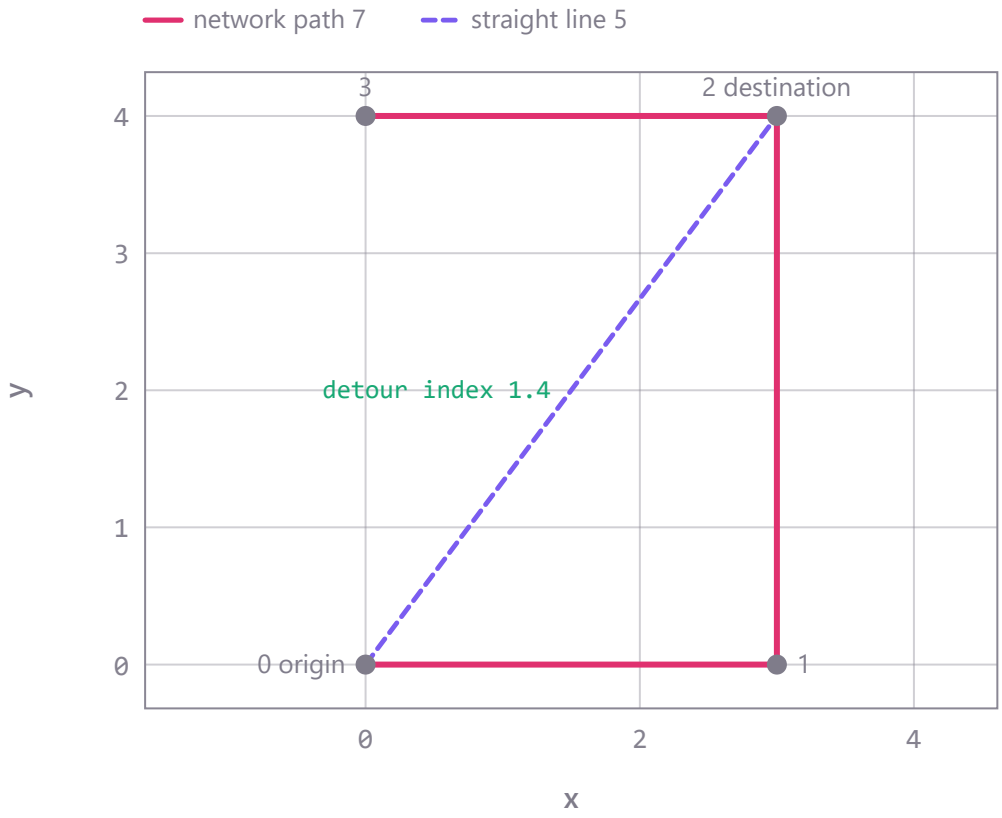
The shortest network distance between two nodes of a street network (edge lengths from the node coordinates), the straight-line distance and their ratio, the detour index or circuity.

$$DI = \frac{d_{net}(o,t)}{\|\mathbf{x}_o - \mathbf{x}_t\|}$$

input	meaning	condition
`coords`	x and y of nodes 0 .. $n-1$ (n x 2, projected coordinates)	
`edges`	street segments between nodes (lengths are computed from the coordinates)	
`origin`	origin node	
`destination`	destination node	

output	meaning
`network_distance`	shortest network distance
`euclidean_distance`	straight-line distance
`detour_index`	network / straight-line distance (>= 1)

- **Method:** Euclidean edge lengths, Dijkstra, ratio to the straight-line distance
- **Assumes:** straight edges between the node coordinates; destination reachable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Nodes at the stated coordinates joined by the edges 0-1; 1-2; 2-3: from node 0 to node 2 the package returns a network distance 7 against a straight-line distance 5, a detour index of 1.4.

```
geospatial_analytics_ai999x.solve('geo_detour_index', coords=[[0, 0], [3, 0], [3, 4], [0, 4]],
edges='0-1; 1-2; 2-3', origin=0, destination=2)
# -> network_distance = 7, euclidean_distance = 5, detour_index = 1.4   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_detour_index', coords=[[0, 0], [1, 1], [2, 0], [3, 1], [4,
0]], edges='0-1; 1-2; 2-3; 3-4; 0-2', origin=0, destination=4)
# -> network_distance = 4.82843, euclidean_distance = 4, detour_index = 1.20711   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Detour index: coords = [[0, 0], [3, 0], [3, 4], [0, 4]], edges = [0-1; 1-2; 2-3], origin = 0, destination = 2	SOLVED	network_distance = 7, euclidean_distance = 5, detour_index = 1.4
Detour index: coords = [[0, 0], [1, 1], [2, 0], [3, 1], [4, 0]], edges = [0-1; 1-2; 2-3; 3-4; 0-2], origin = 0, destination = 4	SOLVED	network_distance = 4.82843, euclidean_distance = 4, detour_index = 1.20711
Detour index: edges = [0-1; 1-2; 2-3], origin = 0, destination = 2	NEEDS_INPUT	geo_detour_index needs coords (x and y of nodes 0 .. n-1 (n x 2, projected coordinates)). Example: solve('geo_detour_ind

question	outcome	result
Detour index: coords = [[0, 0], [3, 0], [3, 4], [0, 4]], edges = [0-1; 1-2; 2-3], origin = 0, destination = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Network connectivity indices: cyclomatic number, alpha, beta and gamma — `geo_network_connectivity_indices`

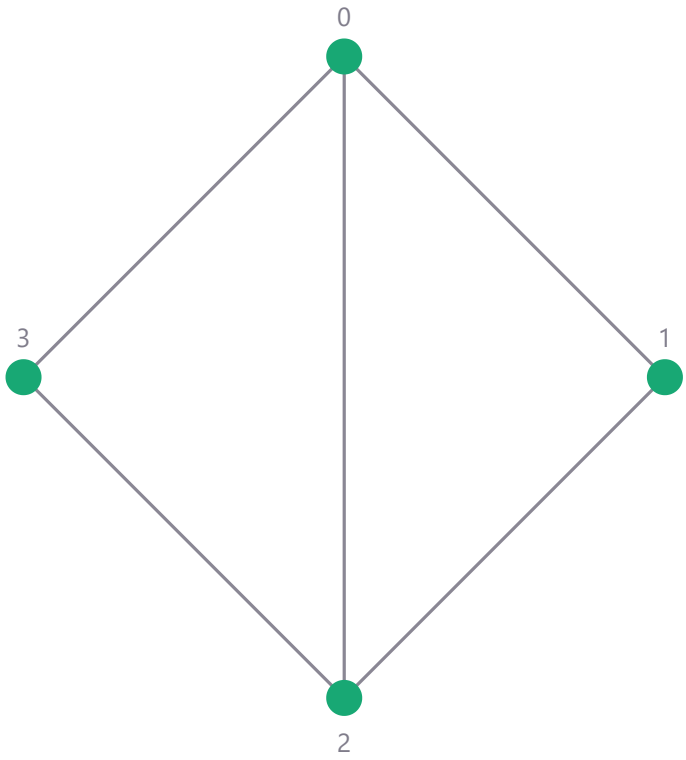
The graph-theoretic connectivity measures of a (planar) transport network: vertices, edges, components, cyclomatic number and Kansky's alpha, beta and gamma indices.

$$\mu = e - v + p, \quad \alpha = \frac{e - v + p}{2v - 5}, \quad \beta = \frac{e}{v}, \quad \gamma = \frac{e}{3(v - 2)}$$

input	meaning	condition
`edges`	network edges between nodes numbered from 0 (lengths, if given, are ignored)	

output	meaning
`vertices`	number of vertices v
`edges`	number of distinct edges e
`components`	connected components p
`cyclomatic`	cyclomatic number e - v + p
`alpha`	$(e - v + p)/(2v - 5)$
`beta`	e / v
`gamma`	e / (3(v - 2))

- **Method:** count vertices and distinct edges; components by union-find
- **Assumes:** at least 3 vertices; planar formulas for alpha and gamma
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (network edges between nodes numbered from 0 (lengths, if given, are ignored)) of the worked example (4 nodes, 5 edges, laid out on a circle). Returned: vertices = 4, edges = 5, components = 1, cyclomatic = 2, alpha = 0.666667, beta = 1.25, gamma = 0.833333.

```
geospatial_analytics_ai999x.solve('geo_network_connectivity_indices', edges='0-1; 1-2; 2-3; 3-0; 0-2')
# -> vertices = 4, edges = 5, components = 1, cyclomatic = 2, alpha = 0.666667, beta = 1.25, gamma = 0.833333   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_network_connectivity_indices', edges='0-1; 1-2; 3-4; 4-5; 5-3')
# -> vertices = 6, edges = 5, components = 2, cyclomatic = 1, alpha = 0.142857, beta = 0.833333, gamma = 0.416667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Network connectivity indices: edges = [0-1; 1-2; 2-3; 3-0; 0-2]	SOLVED	vertices = 4, edges = 5, components = 1, cyclomatic = 2, alpha = 0.666667, beta = 1.25, gamma = 0.833333
Network connectivity indices: edges = [0-1; 1-2; 3-4; 4-5; 5-3]	SOLVED	vertices = 6, edges = 5, components = 2, cyclomatic = 1, alpha = 0.142857, beta = 0.833333, gamma = 0.416667
Network connectivity indices:	NEEDS_INPUT	geo_network_connectivity_indices needs edges (network edges between nodes numbered from 0 (lengths, if given, are ignore
Network connectivity indices: edges = [0-1; 1-2; 2-3; 3-0; 0-2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Shimbel accessibility index of every node — `geo_shimbel_index`

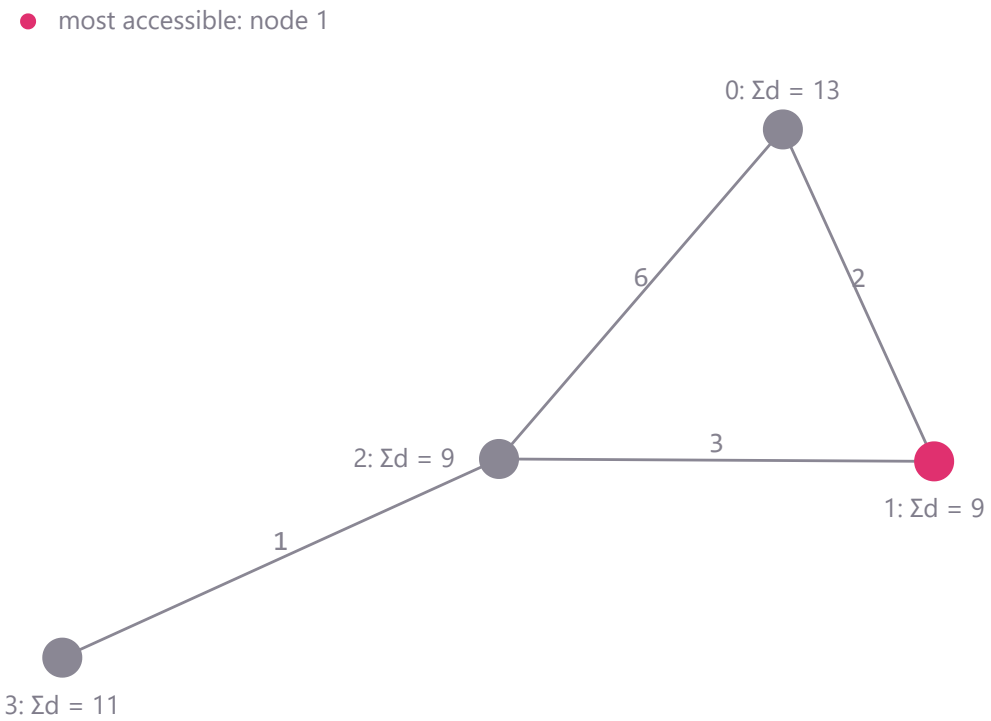
The Shimbel (total shortest-path distance) index of every node of a connected network, the network total (dispersion) and the most accessible node (smallest index).

$$A_i = \sum_j d_{ij}, \quad D(G) = \sum_i A_i$$

input	meaning	condition
`edges`	undirected network as weighted edges between nodes numbered 0 .. $n-1$	

output	meaning
`shimbel`	sum of shortest-path distances from each node
`total`	network dispersion (sum over all nodes)
`most_accessible`	node with the smallest Shimbel index (lowest number on ties)

- **Method:** all-pairs Dijkstra; row sums of the distance matrix
- **Assumes:** connected network with nodes 0 .. $n-1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Shimbel index of each node of the worked example's network (edge lengths printed; layout without meaning): the sum of shortest-path distances to all other nodes, as returned: node 0 13, node 1 9, node 2 9, node 3 11 (total 42); the smallest sum, node 1, is the most accessible.

```
geospatial_analytics_ai999x.solve('geo_shimbel_index', edges='0-1 2; 1-2 3; 0-2 6; 2-3 1')
# -> shimbel = [13.0, 9.0, 9.0, 11.0], total = 42, most_accessible = 1   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_shimbel_index', edges='0-1 1; 1-2 1; 2-3 1; 3-4 1')
# -> shimbel = [10.0, 7.0, 6.0, 7.0, 10.0], total = 40, most_accessible = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Shimbel index: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1]	SOLVED	shimbel = [13.0, 9.0, 9.0, 11.0], total = 42, most_accessible = 1
Shimbel index: edges = [0-1 1; 1-2 1; 2-3 1; 3-4 1]	SOLVED	shimbel = [10.0, 7.0, 6.0, 7.0, 10.0], total = 40, most_accessible = 2
Shimbel index:	NEEDS_INPUT	geo_shimbel_index needs edges (undirected network as weighted edges between nodes numbered 0 .. n-1). Example: solve('ge
Shimbel index: edges = [0-1 2; 1-2 3; 0-2 6; 2-3 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Residential segregation (evenness): dissimilarity, Gini and Theil entropy indices — `geo_segregation_evenness`

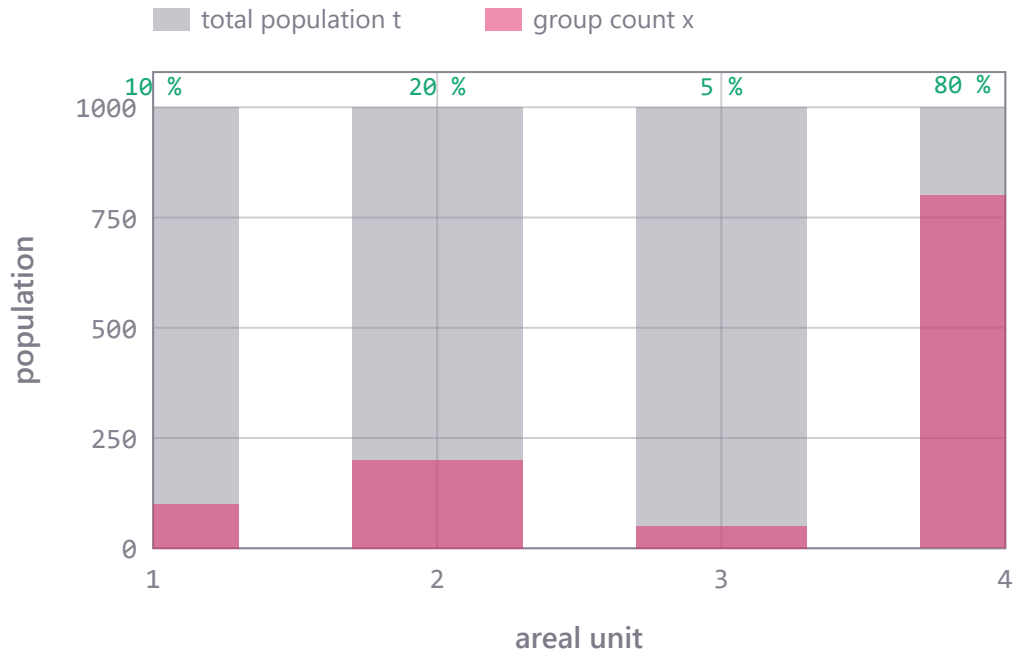
The evenness dimension of residential segregation of a group across areal units (census tracts): Duncan's index of dissimilarity, the segregation Gini index and Theil's entropy index H.

$$D = \frac{\sum_i t_i |p_i - P|}{2TP(1-P)}, \quad G = \frac{\sum_i \sum_j t_i t_j |p_i - p_j|}{2T^2 P(1-P)}, \quad H = \sum_i \frac{t_i (E - E_i)}{ET}$$

input	meaning	condition
`x`	group count in each areal unit	
`t`	total population of each areal unit	

output	meaning
`dissimilarity`	index of dissimilarity D
`gini`	Gini segregation index
`theil_h`	Theil's information theory index H

- **Method:** Massey and Denton (1988): D = sum t_i |p_i - P| / (2TP(1-P)); G = sum_ij t_i t_j |p_i - p_j| / (2T^2P(1-P)); H = sum t_i (E - E_i)/(ET)
- **Assumes:** $0 \le x \le t$ in each unit; the group present but not the whole population
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Group count x = [100, 200, 50, 800] within totals t = [1000, 1000, 1000, 1000] in each areal unit (share printed above each bar, computed here from the inputs). The package returns dissimilarity = 0.625477, gini = 0.71701, theil_h = 0.364724.

```
geospatial_analytics_ai999x.solve('geo_segregation_evenness', x=[100, 200, 50, 800], t=[1000, 1000, 1000, 1000])
# -> dissimilarity = 0.625477, gini = 0.71701, theil_h = 0.364724   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_segregation_evenness', x=[0, 10, 990], t=[500, 500, 1000])
# -> dissimilarity = 0.98, gini = 0.985, theil_h = 0.924243   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Segregation evenness: x = [100, 200, 50, 800], t = [1000, 1000, 1000, 1000]	SOLVED	dissimilarity = 0.625477, gini = 0.71701, theil_h = 0.364724
Segregation evenness: x = [0, 10, 990], t = [500, 500, 1000]	SOLVED	dissimilarity = 0.98, gini = 0.985, theil_h = 0.924243
Segregation evenness: t = [1000, 1000, 1000, 1000]	NEEDS_INPUT	geo_segregation_evenness needs x (group count in each areal unit). Example: solve('geo_segregation_evenness', x= [100, 20
Segregation evenness: x = [100, 200, 50, 800], t = [1000, 1000, 1000, 1000], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ

Residential segregation (exposure): isolation, interaction and correlation ratio — `geo_segregation_exposure`

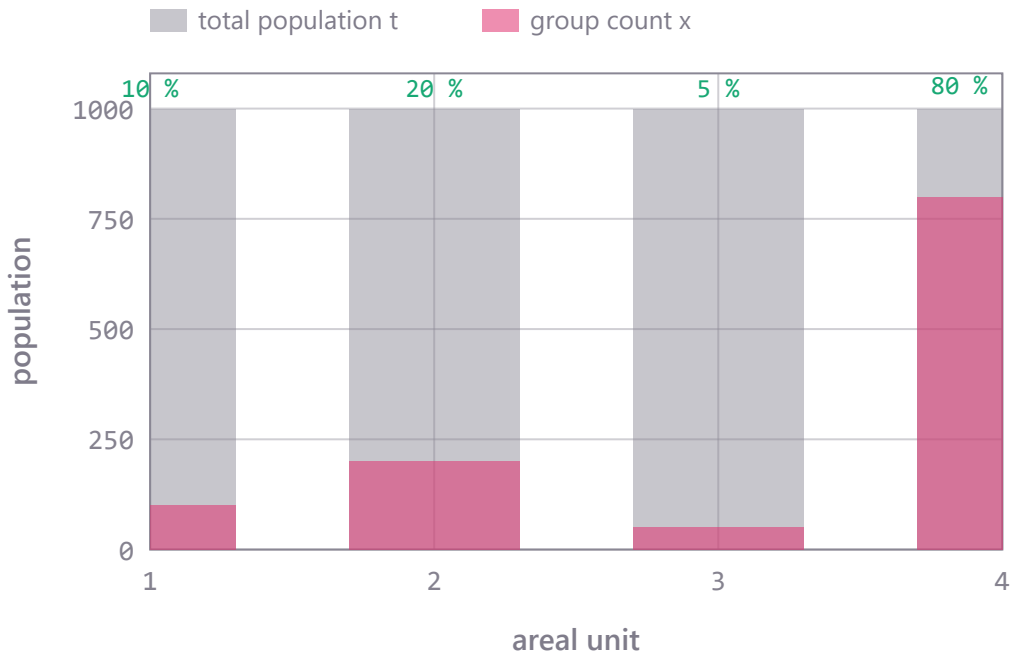
The exposure dimension of residential segregation: the isolation index xPx (probability that a group member meets a member of the same group in their unit), the interaction (exposure) index xPy and the correlation ratio η^2.

$$ {}_xP_x^* = \sum_i \frac{x_i}{X}\frac{x_i}{t_i}, \quad {}_xP_y^* = \sum_i \frac{x_i}{X}\frac{y_i}{t_i}, \quad \eta^2 = \frac{{}_xP_x^* - P}{1 - P} $$

input	meaning	condition
`x`	group count in each areal unit	
`t`	total population of each areal unit	

output	meaning
`isolation`	isolation index xPx
`exposure`	interaction index xPy
`correlation_ratio`	correlation ratio (xPx - P)/(1 - P)

- **Method:** xPx = sum (x_i/X)(x_i/t_i); xPy = sum (x_i/X)(y_i/t_i) with $y = t - x$; eta^2 = (xPx - P)/(1 - P)
- **Assumes:** $0 \le x \le t$ in each unit; the group present but not the whole population
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Group count x = [100, 200, 50, 800] within totals t = [1000, 1000, 1000, 1000] in each areal unit (share printed above each bar, computed here from the inputs). The package returns isolation = 0.602174, exposure = 0.397826, correlation_ratio = 0.441648.

```
geospatial_analytics_ai999x.solve('geo_segregation_exposure', x=[100, 200, 50, 800], t=[1000, 1000, 1000, 1000])
# -> isolation = 0.602174, exposure = 0.397826, correlation_ratio = 0.441648   (SOLVED)
```

```
geospatial_analytics_ai999x.solve('geo_segregation_exposure', x=[0, 10, 990], t=[500, 500, 1000])
# -> isolation = 0.9803, exposure = 0.0197, correlation_ratio = 0.9606   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Segregation exposure: x = [100, 200, 50, 800], t = [1000, 1000, 1000, 1000]	SOLVED	isolation = 0.602174, exposure = 0.397826, correlation_ratio = 0.441648
Segregation exposure: x = [0, 10, 990], t = [500, 500, 1000]	SOLVED	isolation = 0.9803, exposure = 0.0197, correlation_ratio = 0.9606
Segregation exposure: t = [1000, 1000, 1000, 1000]	NEEDS_INPUT	geo_segregation_exposure needs x (group count in each areal unit). Example: solve('geo_segregation_exposure', x= [100, 20
Segregation exposure: x = [100, 200, 50, 800], t = [1000, 1000, 1000, 1000], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: geospatial_analytics_ai999x.templ